

Solving Problems with Python:

**A Practical Guide for
Pupils, Students, and Teachers.**

KLEMENS NGUYEN



Introduction: Why Python?

What Makes Python the Perfect Tool for Problem Solving?

Overview of This Book: Learning Through Practical Examples

Getting Started with Python: A Quick Setup Guide

Who Is This Book For? (Pupils, Students, and Teachers)

How to Use This Book: Tips for Maximizing Your Learning

Part I: Python Basics for Problem Solving

Chapter 1: Python Fundamentals

Understanding Variables, Data Types, and Basic Syntax

How Python Handles Numbers, Text, and Lists

Writing Your First Python Script: Simple Calculations

Chapter 2: Control Structures: If, Else, and Loops

Using Conditional Statements to Make Decisions

Loops: Solving Repetitive Problems Efficiently

Practical Examples: Building a Simple Quiz

Chapter 3: Functions: Breaking Down Complex Problems

Writing and Using Functions for Modularity

Understanding Function Arguments and Return Values

Real-World Application: A Calculator Program

Part II: Problem-Solving with Python

Chapter 4: Solving Math Problems with Python

Basic Arithmetic and Algebra with Python

Finding Factors, Prime Numbers, and Fibonacci Sequences

Real-World Problem: Solving Geometry Problems (Area, Perimeter)

Chapter 5: Working with Text: Strings and Patterns

Manipulating Strings to Solve Text-Based Challenges

Finding Patterns with Regular Expressions

Application: Building a Simple Text-Based Game

Chapter 6: Data Handling with Lists and Dictionaries

[Solving Problems Using Lists, Tuples, and Dictionaries](#)

[Counting, Sorting, and Organizing Data](#)

[Application: Managing a Simple Contact Book with Python](#)

[Chapter 7: File Handling: Reading, Writing, and Managing Files](#)

[Reading from and Writing to Text Files](#)

[Solving Real Problems with Data from Files](#)

[Building a Simple Log System for Classroom Activities](#)

[Part III: Intermediate Python: Working with Libraries](#)

[Chapter 8: Problem Solving with NumPy and Pandas](#)

[Introduction to Data Structures in NumPy and Pandas](#)

[Using Pandas to Solve Real-World Data Problems](#)

[Chapter 9: Visualizing Solutions with Matplotlib](#)

[Creating Simple Charts and Graphs to Understand Data](#)

[Application: Plotting Temperature Changes Over Time](#)

[Visualizing Classroom Attendance Data](#)

[Chapter 10: Automating Everyday Tasks with Python](#)

[Automating Repetitive Tasks with Python](#)

[Application: Automating Email Sending for Class Updates](#)

[Building a Simple Task Scheduler](#)

[Part IV: Problem Solving in Real-World Scenarios](#)

[Chapter 11: Games and Puzzles with Python](#)

[Building Simple Games: Rock, Paper, Scissors, and Tic-Tac-Toe](#)

[Solving Logical Puzzles with Python Algorithms](#)

[Application: A Number Guessing Game for Students](#)

[Chapter 12: Web Scraping for Problem Solving](#)

[Extracting Useful Data from Websites](#)

[Application: Scraping Book Prices for a Library or Classroom](#)

[Ethical Considerations of Web Scraping](#)

[Chapter 13: Introduction to Machine Learning \(for Beginners\)](#)

[Understanding Basic Concepts of Machine Learning](#)

[Using Python to Build Simple Predictive Models](#)

[Example: Predicting Student Grades Based on Past Data](#)

[Part V: Advanced Projects for Students and Teachers](#)

[Chapter 14: Creating Educational Tools with Python](#)

[Building Flashcard Quizzes for Classroom Learning](#)

[Application: A Python-Based Quiz Maker for Teachers](#)

[Tracking Progress with Python](#)

[Chapter 15: Python in Science and Engineering](#)

[Solving Physics and Chemistry Problems with Python](#)

[Chapter 16: Working on Capstone Projects](#)

[How to Approach Larger Problems and Break Them Down](#)

[Capstone Example: A Classroom Management System](#)

[Further Learning: Expanding Your Python Skills Beyond This Book](#)

[Conclusion: Your Journey Ahead](#)

[How to Continue Problem Solving with Python](#)

[Recommended Resources and Communities for Python Learners](#)

[A Note to Teachers: How to Incorporate This Book into Your Curriculum](#)

Introduction: Why Python?

What Makes Python the Perfect Tool for Problem Solving?

Python is a fantastic language for problem solving, especially for students and those just starting out in coding. But why is Python the perfect tool? Let's break it down in a way that's easy to understand and fun to learn.

1. Simple and Easy to Read

Python was designed to be human-friendly. Unlike many other programming languages, Python code reads almost like plain English. This makes it a great choice for beginners because you don't have to worry about confusing symbols and strange syntax. Imagine writing out a shopping list or following a recipe — that's how straightforward Python can feel when you use it to solve problems.

Here's an example:

```
if temperature > 30:
    print("It's hot today!")
else:
    print("It's a bit cooler.")
```

NOTE! This is not a complete program. It is only four lines of code.

Even if you've never programmed before, you can guess what this code does! It checks if the temperature is above 30 degrees, and if it is, it prints "It's hot today!" Otherwise, it says, "It's a bit cooler." This kind of simplicity helps students focus on *how* to solve the problem rather than getting stuck on complicated code.

2. Versatile for All Kinds of Problems

Python is like a Swiss Army knife. Whether you're solving math equations, analyzing data, building a game, or even creating websites, Python can

handle it. It's flexible enough to be used in many different fields, from science and engineering to art and education. As a student, this means you can apply the skills you learn in Python to almost any subject or interest you have.

For example, you can use Python to:

- Solve math problems like finding the area of a circle.
- Analyze data from a science experiment.
- Create a simple game, like "Rock, Paper, Scissors."
- Automate tasks like organizing files on your computer.

3. Huge Support from the Python Community

Python has been around for a while, and it has one of the biggest communities of programmers in the world. This means that there are tons of resources, tutorials, and ready-made code that you can use to solve your problems. If you ever get stuck, a quick search online will likely lead you to a helpful explanation or example.

There's also something called **libraries** in Python. Think of libraries as pre-made tools that can save you a lot of time. Instead of building everything from scratch, you can use libraries to quickly solve problems. For example, if you want to draw a graph, you don't have to write all the code yourself — you can use a library like **Matplotlib** to do it in just a few lines.

4. It Helps You Think Like a Problem Solver

Learning Python isn't just about writing code — it's about learning how to think logically and break down problems step by step. This is a skill that will help you in every part of life, not just in programming. Python encourages you to approach problems in a structured way. Here's a simple process you'll often follow:

- **Understand the problem.** What do you need to solve?
- **Plan a solution.** What steps will you take to solve it?

- **Write the code.** How can you translate those steps into Python?
- **Test it out.** Did your solution work? If not, how can you fix it?

By working through this process, Python teaches you how to approach challenges in a logical and systematic way, which is an incredibly valuable skill for school, work, and beyond.

5. Python Grows with You

Finally, one of the best things about Python is that it grows with you as you become a more experienced coder. You can start small, solving basic problems like math equations or creating a quiz, and then, as you learn more, Python lets you build bigger and more complex projects. You could eventually create games, analyze huge sets of data, or even explore artificial intelligence!

The key is that Python won't overwhelm you right from the start, but it has the power to support your learning and growth as a programmer.

So, in a nutshell, Python is the perfect tool for problem solving because it's easy to understand, versatile, and helps you think in a structured way. Whether you're a beginner or ready to take on more complex challenges, Python will always have your back. You'll find yourself solving problems faster, more efficiently, and maybe even enjoying the process!

Overview of This Book: Learning Through Practical Examples

Welcome to this exciting journey into the world of Python programming! This book is all about learning how to solve problems using Python, and the best way to learn is through **doing**. That's why every chapter is packed with practical examples that will guide you step by step, helping you understand not just how to code, but how to think like a problem solver.

What Can You Expect from This Book?

The goal of this book is to make Python **fun**, **approachable**, and **useful**. Whether you're a pupil just starting out, a student looking to sharpen your skills, or a teacher introducing coding to your classroom, you'll find real-world examples and exercises that match your level.

Here's how we've structured the book to help you learn in the most effective way:

1. Learn Concepts in Simple Steps

Each chapter introduces you to a specific concept or skill. We'll start with the basics, like understanding variables and loops, and gradually build up to more complex ideas like working with data and creating simple games. But don't worry — we won't throw everything at you at once! You'll take it one step at a time, with clear explanations that are easy to follow.

2. Hands-On Practice with Real Problems

You won't just read about how to use Python — you'll **use it** to solve problems. Each concept comes with practical examples that let you see how Python works in real life. From solving simple math problems to automating tasks and analyzing data, you'll learn how to apply Python to things you might encounter in your everyday life or studies.

For example:

- **Math problem:** Use Python to calculate areas and volumes.

- **Game design:** Create a fun quiz or guessing game.
- **Data analysis:** Use Python to sort and organize data from school assignments.

These examples are designed to be interesting, useful, and something you can **immediately put into action**.

3. Step-by-Step Instructions

Every problem-solving example is broken down into simple, easy-to-follow steps. This means you'll never feel lost or overwhelmed. You'll see exactly how to approach a problem, plan a solution, and write the code to solve it. And if you ever get stuck, don't worry! The book will guide you through common mistakes and show you how to fix them.

4. Build Skills as You Go

The examples in this book aren't just random exercises — they're carefully chosen to help you build up your skills gradually. Each chapter builds on what you've already learned, so by the end of the book, you'll be solving more complex problems with confidence. It's like stacking building blocks: the more you learn, the more powerful your Python skills become.

5. Projects You'll Be Proud Of

As you move through the book, you'll create projects that you can show off to your friends, teachers, or even include in your schoolwork. By solving real-world problems, you'll see just how powerful Python is and how it can make your life easier. Whether it's building a simple game, automating a task, or analyzing data, these projects will not only help you learn, but also give you something **tangible** to be proud of.

6. Who Can Use This Book?

This book is designed to be helpful for everyone:

- **Pupils** who are new to Python and want to learn how to solve problems step by step.
- **Students** who have a little experience but want to tackle real-world challenges using Python.

- **Teachers** who want to bring coding into the classroom in a practical, hands-on way.

We've made sure the examples are **relatable** and **easy to understand**, so you don't need any previous coding experience. By the time you finish, you'll have the skills and confidence to tackle problems on your own.

This book is not just about coding — it's about **thinking** and **problem-solving** in a structured way, while having fun in the process. With each chapter, you'll gain knowledge that builds on the previous one, and by the end, you'll be amazed at what you can create with Python. We hope you enjoy learning through practical examples as much as we enjoyed putting them together for you!

Getting Started with Python: A Quick Setup Guide

Before you dive into solving problems with Python, let's get you set up and ready to code! Don't worry — it's easier than you think, and you'll be up and running in no time. Whether you're using a laptop or a school computer, follow these simple steps, and soon, you'll be writing your own Python programs.

1. What Do You Need?

To start coding in Python, you need two main things:

- **A computer** (Windows, Mac, or Linux)
- **Python installed on your computer**

We'll walk you through the process step by step, so you don't have to worry about anything technical. By the end of this guide, you'll be ready to run your first Python program!

2. Installing Python

Python is free, and installing it is simple. Here's what you need to do:

1. **Go to the Python website:** Open your browser and go to python.org.
2. **Download Python:** On the home page, you'll see a big button that says "Download Python." Click it. The website will automatically detect your operating system (Windows, Mac, or Linux) and suggest the correct version.
3. **Install Python:** Once the download is complete, open the file and start the installation process.
 - **Important:** During the installation, make sure you check the box that says "Add Python to PATH."

This makes it easier to run Python from anywhere on your computer.

4. **Finish the installation:** Follow the steps in the installation wizard, and when it's done, Python will be installed on your computer!

3. Opening Your First Python Program

Now that Python is installed, it's time to write your first piece of code!

- **Windows:** In field **Search** type "**cmd**" and press **Enter** to open the command prompt. Once open, type **python** and press **Enter**. You should see something that says "**Python**" with the version number.

```
C:\Users\wrote>python
Python 3.12.5 (tags/v3.12.5:ff3bc82, Aug 6 2024, 20:45:27) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> |
```

NOTE! The illustrations in this book show what a computer screen looks like with the Windows 11 operating system installed.

- **Mac:** Open Terminal (you can find it in Applications > Utilities). Type **python3** and press enter. You'll see Python's version information, just like on Windows.
- **Linux:** Open your terminal and type **python3**. This works just like on Mac.

When you see Python's version information, you're ready to code!

4. Your First Python Program: Hello, World!

Let's start by writing the simplest program ever: one that says "Hello, World!" This is a tradition for beginner programmers — and it's fun because you'll see your first output in no time.

Here's how to do it:

1. In your command prompt or terminal, type the following code:

```
print("Hello, World!")
```

2. Press enter. Python will run the code, and you'll see the words "Hello, World!" printed right on your screen!

```
>>> print ("Hello, World!")
Hello, World!
>>> |
```

5. Using an Integrated Development Environment (IDE)

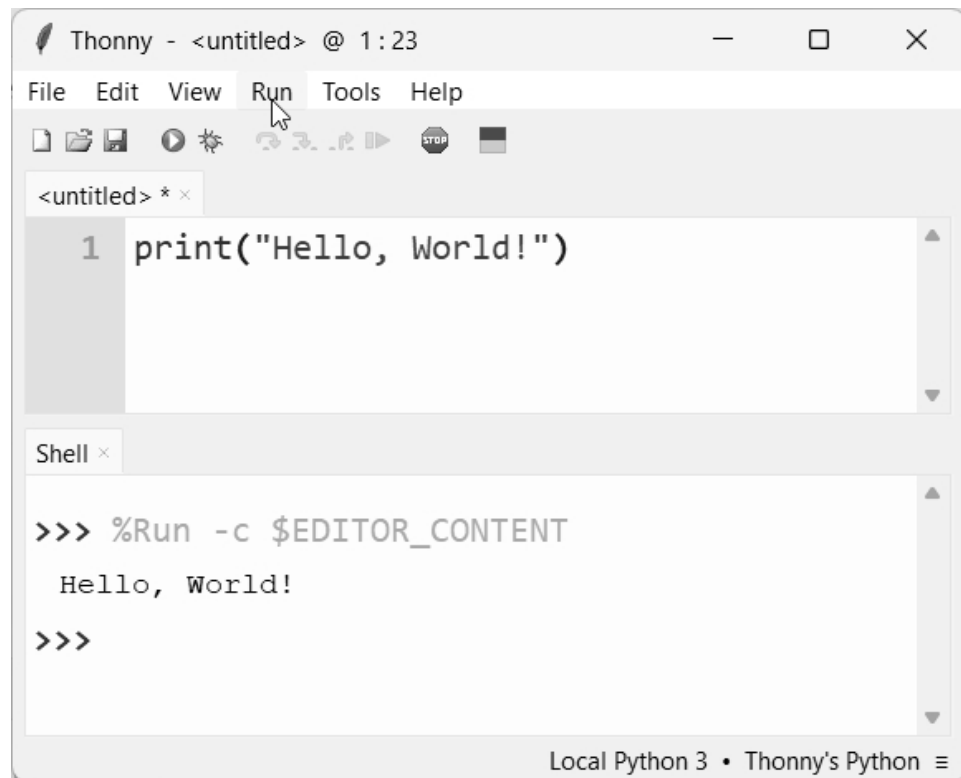
While you can write Python code in the terminal, there are easier and more fun ways to do it using something called an **IDE** (Integrated Development Environment). IDEs make writing, testing, and running Python programs simpler, with lots of helpful features like highlighting errors or auto-completing your code.

A popular and beginner-friendly IDE for Python is **Thonny**. Here's how to get it:

1. **Download Thonny:** Visit thonny.org and download the latest version for your computer.
2. **Install Thonny:** Follow the installation steps. Once it's installed, open Thonny, and you'll be ready to write your Python code in a much friendlier environment!
3. **Write your first program in Thonny:**
 - In Thonny's code editor, type:

```
print("Hello, World!")
```

- Click "Run" at the top of the screen, and you'll see "Hello, World!" appear at the bottom.



Using an IDE like Thonny will make your coding experience smoother and more enjoyable, especially as you start working on bigger projects.

Who Is This Book For? (Pupils, Students, and Teachers)

This book is designed with **you** in mind, whether you're a pupil just starting your coding journey, a student looking to solve real-world problems with Python, or a teacher eager to introduce programming in the classroom. Let's explore how this book can help each of you:

1. For Pupils: Start Learning Python Step by Step

If you've never written a line of code before, don't worry! This book will guide you step by step from the very basics. Python is a great first programming language because it's easy to learn and fun to use. You'll start with simple concepts like writing your first program, and before you know it, you'll be solving problems using Python on your own.

Here's what you can expect:

- **Simple, clear explanations** that won't overwhelm you.
- **Fun exercises** that will help you understand how coding works.
- **Real-world examples** that show how Python can be used in everyday situations, like solving math problems or creating games.

By the end of the book, you'll not only understand Python, but you'll also feel confident about solving challenges creatively!

2. For Students: Solve Practical Problems with Python

If you already have a little experience with coding or want to dive deeper, this book is perfect for you. It's filled with practical examples and projects that apply Python to real-world problems. Whether you're interested in science, math, or even building simple games, this book will help you tackle problems that matter to you.

Here's what you'll find:

- **Real-world projects** like analyzing data, automating tasks, or creating simple apps.
- **Hands-on examples** to build your skills, chapter by chapter.
- **Intermediate topics** like working with data or creating visualizations to help you expand your programming knowledge.

By the end of the book, you'll have the tools to take on more complex projects and the confidence to explore even bigger coding challenges.

3. For Teachers: Bring Python to the Classroom

If you're a teacher looking to introduce Python to your students, this book is a valuable resource. It's designed to be **practical** and **engaging**, with lessons that you can easily adapt for the classroom. Whether you teach math, science, or computer science, Python can become a tool to make learning more interactive and fun.

What this book offers you:

- **Step-by-step explanations** to help students at different levels learn at their own pace.
- **Classroom-ready examples** that align with real-world learning goals.
- **Projects and exercises** you can use to challenge and inspire your students to think critically and solve problems.

With this book, you can introduce Python as more than just a programming language — it becomes a way to teach problem-solving and logical thinking that students can apply across subjects.

Whether you're a **pupil**, **student**, or **teacher**, this book is here to make learning Python enjoyable and accessible. It's about more than just writing code — it's about developing the skills to approach problems creatively and confidently. We're excited for you to begin this journey, and we hope it inspires you to discover how powerful Python can be!

How to Use This Book: Tips for Maximizing Your Learning

Welcome to your Python adventure! To get the most out of this book, it's important to know how to approach the lessons and examples in a way that helps you learn effectively and enjoyably. Whether you're a pupil, student, or teacher, these tips will guide you through using this book as a powerful tool for learning.

1. Take It Step by Step

Each chapter is designed to build on what you've already learned, so it's important to follow the sequence in order. Don't rush ahead — take the time to understand the concepts before moving on. Think of it like climbing a ladder: each step gets you closer to mastering Python, and skipping steps might make you lose your balance.

Here's how you can approach each chapter:

- **Read carefully:** Make sure you understand the examples and explanations before jumping to the exercises.
- **Try the examples yourself:** Don't just read the code — type it out and run it. You'll learn much better by doing rather than watching.
- **Experiment:** Once you've run the example, try changing things! What happens if you tweak a number or change a word in the code? Experimenting helps reinforce what you've learned.

2. Practice with the Exercises

At the end of many chapters, you'll find exercises designed to test your understanding. These aren't just for practice — they're a way to see how much you've really learned. Don't skip them! Doing the exercises will make you feel more confident and capable as you move forward.

Here's how to get the most out of the exercises:

- **Solve them on your own:** Try to solve the problem without looking at the answer first. If you're stuck, break it down into smaller steps.
- **Check your work:** Once you've written your code, run it and see if it works as expected. If it doesn't, that's okay! Debugging is part of the learning process.
- **Review the solution:** After you've attempted the exercise, review the provided solution. Did you approach it the same way? Is there something you could do differently next time?

3. Don't Be Afraid to Make Mistakes

Mistakes are an important part of learning to code. When your program doesn't work, don't get discouraged! It's all part of the process. Instead, try to understand **why** it didn't work and how you can fix it. Python will often give you clues in the form of error messages — use them to your advantage.

If you encounter an error:

- **Read the error message:** Python usually tells you what went wrong and where it happened.
- **Break down the problem:** Go back through your code step by step. Did you miss a comma? Forget a quotation mark? Fixing small mistakes is a normal part of coding.
- **Learn from it:** Every mistake is an opportunity to learn something new. The more errors you encounter, the more skilled you become at solving problems!

4. Apply What You Learn to Real-Life Problems

The exercises in this book are designed to be practical and useful, but don't stop there! Think about how you can apply what you've learned to your own life or schoolwork. Maybe you can use Python to help with a science

project, organize data for a math assignment, or even create a fun game to share with friends.

By using Python to solve real-world problems, you'll find it becomes more meaningful and enjoyable. Here's how you can start:

- **Think of a problem you want to solve:** Is there something you could automate or make easier using Python?
- **Use what you've learned:** Apply the concepts from the book to create your own programs. This could be anything from a simple calculator to a tool that helps with homework.
- **Challenge yourself:** As you get more comfortable, try solving problems that aren't in the book. The more you practice, the better you'll get!

5. Ask for Help When You Need It

If you ever get stuck or feel lost, don't hesitate to ask for help. Whether it's from a teacher, a classmate, or online Python communities, there are plenty of people who can help guide you. Learning to code is a journey, and it's okay to seek support along the way.

Here's how you can find help:

- **Online communities:** Websites like Stack Overflow or Python forums are great places to ask questions and learn from others.
- **Study groups:** If you're in a classroom, work together with your classmates. Teaching others what you've learned can also help solidify your own understanding.
- **Review previous chapters:** Sometimes, going back to earlier sections can help you understand what's going wrong in your current project.

6. Keep Practicing and Enjoy the Process

The more you practice, the more confident you'll become. Don't be afraid to take your time and enjoy the process of learning Python. Every time you solve a problem, no matter how small, you're making progress and developing valuable skills that will help you in many areas of life.

By following these tips, you'll maximize your learning and get the most out of this book. Python is not just a tool — it's a way to unlock creative problem-solving skills that you can use in everyday life. So, take your time, have fun, and watch as your abilities grow with each chapter!

Part I: Python Basics for Problem Solving

Chapter 1: Python Fundamentals

Understanding Variables, Data Types, and Basic Syntax

Before we dive into solving problems with Python, let's take a step back and learn some of the essential building blocks. In this chapter, we're going to explore three important concepts that every Python programmer needs to know: **variables**, **data types**, and **basic syntax**. Don't worry if these terms sound unfamiliar right now — we'll break them down so that they're easy to understand and fun to learn.

1. What Are Variables?

Imagine a variable like a storage box with a label on it. This box can hold different things, like numbers or words, and the label tells you what's inside. In Python, a variable is a name you give to store data, so you can use it later in your program.

For example, let's say you want to store your age in a program. You can create a variable called `age` and give it a value, like this:

```
age = 15
```

Here, **age** is the variable, and 15 is the value it's holding. Now, whenever you use `age` in your program, Python knows that it's 15.

Variables make your code more flexible. Instead of writing the number 15 over and over, you can just use `age`, and if your age changes, you only have to update it in one place.

2. Data Types: What Can Variables Hold?

In Python, variables can store different types of data. These are called **data types**. Think of them like different kinds of boxes for storing different things. There are several common data types you'll use all the time:

- **Integers (int):** Whole numbers, like 10, -3, or 42.

- **Floats (float):** Decimal numbers, like 3.14 or 5.0.
- **Strings (str):** Text or characters, like "Hello, World!" or "Python is fun!".
- **Booleans (bool):** True/False values, like True or False.

Here's an example of each data type in Python:

```
age = 15      # Integer
height = 5.7  # Float
name = "Alice" # String
is_student = True # Boolean
```

You'll notice that Python automatically knows what kind of data each variable is holding. You don't have to tell it!

3. Basic Syntax: How Python Understands Your Instructions

Now that we know about variables and data types, let's talk about **syntax**. Syntax is just a fancy word for the rules of a language. Just like in English, where you need to write sentences in a certain way (with periods, commas, etc.), Python has rules about how you write code. These rules help Python understand what you want it to do.

Here are some key rules to keep in mind:

- **Case Sensitivity:** Python cares about uppercase and lowercase letters. For example, **Name** and **name** are two different variables.
- **Indentation:** Python uses spaces or tabs to show which parts of code belong together. This is especially important when we get to loops and functions.
- **Comments:** You can write notes in your code using the # symbol. Python ignores these lines, so they're just for you to read.

Example:

This is a comment. Python will ignore this line.

name = "Alice" # This stores the name "Alice" in the variable.

4. Putting It All Together: A Simple Example

Let's combine everything we've learned so far. Imagine you're creating a program that stores a person's name, age, and whether they are a student, and then prints out this information. Here's what the code might look like:

```
name = "Alice"
```

```
age = 15
```

```
is_student = True
```

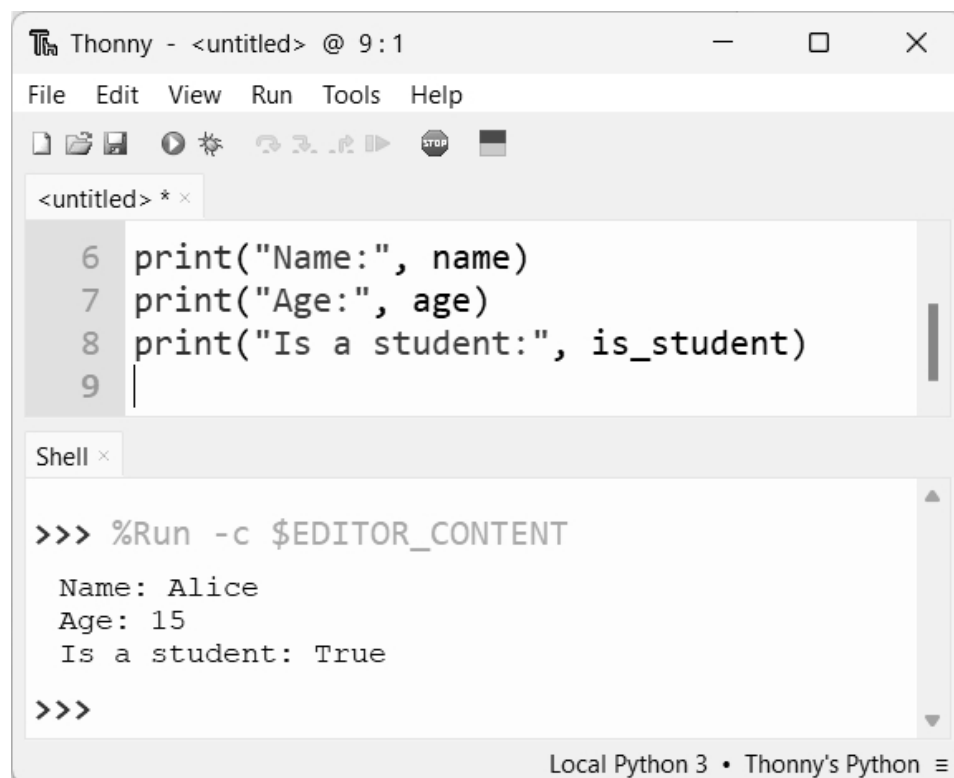
```
# Print the information
```

```
print("Name:", name)
```

```
print("Age:", age)
```

```
print("Is a student:", is_student)
```

When you run this code, Python will output:

A screenshot of the Thonny Python IDE interface. The window title is "Thonny - <untitled> @ 9:1". The menu bar includes "File", "Edit", "View", "Run", "Tools", and "Help". Below the menu bar is a toolbar with icons for file operations and execution. The main editor area shows a Python script with four lines of code:

```
6 print("Name:", name)
7 print("Age:", age)
8 print("Is a student:", is_student)
9 |
```

 Below the editor is a "Shell" window. It shows the command

```
>>> %Run -c $EDITOR_CONTENT
```

 followed by the output:

```
Name: Alice
Age: 15
Is a student: True
>>>
```

 The status bar at the bottom right indicates "Local Python 3 • Thonny's Python".

As you can see, the variables **name**, **age**, and **is_student** each hold different types of data, and the `print()` function displays this information. This is a simple, yet powerful way to use variables to store and display data.

Quick Tips for Success:

- **Name your variables clearly:** Use names that describe what the variable holds. For example, age is better than a, because it's easier to understand.
- **Test as you go:** Run your code often to check for errors or see how your variables are working.
- **Practice!** The more you work with variables, the easier it becomes.

Now that you've learned about variables, data types, and basic syntax, you're ready to start solving problems with Python! Keep these concepts in mind as you move forward — they're the foundation for everything else you'll learn in programming.

How Python Handles Numbers, Text, and Lists

Python is a powerful tool, and one of the reasons it's so useful for solving problems is the way it handles different types of information, such as numbers, text, and lists. In this section, we'll explore how Python works with each of these data types in simple, practical ways.

1. Working with Numbers

Numbers are one of the most basic types of data in Python, and you'll use them all the time, whether for math problems, counting things, or making decisions.

There are two main types of numbers in Python:

- **Integers** (whole numbers like 3, -5, and 100)
- **Floats** (decimal numbers like 3.14, -0.5, and 10.0)

Let's look at how easy it is to work with numbers in Python.

Basic Math Operations

Python can handle all basic math operations, such as addition, subtraction, multiplication, and division:

```
a = 10
```

```
b = 5
```

```
# Addition
```

```
print(a + b) # Output: 15
```

```
# Subtraction
```

```
print(a - b) # Output: 5
```

```
# Multiplication
```

```
print(a * b) # Output: 50
```

```
# Division
```

```
print(a / b) # Output: 2.0
```

Python can also handle more advanced math operations like raising numbers to a power or finding remainders:

```
# Exponentiation (raising a number to a power)
```

```
print(2 ** 3) # Output: 8
```

```
# Modulus (finding the remainder)
```

```
print(10 % 3) # Output: 1
```

You can see that working with numbers in Python is straightforward. Whether you're solving a math problem or calculating something in a game, Python makes it easy to do!

2. Handling Text (Strings)

When it comes to working with text, Python uses a data type called a **string**. A string is simply a sequence of characters — it could be a word, a sentence, or even a whole paragraph.

Here's how you can create and use strings in Python:

```
name = "Alice"
```

```
greeting = "Hello, " + name
```

```
print(greeting) # Output: Hello, Alice
```

In this example, we combined two pieces of text (called "string concatenation"). The "Hello, " part and the name variable come together to form the final greeting. Python is great at handling text and makes it easy to work with names, messages, or any kind of written data.

Useful String Functions

Python comes with several built-in functions for working with strings:

- **Finding the length of a string:** You can find out how many characters are in a string using the `len()` function.

```
print(len("Python")) # Output: 6
```

- **Changing the case of a string:** You can make all the letters in a string uppercase or lowercase:

```
word = "python"
```

```
print(word.upper()) # Output: PYTHON
```

```
print(word.lower()) # Output: python
```

- **Replacing parts of a string:** You can replace certain words or letters in a string:

```
sentence = "I love apples"
new_sentence = sentence.replace("apples", "bananas")
print(new_sentence) # Output: I love bananas
```

These simple tools allow you to do a lot with text, making Python very useful for creating messages, analyzing text, or building interactive programs like quizzes and games.

3. Working with Lists

A **list** in Python is a way to store multiple pieces of data in one place. Think of it like a shopping list — you have a collection of items, and you can easily add, remove, or look at any item in the list.

Here's an example of a simple list:

```
fruits = ["apple", "banana", "cherry"]
print(fruits) # Output: ['apple', 'banana', 'cherry']
```

Lists can hold **any kind of data** — numbers, strings, or even other lists! Here are some of the things you can do with lists:

- **Access items by index:** In Python, list items are numbered starting from 0. So, the first item is at index 0, the second item is at index 1, and so on.

```
print(fruits[0]) # Output: apple
print(fruits[2]) # Output: cherry
```

- **Add items to a list:** You can add new items to a list using the `append()` function.

```
fruits.append("orange")
print(fruits) # Output: ['apple', 'banana', 'cherry', 'orange']
```


- **Remove items from a list:** You can also remove items using the `remove()` function.

```
fruits.remove("banana")  
print(fruits) # Output: ['apple', 'cherry', 'orange']
```

- **Loop through a list:** Sometimes, you'll want to go through every item in a list and do something with it. You can use a loop to do this.

```
for fruit in fruits:
```

```
    print(fruit)
```

This will print each fruit on its own line:

```
apple
```

```
cherry
```

```
orange
```

Lists are extremely useful in programming because they allow you to organize and work with multiple pieces of data in an efficient way. Whether you're keeping track of scores, making to-do lists, or working with large sets of information, lists make it easy to manage data.

Putting It All Together

Python's ability to handle numbers, text, and lists makes it an ideal language for solving problems. Whether you're doing simple math, working with strings, or organizing data, Python gives you powerful tools to make your tasks easier.

Here's an example of how you might use all three in one program:

```
# A list of students and their ages  
students = ["Alice", "Bob", "Charlie"]  
ages = [15, 16, 14]  
  
# Print out each student's name and age  
for i in range(len(students)):  
    print(students[i] + " is " + str(ages[i]) + " years old.")
```

When you run this, Python will output:



The screenshot shows the Thonny Python IDE interface. The top window, titled "<untitled> @ 8:1", contains a Python script. The script defines a list of student names and their corresponding ages, then uses a for loop to print each student's name and age. The bottom window, titled "Shell", shows the output of the script after execution.

```
1 # A list of students and their ages
2 students = ["Alice", "Bob", "Charlie"]
3 ages = [15, 16, 14]
4
5 # Print out each student's name and age
6 for i in range(len(students)):
7     print(students[i] + " is " + str(ages[i]) + " years old.")
8
```

```
>>> %Run -c $EDITOR_CONTENT
Alice is 15 years old.
Bob is 16 years old.
Charlie is 14 years old.
>>>
```

Local Python 3 • Thonny's Python

In this example, you used both strings and numbers, combined them in a meaningful way, and organized the data using lists. This is just a small taste of what Python can do — with these tools, you're well on your way to solving more complex problems!

Writing Your First Python Script: Simple Calculations

Now that you've learned about variables, numbers, text, and lists, it's time to put those skills to work by writing your **first Python script**. Don't worry — writing a script is just a fancy way of saying “writing a set of instructions that Python will follow to do something.” In this section, you'll learn how to write simple scripts that perform basic calculations, which is one of the most useful things you can do with Python.

1. What Is a Python Script?

A Python script is just a file that contains Python code. When you run the file, Python reads and follows the instructions in the code. This is how you'll create programs that can solve problems, perform tasks, or even automate things for you.

To write your first script, all you need is a text editor (like Notepad or Thonny) and Python installed on your computer (which we covered in the setup section).

2. Let's Start Simple: A Calculator Program

We'll start by creating a simple calculator that can add, subtract, multiply, and divide two numbers. You'll see how Python can handle these basic math operations in a program that you control.

Here's what our script will do:

1. Ask the user to input two numbers.
2. Perform the calculations (addition, subtraction, multiplication, and division).
3. Display the results of each calculation.

3. Step-by-Step Guide to Writing Your Script

Step 1: Create the Variables

First, we'll need to get two numbers from the user. You can use the `input()` function to do this, which lets the user type something in. We'll also use the `int()` function to turn the user's input into an integer (a whole number) so that Python can do the math.

```
# Asking the user to enter two numbers
num1 = int(input("Enter the first number: "))
num2 = int(input("Enter the second number: "))
```

Here, we've created two variables, `num1` and `num2`, which will store the numbers the user enters.

Step 2: Perform the Calculations

Next, we'll use Python to perform the four basic calculations: addition, subtraction, multiplication, and division. Python makes this easy with symbols like `+`, `-`, `*`, and `/`.

```
# Performing calculations
addition = num1 + num2
subtraction = num1 - num2
multiplication = num1 * num2
division = num1 / num2
```

In this part of the script, we're creating new variables (`addition`, `subtraction`, `multiplication`, and `division`) to store the results of each calculation.

Step 3: Display the Results

Finally, we'll print out the results of the calculations so the user can see them. We'll use the `print()` function for this.

```
# Displaying the results
print("Addition: ", addition)
print("Subtraction: ", subtraction)
print("Multiplication: ", multiplication)
print("Division: ", division)
```

This code will show the results of the calculations on the screen.

4. Putting It All Together

Now, let's put all the pieces together into one complete script:

```
# Asking the user to enter two numbers
num1 = int(input("Enter the first number: "))
num2 = int(input("Enter the second number: "))

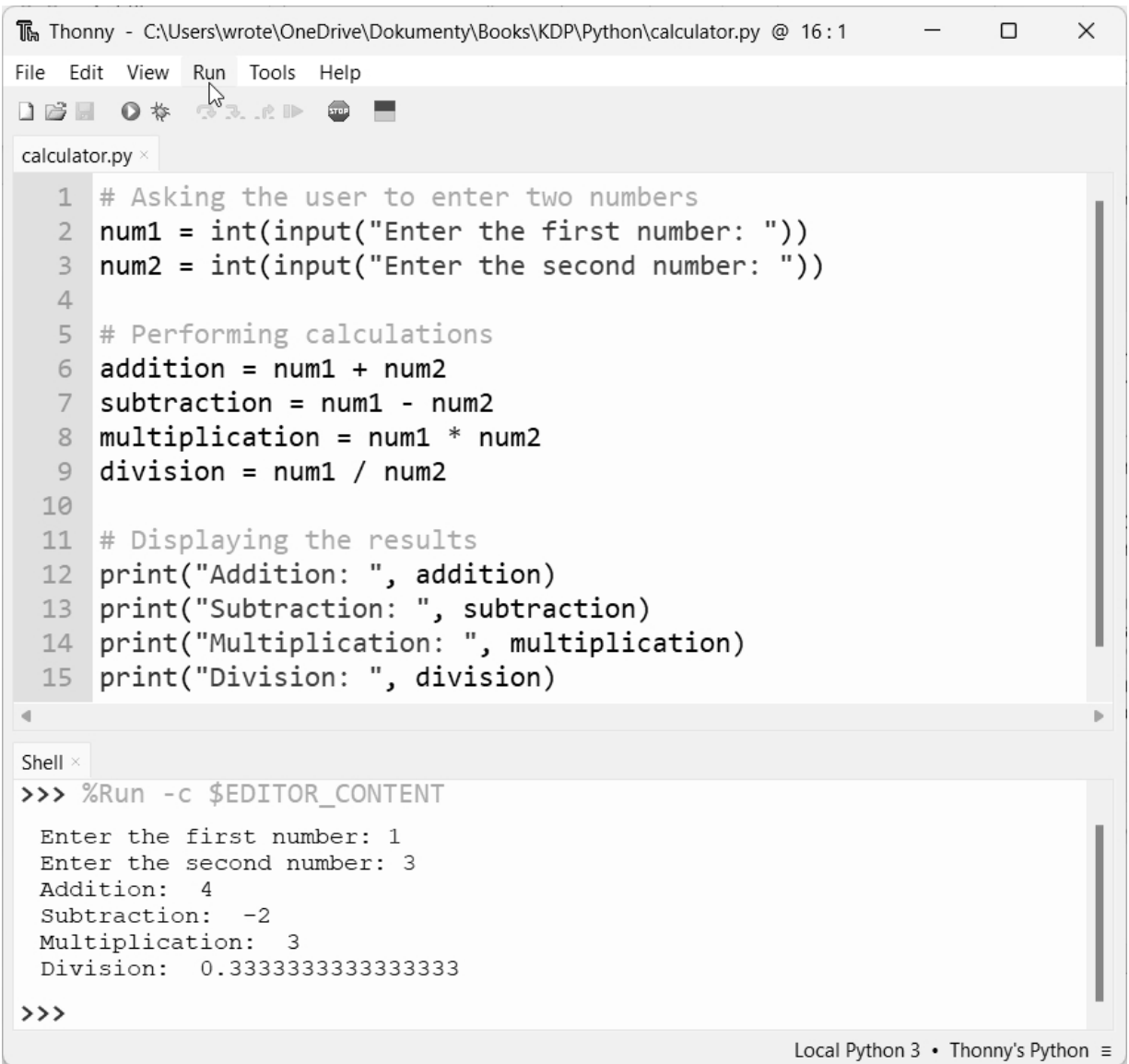
# Performing calculations
addition = num1 + num2
subtraction = num1 - num2
multiplication = num1 * num2
division = num1 / num2

# Displaying the results
print("Addition: ", addition)
print("Subtraction: ", subtraction)
print("Multiplication: ", multiplication)
print("Division: ", division)
```

5. Running Your Script

Once you've written your script, save it as a .py file (for example, calculator.py). Now, let's run it!

- **In Thonny:** If you're using Thonny, just click the green "Run" button at the top of the window. Thonny will run your script, and you'll see the results.



The screenshot shows the Thonny Python IDE interface. The top window, titled 'calculator.py', contains the following Python code:

```
1 # Asking the user to enter two numbers
2 num1 = int(input("Enter the first number: "))
3 num2 = int(input("Enter the second number: "))
4
5 # Performing calculations
6 addition = num1 + num2
7 subtraction = num1 - num2
8 multiplication = num1 * num2
9 division = num1 / num2
10
11 # Displaying the results
12 print("Addition: ", addition)
13 print("Subtraction: ", subtraction)
14 print("Multiplication: ", multiplication)
15 print("Division: ", division)
```

The bottom window, titled 'Shell', shows the output of running the script:

```
>>> %Run -c $EDITOR_CONTENT
Enter the first number: 1
Enter the second number: 3
Addition: 4
Subtraction: -2
Multiplication: 3
Division: 0.3333333333333333
>>>
```

The status bar at the bottom right indicates 'Local Python 3 • Thonny's Python'.

- **In the Command Line:** If you're using the command line, navigate to the folder where you saved your file and type `python calculator.py` (or `python3 calculator.py` if you're on a Mac or Linux).

Python will ask you to enter two numbers, perform the calculations, and then display the results.

6. Experimenting with Your Script

Congratulations! You've just written your first Python script. Now, let's have some fun by experimenting with it:

- **Try different numbers:** Run the script again with different numbers. Does it still work as expected?
- **Add more operations:** Can you add new calculations like finding the remainder (using the % operator)?
- **Include error handling:** What happens if the user tries to divide by zero? Try adding code to prevent errors like this.

Here's an example of handling division by zero:

```
if num2 != 0:
    division = num1 / num2
    print("Division: ", division)
else:
    print("Division by zero is not allowed.")
```

Why This Matters

This script might seem simple, but it's a powerful example of how you can use Python to perform calculations and make decisions based on user input. As you continue to learn Python, you'll be able to create more complex programs that solve real-world problems. But it all starts with these basic steps!

Keep practicing, experiment with different calculations, and soon, writing Python scripts will feel natural. You're well on your way to becoming a Python problem-solver!

Chapter 2: Control Structures: If, Else, and Loops

Using Conditional Statements to Make Decisions

Imagine you're playing a game where you need to make choices based on different situations. For example, if it's raining, you might take an umbrella; if it's sunny, you might wear sunglasses. In programming, we use **conditional statements** to make these kinds of decisions in our code. They allow your program to choose different paths and actions depending on certain conditions.

In this section, we'll explore how to use conditional statements in Python, specifically the `if`, `elif`, and `else` statements. You'll learn how to make your programs smarter and more interactive by teaching them to make decisions!

1. Understanding the `if` Statement

The `if` statement tells your program to execute a block of code **only if** a certain condition is true.

Syntax:

```
if condition:  
    # Code to execute if the condition is true
```

Example:

```
age = 16  
  
if age >= 18:  
    print("You can vote!")
```

In this example, the program checks if `age` is greater than or equal to 18. Since `age` is 16, which is less than 18, the code inside the `if` block doesn't

run, and nothing is printed.

2. Adding an else Statement

What if you want your program to do something when the condition is **not** true? That's where the else statement comes in.

Syntax:

```
if condition:
```

```
    # Code if condition is true
```

```
else:
```

```
    # Code if condition is false
```

Example:

```
age = 16
```

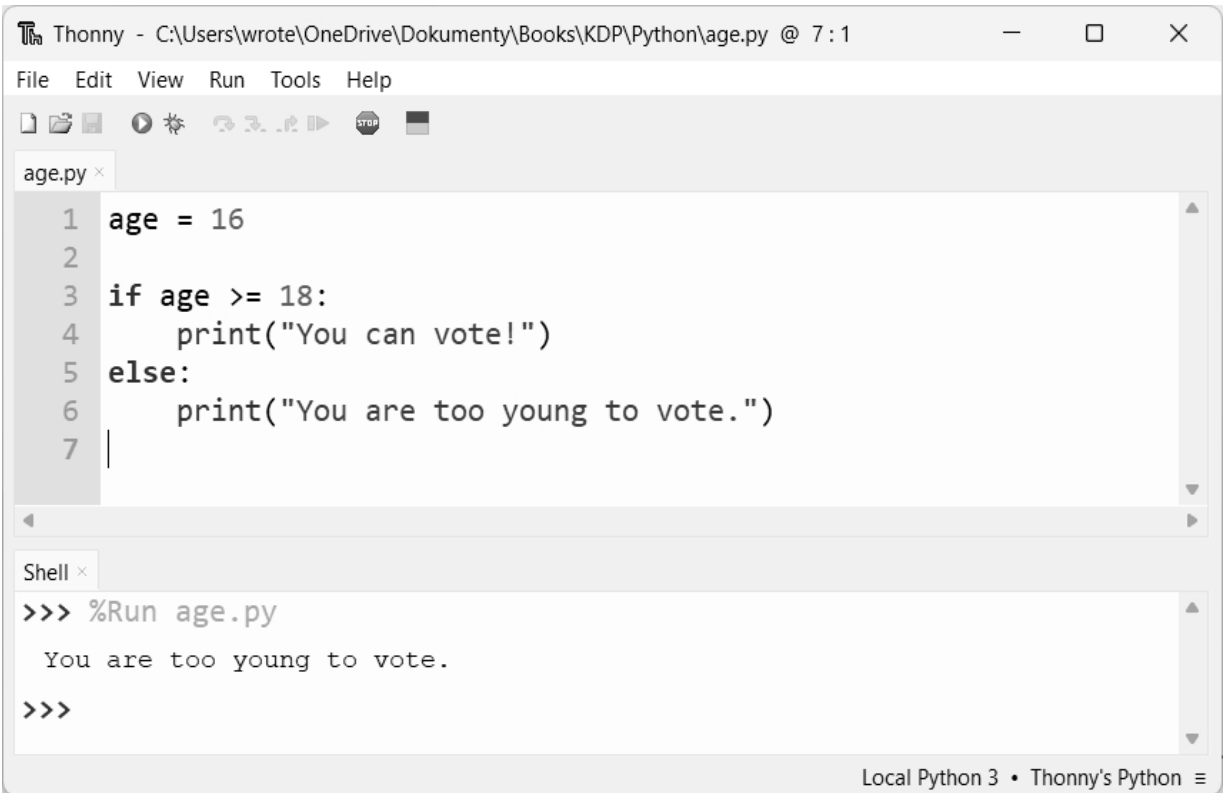
```
if age >= 18:
```

```
    print("You can vote!")
```

```
else:
```

```
    print("You are too young to vote.")
```

Now, since age is 16, the program will print:



The screenshot shows the Thonny Python IDE interface. The top window, titled 'age.py', contains the following Python code:

```
1 age = 16
2
3 if age >= 18:
4     print("You can vote!")
5 else:
6     print("You are too young to vote.")
7
```

The bottom window, titled 'Shell', shows the execution of the script:

```
>>> %Run age.py
You are too young to vote.
>>>
```

The status bar at the bottom right indicates 'Local Python 3 • Thonny's Python'.

3. Using elif for Multiple Conditions

Sometimes, you might have more than two possible situations. You can use `elif` (short for "else if") to check additional conditions.

Syntax:

if condition1:

 # Code if condition1 is true

elif condition2:

 # Code if condition2 is true

else:

 # Code if none of the above conditions are true

Example:

```
score = 85
```

```
if score >= 90:
```

```
    grade = "A"
```

```
elif score >= 80:
```

```
    grade = "B"
```

```
elif score >= 70:
```

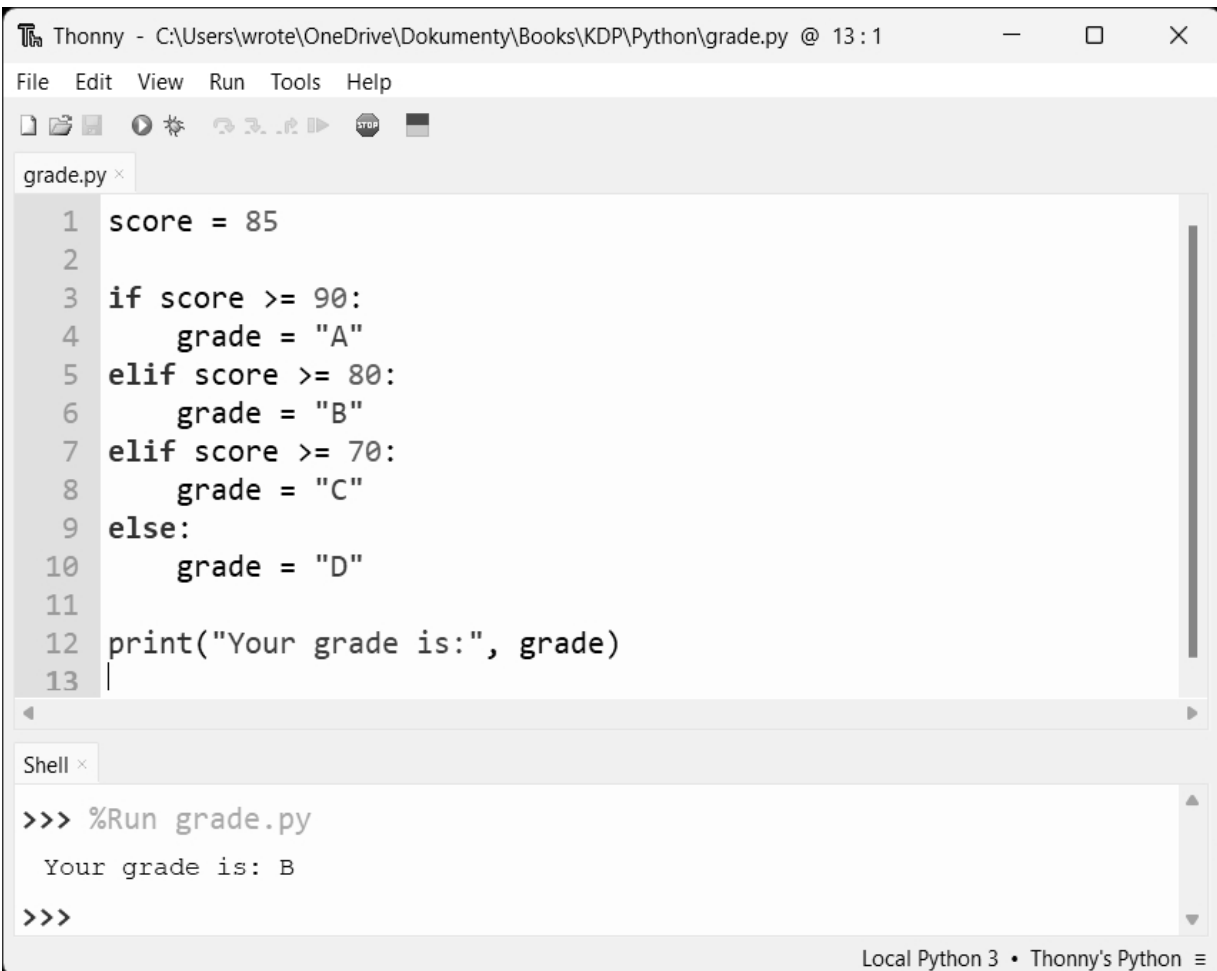
```
    grade = "C"
```

```
else:
```

```
    grade = "D"
```

```
print("Your grade is:", grade)
```

In this case, since score is 85, the program assigns grade as "B" and prints:



The screenshot shows the Thonny Python IDE interface. The top window, titled 'grade.py', contains the following Python code:

```
1 score = 85
2
3 if score >= 90:
4     grade = "A"
5 elif score >= 80:
6     grade = "B"
7 elif score >= 70:
8     grade = "C"
9 else:
10    grade = "D"
11
12 print("Your grade is:", grade)
13
```

The bottom window, titled 'Shell', shows the execution output:

```
>>> %Run grade.py
Your grade is: B
>>>
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

4. Conditions and Comparison Operators

Conditions in if statements usually involve comparison operators. Here are some common ones:

- `==` : Equal to
- `!=` : Not equal to
- `>` : Greater than

- < : Less than
- >= : Greater than or equal to
- <= : Less than or equal to

Example:

```
password = "secret"

if password == "secret":
    print("Access granted.")
else:
    print("Access denied.")
```

5. Logical Operators: and, or, not

You can combine conditions using logical operators to create more complex decisions.

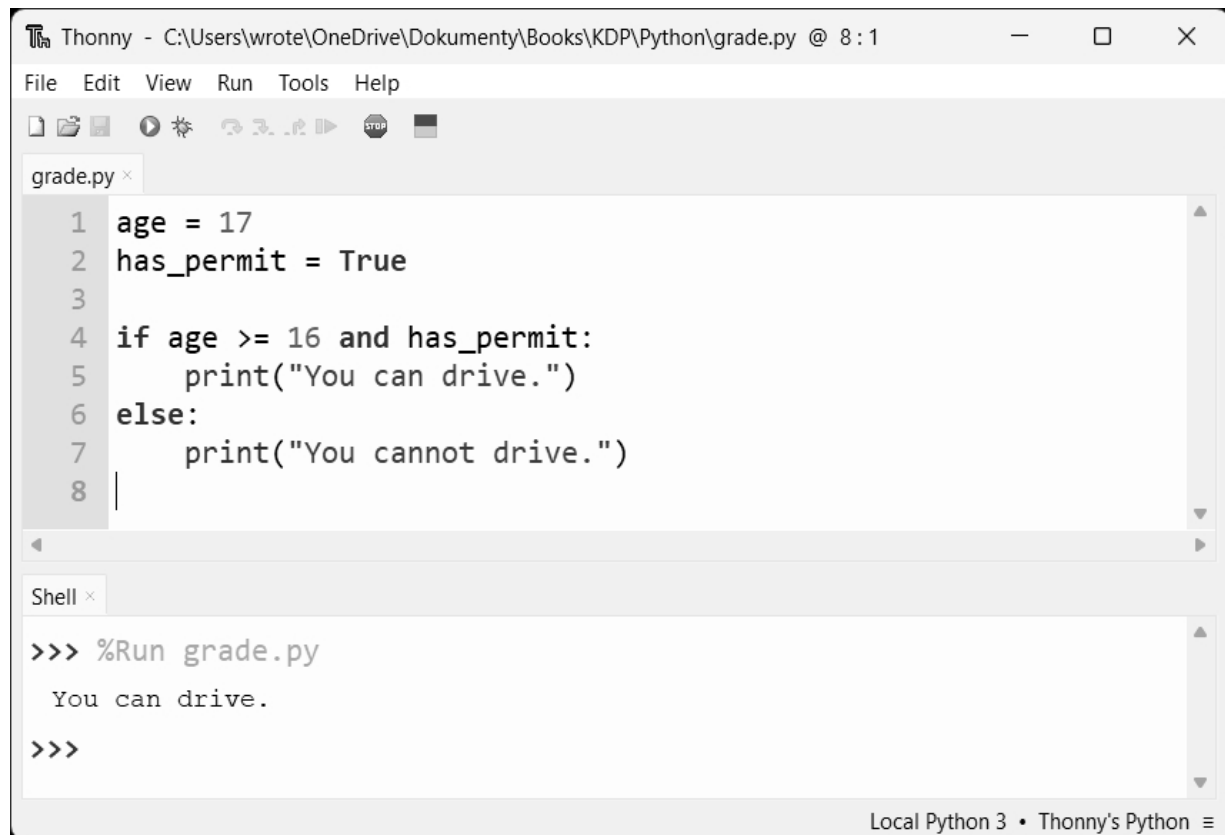
- **and** : True if **both** conditions are true
- **or** : True if **at least one** condition is true
- **not** : Inverts the truth value

Example with and:

```
age = 17
has_permit = True

if age >= 16 and has_permit:
    print("You can drive.")
else:
    print("You cannot drive.")
```

Since both conditions are true (age >= 16 and has_permit is True), the program prints:



The screenshot shows the Thonny Python IDE interface. The title bar reads "Thonny - C:\Users\wrote\OneDrive\Dokumenty\Books\KDP\Python\grade.py @ 8:1". The menu bar includes "File", "Edit", "View", "Run", "Tools", and "Help". Below the menu is a toolbar with icons for file operations and execution. The main editor window, titled "grade.py", contains the following Python code:

```
1 age = 17
2 has_permit = True
3
4 if age >= 16 and has_permit:
5     print("You can drive.")
6 else:
7     print("You cannot drive.")
8 |
```

Below the editor is a "Shell" window showing the execution output:

```
>>> %Run grade.py
You can drive.
>>>
```

The status bar at the bottom right indicates "Local Python 3 • Thonny's Python".

Example with or:

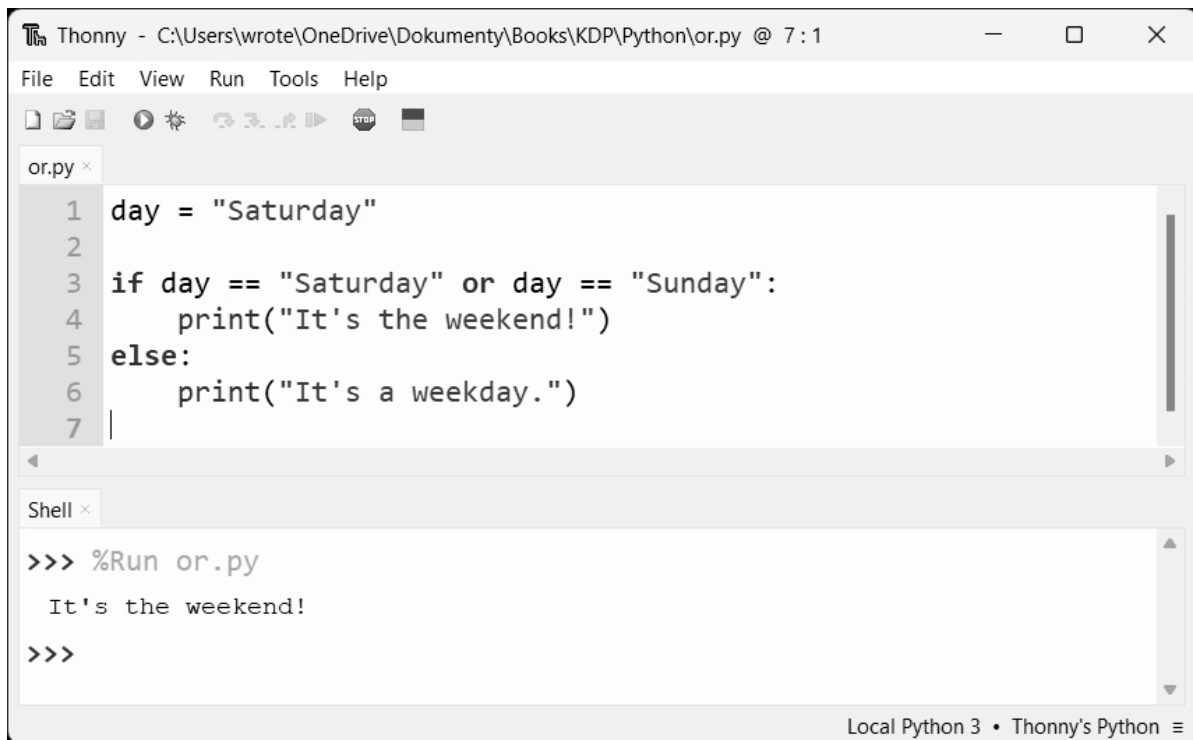
```
day = "Saturday"
```

```
if day == "Saturday" or day == "Sunday":
```

```
    print("It's the weekend!")
```

```
else:
```

```
    print("It's a weekday.")
```

The image shows a screenshot of the Thonny Python IDE. The main window displays a Python script named 'or.py' with the following code:

```
1 day = "Saturday"
2
3 if day == "Saturday" or day == "Sunday":
4     print("It's the weekend!")
5 else:
6     print("It's a weekday.")
7 |
```

The script is being executed, and the output is shown in the 'Shell' window at the bottom:

```
>>> %Run or.py
    It's the weekend!
>>>
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

6. Nested Conditional Statements

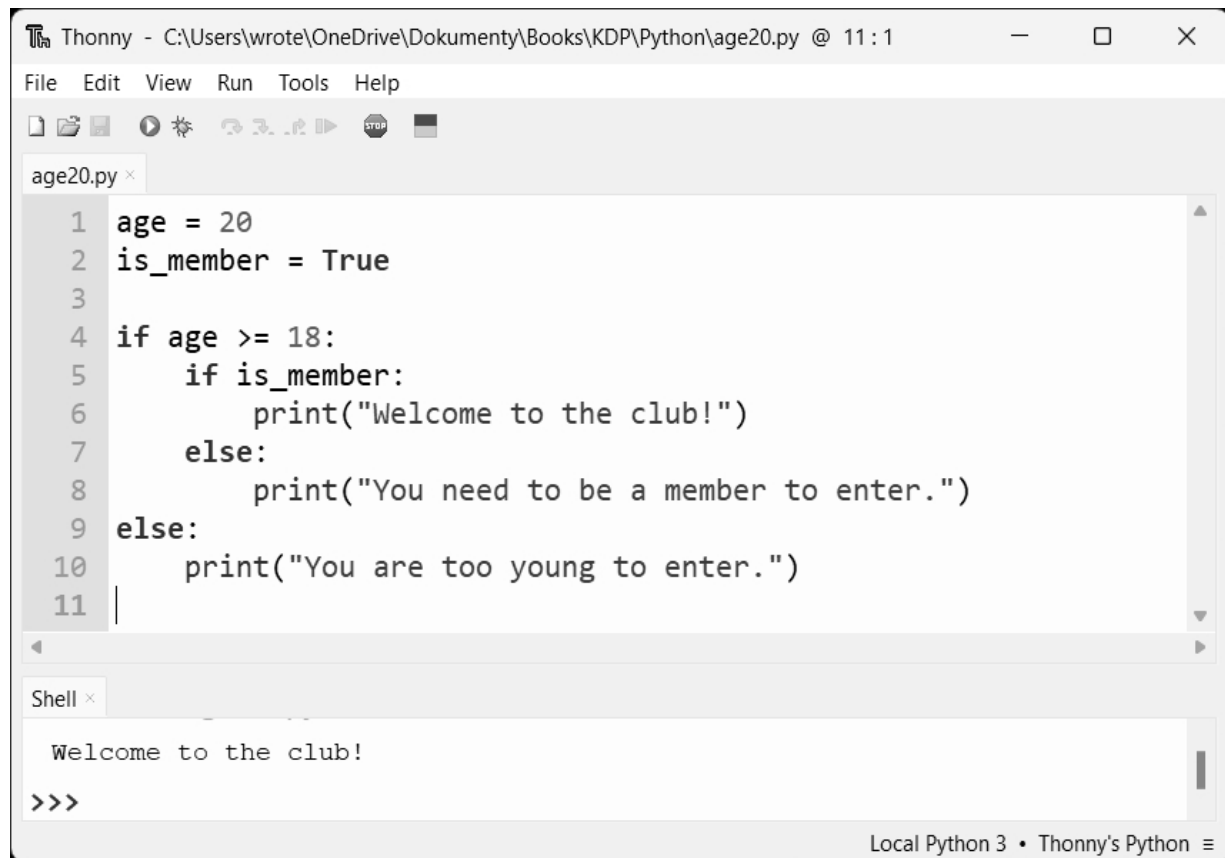
You can place an if statement inside another if or else block to check additional conditions.

Example:

```
age = 20
is_member = True

if age >= 18:
    if is_member:
        print("Welcome to the club!")
    else:
        print("You need to be a member to enter.")
else:
    print("You are too young to enter.")
```

Since age is 20 and is_member is True, the program prints:



The screenshot shows the Thonny Python IDE interface. The title bar reads "Thonny - C:\Users\wrote\OneDrive\Dokumenty\Books\KDP\Python\age20.py @ 11:1". The menu bar includes "File", "Edit", "View", "Run", "Tools", and "Help". Below the menu is a toolbar with icons for file operations and execution. The main editor window, titled "age20.py", contains the following Python code:

```
1 age = 20
2 is_member = True
3
4 if age >= 18:
5     if is_member:
6         print("Welcome to the club!")
7     else:
8         print("You need to be a member to enter.")
9 else:
10    print("You are too young to enter.")
11
```

Below the editor is a "Shell" window showing the output of the program:

```
Welcome to the club!
>>>
```

The status bar at the bottom right indicates "Local Python 3 • Thonny's Python".

7. Practical Example: Weather Advisor

Let's create a simple program that advises you on what to wear based on the weather.

Example:

```
weather = "rainy"
```

```
if weather == "sunny":
```

```
    print("Wear sunglasses!")
```

```
elif weather == "rainy":
```

```
    print("Don't forget your umbrella!")
```

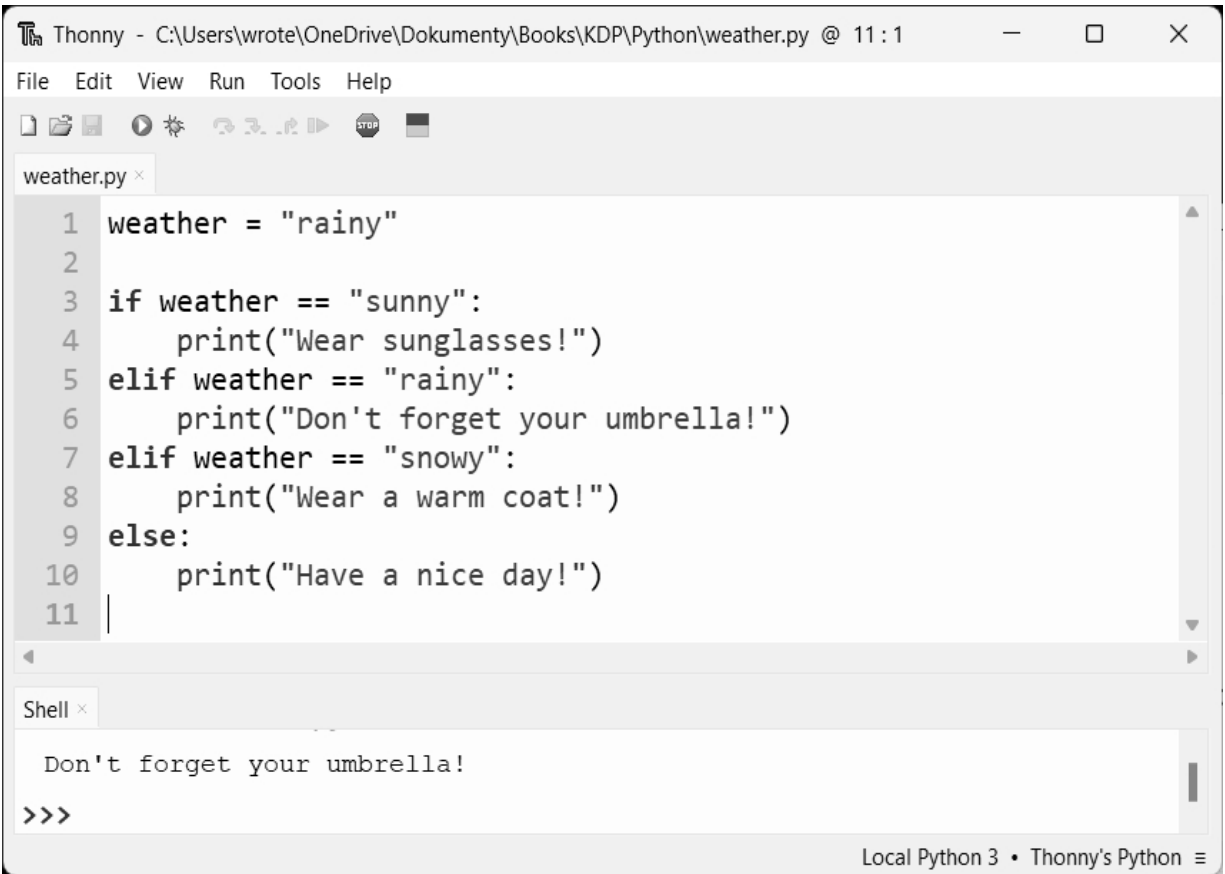
```
elif weather == "snowy":
```

```
    print("Wear a warm coat!")
```

```
else:
```

```
    print("Have a nice day!")
```

Since weather is "rainy", the program prints:



The screenshot shows the Thonny Python IDE interface. The title bar indicates the file path: C:\Users\wrote\OneDrive\Dokumenty\Books\KDP\Python\weather.py. The menu bar includes File, Edit, View, Run, Tools, and Help. Below the menu is a toolbar with icons for file operations and execution. The main editor window displays a Python script named weather.py with the following code:

```
1 weather = "rainy"
2
3 if weather == "sunny":
4     print("Wear sunglasses!")
5 elif weather == "rainy":
6     print("Don't forget your umbrella!")
7 elif weather == "snowy":
8     print("Wear a warm coat!")
9 else:
10    print("Have a nice day!")
11
```

Below the editor is a Shell window showing the output of the script: "Don't forget your umbrella!". The status bar at the bottom indicates "Local Python 3 • Thonny's Python".

8. Tips for Writing Conditional Statements

- **Indentation Matters:** In Python, indentation is crucial. The code inside the if, elif, and else blocks must be indented consistently.

if condition:

```
# Correct indentation
print("Do something")
```

- **Use Meaningful Conditions:** Make sure your conditions make sense and accurately reflect the decision you want the program to make.
- **Test Different Scenarios:** Try different values to ensure all parts of your conditional statements work as expected.

9. Try It Yourself

Let's practice writing a simple program using conditional statements.

Problem: Write a program that asks the user for a number and tells them if it's positive, negative, or zero.

Solution:

```
number = int(input("Enter a number: "))

if number > 0:
    print("The number is positive.")
elif number == 0:
    print("The number is zero.")
else:
    print("The number is negative.")
```

Explanation:

- We use `int(input(...))` to get a number from the user.
- The program checks:
 - If number is greater than zero, it's positive.
 - If number is equal to zero, it's zero.
 - Otherwise, it's negative.

10. Why Conditional Statements Are Powerful

Conditional statements allow your programs to react differently based on the data they receive. This is essential for:

- **Making Decisions:** Control the flow of your program based on conditions.
- **Handling User Input:** Respond appropriately to different inputs.
- **Building Interactive Programs:** Create games, quizzes, and applications that change behavior based on user actions.

By mastering conditional statements, you're adding a powerful tool to your programming toolbox. You'll be able to create programs that think and make decisions, just like you do!

Keep experimenting, and try writing your own programs using `if`, `elif`, and `else`. The more you practice, the more confident you'll become in making your code smart and interactive.

Loops: Solving Repetitive Problems

Efficiently

Imagine you need to solve a problem that involves repeating the same task multiple times, like counting the number of students in a class or printing the same message five times. Doing this manually would be boring and inefficient. Luckily, Python gives you **loops**, which allow you to repeat actions as many times as you need — and they make your programs much more powerful and efficient.

In this section, we'll explore two types of loops that will help you solve repetitive problems: the for loop and the while loop. Both are easy to understand and use, and you'll see just how valuable they are for problem-solving.

1. What Are Loops?

Loops allow you to run the same block of code multiple times without having to write it over and over. With loops, you can automate repetitive tasks, making your code simpler, shorter, and faster.

2. The for Loop: Iterating Over a Sequence

A for loop is used when you know exactly how many times you want to repeat an action, or when you want to go through each item in a sequence, like a list of names or a range of numbers.

Syntax:

```
for item in sequence:
```

```
    # Code to run for each item
```

Here's an example of using a for loop to print each item in a list:

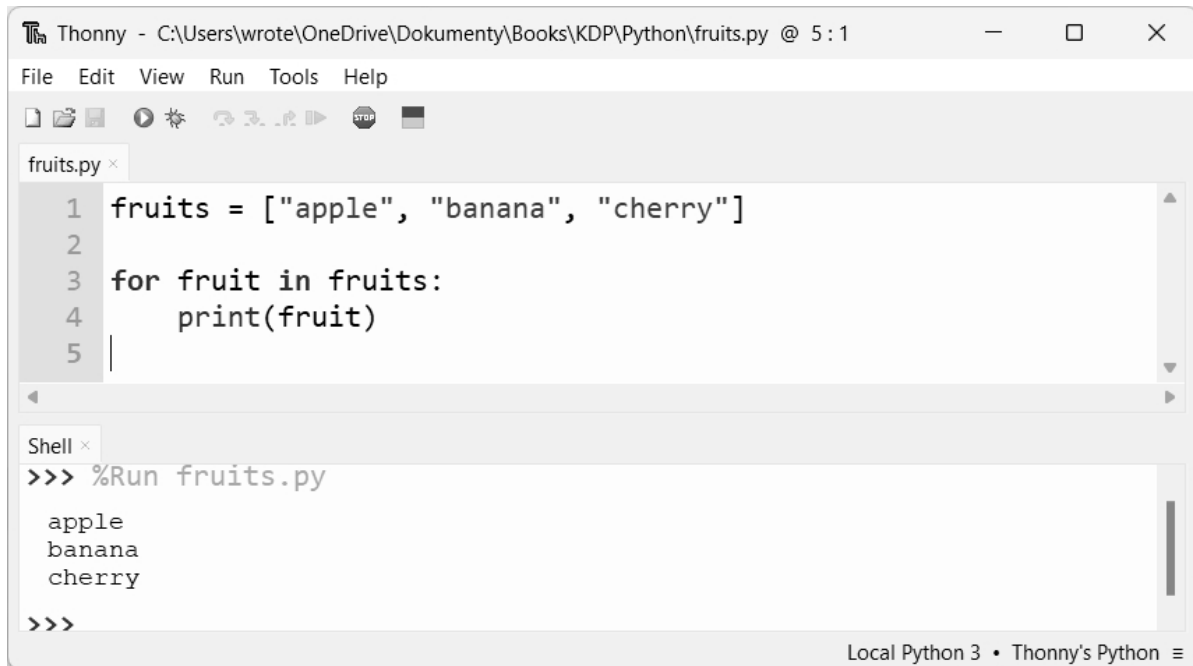
```
fruits = ["apple", "banana", "cherry"]
```

```
for fruit in fruits:
```

```
    print(fruit)
```

Output:

In this example, the loop goes through the fruits list one by one and prints each item. This saves you from having to write `print(fruits[0])`, `print(fruits[1])`, and so on.



The screenshot shows the Thonny Python IDE interface. The top window, titled 'Thonny - C:\Users\wrote\OneDrive\Dokumenty\Books\KDP\Python\fruits.py @ 5 : 1', contains the following Python code:

```
1 fruits = ["apple", "banana", "cherry"]
2
3 for fruit in fruits:
4     print(fruit)
5 |
```

Below the code editor is a 'Shell' window. It shows the command `>>> %Run fruits.py` being executed, followed by the output:

```
apple
banana
cherry
>>>
```

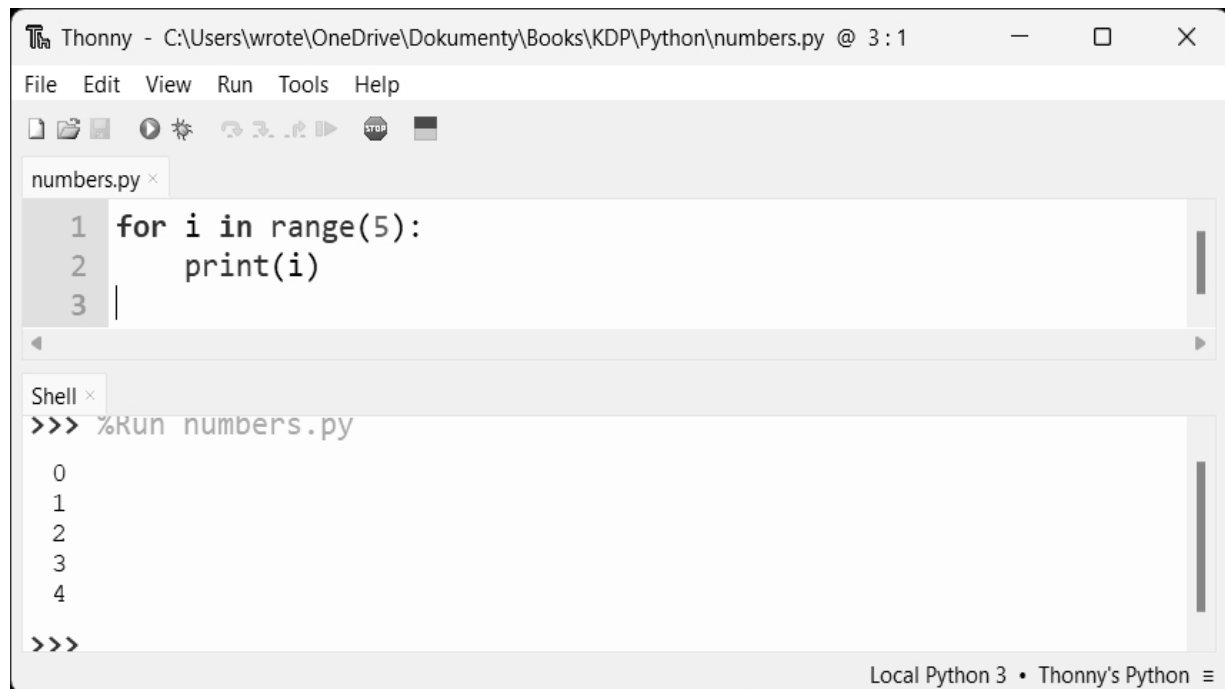
The status bar at the bottom right indicates 'Local Python 3 • Thonny's Python'.

Using for with a Range of Numbers:

You can also use the `range()` function to loop through a sequence of numbers.

```
for i in range(5):
    print(i)
```

Output:



The screenshot shows the Thonny Python IDE interface. The top window, titled 'numbers.py', contains the following Python code:

```
1 for i in range(5):  
2     print(i)  
3
```

The bottom window, titled 'Shell', shows the command prompt with the command `>>> %Run numbers.py` and the output:

```
0  
1  
2  
3  
4  
>>>
```

The status bar at the bottom right indicates 'Local Python 3 • Thonny's Python'.

The `range(5)` generates numbers from 0 to 4, and the `for` loop prints each number. This is a great way to repeat an action a specific number of times.

3. The while Loop: Repeating Until a Condition Is Met

A `while` loop is used when you don't know exactly how many times to repeat something, but you want to keep looping **as long as** a certain condition is true.

Syntax:

`while condition:`

 # Code to run while the condition is true

Here's an example where we use a `while` loop to print numbers until we reach 5:

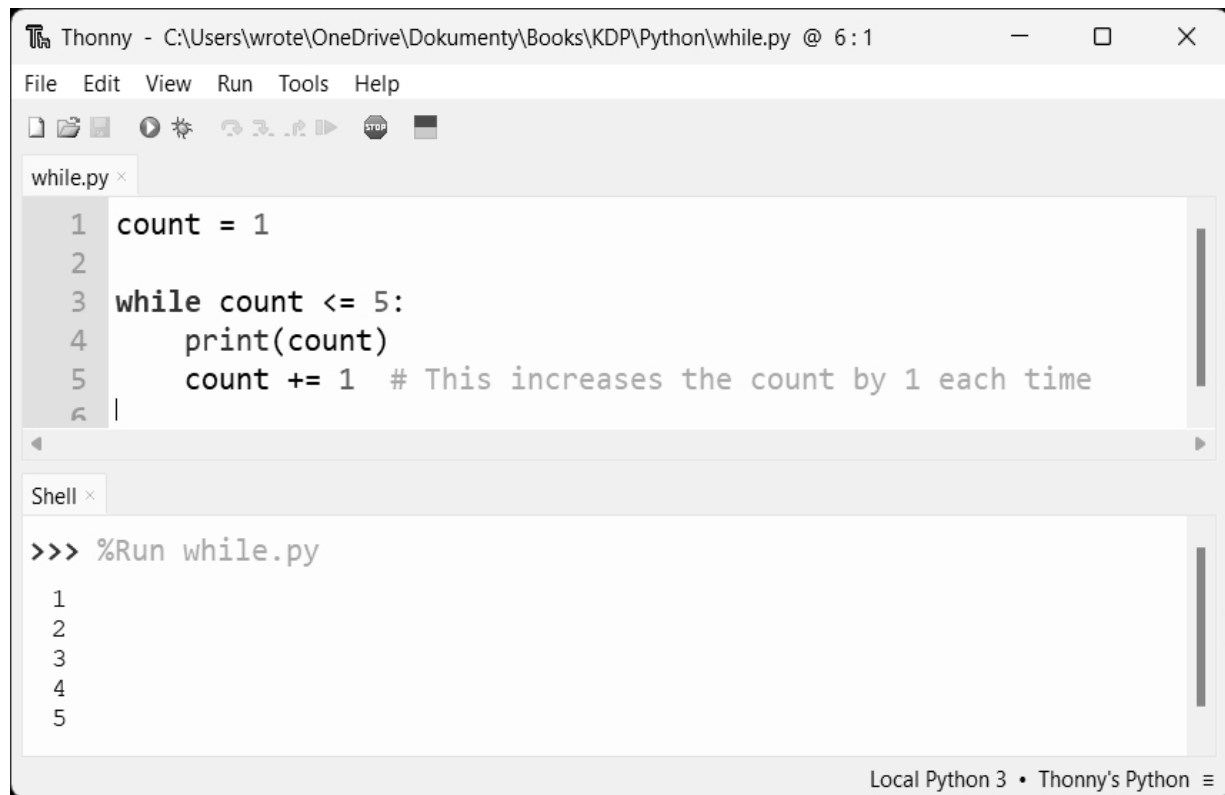
```
count = 1
```

```
while count <= 5:
```

```
    print(count)
```

```
    count += 1 # This increases the count by 1 each time
```

Output:

The image shows a screenshot of the Thonny Python IDE. The title bar at the top reads "Thonny - C:\Users\wrote\OneDrive\Dokumenty\Books\KDP\Python\while.py @ 6:1". Below the title bar is a menu bar with "File", "Edit", "View", "Run", "Tools", and "Help". Underneath the menu bar is a toolbar with icons for file operations and running code. The main editor window shows a Python script in a file named "while.py". The script contains the following code:

```
1 count = 1
2
3 while count <= 5:
4     print(count)
5     count += 1 # This increases the count by 1 each time
6
```

The script is executed, and the output is shown in the "Shell" window at the bottom. The shell prompt is ">>> %Run while.py", and the output consists of the numbers 1, 2, 3, 4, and 5, each on a new line. The status bar at the bottom right indicates "Local Python 3 • Thonny's Python".

In this example, the while loop keeps running as long as the condition (`count <= 5`) is true. Each time the loop runs, `count` increases by 1. Once `count` reaches 6, the condition becomes false, and the loop stops.

4. Avoiding Infinite Loops

Be careful with while loops! If the condition never becomes false, your loop will run forever, creating an **infinite loop**. This can cause your program to freeze or crash. Always make sure your condition will eventually become false.

For example, if we forget to increase `count` in the previous example:

```
count = 1

while count <= 5:
    print(count)
    # Oops! We forgot to increase the count!
```

This would cause an infinite loop because `count` will always stay 1, and the condition `count <= 5` will never change. To avoid this, make sure your loop has a way to stop!

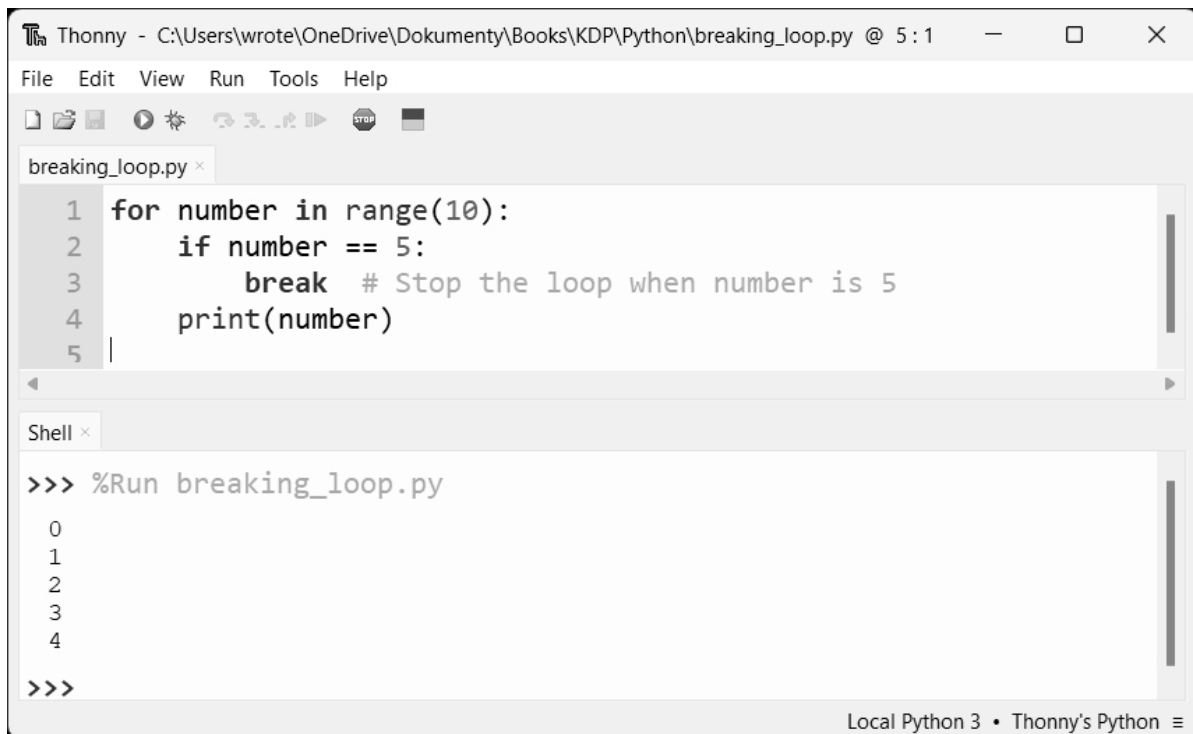
5. Breaking Out of a Loop

Sometimes, you may want to stop a loop before it finishes all the iterations. You can use the break statement to exit a loop early.

Example:

```
for number in range(10):  
    if number == 5:  
        break # Stop the loop when number is 5  
    print(number)
```

Output:



The screenshot shows the Thonny Python IDE interface. The top window, titled 'breaking_loop.py', contains the following Python code:

```
1 for number in range(10):  
2     if number == 5:  
3         break # Stop the loop when number is 5  
4     print(number)  
5
```

Below the code editor is a 'Shell' window showing the execution of the script. The prompt is '>>> %Run breaking_loop.py', and the output is:

```
0  
1  
2  
3  
4  
>>>
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

Here, the loop stops when number reaches 5 because of the break statement.

6. Practical Example: Sum of Numbers

Let's use a loop to solve a simple problem: adding up all the numbers from 1 to 10.

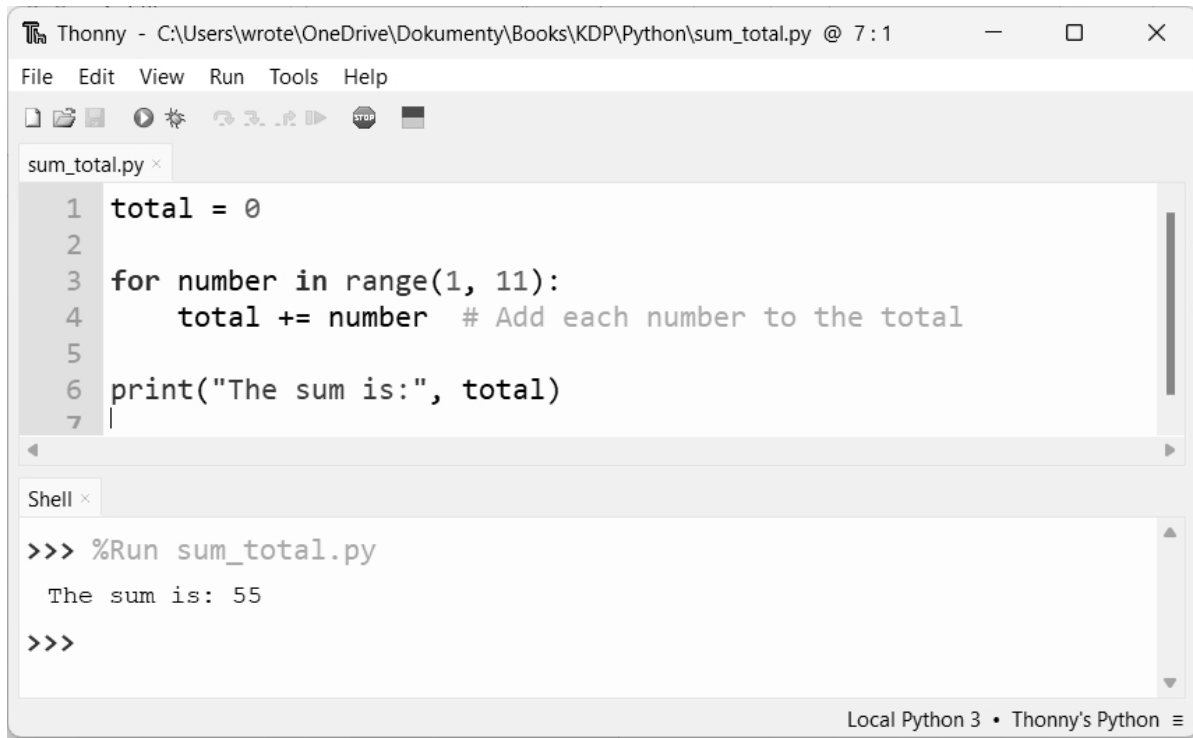
```
total = 0
```

```
for number in range(1, 11):
```

```
    total += number # Add each number to the total
```

```
print("The sum is:", total)
```

Output:



The screenshot shows the Thonny Python IDE interface. The top window, titled 'sum_total.py', contains the following Python code:

```
1 total = 0
2
3 for number in range(1, 11):
4     total += number # Add each number to the total
5
6 print("The sum is:", total)
7
```

The bottom window, titled 'Shell', shows the execution output:

```
>>> %Run sum_total.py
The sum is: 55
>>>
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

In this example, the loop goes through each number from 1 to 10, adding each one to total. By the end of the loop, total contains the sum of all the numbers.

7. Practical Example: Guess the Number Game

Let's build a simple game using a while loop. The game will ask the player to guess a secret number between 1 and 10. The loop will keep going until the player guesses the number correctly.

```
secret_number = 7
guess = 0

while guess != secret_number:
    guess = int(input("Guess the number between 1 and 10: "))

    if guess < secret_number:
        print("Too low!")
    elif guess > secret_number:
        print("Too high!")
```



```
else:  
    print("You guessed it!")
```

In this game, the while loop keeps asking the player to guess until they get it right. If their guess is too low or too high, the program gives them a hint.

8. Using else with Loops

You can also use an else block with loops. The else part will run after the loop finishes, unless you break out of the loop early.

Example:

```
for i in range(5):  
    print(i)  
else:  
    print("Loop finished!")
```

Output:



The screenshot shows the Thonny Python IDE interface. The top window displays the code for 'loop_finished.py':

```
1 for i in range(5):  
2     print(i)  
3 else:  
4     print("Loop finished!")  
5
```

The bottom window, labeled 'Shell', shows the output of running the code:

```
>>> %Run loop_finished.py  
0  
1  
2  
3  
4  
Loop finished!  
>>>
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

In this case, the else block runs after the loop completes all its iterations. However, if we had a break statement in the loop, the else block wouldn't run.

Why Loops Are Powerful

Loops are a huge time-saver in programming. Instead of repeating the same code over and over, you can write a loop to do the work for you. They help you:

- **Solve repetitive problems:** You can repeat tasks efficiently, whether you're printing a list of names, adding numbers, or building a game.
- **Work with large amounts of data:** Loops allow you to process each item in a list or range of numbers without writing extra code.

With loops, you can automate repetitive tasks and solve problems much faster. Keep experimenting with for and while loops, and try writing your own programs to see just how powerful and flexible they are. You're one step closer to mastering Python!

Practical Examples: Building a Simple Quiz

Now that you've learned how to use conditional statements (if, else, elif) and loops (for, while), it's time to put everything together in a fun and practical project: building a **simple quiz** in Python! This will help you see how the concepts you've learned come to life in a real program. By the end of this section, you'll have created an interactive quiz that tests the user's knowledge and provides instant feedback.

1. Planning the Quiz

Before writing the code, let's think about what we want the quiz to do:

- Ask the user several questions.
- Allow the user to answer each question.
- Give feedback on whether the answer is correct or not.
- Keep track of how many questions the user gets right.
- Display the final score at the end.

We'll start simple, with three questions, and build from there.

2. Writing the Quiz: Step by Step

Step 1: Setting Up the Questions and Answers

We need a way to store the quiz questions and their correct answers. An easy way to do this is by using a **list** to store the questions and another list for the answers.

Here's how to set that up:

```
# List of questions
questions = [
    "What is the capital of France?",
    "What is 5 + 7?",
```

```
"What color is the sky?"  
]
```

```
# List of correct answers
```

```
answers = [  
    "Paris",  
    "12",  
    "blue"  
]
```

Each item in the questions list matches up with an item in the answers list. The first question, "What is the capital of France?", has the answer "Paris", and so on.

Step 2: Asking the Questions

Next, we'll use a **for loop** to go through each question and ask the user for their answer.

```
score = 0 # Keep track of how many questions the user gets right
```

```
for i in range(len(questions)):
```

```
    # Ask the question
```

```
    user_answer = input(questions[i] + " ")
```

```
    # Check if the user's answer is correct
```

```
    if user_answer.lower() == answers[i].lower():
```

```
        print("Correct!")
```

```
        score += 1 # Increase the score if they are correct
```

```
    else:
```

```
        print("Sorry, the correct answer is", answers[i])
```

In this code:

- We loop through each question using the for loop.
- The input() function is used to get the user's answer.
- We compare the user's answer with the correct answer, ignoring case differences by converting both to lowercase (.lower()).

- If the answer is correct, we print "Correct!" and increase the score.
- If the answer is incorrect, we print the correct answer.

Step 3: Displaying the Final Score

After the user has answered all the questions, we'll display their final score using a simple print() statement:

```
print("You got", score, "out of", len(questions), "correct!")
```

This line tells the user how many questions they got right out of the total number of questions.

3. The Complete Quiz Code

Here's the complete code for your simple quiz:

```
# List of questions
questions = [
    "What is the capital of France?",
    "What is 5 + 7?",
    "What color is the sky?"
]

# List of correct answers
answers = [
    "Paris",
    "12",
    "blue"
]

score = 0 # Keep track of how many questions the user gets right

# Ask each question in the list
for i in range(len(questions)):
    # Ask the question
    user_answer = input(questions[i] + " ")

    # Check if the user's answer is correct
    if user_answer.lower() == answers[i].lower():
```

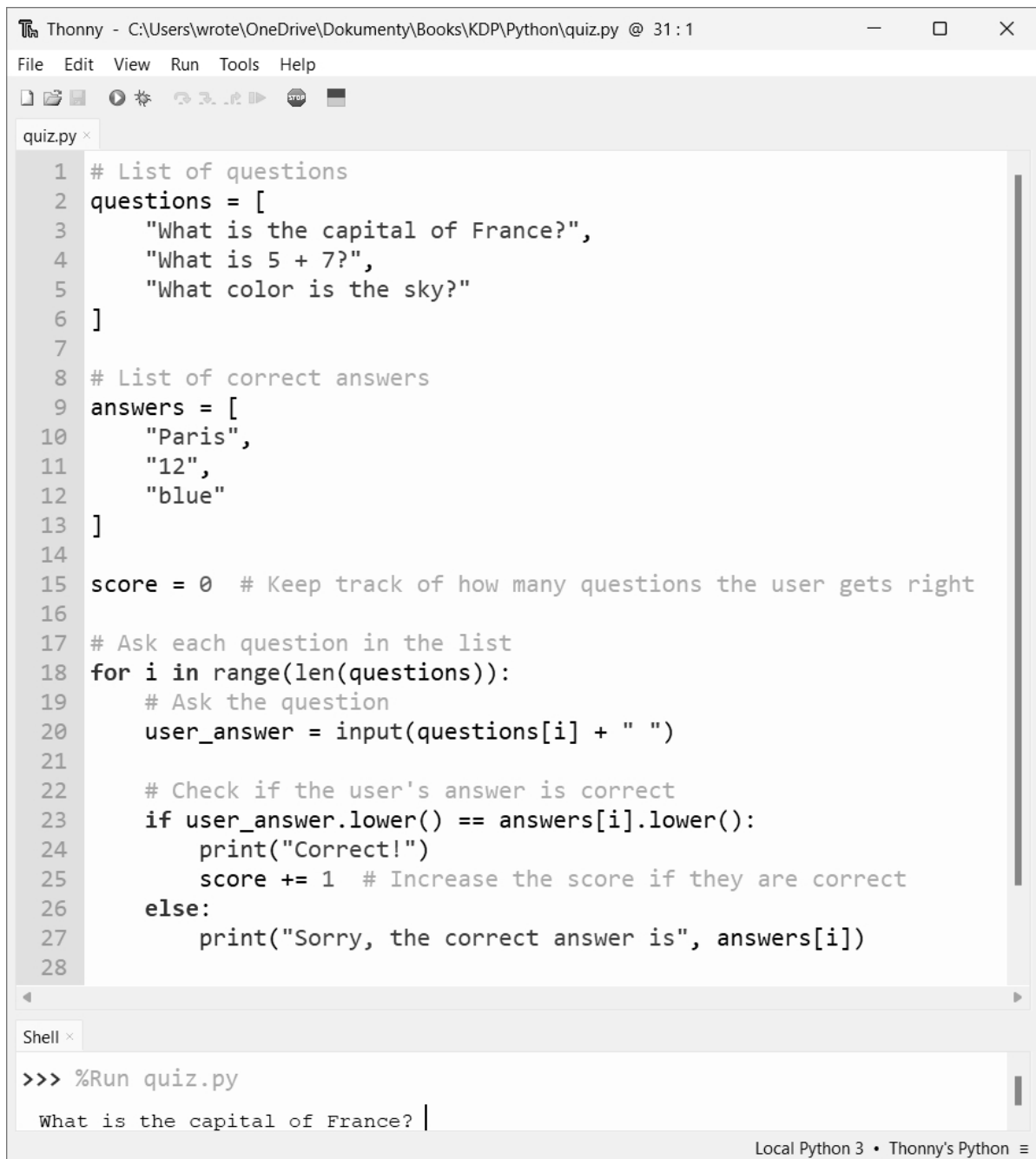
```
        print("Correct!")
        score += 1 # Increase the score if they are correct
    else:
        print("Sorry, the correct answer is", answers[i])

# Display the final score
print("You got", score, "out of", len(questions), "correct!")
```

4. Running the Quiz

When you run the quiz, Python will ask each question, wait for the user to type their answer, and then give feedback. After all the questions have been answered, it will show the final score.

Example run:



The screenshot shows the Thonny Python IDE interface. The top window, titled 'quiz.py', contains the following Python code:

```
1 # List of questions
2 questions = [
3     "What is the capital of France?",
4     "What is 5 + 7?",
5     "What color is the sky?"
6 ]
7
8 # List of correct answers
9 answers = [
10     "Paris",
11     "12",
12     "blue"
13 ]
14
15 score = 0 # Keep track of how many questions the user gets right
16
17 # Ask each question in the list
18 for i in range(len(questions)):
19     # Ask the question
20     user_answer = input(questions[i] + " ")
21
22     # Check if the user's answer is correct
23     if user_answer.lower() == answers[i].lower():
24         print("Correct!")
25         score += 1 # Increase the score if they are correct
26     else:
27         print("Sorry, the correct answer is", answers[i])
28
```

The bottom window, titled 'Shell', shows the execution of the program:

```
>>> %Run quiz.py
What is the capital of France? |
```

The status bar at the bottom right indicates 'Local Python 3 • Thonny's Python'.

5. Making the Quiz More Interesting

Now that you have the basic quiz working, here are a few ways you can make it more interesting:

- **Add More Questions:** Expand your lists of questions and answers to make the quiz longer.

```
questions.append("What is the largest planet in our solar system?")
answers.append("Jupiter")
```

- **Randomize the Questions:** Use the random module to shuffle the questions so they appear in a different order each time the quiz is played.

```
import random
combined = list(zip(questions, answers)) # Combine questions and answers
random.shuffle(combined) # Shuffle the combined list
questions, answers = zip(*combined) # Unpack the shuffled list
```

- **Limit the Number of Tries:** Give the user only a certain number of tries to get the answer right.

```
attempts = 3
while attempts > 0:
    user_answer = input(questions[i] + " ")
    if user_answer.lower() == answers[i].lower():
        print("Correct!")
        score += 1
        break
    else:
        attempts -= 1
        print("Try again. Attempts left:", attempts)
        if attempts == 0:
            print("Sorry, the correct answer is", answers[i])
```

- **Add a Timer:** You can give players a limited amount of time to answer each question using the time module.

```
import time
start_time = time.time() # Record the start time

# Example of a question
user_answer = input("What is 5 + 7? ")
```



```
# Check the time taken
if time.time() - start_time > 10: # 10 seconds limit
    print("Time's up!")
```

6. Why Build a Quiz?

Building a quiz might seem simple, but it's a great way to practice using Python's **control structures** — loops and conditional statements — in a real-world context. This project combines everything you've learned:

- **Conditional statements** for checking answers.
- **Loops** for going through the list of questions.
- **Input** for interacting with the user.

It's also a project that you can easily expand and customize, making it more fun as you add your own creativity.

Now that you've built a simple quiz, you've taken a big step in understanding how Python can be used to solve real-world problems. Keep practicing, experiment with different features, and soon you'll be creating even more complex projects!

Chapter 3: Functions: Breaking Down Complex Problems

Writing and Using Functions for Modularity

As your programs get more complex, writing everything in one big chunk of code can quickly become messy and hard to manage. That's where **functions** come in! Functions help you break down complicated problems into smaller, more manageable pieces. This makes your code easier to understand, test, and reuse. In this section, we'll explore how to write and use functions to make your code more modular and organized.

1. What Is a Function?

A **function** is like a mini-program within your program. It's a block of code that performs a specific task, and you can "call" it whenever you need that task to be done. You've already been using functions like `print()` and `input()`, but now you'll learn how to create your own.

Think of a function as a recipe. When you want to cook something, you don't write out every step each time — you follow the same recipe. Similarly, when you write a function, you create it once and then use it whenever needed without rewriting the same code.

2. Why Use Functions?

There are many benefits to using functions:

- **Modularity:** Functions help you break down big problems into smaller, manageable tasks. Each task can have its own function, making your code easier to understand.

- **Reusability:** Once you've written a function, you can use it multiple times without rewriting the code.
- **Organization:** Functions keep your code neat and organized. It's easier to spot mistakes and make updates when your code is divided into logical sections.

3. Writing Your First Function

Let's start by creating a simple function that greets the user. Here's how you define a function in Python:

Syntax:

```
def function_name():  
    # Code inside the function
```

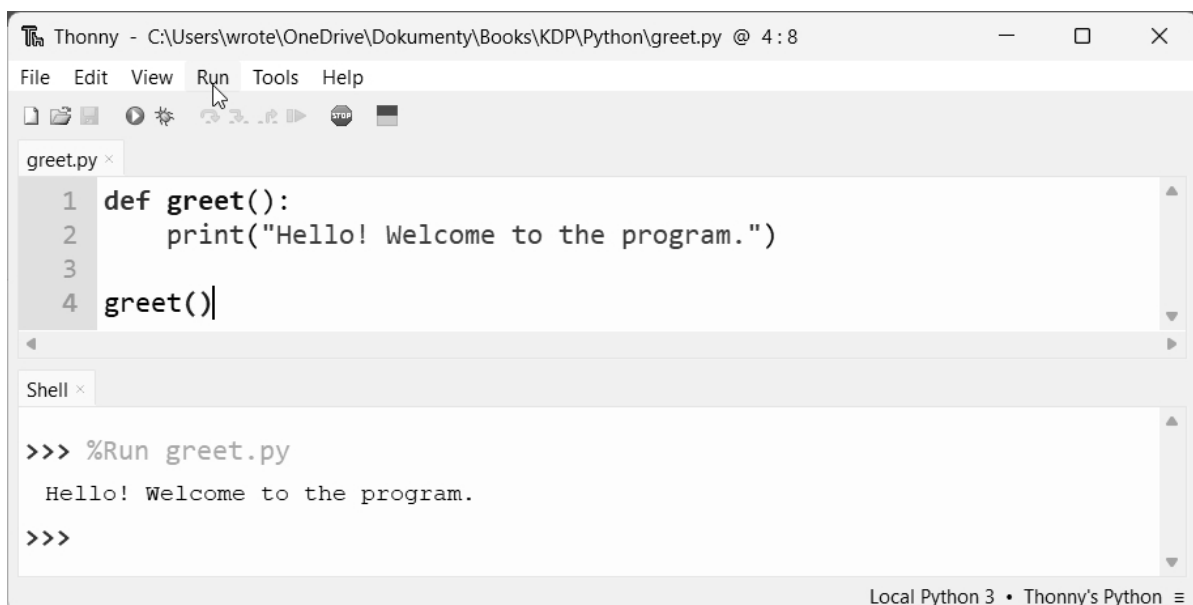
Example:

```
def greet():  
    print("Hello! Welcome to the program.")
```

This is a very basic function named `greet()`. The keyword `def` tells Python that we're defining a new function, and the code inside the function is what will run when we call it. Now, let's **call** the function to see it in action:

```
greet()
```

Output:



The screenshot shows the Thonny Python IDE interface. The main editor window displays a Python file named `greet.py` with the following code:

```
1 def greet():  
2     print("Hello! Welcome to the program.")  
3  
4 greet()
```

Below the editor is a Shell window showing the execution of the script:

```
>>> %Run greet.py  
Hello! Welcome to the program.  
>>>
```

The status bar at the bottom indicates "Local Python 3 • Thonny's Python".

As you can see, when we call the `greet()` function, Python runs the code inside it and prints the greeting. You can call this function anytime you want to greet the user without having to rewrite the `print()` statement.

4. Making Functions More Useful with Parameters

Functions become even more powerful when you can pass in data for them to work with. These pieces of data are called **parameters**, and they allow your functions to be more flexible.

Let's modify the `greet()` function to accept a parameter so it can greet a specific person:

```
def greet(name):  
    print("Hello, " + name + "! Welcome to the program.")
```

Now, the `greet()` function takes a **parameter** called `name`. When we call the function, we can pass in a name, and the function will greet that person:

```
greet("Alice")  
greet("Bob")
```

Output:



```
Thonny - C:\Users\wrote\OneDrive\Dokumenty\Books\KDP\Python\greet1.py @ 5:13  
File Edit View Run Tools Help  
greet1.py x  
1 def greet(name):  
2     print("Hello, " + name + "! Welcome to the program.")  
3  
4 greet("Alice")  
5 greet("Bob")  
  
Shell x  
>>> %Run greet1.py  
Hello, Alice! Welcome to the program.  
Hello, Bob! Welcome to the program.  
>>>  
Local Python 3 • Thonny's Python
```

By using parameters, the same function can do different things based on the input it receives. This makes your functions much more flexible and reusable.

5. Returning Values from Functions

Sometimes, you'll want your function to calculate or create something and then send that result back to the rest of your program. To do this, you use a **return** statement. A function can "return" a value to the part of the program that called it.

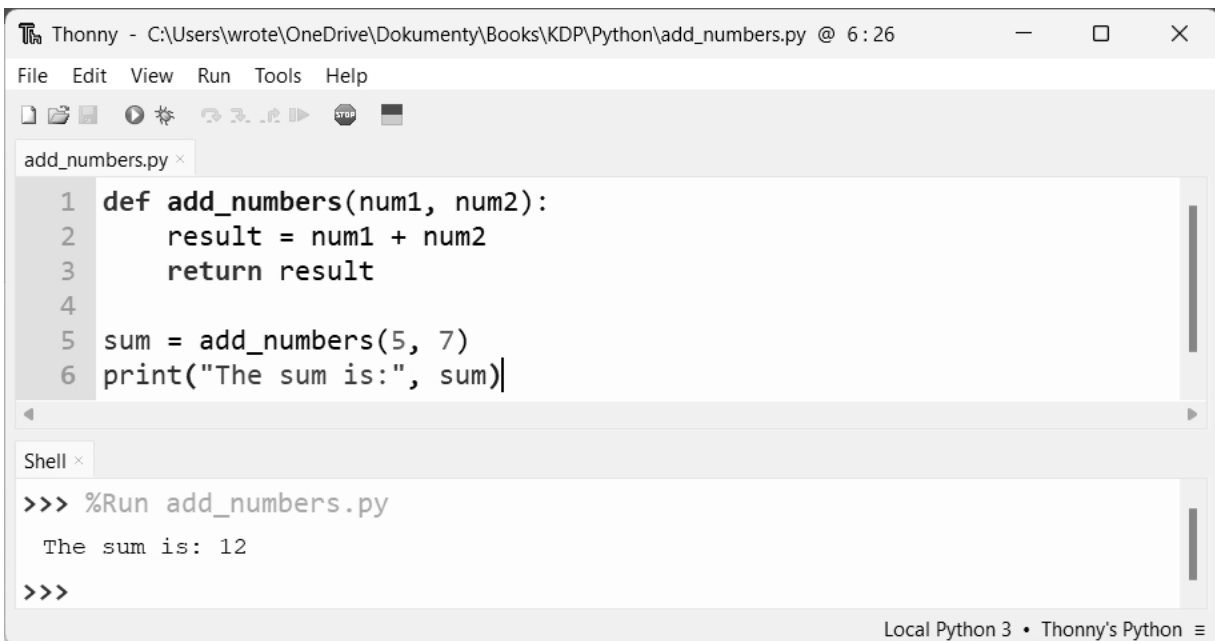
Let's create a function that adds two numbers and returns the result:

```
def add_numbers(num1, num2):  
    result = num1 + num2  
    return result
```

In this function, `num1` and `num2` are parameters, and `result` stores the sum of those two numbers. The `return` statement sends the result back to the caller. Now we can call the function and store the result in a variable:

```
sum = add_numbers(5, 7)  
print("The sum is:", sum)
```

Output:



The screenshot shows the Thonny Python IDE interface. The top window, titled 'add_numbers.py', contains the following code:

```
1 def add_numbers(num1, num2):  
2     result = num1 + num2  
3     return result  
4  
5 sum = add_numbers(5, 7)  
6 print("The sum is:", sum)
```

Below the code editor is a 'Shell' window showing the execution output:

```
>>> %Run add_numbers.py  
The sum is: 12  
>>>
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

The `add_numbers()` function calculates the sum of 5 and 7, then returns the result, which is stored in the variable `sum`. You can now use this result in other parts of your program.

6. Breaking Down Complex Problems with Functions

One of the main reasons to use functions is to break down complex problems into smaller, more manageable parts. Let's say you're building a program that calculates the area of both a rectangle and a circle. Instead of writing all the code in one long block, you can create two separate functions for each calculation.

Example:

```
# Function to calculate the area of a rectangle
```

```
def rectangle_area(length, width):
```

```
    return length * width
```

```
# Function to calculate the area of a circle
```

```
def circle_area(radius):
```

```
    return 3.14 * radius * radius
```

Now, whenever you need to calculate the area of a rectangle or circle, you just call the appropriate function:

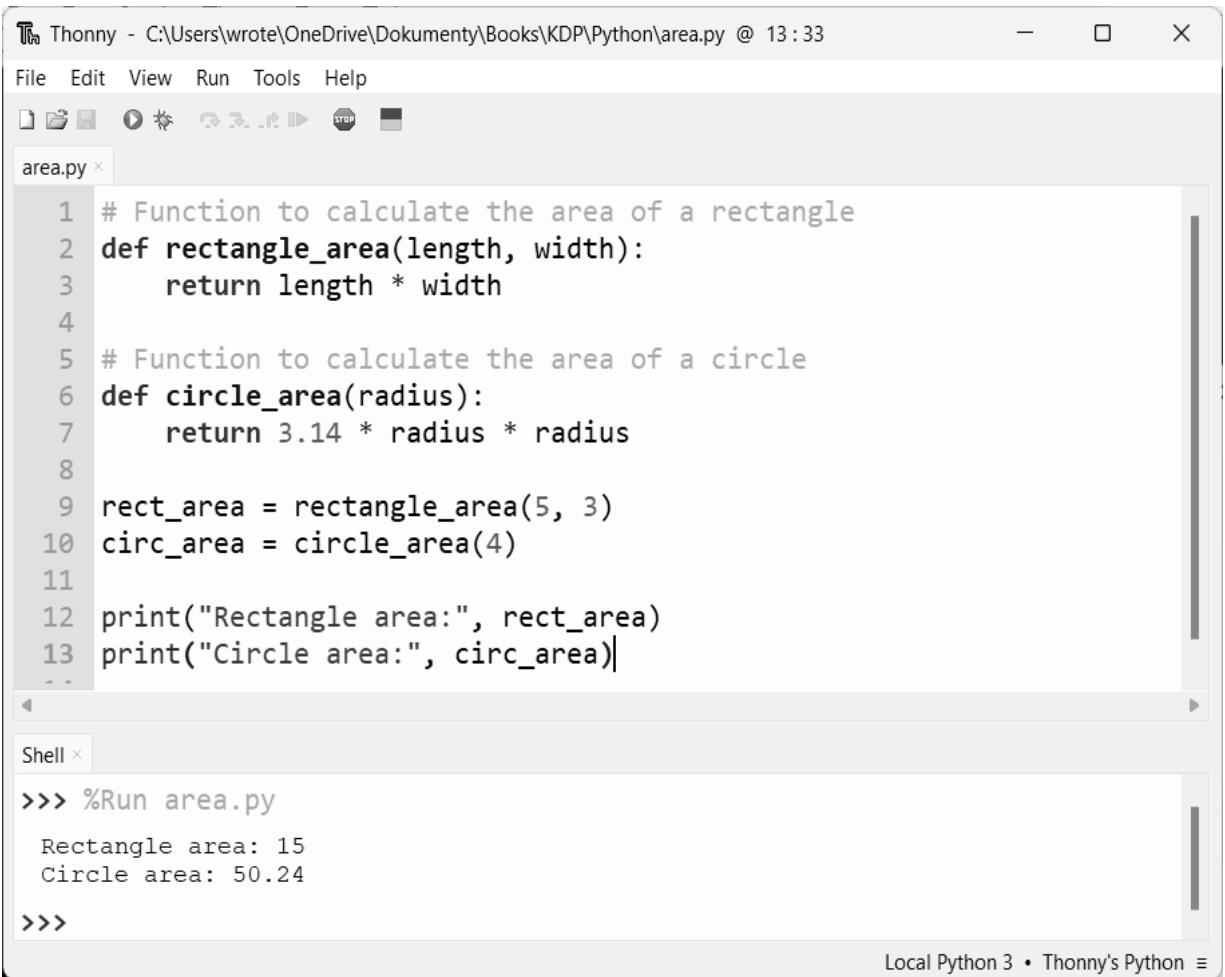
```
rect_area = rectangle_area(5, 3)
```

```
circ_area = circle_area(4)
```

```
print("Rectangle area:", rect_area)
```

```
print("Circle area:", circ_area)
```

Output:



The screenshot shows the Thonny Python IDE interface. The top window, titled 'area.py', contains the following Python code:

```
1 # Function to calculate the area of a rectangle
2 def rectangle_area(length, width):
3     return length * width
4
5 # Function to calculate the area of a circle
6 def circle_area(radius):
7     return 3.14 * radius * radius
8
9 rect_area = rectangle_area(5, 3)
10 circ_area = circle_area(4)
11
12 print("Rectangle area:", rect_area)
13 print("Circle area:", circ_area)
```

The bottom window, titled 'Shell', shows the execution of the script:

```
>>> %Run area.py
Rectangle area: 15
Circle area: 50.24
>>>
```

The status bar at the bottom right indicates 'Local Python 3 • Thonny's Python'.

By using functions, your code is cleaner, easier to read, and much easier to maintain. If you ever need to update how the area is calculated, you only need to change the function itself, not every place you've used it.

7. Keeping Your Code Modular

Modular code is like building with Lego blocks. Each function is a block that does one specific job. When you need to solve a bigger problem, you can combine these blocks (functions) to build something more complex.

For example, if you're creating a program to process orders at a restaurant, you could have separate functions for:

- Taking the order
- Calculating the total price
- Applying a discount

- Printing the receipt

Each function handles its own task, and together, they form a complete program.

Example:

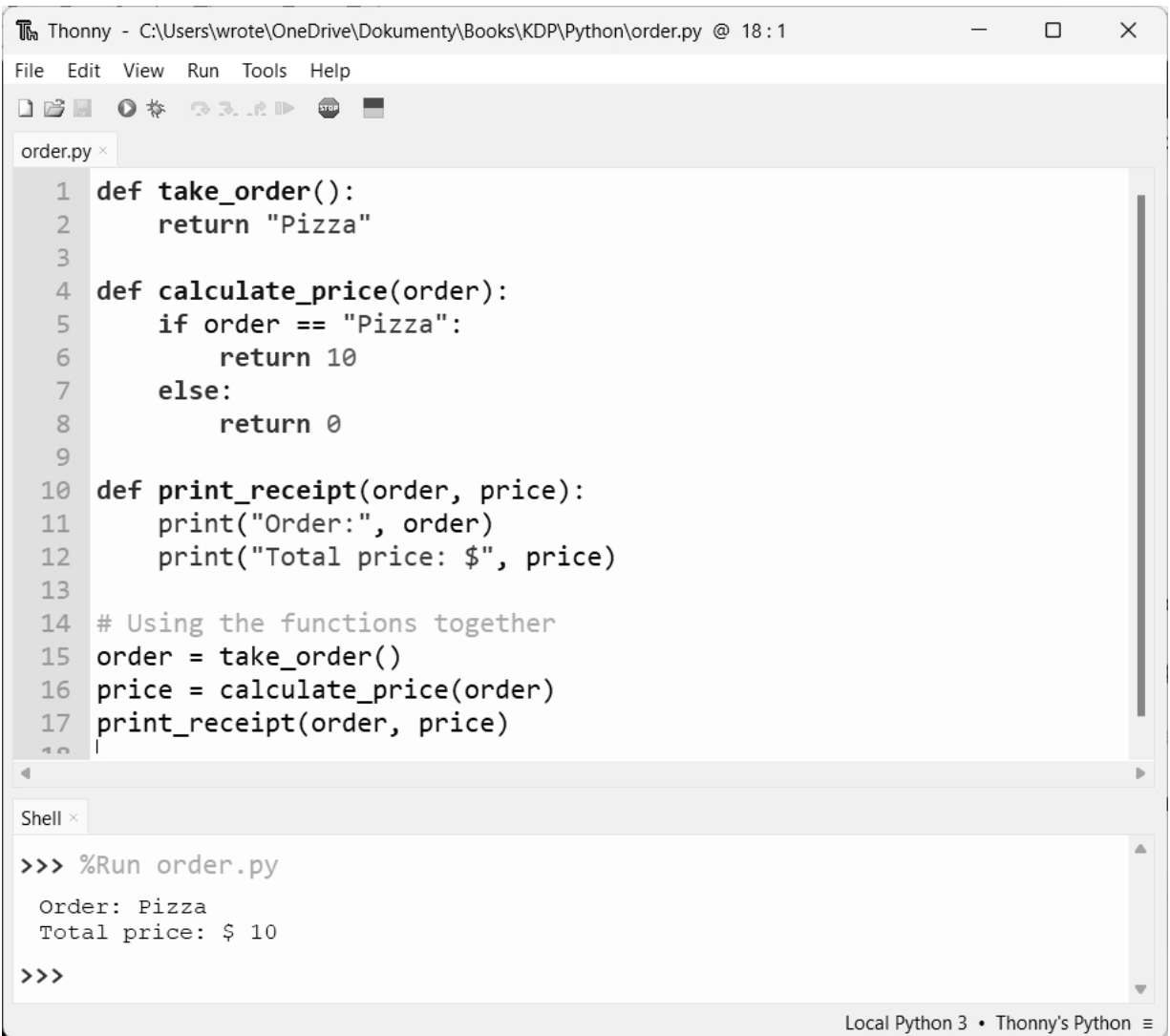
```
def take_order():
    return "Pizza"

def calculate_price(order):
    if order == "Pizza":
        return 10
    else:
        return 0

def print_receipt(order, price):
    print("Order:", order)
    print("Total price: $", price)

# Using the functions together
order = take_order()
price = calculate_price(order)
print_receipt(order, price)
```

By dividing each task into its own function, your code is much easier to manage and update.



The screenshot shows the Thonny Python IDE interface. The top window, titled 'order.py', contains the following Python code:

```
1 def take_order():
2     return "Pizza"
3
4 def calculate_price(order):
5     if order == "Pizza":
6         return 10
7     else:
8         return 0
9
10 def print_receipt(order, price):
11     print("Order:", order)
12     print("Total price: $", price)
13
14 # Using the functions together
15 order = take_order()
16 price = calculate_price(order)
17 print_receipt(order, price)
```

The bottom window, titled 'Shell', shows the execution output:

```
>>> %Run order.py
Order: Pizza
Total price: $ 10
>>>
```

The status bar at the bottom right indicates 'Local Python 3 • Thonny's Python'.

Key Takeaways

- **Functions** help you break down complex problems into smaller, manageable parts.
- **Parameters** allow you to pass data into a function, making it flexible and reusable.
- **Return values** let you send results from a function back to the main program.
- Functions help keep your code **modular**, **organized**, and **easy to maintain**.

By writing and using functions, you'll be able to solve problems more efficiently and write cleaner, more organized code. Keep practicing, and soon, functions will become one of your most powerful tools in Python!

Understanding Function Arguments and Return Values

Functions are a powerful way to break down problems into smaller, manageable pieces. But to make the most of functions, it's important to understand how they work with **arguments** and **return values**. These two concepts allow functions to take in data, do something with it, and send the result back. In this section, we'll explore how function arguments and return values work, with clear examples to help you grasp these essential concepts.

1. What Are Function Arguments?

Function arguments (also called **parameters**) are pieces of information you pass into a function so that the function can work with them. Think of arguments as inputs that a function needs to perform its task.

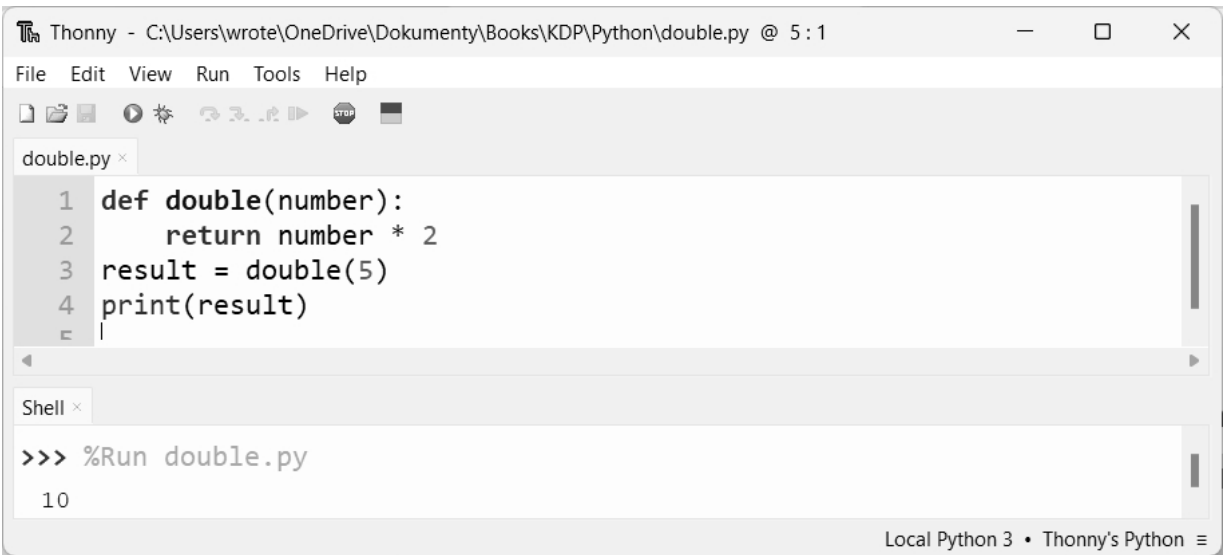
Here's a simple function that takes one argument — a number — and doubles it:

```
def double(number):  
    return number * 2
```

In this example, `number` is the argument. When you call the function and provide an actual number, like this:

```
result = double(5)  
print(result)
```

The function will take 5 as the argument, double it, and return the result. The output will be:



The screenshot shows the Thonny Python IDE interface. The title bar indicates the file path: C:\Users\wrote\OneDrive\Dokumenty\Books\KDP\Python\double.py @ 5 : 1. The menu bar includes File, Edit, View, Run, Tools, and Help. Below the menu is a toolbar with icons for file operations and execution. The main editor window displays the following Python code in double.py:

```
1 def double(number):  
2     return number * 2  
3 result = double(5)  
4 print(result)
```

Below the editor is a Shell window showing the execution command and output:

```
>>> %Run double.py  
10
```

The status bar at the bottom right indicates 'Local Python 3 • Thonny's Python'.

Why Are Arguments Useful?

Arguments make your functions more flexible because they allow you to work with different values. Instead of writing a function that always works with a specific number, you can create a function that works with any number by using arguments.

2. Multiple Arguments

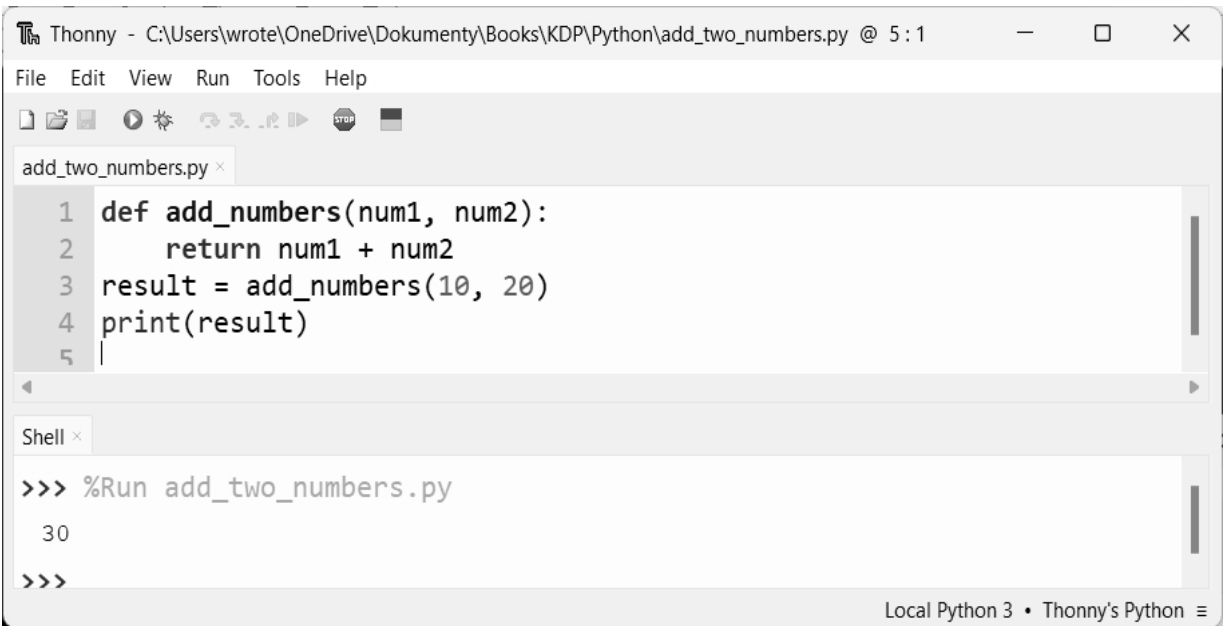
Functions can take more than one argument. For example, let's write a function that takes two numbers and adds them together:

```
def add_numbers(num1, num2):  
    return num1 + num2
```

Now, you can call the function with two arguments:

```
result = add_numbers(10, 20)  
print(result)
```

Output:

The screenshot shows the Thonny Python IDE interface. The top window, titled 'add_two_numbers.py', contains the following Python code:

```
1 def add_numbers(num1, num2):  
2     return num1 + num2  
3 result = add_numbers(10, 20)  
4 print(result)  
5
```

The bottom window, titled 'Shell', shows the execution of the script with the command `>>> %Run add_two_numbers.py` and the output `30`. The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

In this case, num1 is 10 and num2 is 20. The function adds them together and returns the result, which is 30. You can use this function with any two numbers, making it versatile and reusable.

3. Default Arguments

Sometimes, you might want to give a function a **default value** for an argument, so that if the caller doesn't provide one, the function still works. You can do this by setting a default value in the function definition.

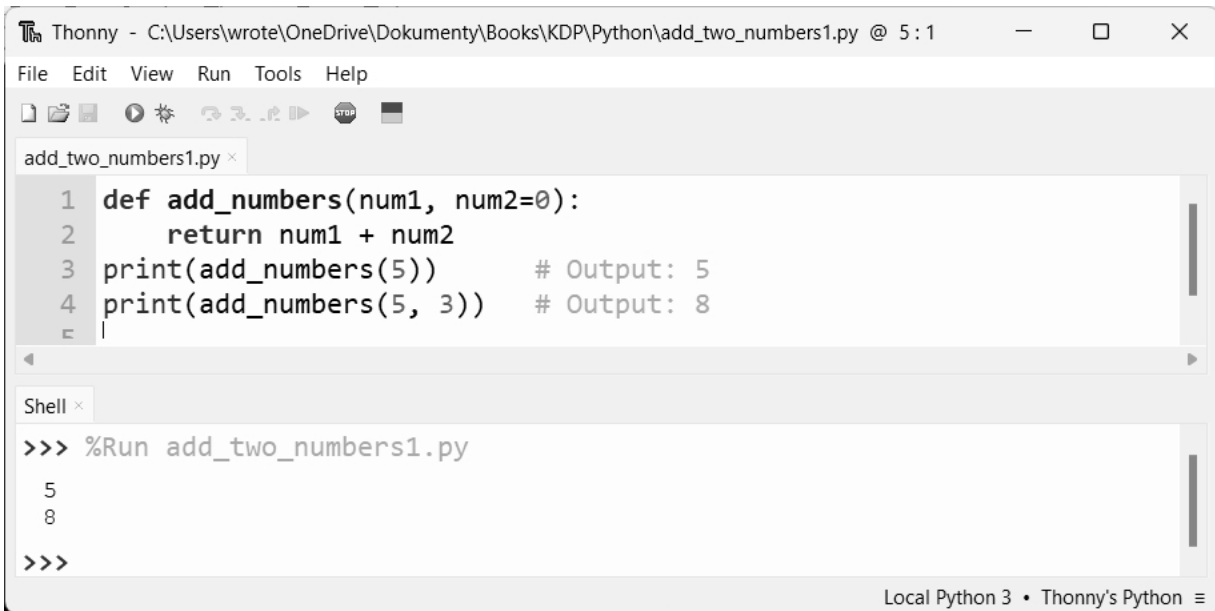
Let's modify our `add_numbers()` function to give num2 a default value of 0:

```
def add_numbers(num1, num2=0):  
    return num1 + num2
```

Now, you can call the function with either one or two arguments:

```
print(add_numbers(5))    # Output: 5  
print(add_numbers(5, 3)) # Output: 8
```

If you only provide one argument, num2 will automatically be set to 0. This makes your function more flexible.



The screenshot shows the Thonny Python IDE interface. The title bar indicates the file path: C:\Users\wrote\OneDrive\Dokumenty\Books\KDP\Python\add_two_numbers1.py @ 5:1. The menu bar includes File, Edit, View, Run, Tools, and Help. The toolbar contains icons for file operations and execution. The editor window displays the following Python code:

```
1 def add_numbers(num1, num2=0):
2     return num1 + num2
3 print(add_numbers(5))      # Output: 5
4 print(add_numbers(5, 3))  # Output: 8
```

Below the editor is a Shell window with the following content:

```
>>> %Run add_two_numbers1.py
5
8
>>>
```

The status bar at the bottom right shows "Local Python 3 • Thonny's Python".

4. What Are Return Values?

When a function finishes its task, it can send a result back to the part of the program that called it. This result is called a **return value**. You use the return statement to send this value back.

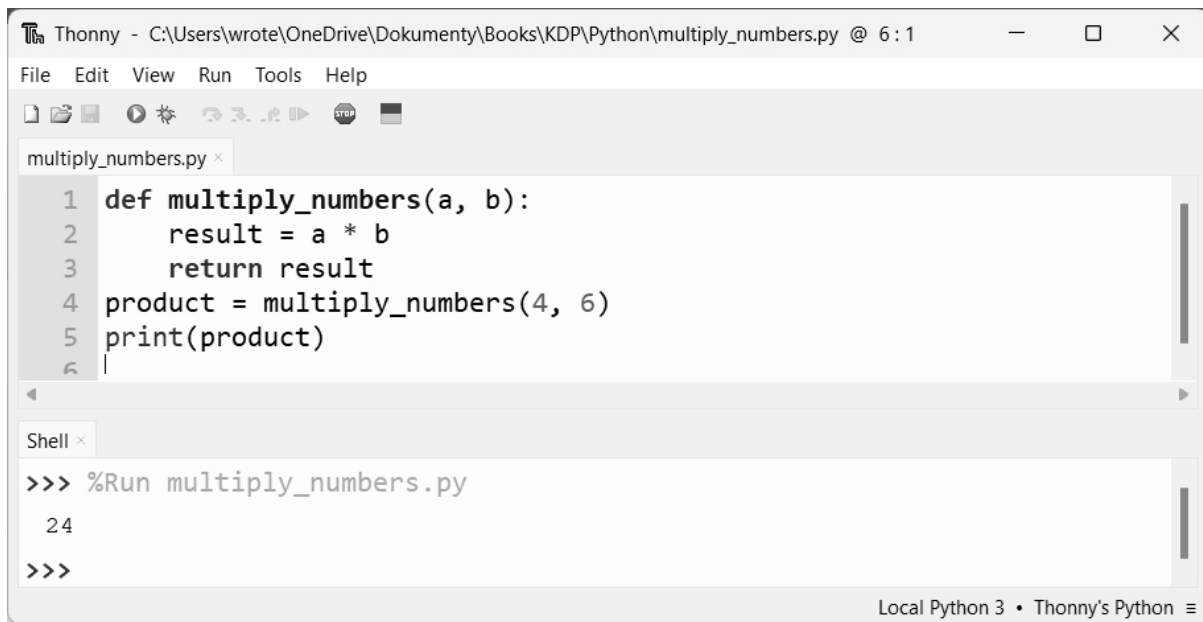
Here's a simple function that multiplies two numbers and returns the result:

```
def multiply_numbers(a, b):
    result = a * b
    return result
```

When you call this function, it will return the result of multiplying a and b:

```
product = multiply_numbers(4, 6)
print(product)
```

Output:

The image shows a screenshot of the Thonny Python IDE. The top window, titled 'multiply_numbers.py', contains the following Python code:

```
1 def multiply_numbers(a, b):  
2     result = a * b  
3     return result  
4 product = multiply_numbers(4, 6)  
5 print(product)  
6
```

The bottom window, titled 'Shell', shows the execution of the script with the command `>>> %Run multiply_numbers.py`. The output is `24`, followed by a prompt `>>>`. The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

The value 24 is the **return value**, and it's stored in the variable `product`. You can now use `product` in other parts of your program.

5. Why Use Return Values?

Return values are important because they allow you to:

- **Reuse the result:** Once a function returns a value, you can use that value in other parts of your program.
- **Make functions more modular:** By returning a result, your function doesn't have to handle everything itself — it can just do its job and pass the result back to the rest of the program.

For example, let's say you want to calculate the area of a rectangle. You could write one function to calculate the area and return the result, and another function to print it:

```
def rectangle_area(length, width):  
    return length * width
```

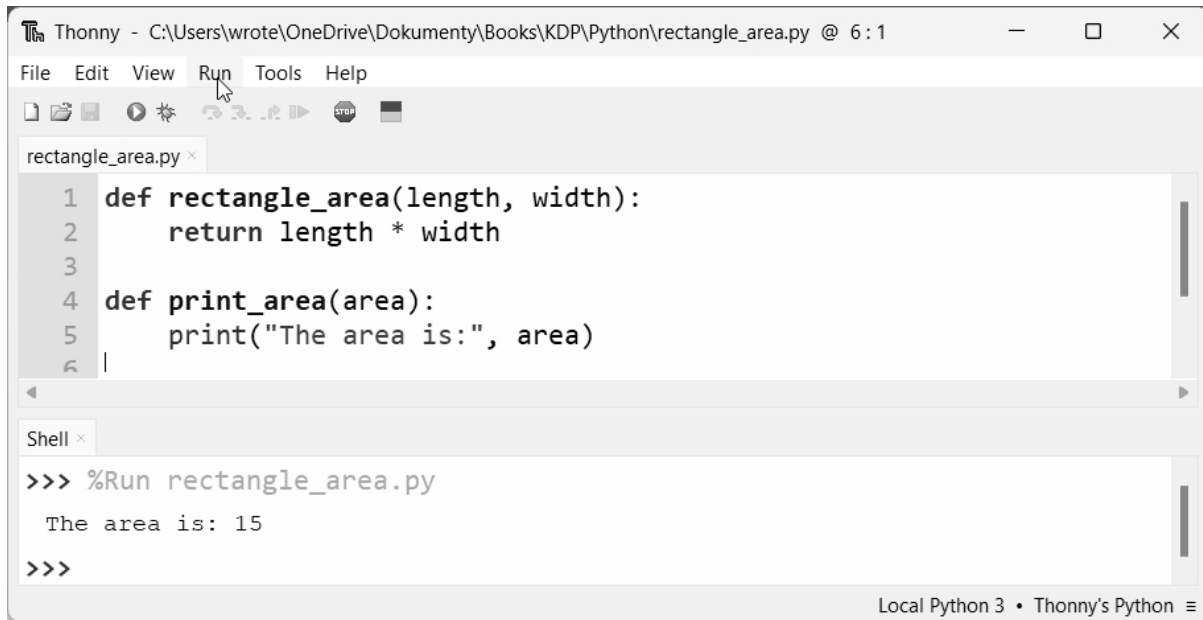
```
def print_area(area):  
    print("The area is:", area)
```

Now we can use both functions together

```
area = rectangle_area(5, 3)
```

```
print_area(area)
```

Output:



The screenshot shows the Thonny Python IDE interface. The top window displays a Python script named `rectangle_area.py` with the following code:

```
1 def rectangle_area(length, width):  
2     return length * width  
3  
4 def print_area(area):  
5     print("The area is:", area)  
6
```

The bottom window, titled "Shell", shows the execution of the script using the command `>>> %Run rectangle_area.py`. The output is `The area is: 15`, followed by a prompt `>>>`. The status bar at the bottom indicates "Local Python 3 • Thonny's Python".

By separating the calculation and the printing into two functions, your code is more organized and easier to manage.

6. Returning Multiple Values

Sometimes, you might want to return more than one value from a function. You can do this in Python by returning the values as a **tuple**.

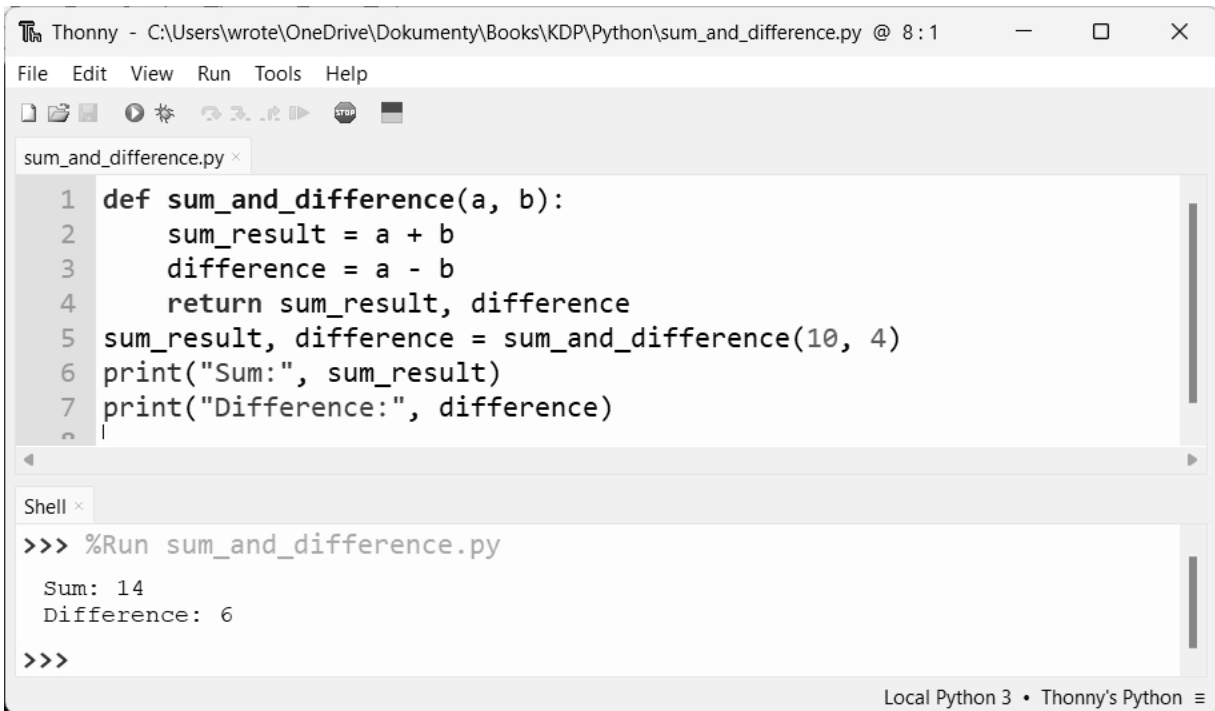
Here's an example of a function that returns both the sum and the difference of two numbers:

```
def sum_and_difference(a, b):  
    sum_result = a + b  
    difference = a - b  
    return sum_result, difference
```

When you call this function, you can capture both return values:

```
sum_result, difference = sum_and_difference(10, 4)  
print("Sum:", sum_result)  
print("Difference:", difference)
```

Output:



The screenshot shows the Thonny Python IDE interface. The top window displays the code for `sum_and_difference.py`:

```
1 def sum_and_difference(a, b):
2     sum_result = a + b
3     difference = a - b
4     return sum_result, difference
5 sum_result, difference = sum_and_difference(10, 4)
6 print("Sum:", sum_result)
7 print("Difference:", difference)
```

The bottom window, titled "Shell", shows the execution output:

```
>>> %Run sum_and_difference.py
Sum: 14
Difference: 6
>>>
```

The status bar at the bottom indicates "Local Python 3 • Thonny's Python".

This is useful when your function needs to return more than one piece of information.

7. Putting It All Together: A Simple Calculator

Let's build a simple calculator program that uses functions to perform basic arithmetic operations (addition, subtraction, multiplication, and division) and returns the result.

Step 1: Define the Functions

We'll define four separate functions, one for each operation:

```
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    if b != 0:
        return a / b
```

```
else:
```

```
    return "Error! Division by zero."
```

Each function takes two arguments, performs the appropriate calculation, and returns the result.

Step 2: Use the Functions

Now, we can use these functions to perform calculations based on user input:

```
num1 = float(input("Enter the first number: "))
num2 = float(input("Enter the second number: "))

operation = input("Enter the operation (+, -, *, /): ")

if operation == "+":
    print("Result:", add(num1, num2))
elif operation == "-":
    print("Result:", subtract(num1, num2))
elif operation == "*":
    print("Result:", multiply(num1, num2))
elif operation == "/":
    print("Result:", divide(num1, num2))
else:
    print("Invalid operation!")
```

This program takes two numbers and an operator from the user, then calls the appropriate function to perform the calculation.

Key Takeaways:

- **Arguments** allow you to pass data into functions, making them flexible and reusable.
- **Return values** let functions send results back to the main program, so the results can be used elsewhere.
- Functions can return a single value or multiple values, depending on your needs.

- Using arguments and return values keeps your code modular, organized, and easier to maintain.

By mastering function arguments and return values, you'll unlock the full potential of functions and be able to build more complex, powerful programs. Keep experimenting and practicing — soon, functions will be your go-to tool for solving problems with Python!

Real-World Application: A Calculator Program

Now that you've learned how to write functions, pass arguments, and return values, let's put all these concepts together in a practical project — building a **calculator program**! This calculator will allow users to perform basic arithmetic operations, like addition, subtraction, multiplication, and division, using Python functions. By the end of this section, you'll see how the power of functions can help you solve real-world problems in an organized and efficient way.

1. Understanding the Calculator's Structure

Before diving into the code, let's break down what we want our calculator to do:

- It will ask the user for two numbers.
- It will ask the user to choose an operation (add, subtract, multiply, or divide).
- It will use separate functions to handle each operation.
- It will display the result of the chosen operation.

By separating each task into its own function, the code will stay clean and easy to modify. If you ever want to add more operations later, you can simply add new functions without changing the core structure of the program.

2. Step-by-Step Guide to Building the Calculator

Step 1: Writing the Arithmetic Functions

First, we'll create four functions to handle the basic operations: addition, subtraction, multiplication, and division. Each function will take two arguments (the numbers) and return the result.

```
def add(a, b):
```

```

    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    if b != 0:
        return a / b
    else:
        return "Error! Division by zero."

```

Each function is simple and focused on just one task. For example, `add()` returns the sum of `a` and `b`, while `divide()` checks if the second number (`b`) is zero to avoid dividing by zero, which would cause an error.

Step 2: Asking the User for Input

Next, we'll write code to ask the user for two numbers and what operation they'd like to perform. The `input()` function will help us get this information from the user:

```

num1 = float(input("Enter the first number: "))
num2 = float(input("Enter the second number: "))
operation = input("Choose an operation (+, -, *, /): ")

```

Here, we're using `float()` to convert the user's input into numbers with decimals, which makes the calculator more versatile (so it can handle numbers like 2.5 or 7.3).

Step 3: Performing the Chosen Operation

Now that we've gathered the numbers and the chosen operation, we need to decide which function to call based on what the user entered. We'll use `if` statements to check which operation was selected:

```

if operation == "+":
    result = add(num1, num2)
elif operation == "-":
    result = subtract(num1, num2)
elif operation == "*":

```

```
    result = multiply(num1, num2)
elif operation == "/":
    result = divide(num1, num2)
else:
    result = "Invalid operation!"
```

Here's what's happening:

- If the user chose "+", the add() function is called with num1 and num2 as arguments.
- If they chose "-", the subtract() function is called, and so on.
- If the user enters an invalid operation (anything other than +, -, *, or /), the program will return a message saying "Invalid operation!"

Step 4: Displaying the Result

Finally, we'll display the result to the user using the print() function:

```
print("The result is:", result)
```

Now, no matter which operation was chosen, the calculator will display the correct result (or an error message if something went wrong).

3. The Complete Calculator Program

Let's put all the pieces together to create a working calculator program:

```
# Defining the arithmetic functions
```

```
def add(a, b):
```

```
    return a + b
```

```
def subtract(a, b):
```

```
    return a - b
```

```
def multiply(a, b):
```

```
    return a * b
```

```
def divide(a, b):
```

```
    if b != 0:
```

```
        return a / b
```

```

    else:
        return "Error! Division by zero."

# Asking the user for input
num1 = float(input("Enter the first number: "))
num2 = float(input("Enter the second number: "))
operation = input("Choose an operation (+, -, *, /): ")

# Performing the chosen operation
if operation == "+":
    result = add(num1, num2)
elif operation == "-":
    result = subtract(num1, num2)
elif operation == "*":
    result = multiply(num1, num2)
elif operation == "/":
    result = divide(num1, num2)
else:
    result = "Invalid operation!"

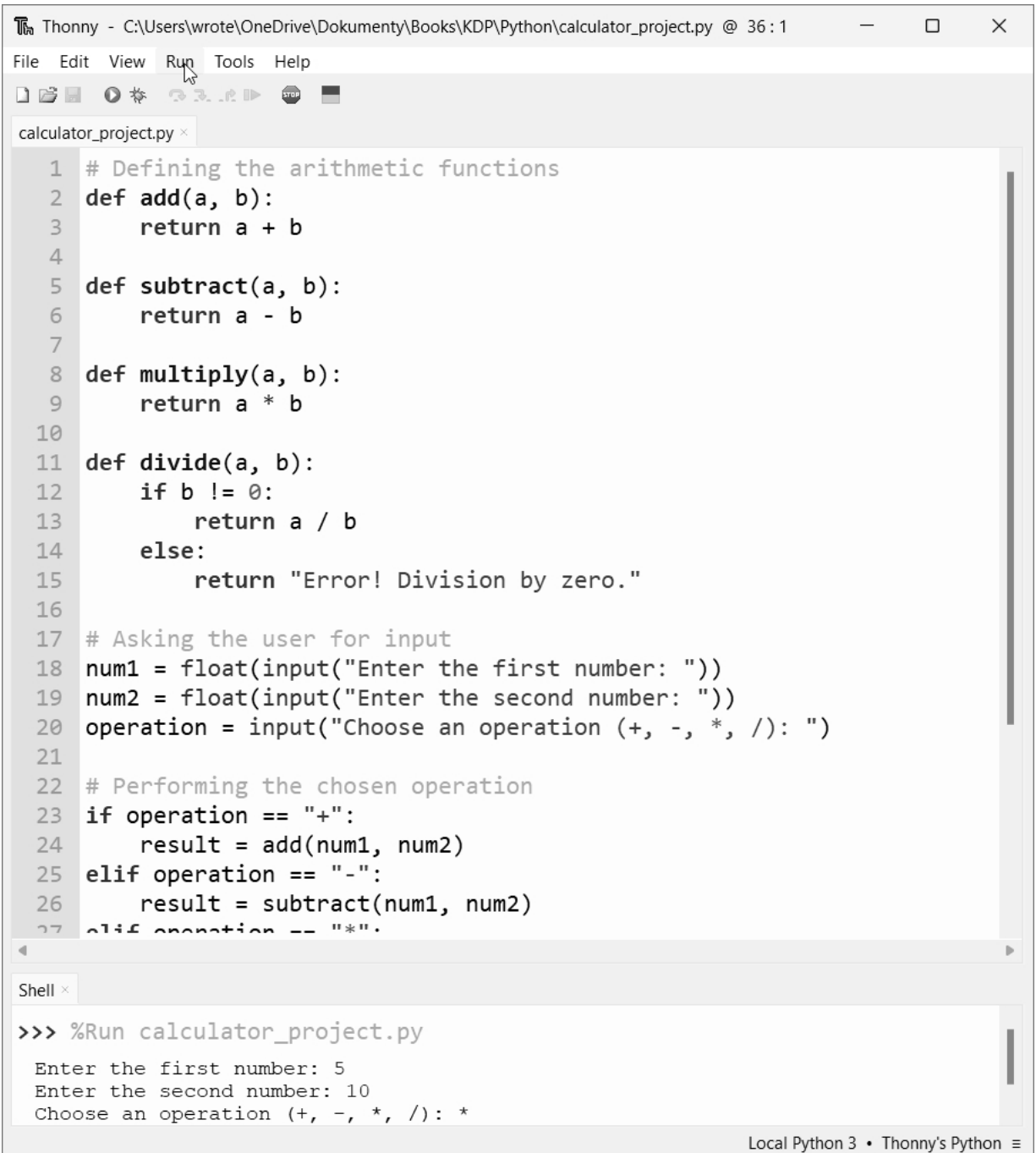
# Displaying the result
print("The result is:", result)

```

4. Running the Calculator

When you run this program, Python will ask you for two numbers and an operation, then perform the calculation and display the result.

Example:



```
Thonny - C:\Users\wrote\OneDrive\Dokumenty\Books\KDP\Python\calculator_project.py @ 36:1
File Edit View Run Tools Help
calculator_project.py x
1 # Defining the arithmetic functions
2 def add(a, b):
3     return a + b
4
5 def subtract(a, b):
6     return a - b
7
8 def multiply(a, b):
9     return a * b
10
11 def divide(a, b):
12     if b != 0:
13         return a / b
14     else:
15         return "Error! Division by zero."
16
17 # Asking the user for input
18 num1 = float(input("Enter the first number: "))
19 num2 = float(input("Enter the second number: "))
20 operation = input("Choose an operation (+, -, *, /): ")
21
22 # Performing the chosen operation
23 if operation == "+":
24     result = add(num1, num2)
25 elif operation == "-":
26     result = subtract(num1, num2)
27 elif operation == "*":
28     result = multiply(num1, num2)
29 elif operation == "/":
30     result = divide(num1, num2)
31 else:
32     result = "Error! Invalid operation."
33
34 print("The result is:", result)

Shell x
>>> %Run calculator_project.py
Enter the first number: 5
Enter the second number: 10
Choose an operation (+, -, *, /): *
The result is: 50.0

Local Python 3 • Thonny's Python
```

In this example, the user chose multiplication (*), and the calculator correctly multiplies 10 by 5 to get 50.0.

5. Making the Calculator More Advanced

Now that you have a basic calculator, here are a few ways to make it more advanced:

- **Adding More Operations:** You can add more mathematical functions, such as exponentiation or square root.

```
def exponent(a, b):  
    return a ** b
```

- **Handling More Errors:** For example, you could check if the user entered non-numeric values and display an error message.

```
try:  
    num1 = float(input("Enter the first number: "))  
except ValueError:  
    print("Please enter a valid number.")
```

- **Looping the Program:** You can use a loop to allow the user to perform multiple calculations without restarting the program.

```
while True:  
    # Your calculator code here  
    again = input("Do you want to perform another calculation? (yes/no): ")  
    if again.lower() != "yes":  
        break
```

6. Why Build a Calculator?

Building a calculator might seem simple, but it's an excellent way to practice using functions and handling user input. This project teaches you how to:

- **Break down a real-world problem** into smaller parts using functions.
- **Use arguments and return values** to make your code flexible and reusable.
- **Handle user input and errors** effectively, making your programs more interactive.

By building this calculator, you've applied key programming concepts to a real-world task. You can now take this idea and expand it with more features, turning a simple program into a more complex and useful tool. Keep experimenting with Python functions — you'll soon find they're one of the most valuable tools in your coding toolkit!

Part II: Problem-Solving with Python

Chapter 4: Solving Math Problems with Python

Basic Arithmetic and Algebra with Python

Python is a fantastic tool for solving math problems, whether you're doing simple calculations or tackling more complex algebra. In this section, you'll learn how Python handles **basic arithmetic** and **algebraic operations** with ease. By the end, you'll see how you can use Python to make math quicker and more efficient — no calculator needed!

1. Basic Arithmetic Operations

Python makes it easy to perform all the basic arithmetic operations you're already familiar with:

- **Addition (+)**
- **Subtraction (-)**
- **Multiplication (*)**
- **Division (/)**

Let's take a look at how you can use Python to do these calculations.

Example:

```
# Addition
sum_result = 5 + 3
print("5 + 3 =", sum_result)

# Subtraction
difference = 10 - 4
```

```
print("10 - 4 =", difference)
```

```
# Multiplication
```

```
product = 7 * 6
```

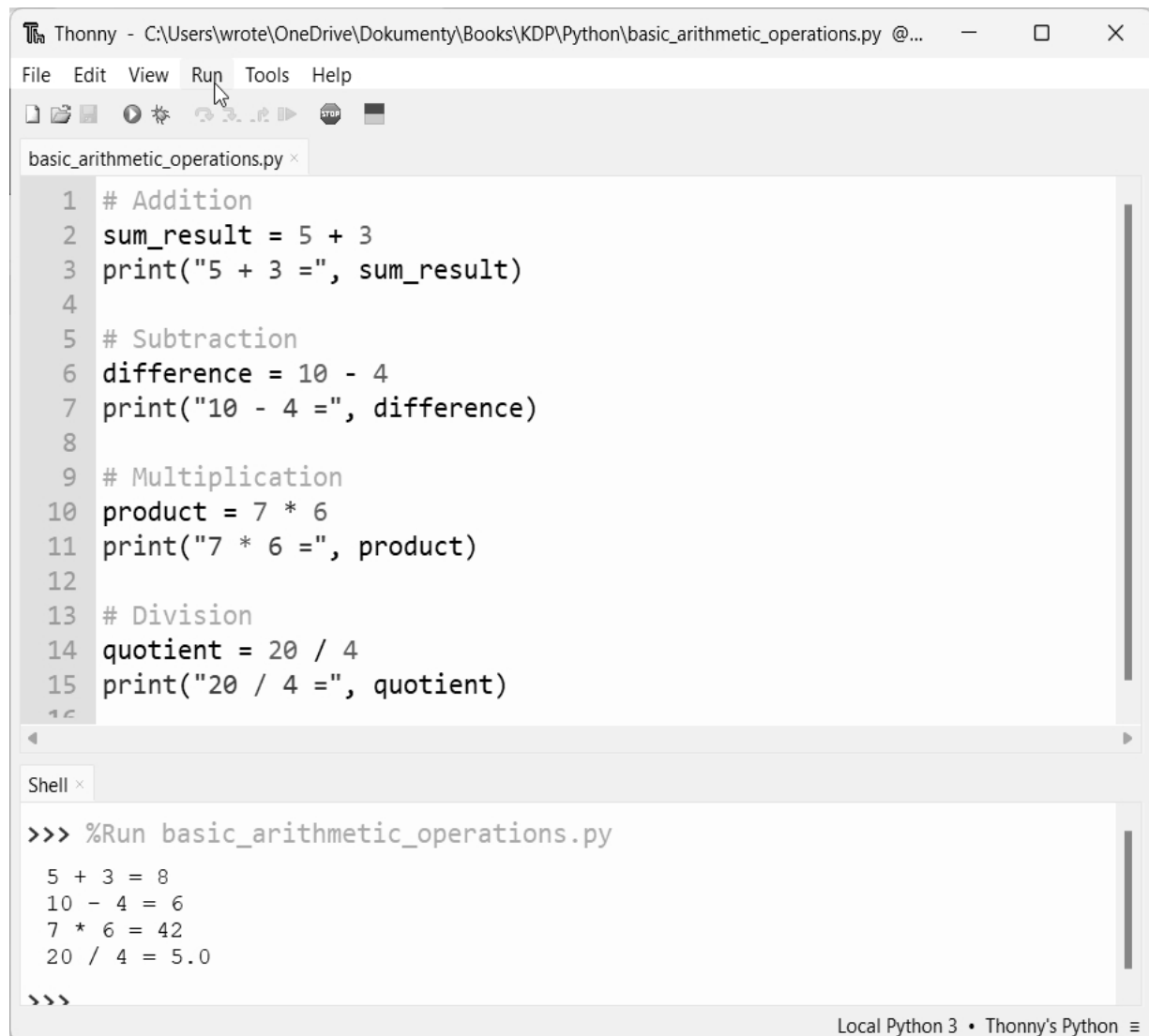
```
print("7 * 6 =", product)
```

```
# Division
```

```
quotient = 20 / 4
```

```
print("20 / 4 =", quotient)
```

Output:



The screenshot shows the Thonny Python IDE interface. The top window displays a Python script named `basic_arithmetic_operations.py` with the following code:

```
1 # Addition
2 sum_result = 5 + 3
3 print("5 + 3 =", sum_result)
4
5 # Subtraction
6 difference = 10 - 4
7 print("10 - 4 =", difference)
8
9 # Multiplication
10 product = 7 * 6
11 print("7 * 6 =", product)
12
13 # Division
14 quotient = 20 / 4
15 print("20 / 4 =", quotient)
```

The bottom window, titled "Shell", shows the output of running the script:

```
>>> %Run basic_arithmetic_operations.py
5 + 3 = 8
10 - 4 = 6
7 * 6 = 42
20 / 4 = 5.0
>>>
```

The status bar at the bottom indicates "Local Python 3 • Thonny's Python".

As you can see, Python does the math just like you would on paper. Notice that division gives a decimal result (5.0), even when the numbers divide evenly.

2. More Advanced Operations

Python can also handle more advanced mathematical operations, such as:

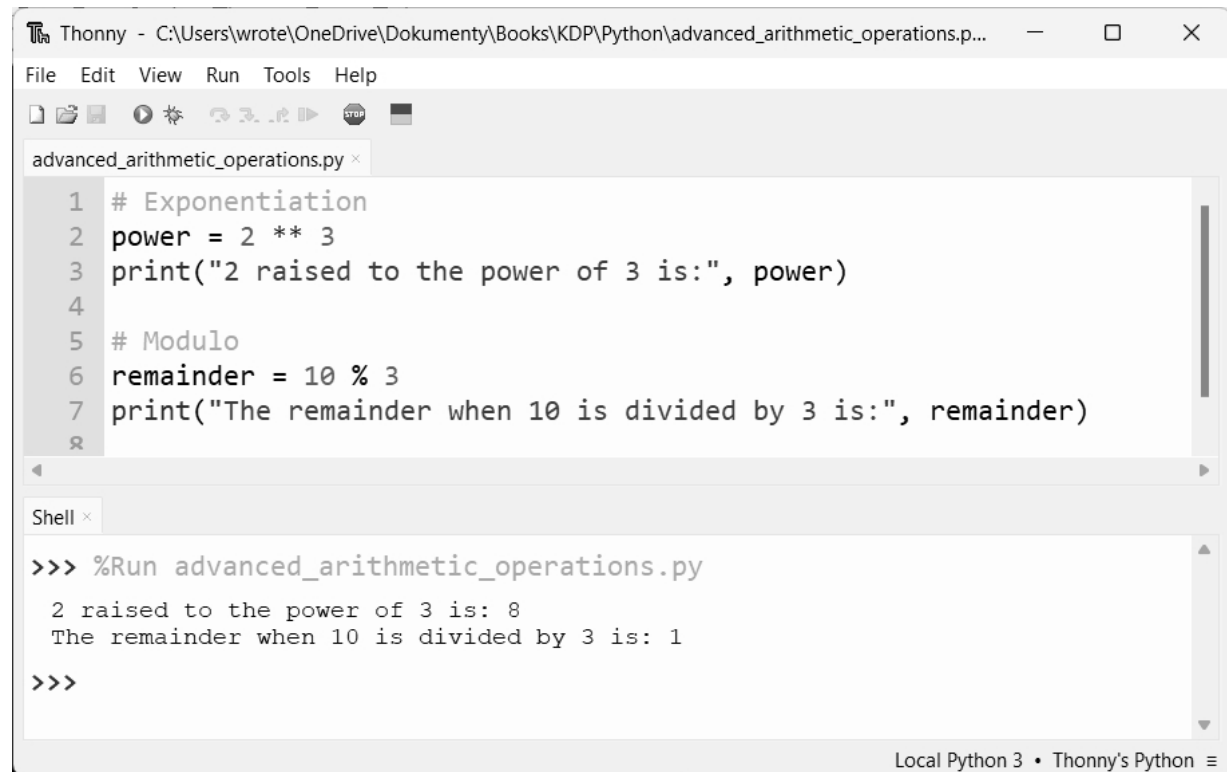
- **Exponentiation** (**) for raising a number to a power.
- **Modulo** (%) for finding the remainder when one number is divided by another.

Example:

```
# Exponentiation
power = 2 ** 3
print("2 raised to the power of 3 is:", power)

# Modulo
remainder = 10 % 3
print("The remainder when 10 is divided by 3 is:", remainder)
```

Output:



The screenshot shows the Thonny Python IDE interface. The top window displays a Python script named `advanced_arithmetic_operations.py` with the following code:

```
1 # Exponentiation
2 power = 2 ** 3
3 print("2 raised to the power of 3 is:", power)
4
5 # Modulo
6 remainder = 10 % 3
7 print("The remainder when 10 is divided by 3 is:", remainder)
```

Below the script editor is a "Shell" window showing the output of running the script:

```
>>> %Run advanced_arithmetic_operations.py
2 raised to the power of 3 is: 8
The remainder when 10 is divided by 3 is: 1
>>>
```

The status bar at the bottom indicates "Local Python 3 • Thonny's Python".

The `**` operator raises 2 to the power of 3, giving 8. The `%` operator tells you that when 10 is divided by 3, the remainder is 1. This is useful in many

math problems, especially in programming challenges.

3. Working with Variables

In math, you often work with unknowns or variables. In Python, you can assign values to variables and use them in calculations, just like in algebra. This makes it easy to set up equations and solve them programmatically.

Example:

```
x = 4
```

```
y = 7
```

```
# Add variables
```

```
sum_result = x + y
```

```
print("x + y =", sum_result)
```

```
# Solve an algebraic equation
```

```
z = (x * 2) + (y - 3)
```

```
print("z = 2x + (y - 3) =", z)
```

Output:



```
Thonny - C:\Users\wrote\OneDrive\Documents\Books\KDP\Python\variables.py @ 11:1
File Edit View Run Tools Help
variables.py x
1 x = 4
2 y = 7
3
4 # Add variables
5 sum_result = x + y
6 print("x + y =", sum_result)
7
8 # Solve an algebraic equation
9 z = (x * 2) + (y - 3)
10 print("z = 2x + (y - 3) =", z)
11
Shell x
>>> %Run variables.py
x + y = 11
z = 2x + (y - 3) = 12
>>>
Local Python 3 • Thonny's Python
```

Here, Python handles the variables x and y as if they were numbers, making it easy to plug them into algebraic equations. You can set up formulas, calculate unknowns, and even update variables later in your program.

4. Using Python for Algebra

Python can be used to solve more complex algebraic problems, too. You can solve equations, simplify expressions, and even factor numbers. While Python's basic operators can handle a lot, you can also use the **sympy** library, which is designed for symbolic math and algebraic calculations.

First, let's install and import sympy. It are two steps:

First - in Terminal write **pip3 install mpmath** and press **Enter**.

```
C:\Users\wrote>pip3 install mpmath
Collecting mpmath
  Downloading mpmath-1.3.0-py3-none-any.whl.metadata (8.6 kB)
  Downloading mpmath-1.3.0-py3-none-any.whl (536 kB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 536.2/536.2 kB 5.8 MB/s eta 0:00:00
Installing collected packages: mpmath
Successfully installed mpmath-1.3.0

C:\Users\wrote>
```


Second – in Terminal write **pip3 install sympy** and press **Enter**.

```
C:\Users\wrote>pip install sympy
Collecting sympy
  Downloading sympy-1.13.2-py3-none-any.whl.metadata (12 kB)
Requirement already satisfied: mpmath<1.4, >=1.1.0 in c:\users\wrote\appdata\local\programs\python\python312\lib\site-packages (from sympy) (1.3.0)
Downloading sympy-1.13.2-py3-none-any.whl (6.2 MB)
6.2/6.2 MB 4.9 MB/s eta 0:00:00
Installing collected packages: sympy
Successfully installed sympy-1.13.2
C:\Users\wrote>
```

Then use code

from sympy import symbols, solve

Define the variable x

x = symbols('x')

Solve the equation $x^2 - 5x + 6 = 0$

equation = x**2 - 5*x + 6

solutions = solve(equation)

print("Solutions to the equation are:", solutions)

In this example, sympy allows us to solve the quadratic equation $x^2 - 5x + 6 = 0$ and find that the solutions are $x = 2$ and $x = 3$.

You can use sympy to perform all kinds of algebraic operations, such as expanding, factoring, or solving equations with multiple variables.

5. Real-World Application: Solving Algebraic Problems

Let's say you have the following word problem:

"A rectangle has a length that is 5 units longer than its width. If the area of the rectangle is 36 square units, what are the length and width of the rectangle?"

We can use Python to set up and solve this algebraic problem. First, let's define the relationship between the length and width:

- Let the width of the rectangle be w .
- The length of the rectangle is $w + 5$.

- The area is length * width, so the equation is: $w * (w + 5) = 36$.

We can solve this equation using Python:

```
from sympy import symbols, solve

# Define the variable w for width
w = symbols('w')

# Set up the equation for area
equation = w * (w + 5) - 36

# Solve for w
width_solutions = solve(equation)
print("Possible widths are:", width_solutions)

# Calculate the corresponding lengths
for width in width_solutions:
    length = width + 5
    print(f"Width: {width}, Length: {length}")
```

Output:

Possible widths are: [-9, 4]

Width: -9, Length: -4

Width: 4, Length: 9

Since the width of a rectangle can't be negative, we know that the correct solution is width = 4 and length = 9. Python helps us quickly solve this algebraic problem and find the dimensions of the rectangle.

6. Conclusion

Python is a fantastic tool for solving both basic and advanced math problems. From simple arithmetic operations to complex algebraic equations, Python can handle it all. Using Python for math makes it easy to automate calculations, test multiple scenarios, and solve real-world problems efficiently.

With a little practice, you'll find that Python can make math easier and more enjoyable. Keep experimenting with different problems, and soon,

Python will become your go-to tool for solving all kinds of mathematical challenges!

Finding Factors, Prime Numbers, and Fibonacci Sequences

In this section, you'll learn how to solve some interesting mathematical problems using Python. We'll explore how to find the **factors** of a number, check if a number is **prime**, and generate the famous **Fibonacci sequence**. These concepts not only teach you more about Python's power in math, but also help you develop problem-solving skills. Let's dive into each topic, step by step.

1. Finding Factors of a Number

A **factor** of a number is any whole number that divides it evenly (without leaving a remainder). For example, the factors of 12 are 1, 2, 3, 4, 6, and 12. To find the factors of a number, we need to check which numbers divide it without leaving a remainder.

Here's a Python function to find and print all the factors of a given number:

```
def find_factors(number):
    factors = []
    for i in range(1, number + 1):
        if number % i == 0:
            factors.append(i)
    return factors

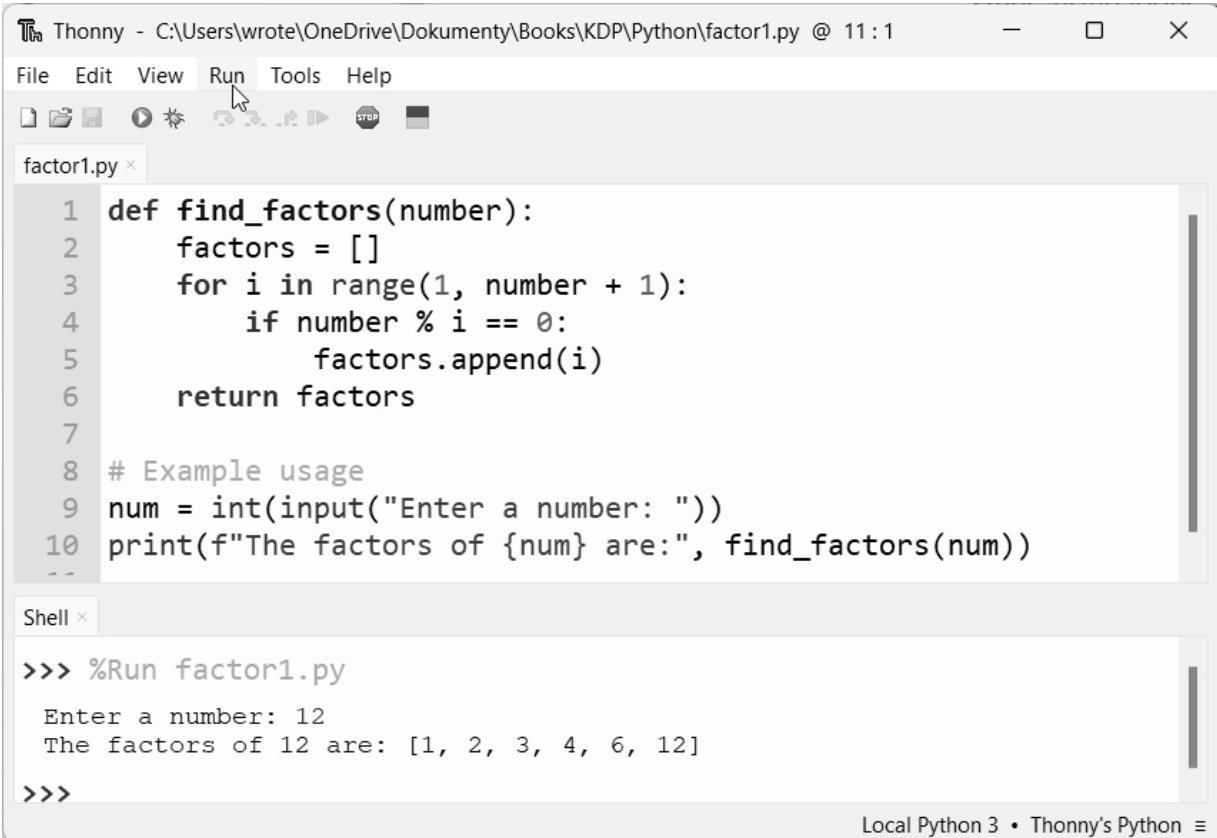
# Example usage
num = int(input("Enter a number: "))
print(f"The factors of {num} are:", find_factors(num))
```

Explanation:

- We use a loop to check all numbers from 1 to the given number.
- For each number, we check if it divides the given number without a remainder using the modulo operator (%).

- If the remainder is 0, it means the number is a factor, and we add it to the list of factors.

Example output:



The screenshot shows the Thonny Python IDE interface. The top window, titled 'factor1.py', contains the following Python code:

```
1 def find_factors(number):
2     factors = []
3     for i in range(1, number + 1):
4         if number % i == 0:
5             factors.append(i)
6     return factors
7
8 # Example usage
9 num = int(input("Enter a number: "))
10 print(f"The factors of {num} are:", find_factors(num))
```

The bottom window, titled 'Shell', shows the execution of the script:

```
>>> %Run factor1.py
Enter a number: 12
The factors of 12 are: [1, 2, 3, 4, 6, 12]
>>>
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

2. Checking for Prime Numbers

A **prime number** is a number greater than 1 that has no factors other than 1 and itself. For example, 7 is a prime number because its only factors are 1 and 7, but 8 is not prime because it has other factors (2 and 4).

To check if a number is prime, we can write a function that checks whether the number has any factors other than 1 and itself.

```
def is_prime(number):
    if number < 2:
        return False
    for i in range(2, int(number ** 0.5) + 1):
        if number % i == 0:
            return False
```

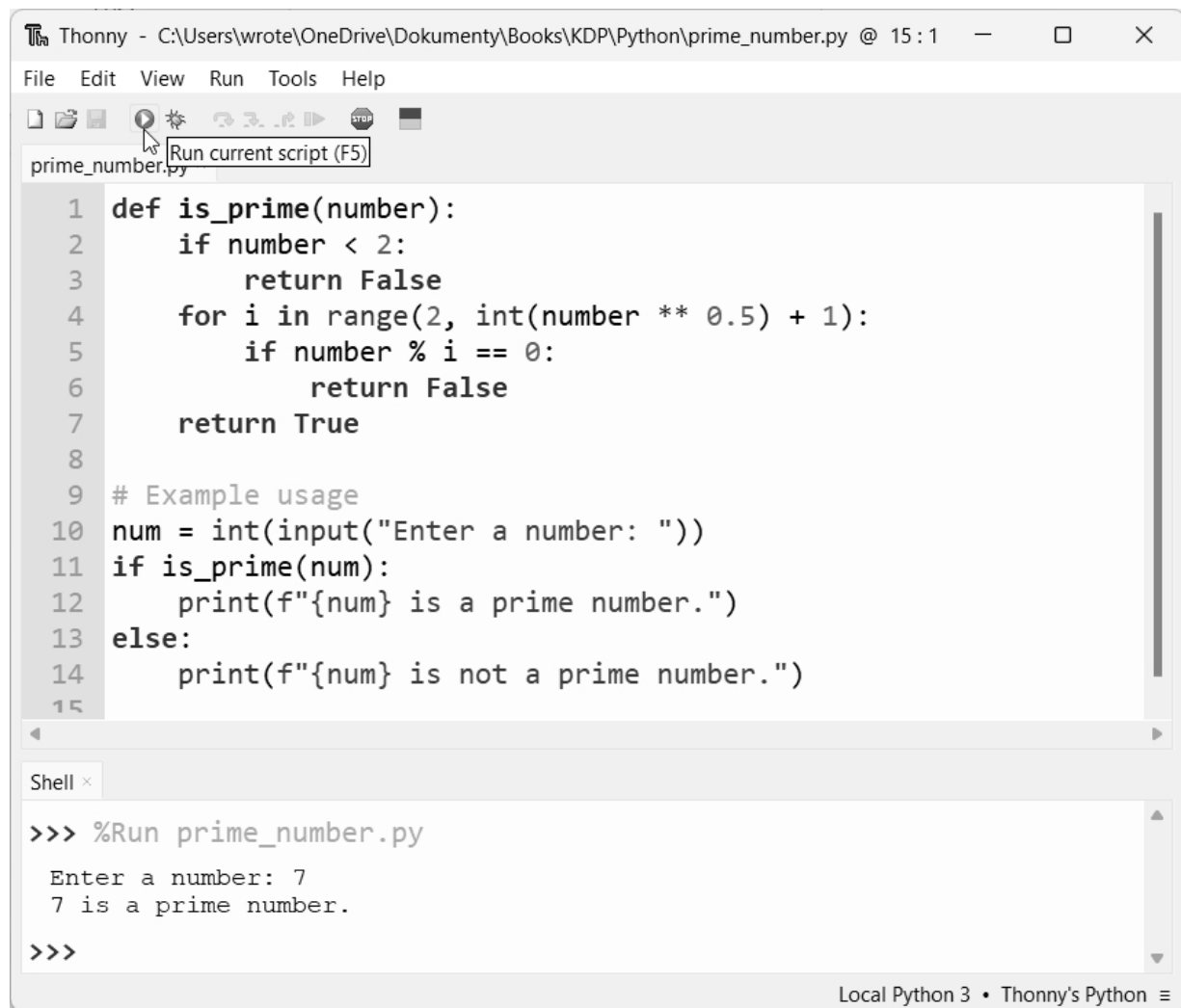
```
        return True

# Example usage
num = int(input("Enter a number: "))
if is_prime(num):
    print(f"{num} is a prime number.")
else:
    print(f"{num} is not a prime number.")
```

Explanation:

- We first check if the number is less than 2. If so, it's not prime.
- Then we check for divisibility starting from 2 up to the square root of the number (since any factor larger than the square root would have already been checked).
- If we find a factor, the number is not prime; otherwise, it is prime.

Example output:



The screenshot shows the Thonny Python IDE interface. The top window displays a Python script named `prime_number.py` with the following code:

```
1 def is_prime(number):
2     if number < 2:
3         return False
4     for i in range(2, int(number ** 0.5) + 1):
5         if number % i == 0:
6             return False
7     return True
8
9 # Example usage
10 num = int(input("Enter a number: "))
11 if is_prime(num):
12     print(f"{num} is a prime number.")
13 else:
14     print(f"{num} is not a prime number.")
15
```

The bottom window, titled "Shell", shows the execution of the script:

```
>>> %Run prime_number.py
Enter a number: 7
7 is a prime number.
>>>
```

The status bar at the bottom indicates "Local Python 3 • Thonny's Python".

3. Generating Fibonacci Sequences

The **Fibonacci sequence** is a series of numbers where each number is the sum of the two preceding ones. It starts with 0 and 1, so the sequence looks like this: 0, 1, 1, 2, 3, 5, 8, 13, This sequence appears in nature, art, and many mathematical problems.

Here's how you can generate the Fibonacci sequence using Python:

```
def fibonacci(n):
    fib_sequence = [0, 1]
    for i in range(2, n):
        next_number = fib_sequence[-1] + fib_sequence[-2]
        fib_sequence.append(next_number)
    return fib_sequence
```

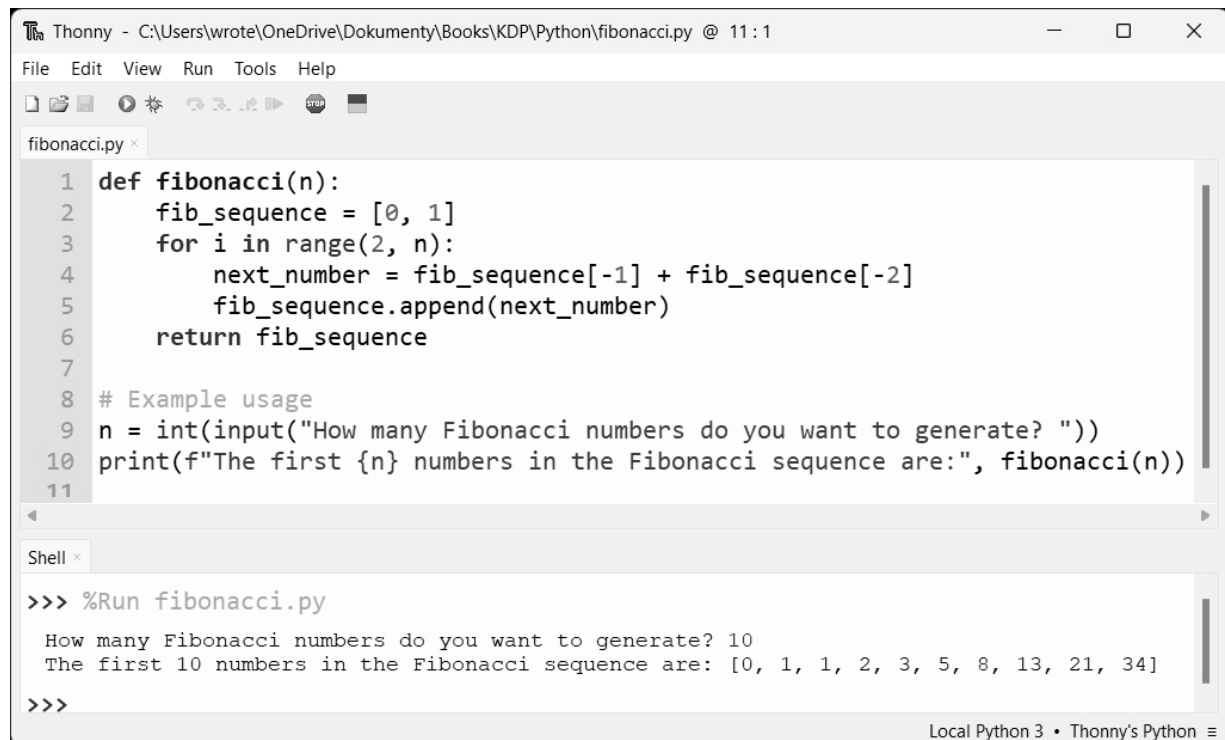
Example usage

```
n = int(input("How many Fibonacci numbers do you want to generate? "))
print(f"The first {n} numbers in the Fibonacci sequence are:", fibonacci(n))
```

Explanation:

- We start by initializing the list `fib_sequence` with the first two numbers of the Fibonacci sequence, 0 and 1.
- We then use a loop to generate the next numbers by adding the last two numbers in the sequence.
- The loop continues until we've generated `n` numbers.

Example output:



The screenshot shows the Thonny Python IDE interface. The top window, titled 'fibonacci.py', contains the following Python code:

```
1 def fibonacci(n):
2     fib_sequence = [0, 1]
3     for i in range(2, n):
4         next_number = fib_sequence[-1] + fib_sequence[-2]
5         fib_sequence.append(next_number)
6     return fib_sequence
7
8 # Example usage
9 n = int(input("How many Fibonacci numbers do you want to generate? "))
10 print(f"The first {n} numbers in the Fibonacci sequence are:", fibonacci(n))
11
```

The bottom window, titled 'Shell', shows the execution of the script:

```
>>> %Run fibonacci.py
How many Fibonacci numbers do you want to generate? 10
The first 10 numbers in the Fibonacci sequence are: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
>>>
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

4. Why Are These Concepts Important?

Finding factors, checking prime numbers, and generating Fibonacci sequences are not only classic mathematical problems, but they also help you practice breaking down problems into manageable steps, using loops, and applying logical conditions.

- **Finding factors** helps you understand how numbers are structured and how they relate to other numbers.
- **Prime numbers** play a huge role in cryptography and number theory, which are essential in fields like cybersecurity.
- **The Fibonacci sequence** is used in nature, computer algorithms, and even financial markets. It's a great way to practice recursion and iterative solutions.

These problems are a great way to sharpen your problem-solving skills in Python. Keep experimenting with these concepts, and try creating more variations of these programs to deepen your understanding!

Real-World Problem: Solving Geometry

Problems (Area, Perimeter)

Geometry problems are all around us — from finding the area of a classroom to calculating the perimeter of a garden. With Python, solving these problems becomes fast and simple. In this section, we'll learn how to use Python to calculate the **area** and **perimeter** of common shapes like rectangles, circles, and triangles. By the end, you'll see how Python can help with real-world geometry tasks, making problem-solving fun and practical!

1. Finding the Area and Perimeter of a Rectangle

The formula for finding the **area** of a rectangle is simple:

$$\text{Area} = \text{length} \times \text{width}$$

The **perimeter** of a rectangle is the sum of all its sides:

$$\text{Perimeter} = 2 \times (\text{length} + \text{width})$$

Let's write a Python program that calculates both the area and perimeter of a rectangle:

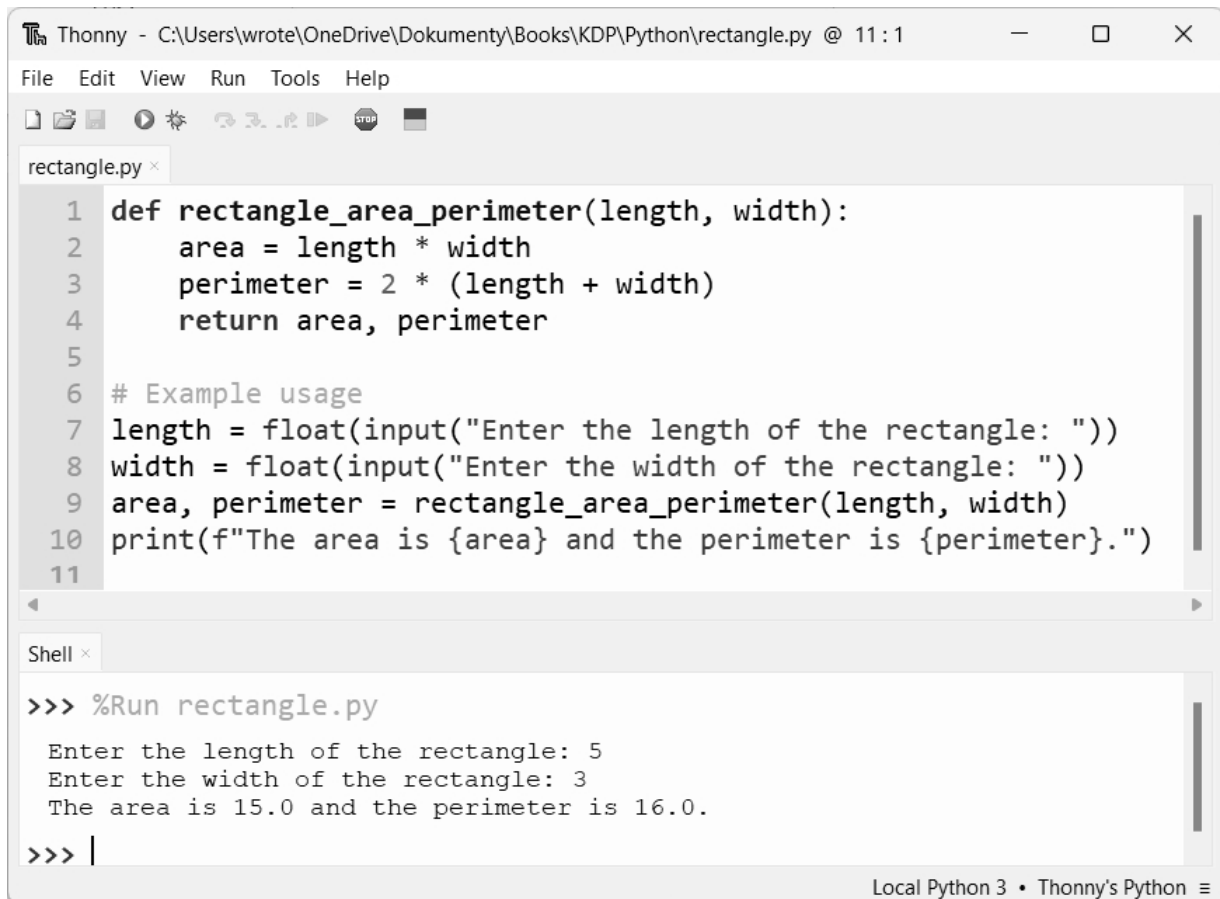
```
def rectangle_area_perimeter(length, width):  
    area = length * width  
    perimeter = 2 * (length + width)  
    return area, perimeter  
  
# Example usage  
length = float(input("Enter the length of the rectangle: "))  
width = float(input("Enter the width of the rectangle: "))  
area, perimeter = rectangle_area_perimeter(length, width)  
print(f"The area is {area} and the perimeter is {perimeter}.")
```

Explanation:

- We define a function `rectangle_area_perimeter()` that takes the length and width as arguments.

- The function calculates the area and perimeter and returns both values.
- We ask the user for the length and width, then use the function to calculate and display the results.

Example output:



```

Thonny - C:\Users\wrote\OneDrive\Dokumenty\Books\KDP\Python\rectangle.py @ 11:1
File Edit View Run Tools Help
rectangle.py x
1 def rectangle_area_perimeter(length, width):
2     area = length * width
3     perimeter = 2 * (length + width)
4     return area, perimeter
5
6 # Example usage
7 length = float(input("Enter the length of the rectangle: "))
8 width = float(input("Enter the width of the rectangle: "))
9 area, perimeter = rectangle_area_perimeter(length, width)
10 print(f"The area is {area} and the perimeter is {perimeter}.")
11

Shell x
>>> %Run rectangle.py
Enter the length of the rectangle: 5
Enter the width of the rectangle: 3
The area is 15.0 and the perimeter is 16.0.
>>> |
Local Python 3 • Thonny's Python

```

2. Finding the Area and Circumference of a Circle

For a **circle**, the formula for the **area** is:

$$\text{Area} = \pi \times \text{radius}^2$$

The **circumference** (which is like the perimeter for a circle) is:

$$\text{Circumference} = 2 \times \pi \times \text{radius}$$

Here's a Python program to calculate the area and circumference of a circle:

```
import math
```

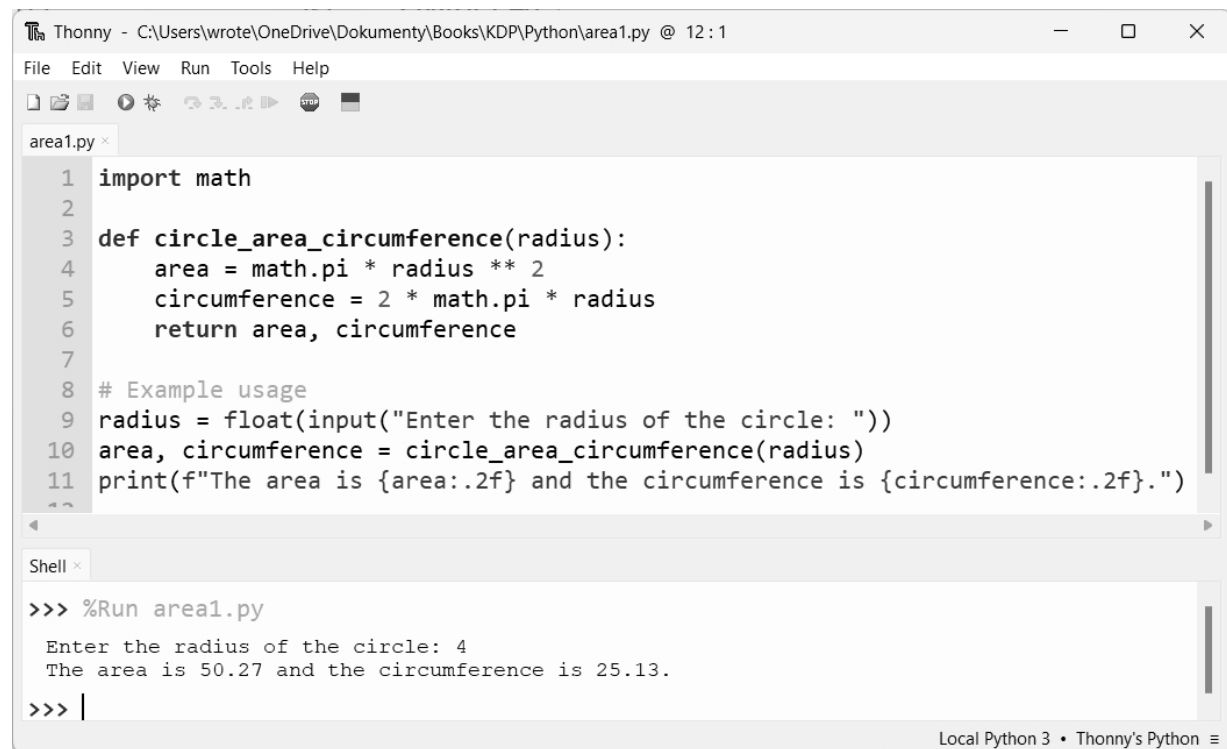
```
def circle_area_circumference(radius):
    area = math.pi * radius ** 2
    circumference = 2 * math.pi * radius
    return area, circumference

# Example usage
radius = float(input("Enter the radius of the circle: "))
area, circumference = circle_area_circumference(radius)
print(f"The area is {area:.2f} and the circumference is {circumference:.2f}.")
```

Explanation:

- We import the math library to access the value of π (pi).
- The function `circle_area_circumference()` calculates the area and circumference using the radius provided.
- The results are displayed with two decimal places using `:.2f`.

Example output:



The screenshot shows the Thonny Python IDE interface. The top window displays the Python code for calculating the area and circumference of a circle. The bottom window, labeled 'Shell', shows the execution output. The user has entered a radius of 4, and the program has calculated the area as 50.27 and the circumference as 25.13.

```
Thonny - C:\Users\wrote\OneDrive\Dokumenty\Books\KDP\Python\area1.py @ 12:1
File Edit View Run Tools Help
area1.py x
1 import math
2
3 def circle_area_circumference(radius):
4     area = math.pi * radius ** 2
5     circumference = 2 * math.pi * radius
6     return area, circumference
7
8 # Example usage
9 radius = float(input("Enter the radius of the circle: "))
10 area, circumference = circle_area_circumference(radius)
11 print(f"The area is {area:.2f} and the circumference is {circumference:.2f}.")
12

Shell x
>>> %Run area1.py
Enter the radius of the circle: 4
The area is 50.27 and the circumference is 25.13.
>>> |
Local Python 3 • Thonny's Python
```

3. Finding the Area and Perimeter of a Triangle

For a **triangle**, if you know the base and height, you can find the **area** using the formula:

$$\text{Area} = 0.5 \times \text{base} \times \text{height}$$

For the **perimeter**, you need to know the lengths of all three sides:

$$\text{Perimeter} = \text{side1} + \text{side2} + \text{side3}$$

Here's how to calculate both for a triangle in Python:

```
def triangle_area_perimeter(base, height, side1, side2, side3):
    area = 0.5 * base * height
    perimeter = side1 + side2 + side3
    return area, perimeter

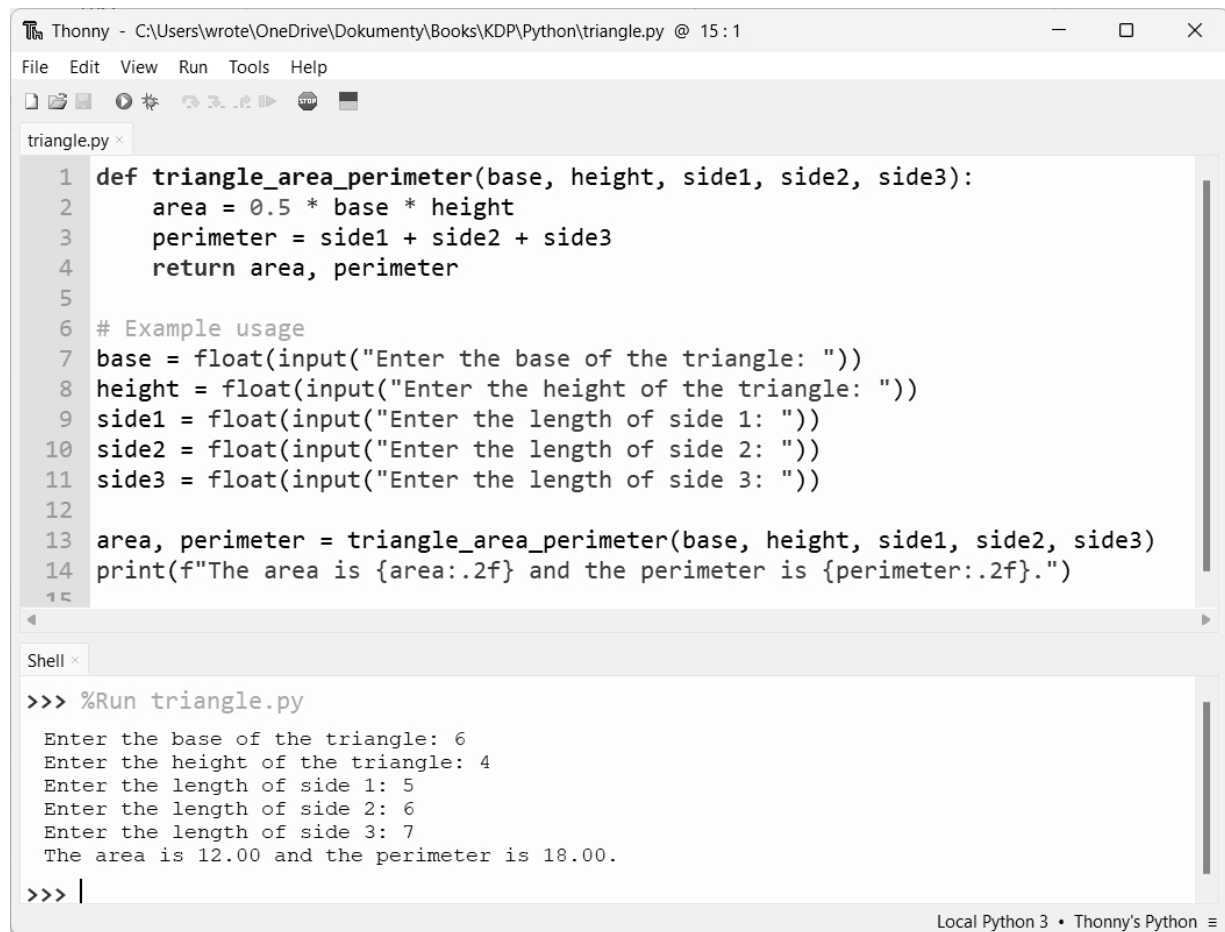
# Example usage
base = float(input("Enter the base of the triangle: "))
height = float(input("Enter the height of the triangle: "))
side1 = float(input("Enter the length of side 1: "))
side2 = float(input("Enter the length of side 2: "))
side3 = float(input("Enter the length of side 3: "))

area, perimeter = triangle_area_perimeter(base, height, side1, side2, side3)
print(f"The area is {area:.2f} and the perimeter is {perimeter:.2f}.")
```

Explanation:

- The function `triangle_area_perimeter()` calculates the area using the base and height and the perimeter using all three side lengths.
- The program asks for user input for the base, height, and sides, then calculates and prints the results.

Example output:



```
Thonny - C:\Users\wrote\OneDrive\Dokumenty\Books\KDP\Python\triangle.py @ 15:1
File Edit View Run Tools Help
triangle.py x
1 def triangle_area_perimeter(base, height, side1, side2, side3):
2     area = 0.5 * base * height
3     perimeter = side1 + side2 + side3
4     return area, perimeter
5
6 # Example usage
7 base = float(input("Enter the base of the triangle: "))
8 height = float(input("Enter the height of the triangle: "))
9 side1 = float(input("Enter the length of side 1: "))
10 side2 = float(input("Enter the length of side 2: "))
11 side3 = float(input("Enter the length of side 3: "))
12
13 area, perimeter = triangle_area_perimeter(base, height, side1, side2, side3)
14 print(f"The area is {area:.2f} and the perimeter is {perimeter:.2f}.")
15

Shell x
>>> %Run triangle.py
Enter the base of the triangle: 6
Enter the height of the triangle: 4
Enter the length of side 1: 5
Enter the length of side 2: 6
Enter the length of side 3: 7
The area is 12.00 and the perimeter is 18.00.
>>> |

Local Python 3 • Thonny's Python
```

4. Real-World Applications of Geometry

Knowing how to calculate the area and perimeter of shapes is useful in many real-life scenarios. Here are a few examples:

- **Building a garden:** You can calculate how much material is needed for a fence (perimeter) or how much grass seed to buy (area).
- **Painting a room:** Knowing the area of walls can help you figure out how much paint you need.
- **Designing a park:** Calculate the area for different sections like playgrounds, walking paths, or lakes.

With Python, these tasks become easy, and you can quickly adjust the calculations if the dimensions change.

Summary

Solving geometry problems with Python makes it easier to handle real-world tasks that involve shapes, sizes, and space. Whether you're calculating the area of a garden, the perimeter of a room, or the circumference of a round swimming pool, Python can help you find the answer quickly and accurately. Keep practicing these problems, and soon you'll be able to solve more complex geometry challenges with confidence!

Chapter 5: Working with Text: Strings and Patterns

Manipulating Strings to Solve Text- Based Challenges

In programming, working with text is just as important as working with numbers. Text data — like names, sentences, or even entire documents — is stored as **strings** in Python. Being able to manipulate and work with strings allows you to solve a wide range of real-world problems, from organizing data to building chatbots or games. In this section, we'll explore how to handle and manipulate strings to solve text-based challenges in Python.

1. What Are Strings in Python?

A **string** is a sequence of characters enclosed in quotes. You can use either single quotes (') or double quotes ("), and Python will treat them the same way.

Example:

```
name = "Alice"
```

```
greeting = 'Hello, how are you?'
```

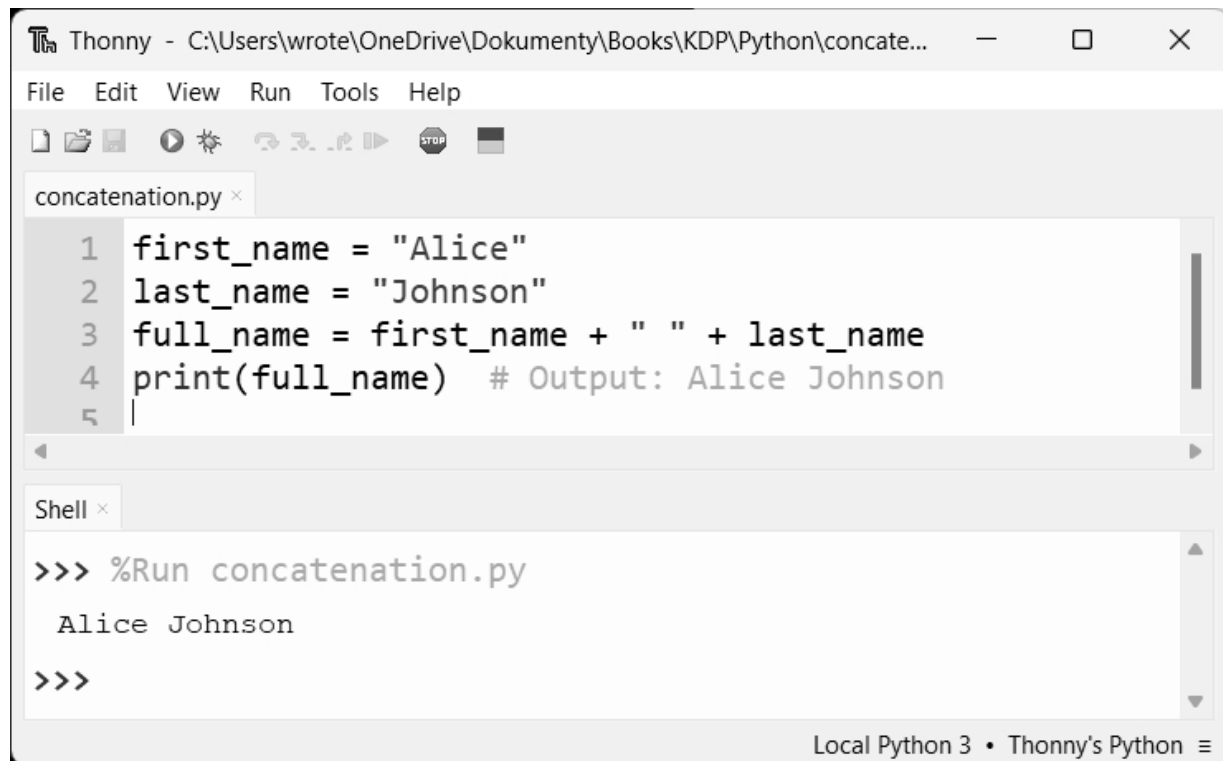
Here, "Alice" and 'Hello, how are you?' are both strings. Strings can contain letters, numbers, symbols, or even spaces. The power of strings lies in the fact that Python provides many tools to manipulate them.

2. Common String Operations

Let's start with some basic operations you can perform on strings. These operations help you manipulate and work with text in various ways.

- **Concatenation:** This is just a fancy word for **combining** two or more strings.


```
first_name = "Alice"
last_name = "Johnson"
full_name = first_name + " " + last_name
print(full_name) # Output: Alice Johnson
```



The screenshot shows the Thonny Python IDE interface. The top window displays a file named 'concatenation.py' with the following code:

```
1 first_name = "Alice"
2 last_name = "Johnson"
3 full_name = first_name + " " + last_name
4 print(full_name) # Output: Alice Johnson
5
```

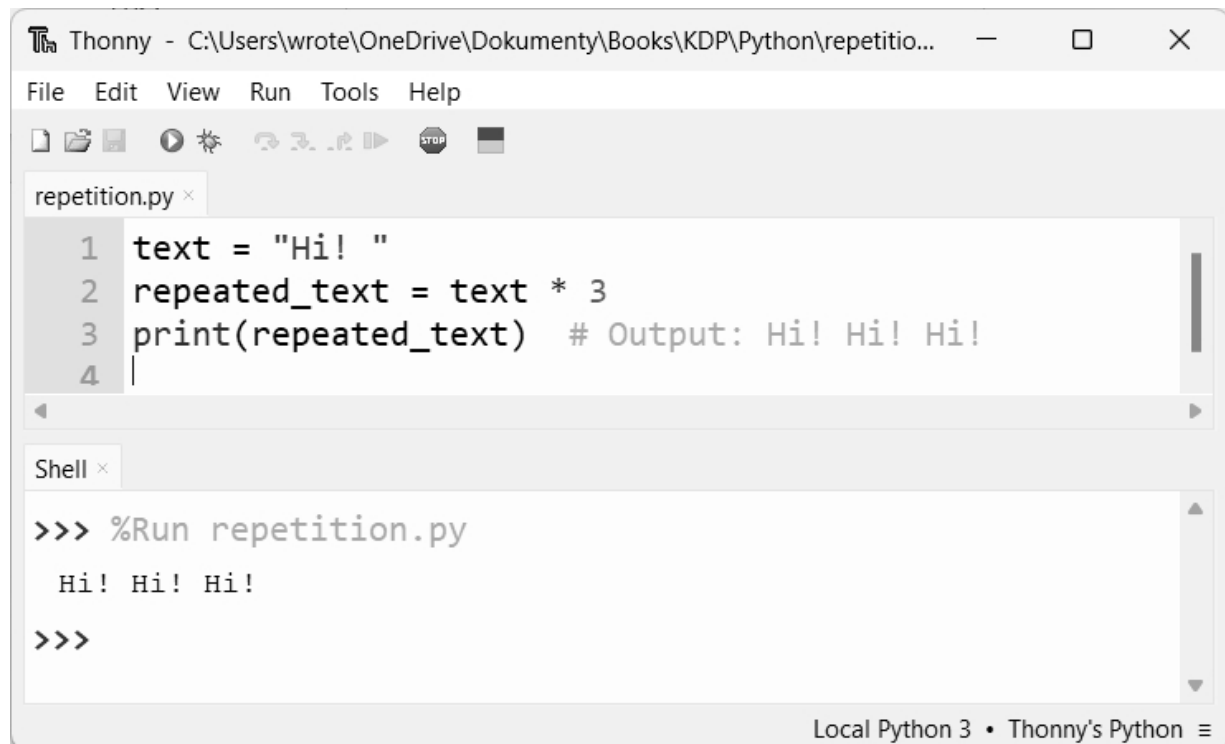
Below the code editor is a 'Shell' window showing the execution of the script:

```
>>> %Run concatenation.py
Alice Johnson
>>>
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

- **Repetition:** You can repeat a string multiple times using the * operator.

```
text = "Hi! "
repeated_text = text * 3
print(repeated_text) # Output: Hi! Hi! Hi!
```



The screenshot shows the Thonny Python IDE interface. The top window, titled 'repetition.py', contains the following Python code:

```
1 text = "Hi! "  
2 repeated_text = text * 3  
3 print(repeated_text) # Output: Hi! Hi! Hi!  
4 |
```

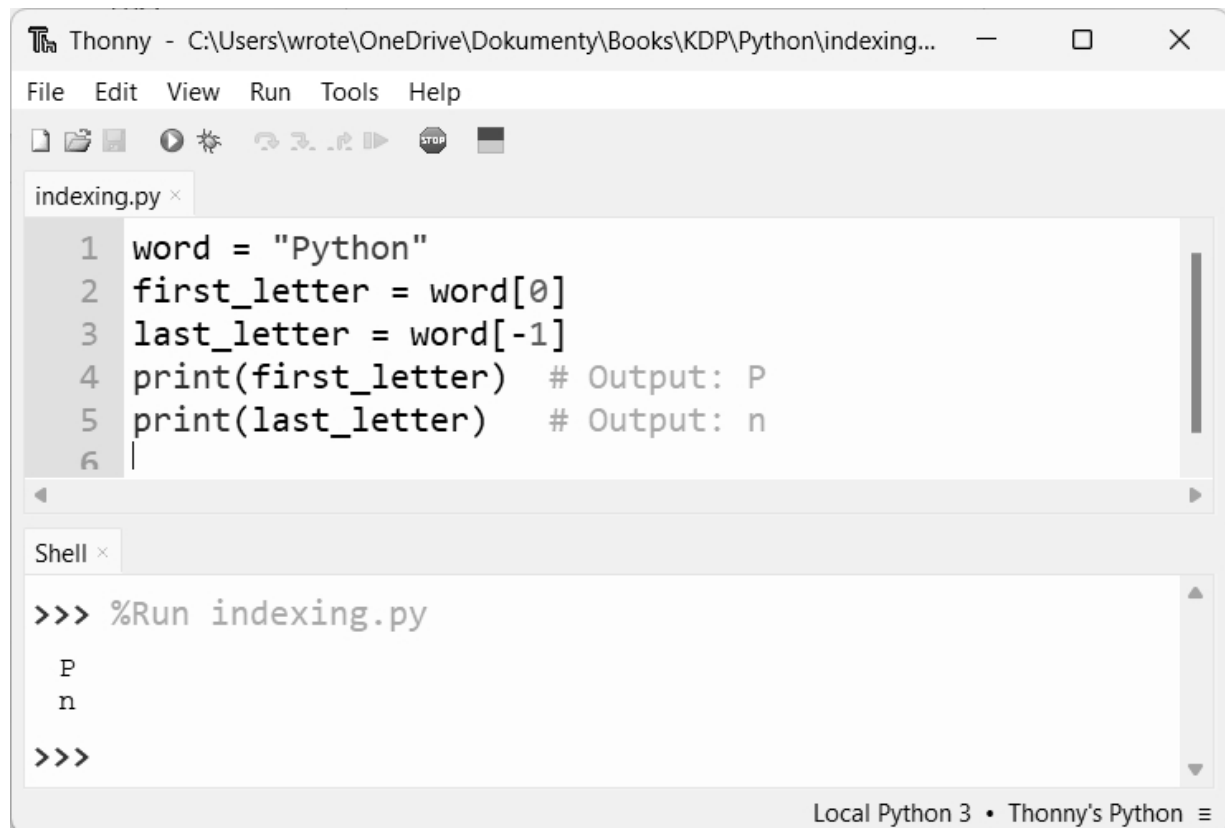
The bottom window, titled 'Shell', shows the execution of the script:

```
>>> %Run repetition.py  
Hi! Hi! Hi!  
>>>
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

- **Indexing:** You can access individual characters in a string using their position (index). In Python, indexing starts at 0.

```
word = "Python"  
first_letter = word[0]  
last_letter = word[-1]  
print(first_letter) # Output: P  
print(last_letter) # Output: n
```



The screenshot shows the Thonny Python IDE interface. The top window, titled 'indexing.py', contains the following Python code:

```
1 word = "Python"
2 first_letter = word[0]
3 last_letter = word[-1]
4 print(first_letter) # Output: P
5 print(last_letter) # Output: n
6
```

Below the code editor is a 'Shell' window showing the execution of the script:

```
>>> %Run indexing.py
P
n
>>>
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

- **Slicing:** You can extract parts of a string by slicing it.

```
word = "Programming"
slice_word = word[0:5]
print(slice_word) # Output: Progr
```

The screenshot shows the Thonny Python IDE interface. The title bar reads "Thonny - C:\Users\wrote\OneDrive\Dokumenty\Books\KDP\Python\slicing.p...". The menu bar includes "File", "Edit", "View", "Run", "Tools", and "Help". Below the menu is a toolbar with icons for file operations and execution. The editor window, titled "slicing.py", contains the following Python code:

```
1 word = "Programming"
2 slice_word = word[0:5]
3 print(slice_word) # Output: Progr
```

The Shell window at the bottom shows the execution process:

```
>>> %Run slicing.py
Progr
>>>
```

The status bar at the bottom right indicates "Local Python 3 • Thonny's Python".

In this example, `word[0:5]` extracts the first five characters of the string "Programming".

3. Transforming Strings

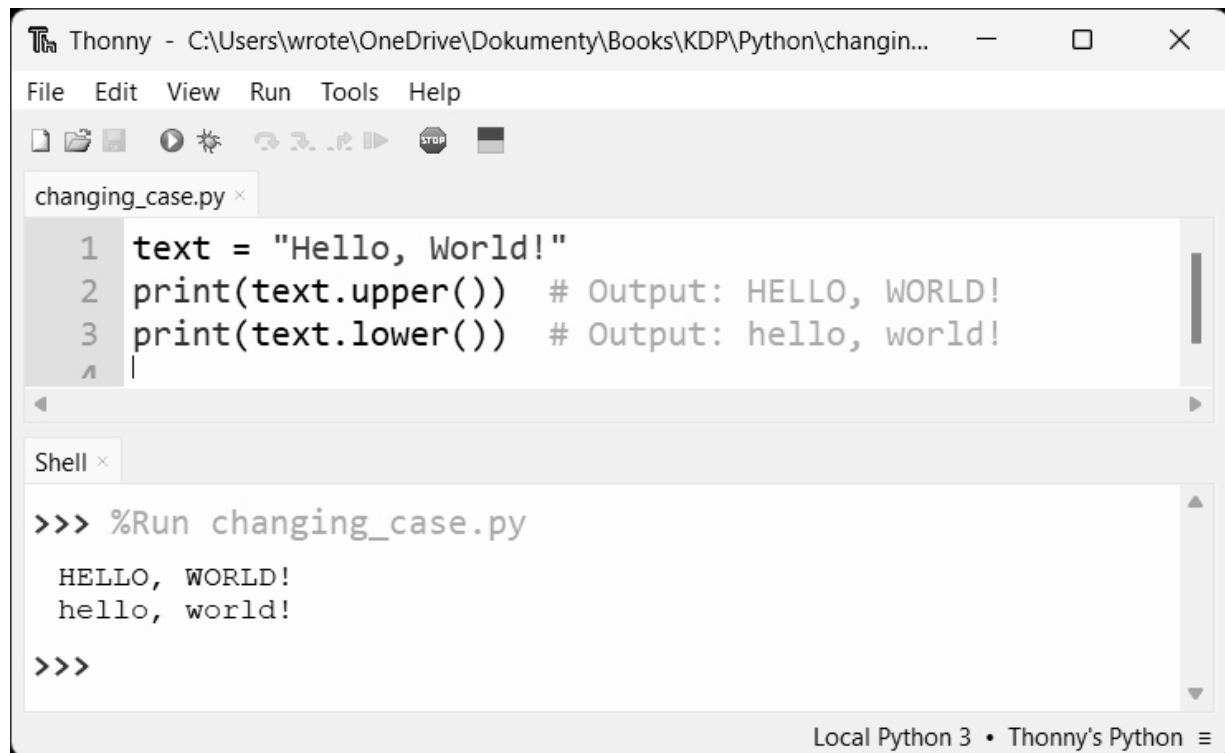
You can also **modify** or **transform** strings in various ways. Python provides several built-in string methods to help you do this:

- **Changing case:** Convert strings to uppercase or lowercase.

```
text = "Hello, World!"
```

```
print(text.upper()) # Output: HELLO, WORLD!
```

```
print(text.lower()) # Output: hello, world!
```



The screenshot shows the Thonny Python IDE interface. The top menu bar includes File, Edit, View, Run, Tools, and Help. Below the menu is a toolbar with icons for file operations and running code. The main editor window displays a file named 'changing_case.py' with the following code:

```
1 text = "Hello, World!"
2 print(text.upper()) # Output: HELLO, WORLD!
3 print(text.lower()) # Output: hello, world!
```

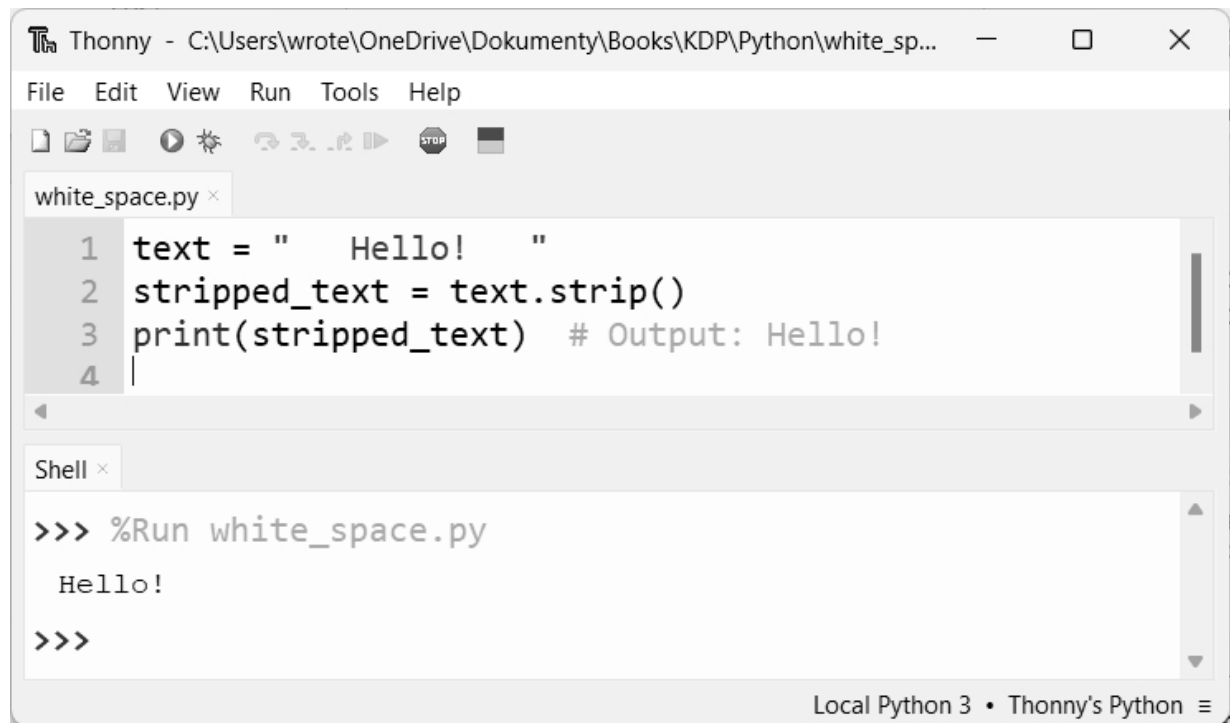
Below the editor is a 'Shell' window showing the execution output:

```
>>> %Run changing_case.py
HELLO, WORLD!
hello, world!
>>>
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

- **Stripping whitespace:** Remove any extra spaces from the beginning or end of a string.

```
text = " Hello! "  
stripped_text = text.strip()  
print(stripped_text) # Output: Hello!
```



The screenshot shows the Thonny Python IDE interface. The top window, titled 'white_space.py', contains the following Python code:

```
1 text = "  Hello!  "
2 stripped_text = text.strip()
3 print(stripped_text) # Output: Hello!
4 |
```

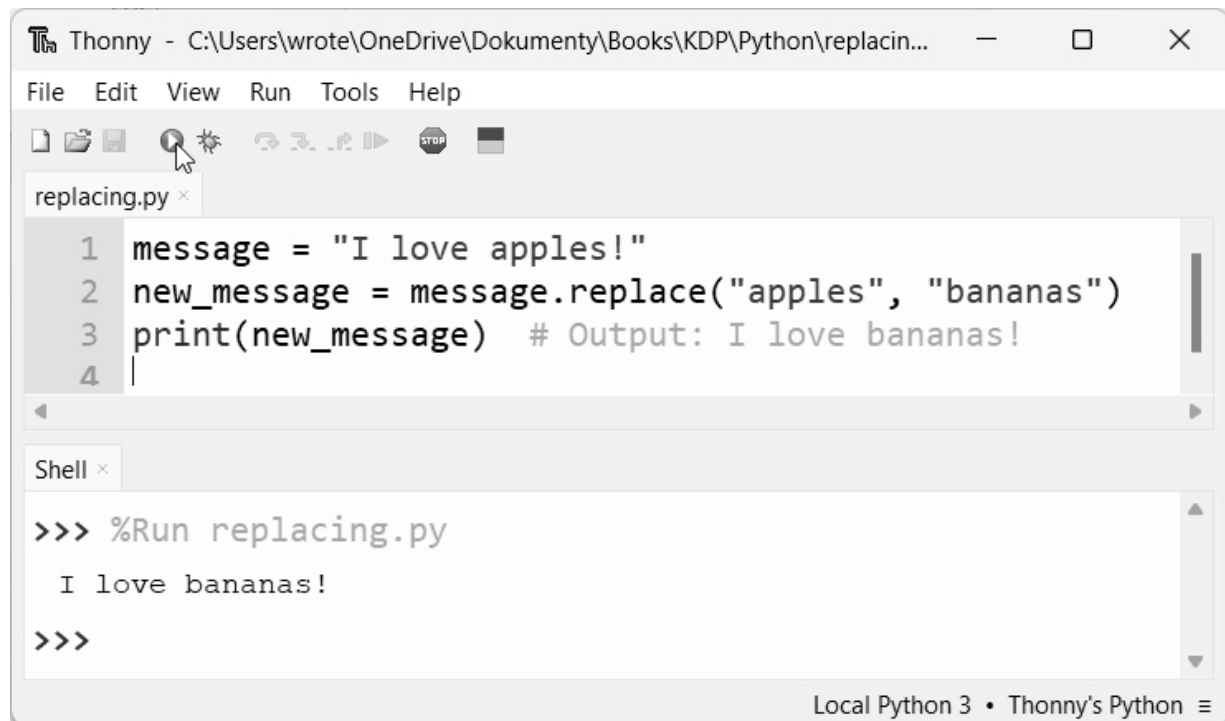
Below the code editor is a 'Shell' window showing the execution of the script:

```
>>> %Run white_space.py
Hello!
>>>
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

- **Replacing parts of a string:** Replace specific characters or words with something else.

```
message = "I love apples!"
new_message = message.replace("apples", "bananas")
print(new_message) # Output: I love bananas!
```



```
Thonny - C:\Users\wrote\OneDrive\Dokumenty\Books\KDP\Python\replacin...
File Edit View Run Tools Help
replacing.py x
1 message = "I love apples!"
2 new_message = message.replace("apples", "bananas")
3 print(new_message) # Output: I love bananas!
4 |
Shell x
>>> %Run replacing.py
I love bananas!
>>>
Local Python 3 • Thonny's Python
```

4. Solving Text-Based Challenges

Now that you understand basic string operations, let's apply them to solve some text-based problems.

Problem 1: Reversing a String

Let's say you need to reverse a word. Python makes this easy with slicing:

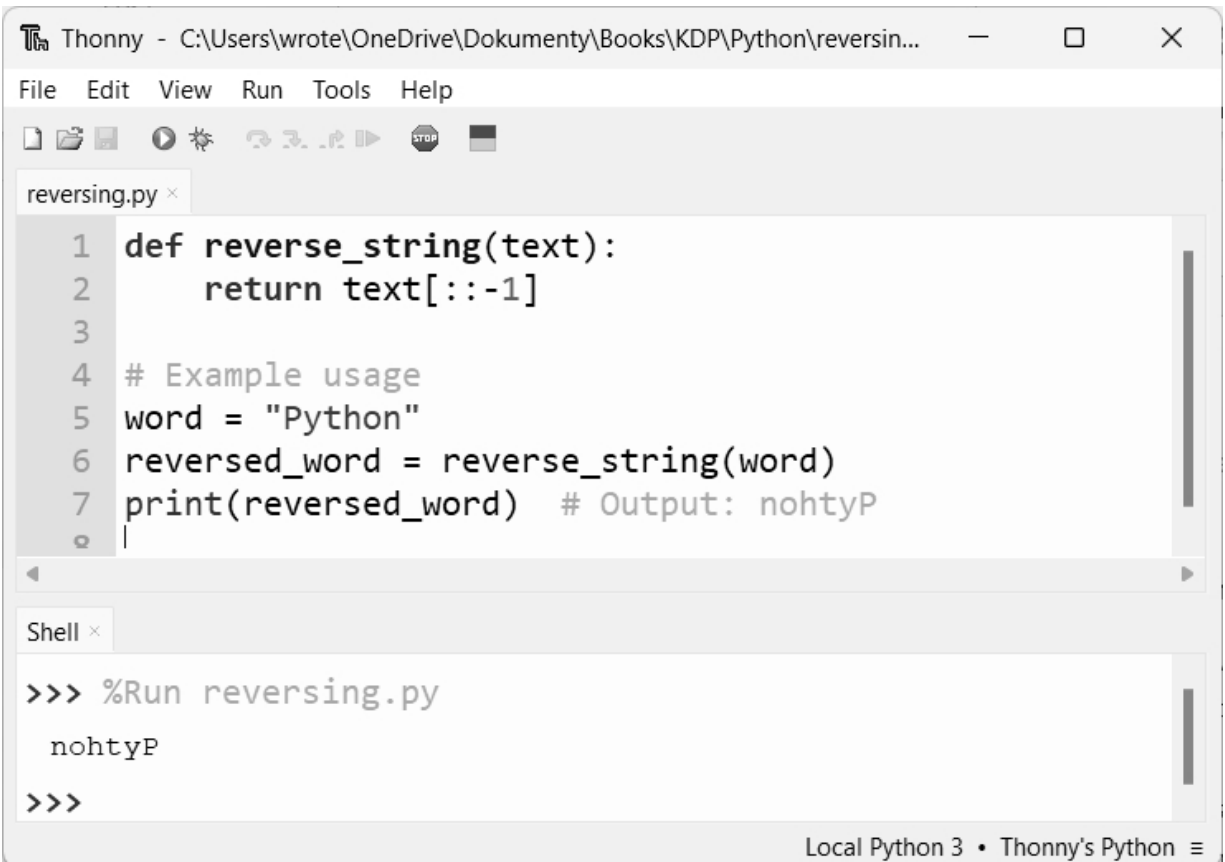
```
def reverse_string(text):
    return text[::-1]
```

Example usage

```
word = "Python"
```

```
reversed_word = reverse_string(word)
```

```
print(reversed_word) # Output: nohtyP
```



The screenshot shows the Thonny Python IDE interface. The top window, titled 'reversing.py', contains the following Python code:

```
1 def reverse_string(text):
2     return text[::-1]
3
4 # Example usage
5 word = "Python"
6 reversed_word = reverse_string(word)
7 print(reversed_word) # Output: nohtyP
```

Below the code editor is a 'Shell' window showing the execution of the script:

```
>>> %Run reversing.py
nohtyP
>>>
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

Here, `text[::-1]` is a slice that tells Python to take the string and go from the end to the beginning, effectively reversing it.

Problem 2: Counting Vowels in a String

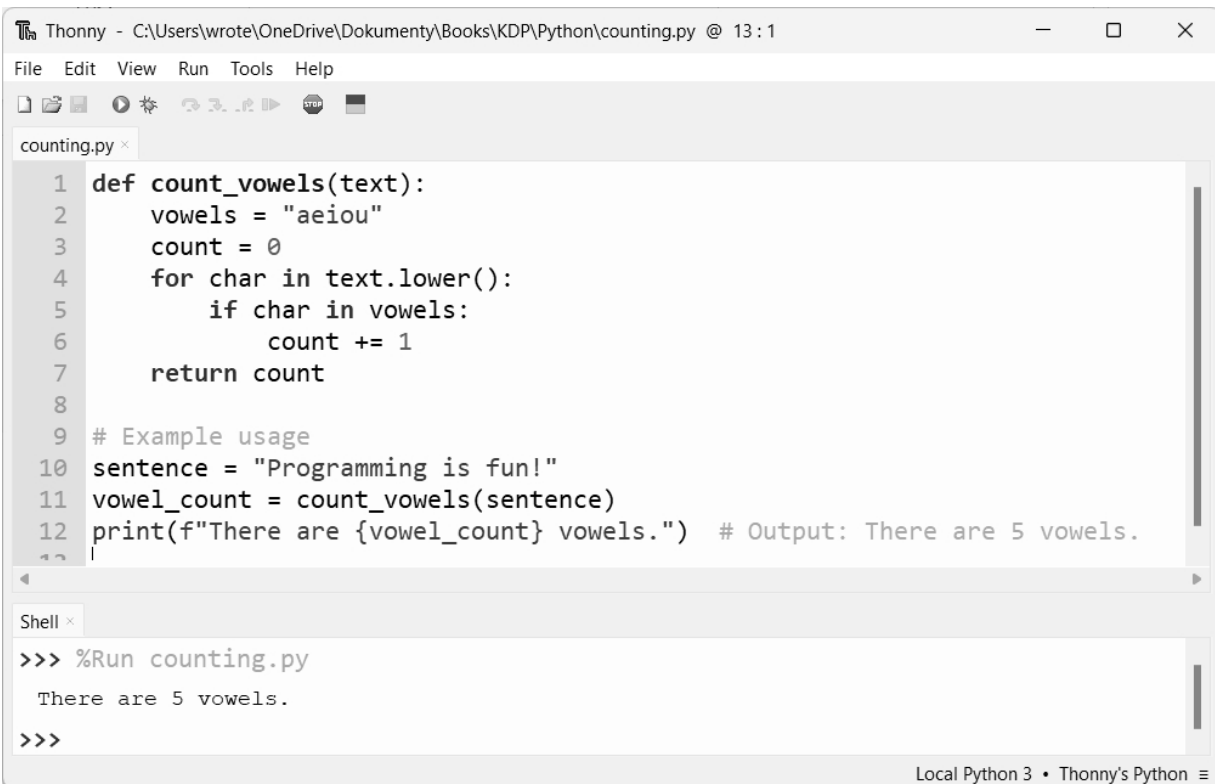
You can solve this problem by checking each character in the string to see if it's a vowel (a, e, i, o, u):

```
def count_vowels(text):
    vowels = "aeiou"
    count = 0
    for char in text.lower():
        if char in vowels:
            count += 1
    return count

# Example usage
sentence = "Programming is fun!"
vowel_count = count_vowels(sentence)
```



```
print(f"There are {vowel_count} vowels.") # Output: There are 5 vowels.
```



The screenshot shows the Thonny Python IDE interface. The main editor window displays a Python script named 'counting.py'. The script defines a function 'count_vowels(text)' that iterates through each character in the input text (converted to lowercase) and counts the vowels ('a', 'e', 'i', 'o', 'u'). An example usage is provided, where the sentence 'Programming is fun!' is processed, resulting in a vowel count of 5. The output is printed as 'There are 5 vowels.' Below the editor, the Shell window shows the command '%Run counting.py' being executed, which produces the same output: 'There are 5 vowels.'

```
1 def count_vowels(text):
2     vowels = "aeiou"
3     count = 0
4     for char in text.lower():
5         if char in vowels:
6             count += 1
7     return count
8
9 # Example usage
10 sentence = "Programming is fun!"
11 vowel_count = count_vowels(sentence)
12 print(f"There are {vowel_count} vowels.") # Output: There are 5 vowels.
```

```
>>> %Run counting.py
There are 5 vowels.
>>>
```

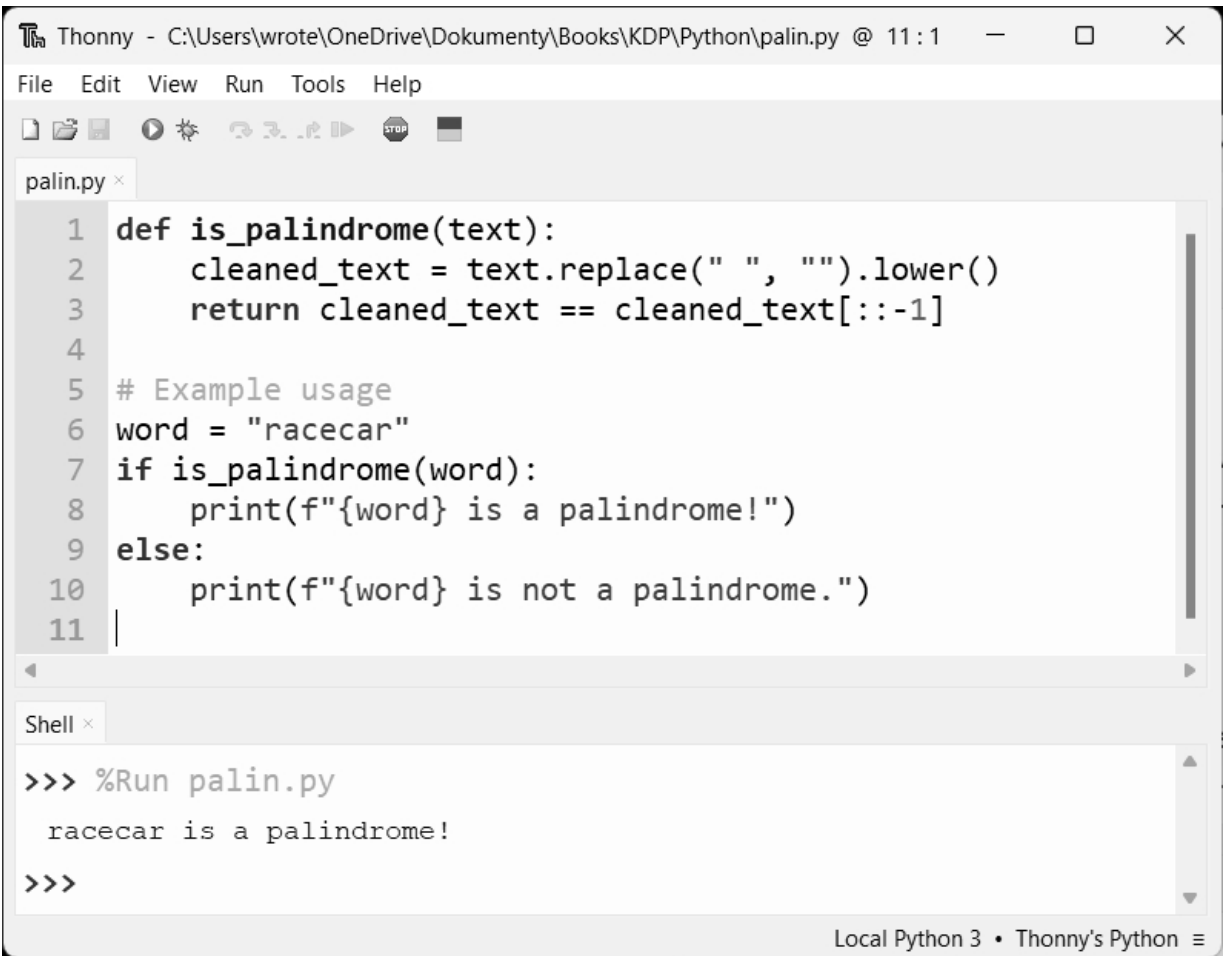
This function loops through the string and increases the count each time it finds a vowel.

Problem 3: Checking for Palindromes

A **palindrome** is a word or phrase that reads the same forward and backward (ignoring spaces and punctuation). For example, "racecar" is a palindrome. Let's write a function to check if a string is a palindrome:

```
def is_palindrome(text):
    cleaned_text = text.replace(" ", "").lower()
    return cleaned_text == cleaned_text[::-1]

# Example usage
word = "racecar"
if is_palindrome(word):
    print(f"{word} is a palindrome!")
else:
    print(f"{word} is not a palindrome.")
```



The screenshot shows the Thonny Python IDE interface. The top window, titled 'palin.py', contains the following Python code:

```
1 def is_palindrome(text):
2     cleaned_text = text.replace(" ", "").lower()
3     return cleaned_text == cleaned_text[::-1]
4
5 # Example usage
6 word = "racecar"
7 if is_palindrome(word):
8     print(f"{word} is a palindrome!")
9 else:
10    print(f"{word} is not a palindrome.")
11
```

The bottom window, titled 'Shell', shows the execution of the script:

```
>>> %Run palin.py
racecar is a palindrome!
>>>
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

This function first removes spaces and converts the string to lowercase. Then, it checks if the cleaned string is the same as its reverse.

5. Real-World Application: Text Manipulation in Everyday Tasks

Working with strings has countless real-world applications. Here are just a few ways string manipulation can be useful:

- **Cleaning data:** When working with user input or large text files, string manipulation allows you to clean up and organize text by removing unwanted spaces, characters, or words.
- **Analyzing text:** You can write programs to count words, identify patterns, or extract important information from text, which is useful in fields like data science, journalism, and research.

- **Building interactive programs:** Many interactive programs like quizzes, games, or chatbots require string manipulation to handle user input, generate responses, and create dynamic content.

Summary

Manipulating strings is an essential skill in Python, and it opens up the ability to solve a wide range of text-based challenges. From reversing strings to checking for palindromes, these operations help you clean, transform, and analyze text in meaningful ways. Whether you're working on a small project or a large application, mastering string manipulation will make you a more powerful and efficient problem solver. Keep practicing, and soon you'll be able to tackle any text-based challenge with confidence!

Finding Patterns with Regular Expressions

In many real-world problems, you'll need to search for specific patterns in text. Whether you're extracting information from a document, verifying email addresses, or finding certain words in a paragraph, **regular expressions** (often called **regex**) are an incredibly powerful tool to help you solve these challenges efficiently.

In this section, we'll explore how to use regular expressions in Python to find patterns in text. Don't worry if it seems a little tricky at first — once you understand how regular expressions work, you'll see that they can solve a wide range of text-based problems with ease!

1. What Are Regular Expressions?

A **regular expression** is a special sequence of characters that forms a search pattern. You can use regex to find specific patterns of characters in a string. For example, you could use regex to:

- Find all occurrences of a word in a document.
- Check if a string matches a specific format (like an email address or phone number).
- Replace parts of a string based on a pattern.

Python has a built-in module called `re` that allows you to work with regular expressions easily.

2. Basic Regular Expression Syntax

To get started with regex, let's look at some common patterns and their meanings:

- `.` : Matches any character (except newline).
- `^` : Matches the start of a string.

- `$` : Matches the end of a string.
- `\d` : Matches any digit (0-9).
- `\w` : Matches any word character (letters, digits, and underscore).
- `+` : Matches one or more occurrences of the preceding character or pattern.
- `*` : Matches zero or more occurrences of the preceding character or pattern.
- `[]` : Matches any one of the characters inside the brackets (e.g., `[abc]` matches 'a', 'b', or 'c').
- `|` : Acts as an "or" operator (e.g., `cat|dog` matches either "cat" or "dog").

3. Using Regular Expressions in Python

Let's dive into how you can use regular expressions with Python's `re` module. The most common tasks are **searching**, **matching**, and **replacing** patterns in text.

Example 1: Searching for a Word in a String

You can use the `re.search()` function to check if a specific word or pattern exists in a string.

```
import re

# Example text
text = "Python is a powerful programming language."

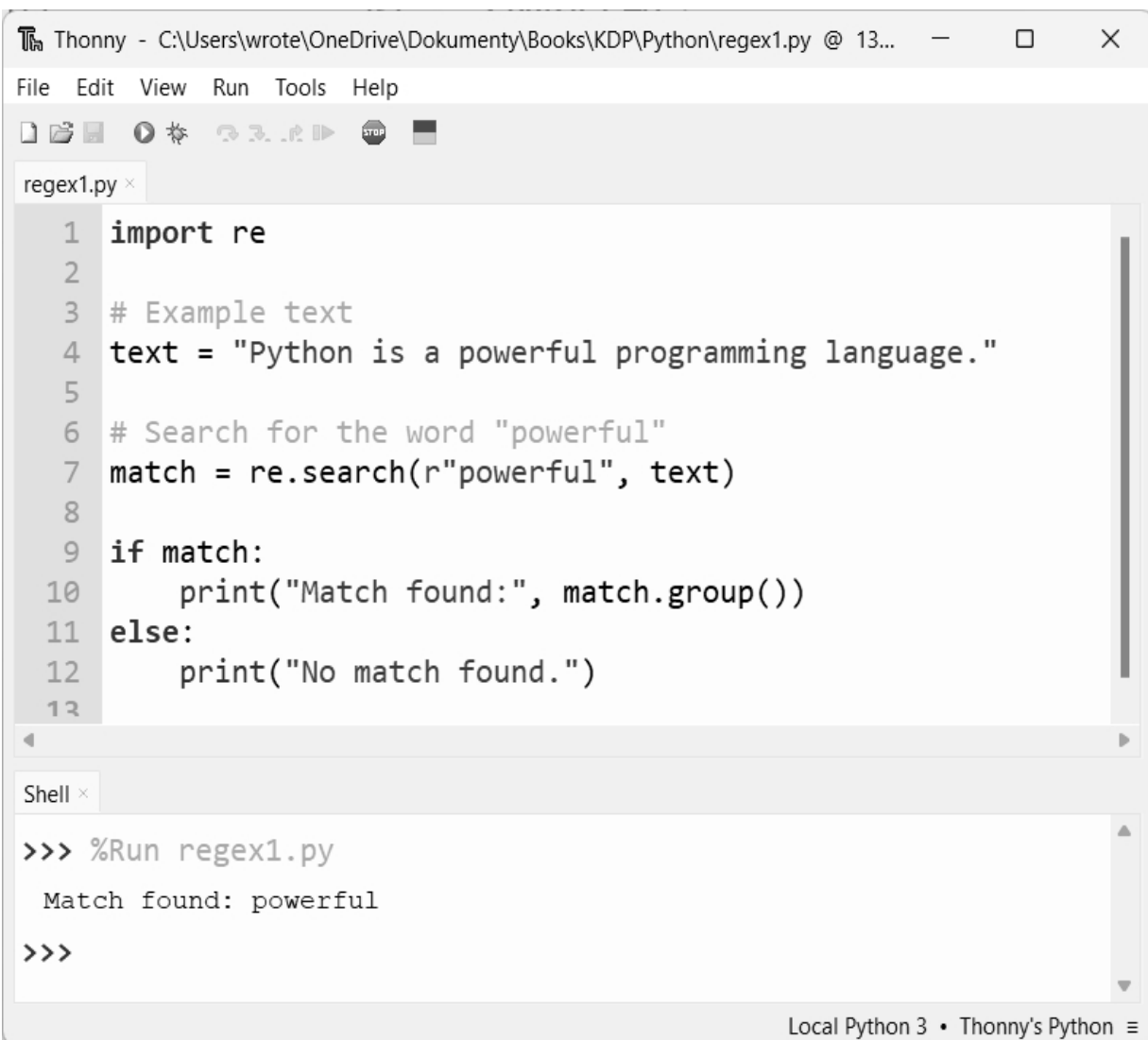
# Search for the word "powerful"
match = re.search(r"powerful", text)

if match:
    print("Match found:", match.group())
else:
    print("No match found.")
```

Explanation:

- `r"powerful"` is a **raw string** that tells Python to treat the backslashes (if any) as part of the regular expression, not escape characters.
- `re.search()` returns a match object if it finds the pattern in the string; otherwise, it returns `None`.

Output:



The screenshot shows the Thonny Python IDE interface. The top window, titled 'regex1.py', contains the following Python code:

```
1 import re
2
3 # Example text
4 text = "Python is a powerful programming language."
5
6 # Search for the word "powerful"
7 match = re.search(r"powerful", text)
8
9 if match:
10     print("Match found:", match.group())
11 else:
12     print("No match found.")
13
```

Below the code editor is a 'Shell' window showing the execution output:

```
>>> %Run regex1.py
Match found: powerful
>>>
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

Example 2: Finding All Occurrences of a Pattern

If you want to find **all** occurrences of a pattern in a string, you can use `re.findall()`.

```
import re

# Example text
text = "The rain in Spain falls mainly in the plain."

# Find all occurrences of "ain"
matches = re.findall(r"ain", text)
print("Matches found:", matches)
```

Output:

Here, `re.findall()` searches the string and returns a list of all occurrences of the pattern "ain".

The image is a screenshot of the Thonny Python IDE. The top window, titled 'find1.py', contains the following Python code:

```
1 import re
2
3 # Example text
4 text = "The rain in Spain falls mainly in the plain."
5
6 # Find all occurrences of "ain"
7 matches = re.findall(r"ain", text)
8 print("Matches found:", matches)
```

The bottom window, titled 'Shell', shows the execution of the script:

```
>>> %Run find1.py
Matches found: ['ain', 'ain', 'ain', 'ain']
>>>
```

The status bar at the bottom right indicates 'Local Python 3 • Thonny's Python'.

Example 3: Extracting Digits from a String

You can use regex to extract specific types of characters, like digits, from a string.

```
import re

# Example text with numbers
text = "My phone number is 123-456-7890 and my zip code is 98765."

# Extract all digits
digits = re.findall(r"\d+", text)
print("Found digits:", digits)
```

Explanation:

- `\d+` means "one or more digits."
- `re.findall()` returns a list of all the numbers it finds.

Output:



The screenshot shows the Thonny IDE interface. The top window, titled 'digits1.py', contains the following Python code:

```
1 import re
2
3 # Example text with numbers
4 text = "My phone number is 123-456-7890 and my zip code is 98765."
5
6 # Extract all digits
7 digits = re.findall(r"\d+", text)
8 print("Found digits:", digits)
```

Below the code editor is a 'Shell' window showing the execution output:

```
>>> %Run digits1.py
Found digits: ['123', '456', '7890', '98765']
>>>
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

Example 4: Validating an Email Address

You can use regular expressions to check if an email address is valid by searching for a pattern that matches the typical structure of an email.

```
import re
```



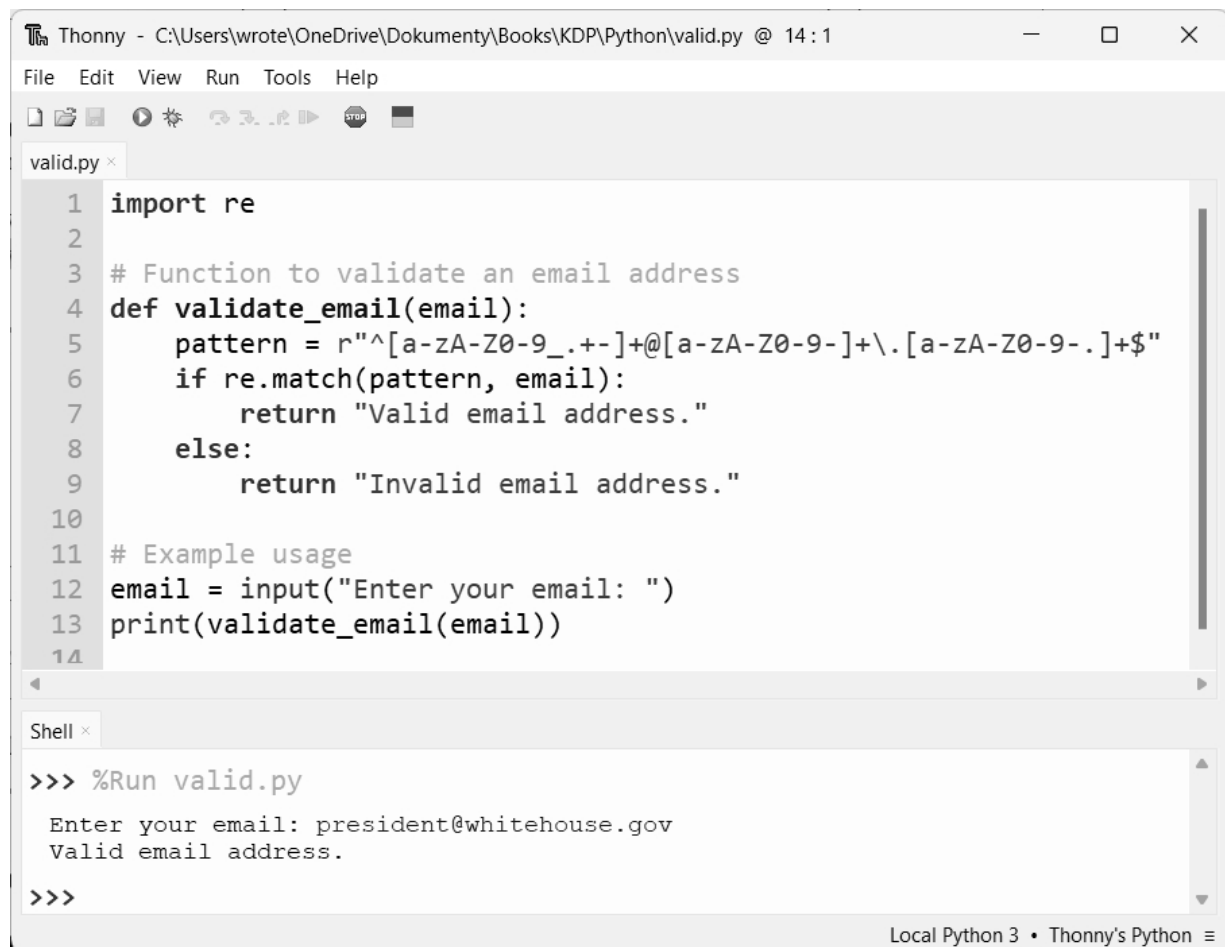
```
# Function to validate an email address
def validate_email(email):
    pattern = r"^[a-zA-Z0-9_+-.]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-]+$"
    if re.match(pattern, email):
        return "Valid email address."
    else:
        return "Invalid email address."

# Example usage
email = input("Enter your email: ")
print(validate_email(email))
```

Explanation:

- The regex pattern checks for a typical email structure like username@domain.com.
- re.match() checks if the entire string matches the pattern from the start.

Example output:



The screenshot shows the Thonny Python IDE interface. The top window, titled 'valid.py', contains the following Python code:

```
1 import re
2
3 # Function to validate an email address
4 def validate_email(email):
5     pattern = r"^[a-zA-Z0-9_+-.]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-.]+$"
6     if re.match(pattern, email):
7         return "Valid email address."
8     else:
9         return "Invalid email address."
10
11 # Example usage
12 email = input("Enter your email: ")
13 print(validate_email(email))
14
```

The bottom window, titled 'Shell', shows the execution of the script:

```
>>> %Run valid.py
Enter your email: president@whitehouse.gov
Valid email address.
>>>
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

Example 5: Replacing Text with Regular Expressions

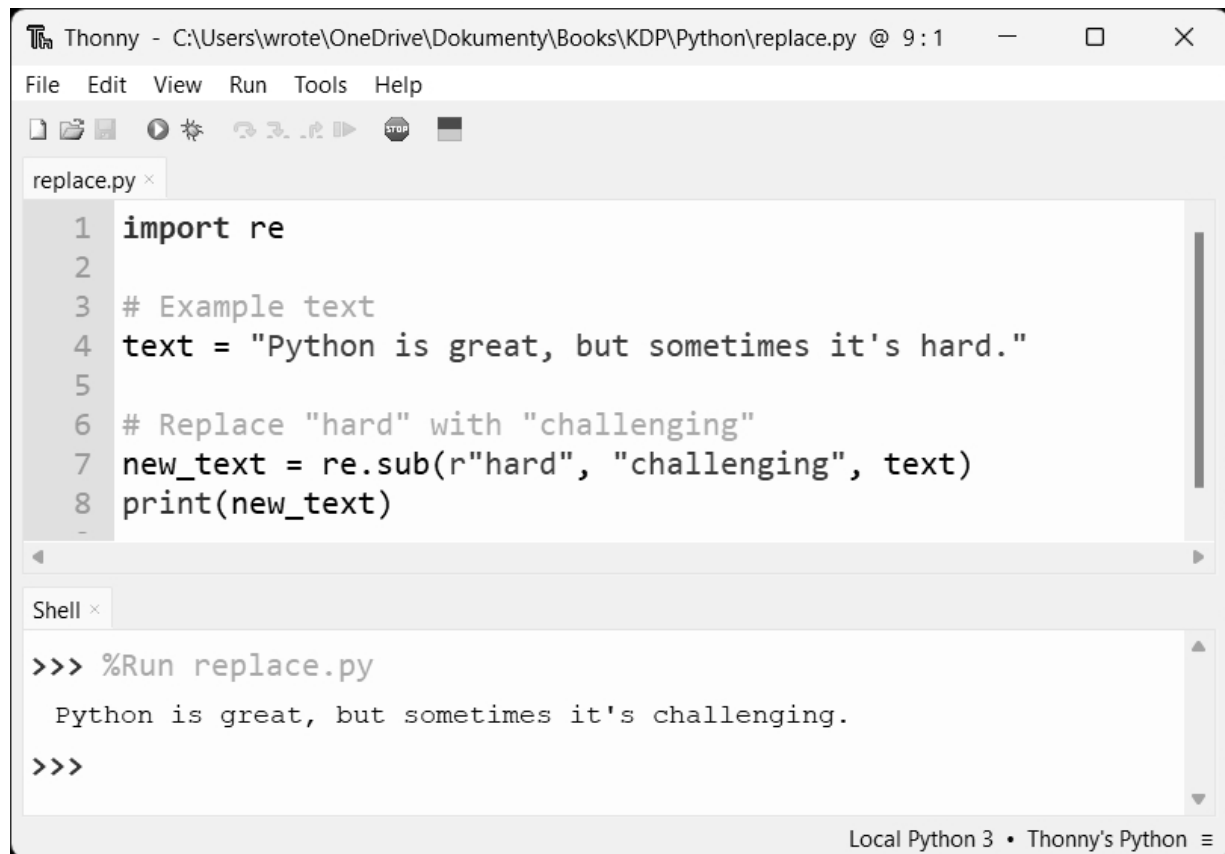
You can use the `re.sub()` function to **replace** parts of a string that match a pattern. This is useful for tasks like removing unwanted characters or censoring certain words.

```
import re

# Example text
text = "Python is great, but sometimes it's hard."

# Replace "hard" with "challenging"
new_text = re.sub(r"hard", "challenging", text)
print(new_text)
```

Output:



The screenshot shows the Thonny Python IDE interface. The title bar indicates the file path: C:\Users\wrote\OneDrive\Dokumenty\Books\KDP\Python\replace.py @ 9:1. The menu bar includes File, Edit, View, Run, Tools, and Help. Below the menu is a toolbar with icons for file operations and execution. The main editor window, titled 'replace.py', contains the following Python code:

```
1 import re
2
3 # Example text
4 text = "Python is great, but sometimes it's hard."
5
6 # Replace "hard" with "challenging"
7 new_text = re.sub(r"hard", "challenging", text)
8 print(new_text)
```

Below the editor is a 'Shell' window showing the execution of the script:

```
>>> %Run replace.py
Python is great, but sometimes it's challenging.
>>>
```

The status bar at the bottom right shows 'Local Python 3 • Thonny's Python'.

4. Real-World Applications of Regular Expressions

Regular expressions are used in many real-world scenarios:

- **Data cleaning:** Extracting useful information from messy data, like finding phone numbers or email addresses in a document.
- **Form validation:** Ensuring that user input follows a specific format, like checking if a password contains both letters and numbers.
- **Search and replace:** Finding and replacing patterns of text in large files, such as changing formatting in documents.

Regular expressions can seem complicated at first, but they are incredibly powerful tools for solving text-based problems. With a little practice, you'll find that they can save you a lot of time and effort when working with patterns in text.

Summary

Regular expressions (regex) allow you to find, extract, and manipulate patterns in text, making them a vital tool for text-based problem-solving. From searching for specific words to validating email addresses, regex can handle complex tasks with just a few lines of code. With Python's `re` module, you can apply these powerful patterns in your programs, making your text processing both faster and more efficient.

Practice working with regular expressions, and soon you'll be able to solve all kinds of pattern-based challenges with ease!

Application: Building a Simple Text-Based Game

Now that you've learned how to manipulate strings and find patterns with regular expressions, it's time to apply those skills in a fun and creative way — by building your very own **text-based game**! Text-based games are an excellent way to practice programming concepts like loops, conditionals, and user input while having fun creating an interactive experience.

In this section, we'll walk through how to build a simple game where the player explores a world, interacts with objects, and makes decisions that affect the outcome. By the end, you'll have a fully functioning game that you can expand and customize as you wish!

1. What Is a Text-Based Game?

A **text-based game** is a game where the player interacts with the game world by typing commands and receiving responses in text form. Think of it like a virtual choose-your-own-adventure book where the player's decisions influence what happens next.

In this game, the player will:

- Explore different locations.
- Pick up or interact with items.

- Solve simple challenges to progress.

2. Planning the Game

Before jumping into the code, let's plan out the basic structure of the game:

- **The world:** We'll create simple locations (like a room, a forest, etc.) that the player can explore.
- **Actions:** The player will be able to perform actions like "look," "go," and "pick up" to interact with the world.
- **Goal:** The player's goal will be to collect an item or solve a challenge to win the game.

3. Creating the Game's World

First, we'll define the locations in the game. Let's say the player starts in a room and can explore a forest and a cave. We'll use a dictionary to store the descriptions of each location:

```
# Define the game world
```

```
world = {  
    "room": "You are in a small room with a wooden door. There is a key on the floor.",  
    "forest": "You are in a dense forest. You hear birds chirping around you.",  
    "cave": "You enter a dark cave. It's cold and damp."  
}
```

```
# Set the starting location
```

```
current_location = "room"
```

Each location has a description that the player will see when they explore it. The player will start in the room.

4. Handling Player Actions

Next, we'll handle the player's actions. The player should be able to **look** around, **move** between locations, and **pick up** items. We'll start by implementing the basic commands:

- **look:** Displays the description of the current location.

- **go**: Moves the player to a new location if it exists.
- **pick up**: Allows the player to pick up items.

```
# List of items the player can pick up
inventory = []

def look():
    print(world[current_location])

def go(direction):
    global current_location
    if direction in world:
        current_location = direction
        print(f"You move to the {direction}.")
        look()
    else:
        print("You can't go there.")

def pick_up(item):
    if item == "key" and current_location == "room":
        inventory.append(item)
        print("You picked up the key.")
    else:
        print("There is nothing to pick up here.")
```

Explanation:

- The look() function shows the player the description of their current location.
- The go(direction) function changes the player's location if the direction is valid.
- The pick_up(item) function allows the player to pick up the key if they are in the correct location.

5. Game Loop and Player Input

Now we need to create a **game loop** that repeatedly asks the player for input and performs actions based on their commands. The loop will continue until the player wins the game by finding an item or solving a challenge.

```
# Game loop
while True:
    command = input("> ").lower()

    if command == "look":
        look()
    elif command.startswith("go"):
        direction = command.split()[1]
        go(direction)
    elif command.startswith("pick up"):
        item = command.split()[2]
        pick_up(item)
    elif command == "quit":
        print("Thanks for playing!")
        break
    else:
        print("I don't understand that command.")

# Check if the player has won the game
if "key" in inventory and current_location == "cave":
    print("You use the key to unlock the treasure in the cave. You win!")
    break
```

Explanation:

- The game waits for the player to input a command using `input()`.
- Based on the command, the game calls the appropriate function (`look()`, `go()`, or `pick_up()`).
- The loop continues until the player types quit or wins by finding the key and entering the cave.

6. Expanding the Game

Now that we have the basic structure of the game, here are a few ways you can expand and customize it:

- **Add more locations:** Expand the game world by adding new locations with unique descriptions.

Skopiuj kod

```
world["village"] = "You arrive at a peaceful village with friendly people."
```

- **Add more items:** Add more items that the player can pick up and use in different locations.

```
def pick_up(item):
    if item == "key" and current_location == "room":
        inventory.append(item)
        print("You picked up the key.")
    elif item == "torch" and current_location == "cave":
        inventory.append(item)
        print("You picked up the torch.")
    else:
        print("There is nothing to pick up here.")
```

- **Add challenges or puzzles:** Include challenges the player must solve to move forward, such as answering a riddle or finding a hidden item.

```
def solve_riddle(answer):
    if answer == "fire":
        print("Correct! You may pass.")
    else:
        print("Wrong answer! Try again.")
```

7. Final Version of the Game

Here's the full game code that includes the basic structure we've discussed:

```
# Define the game world
```



```

world = {
    "room": "You are in a small room with a wooden door. There is a key on the floor.",
    "forest": "You are in a dense forest. You hear birds chirping around you.",
    "cave": "You enter a dark cave. It's cold and damp."
}

# Set the starting location
current_location = "room"

# List of items the player can pick up
inventory = []

def look():
    print(world[current_location])

def go(direction):
    global current_location
    if direction in world:
        current_location = direction
        print(f"You move to the {direction}.")
        look()
    else:
        print("You can't go there.")

def pick_up(item):
    if item == "key" and current_location == "room":
        inventory.append(item)
        print("You picked up the key.")
    else:
        print("There is nothing to pick up here.")

# Game loop
while True:
    command = input("> ").lower()

    if command == "look":
        look()
    elif command.startswith("go"):

```

```

        direction = command.split()[1]
        go(direction)
    elif command.startswith("pick up"):
        item = command.split()[2]
        pick_up(item)
    elif command == "quit":
        print("Thanks for playing!")
        break
    else:
        print("I don't understand that command.")

# Check if the player has won the game
if "key" in inventory and current_location == "cave":
    print("You use the key to unlock the treasure in the cave. You win!")
    break

```

The screenshot shows the Thonny Python IDE interface. The top window, titled 'game.py', contains the following Python code:

```

1 # Define the game world
2 world = {
3     "room": "You are in a small room with a wooden door. T
4     "forest": "You are in a dense forest. You hear birds c
5     "cave": "You enter a dark cave. It's cold and damp."
6 }
7
8 # Set the starting location

```

Below the code editor is a 'Shell' window showing the execution of the game. It starts with the command prompt `>>> %Run game.py`. The user enters the command `> look`, and the program outputs: `You are in a small room with a wooden door. There is a key on the floor.`. The prompt `>` is shown again, indicating the program is waiting for the next command.

At the bottom right of the IDE, the status bar indicates 'Local Python 3 • Thonny's Python'.

8. Why Build a Text-Based Game?

Building a text-based game is a great way to practice your Python skills in a creative and interactive way. Through this project, you've learned how to:

- Handle user input and process commands.
- Work with functions to organize game logic.
- Use loops and conditionals to create a dynamic, interactive experience.

Text-based games are also easy to expand, so you can keep adding new features, puzzles, and locations as your programming skills grow.

With this game, you've created a fun, interactive experience using the power of Python. Whether you choose to expand this game into something bigger or use it as a foundation for other projects, you've learned valuable skills for solving problems with text-based input and logic. Keep practicing, and soon you'll be able to build even more complex and exciting games!

Chapter 6: Data Handling with Lists and Dictionaries

Solving Problems Using Lists, Tuples, and Dictionaries

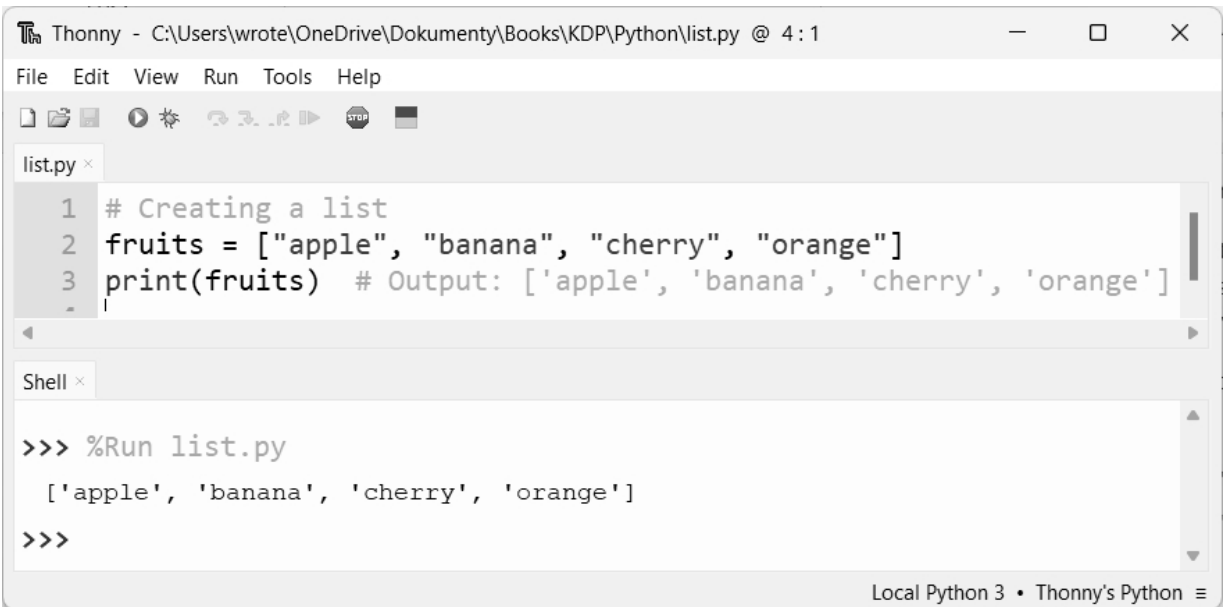
In Python, **lists**, **tuples**, and **dictionaries** are some of the most powerful tools for organizing and handling data. Whether you're managing a collection of items, storing related information together, or creating a fast lookup table, these data structures will help you solve problems more efficiently. In this section, we'll explore how to use lists, tuples, and dictionaries to organize and manipulate data effectively, with practical examples to guide you along the way.

1. Lists: Organizing and Accessing Data

A **list** is a collection of items that are stored in a specific order. You can think of it like a shopping list or a to-do list — each item has a place, and you can access, modify, or add items whenever you need.

Here's how to create a list in Python:

```
# Creating a list
fruits = ["apple", "banana", "cherry", "orange"]
print(fruits) # Output: ['apple', 'banana', 'cherry', 'orange']
```

The image shows a screenshot of the Thonny Python IDE. The top window, titled 'list.py', contains the following Python code:

```
1 # Creating a list
2 fruits = ["apple", "banana", "cherry", "orange"]
3 print(fruits) # Output: ['apple', 'banana', 'cherry', 'orange']
```

The bottom window, titled 'Shell', shows the execution output:

```
>>> %Run list.py
['apple', 'banana', 'cherry', 'orange']
>>>
```

The status bar at the bottom right indicates 'Local Python 3 • Thonny's Python'.

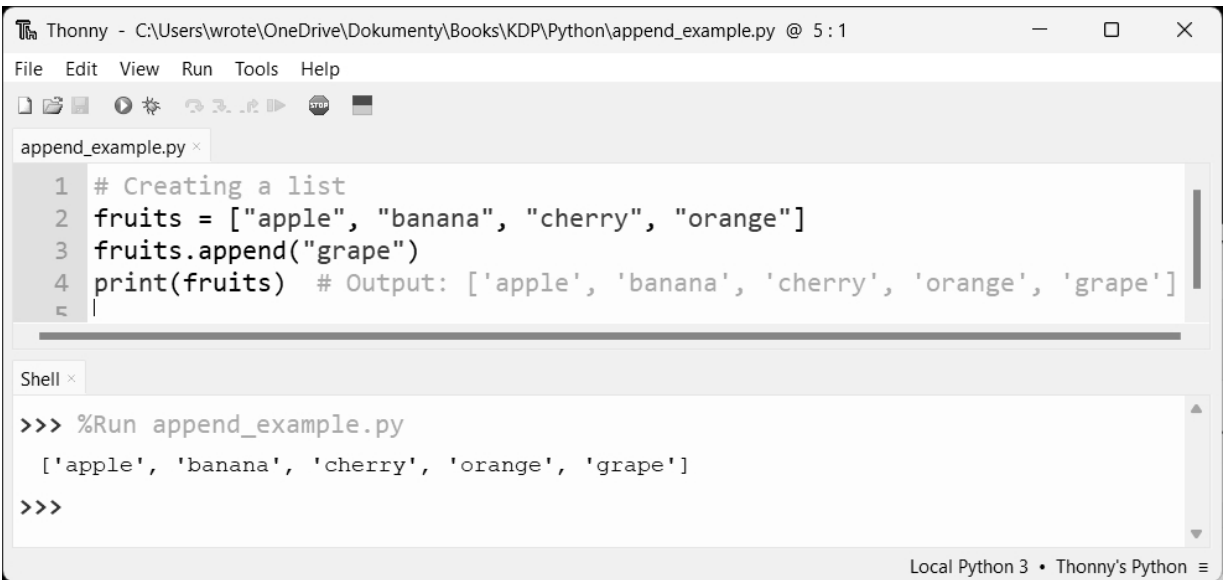
Key Features of Lists:

- Lists can hold items of any type (e.g., numbers, strings, or even other lists).
- Lists are **mutable**, meaning you can change them after they're created (add, remove, or modify items).
- Lists maintain **order**, so the items stay in the order you added them.

Basic List Operations:

- **Adding items:** You can add items to a list using the `append()` method.

```
fruits.append("grape")
print(fruits) # Output: ['apple', 'banana', 'cherry', 'orange', 'grape']
```



The screenshot shows the Thonny IDE interface. The main editor window displays a Python script named `append_example.py` with the following code:

```
1 # Creating a list
2 fruits = ["apple", "banana", "cherry", "orange"]
3 fruits.append("grape")
4 print(fruits) # Output: ['apple', 'banana', 'cherry', 'orange', 'grape']
```

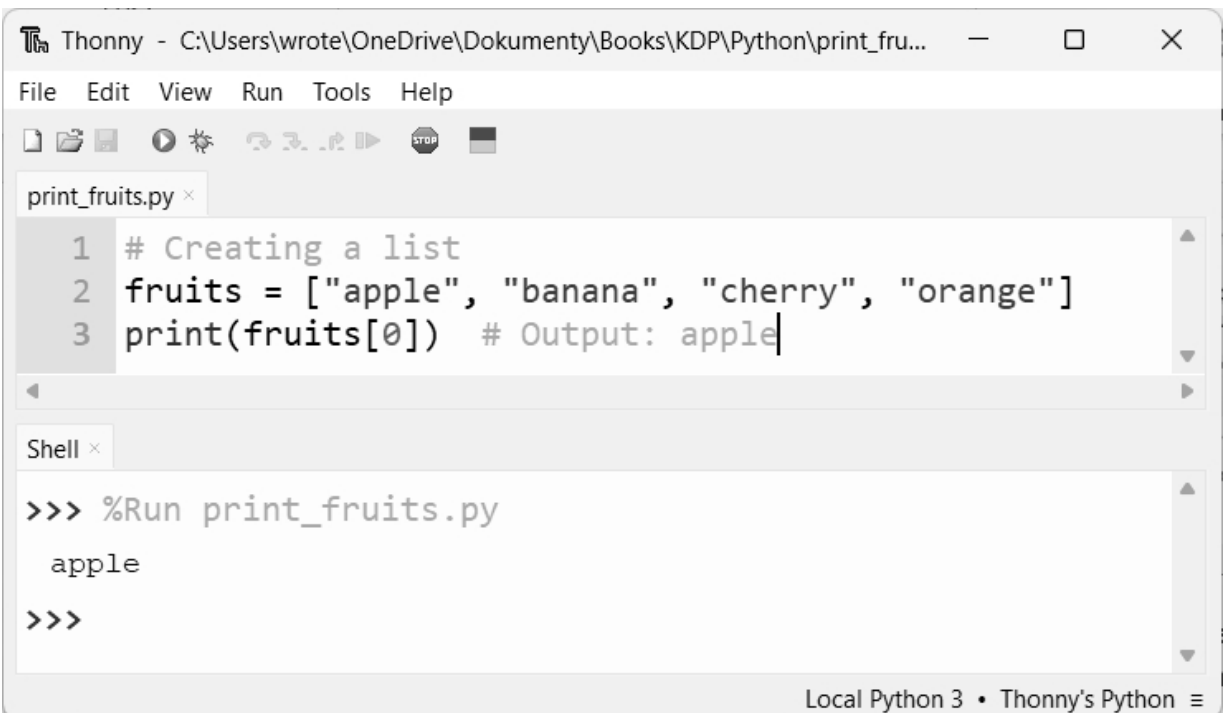
Below the editor is a Shell window showing the execution of the script:

```
>>> %Run append_example.py
['apple', 'banana', 'cherry', 'orange', 'grape']
>>>
```

The status bar at the bottom indicates "Local Python 3 • Thonny's Python".

- **Accessing items:** You can access items in a list by their index (position). Remember that indexing starts at 0.

`print(fruits[0])` # Output: apple



The screenshot shows the Thonny IDE interface. The main editor window displays a Python script named `print_fruits.py` with the following code:

```
1 # Creating a list
2 fruits = ["apple", "banana", "cherry", "orange"]
3 print(fruits[0]) # Output: apple
```

Below the editor is a Shell window showing the execution of the script:

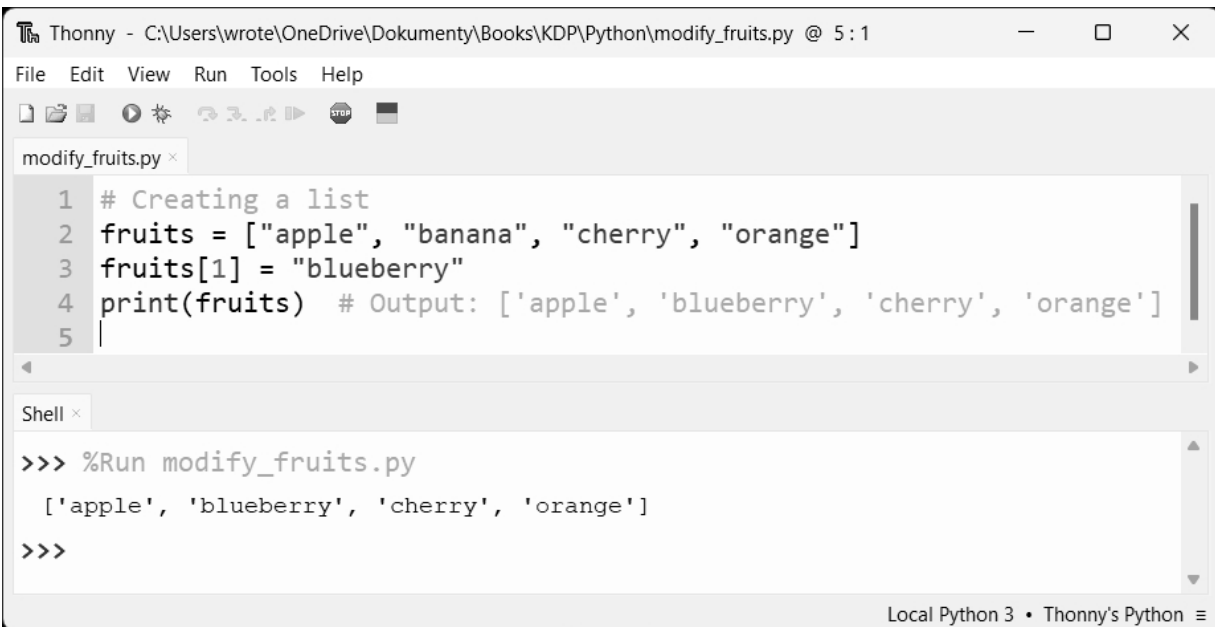
```
>>> %Run print_fruits.py
apple
>>>
```

The status bar at the bottom indicates "Local Python 3 • Thonny's Python".

- **Modifying items:** Since lists are mutable, you can change items directly by assigning a new value.

```
fruits[1] = "blueberry"
```

```
print(fruits) # Output: ['apple', 'blueberry', 'cherry', 'orange']
```



The screenshot shows the Thonny IDE interface. The top window, titled 'modify_fruits.py', contains the following code:

```
1 # Creating a list
2 fruits = ["apple", "banana", "cherry", "orange"]
3 fruits[1] = "blueberry"
4 print(fruits) # Output: ['apple', 'blueberry', 'cherry', 'orange']
5
```

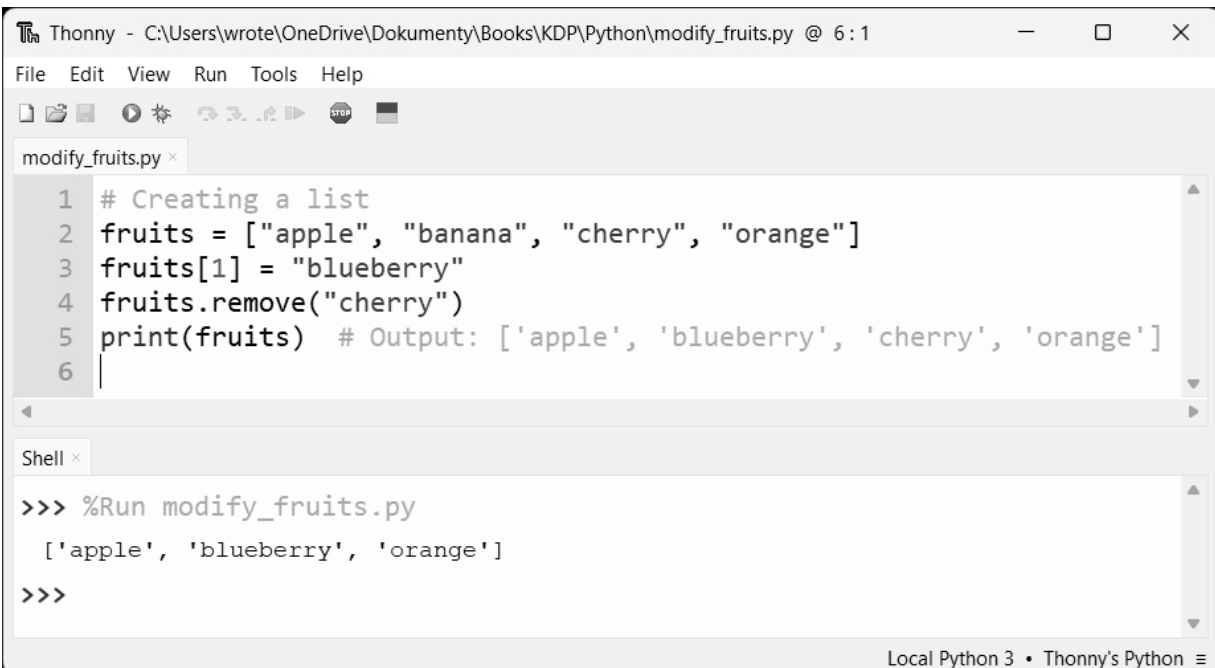
The bottom window, titled 'Shell', shows the execution output:

```
>>> %Run modify_fruits.py
['apple', 'blueberry', 'cherry', 'orange']
>>>
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

- **Removing items:** Use the `remove()` method to remove an item by its value or the `pop()` method to remove an item by its index.

```
fruits.remove("cherry")
```



The screenshot shows the Thonny IDE interface. The top window, titled 'modify_fruits.py', contains the following code:

```
1 # Creating a list
2 fruits = ["apple", "banana", "cherry", "orange"]
3 fruits[1] = "blueberry"
4 fruits.remove("cherry")
5 print(fruits) # Output: ['apple', 'blueberry', 'cherry', 'orange']
6
```

The bottom window, titled 'Shell', shows the execution output:

```
>>> %Run modify_fruits.py
['apple', 'blueberry', 'orange']
>>>
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

Example Problem: Organizing Student Scores

Let's say you have a list of student scores, and you want to calculate the average score:

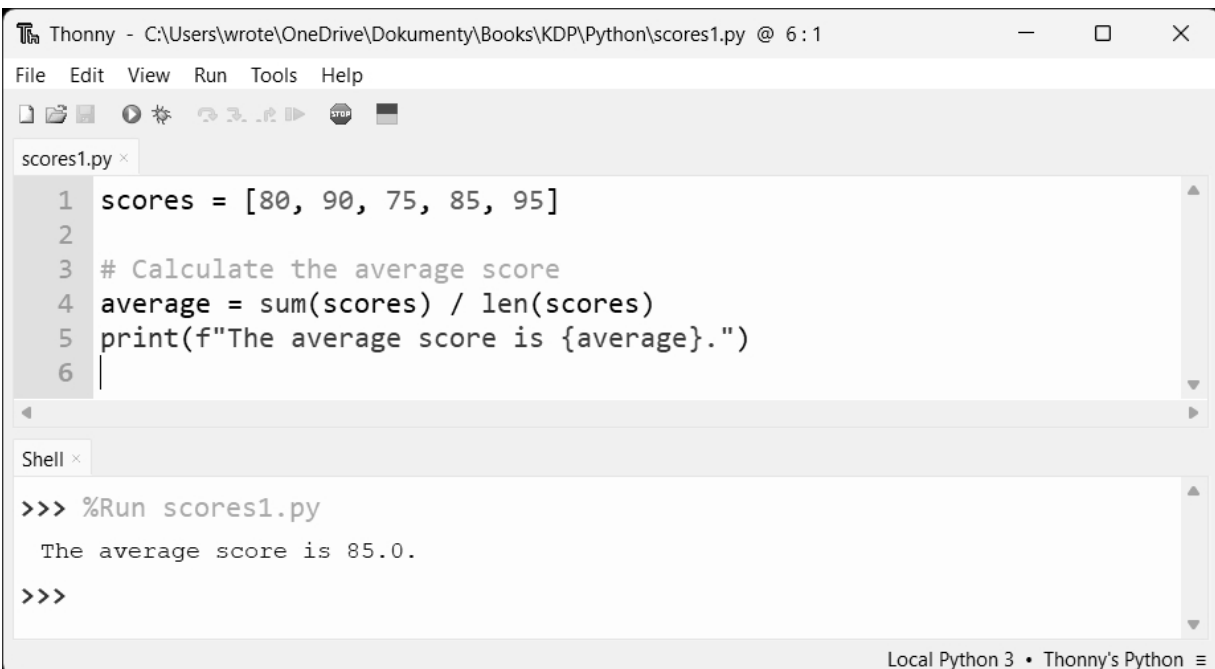
```
scores = [80, 90, 75, 85, 95]
```

```
# Calculate the average score
```

```
average = sum(scores) / len(scores)
```

```
print(f"The average score is {average}.")
```

In this example, we use the `sum()` function to get the total of all scores and the `len()` function to get the number of students. Dividing the total by the number of students gives the average.



The screenshot shows the Thonny Python IDE interface. The top window, titled 'scores1.py', contains the following Python code:

```
1 scores = [80, 90, 75, 85, 95]
2
3 # Calculate the average score
4 average = sum(scores) / len(scores)
5 print(f"The average score is {average}.")
6
```

Below the code editor is a 'Shell' window showing the execution of the script:

```
>>> %Run scores1.py
The average score is 85.0.
>>>
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

2. Tuples: Fixed Collections of Data

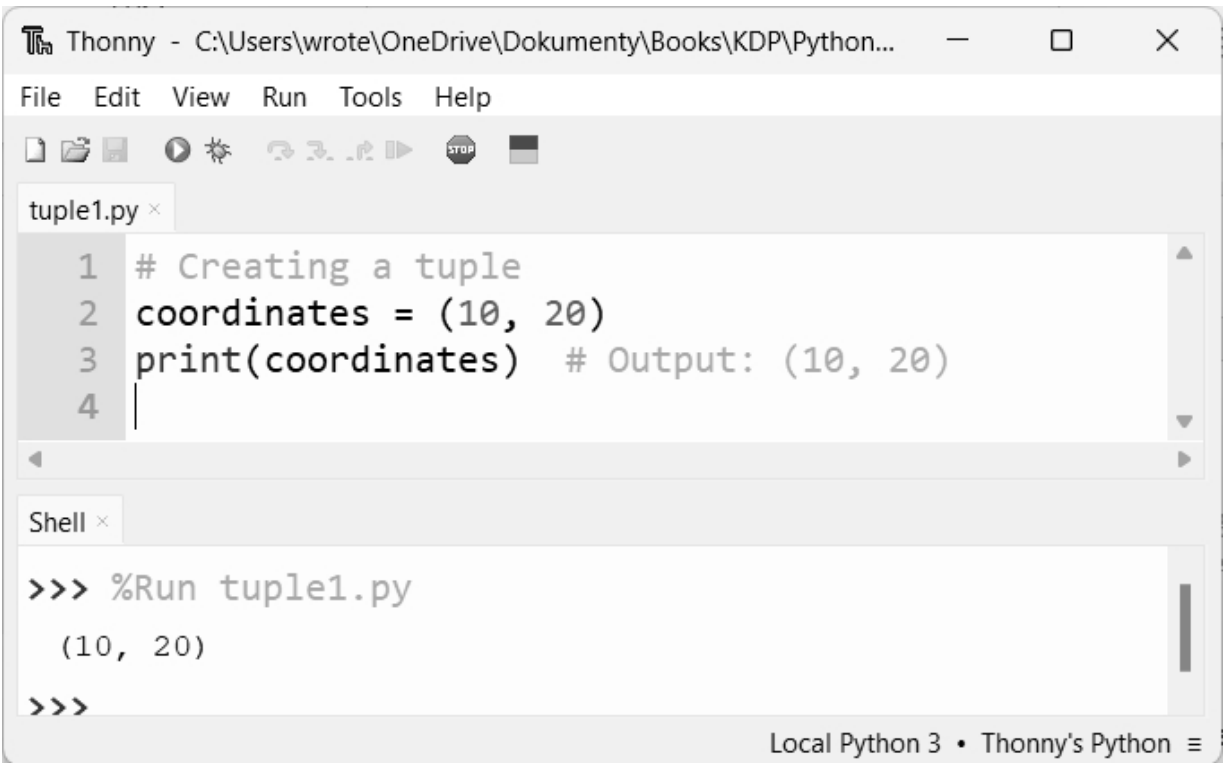
A **tuple** is similar to a list, but there's one key difference: tuples are **immutable**. This means once you create a tuple, you can't change it. Tuples are useful when you want to store data that should not be modified.

Here's how to create a tuple:

```
# Creating a tuple
```

```
coordinates = (10, 20)
```

```
print(coordinates) # Output: (10, 20)
```

The screenshot shows the Thonny Python IDE interface. The top window, titled 'tuple1.py', contains the following Python code:

```
1 # Creating a tuple
2 coordinates = (10, 20)
3 print(coordinates) # Output: (10, 20)
4
```

The bottom window, titled 'Shell', shows the execution of the script:

```
>>> %Run tuple1.py
(10, 20)
>>>
```

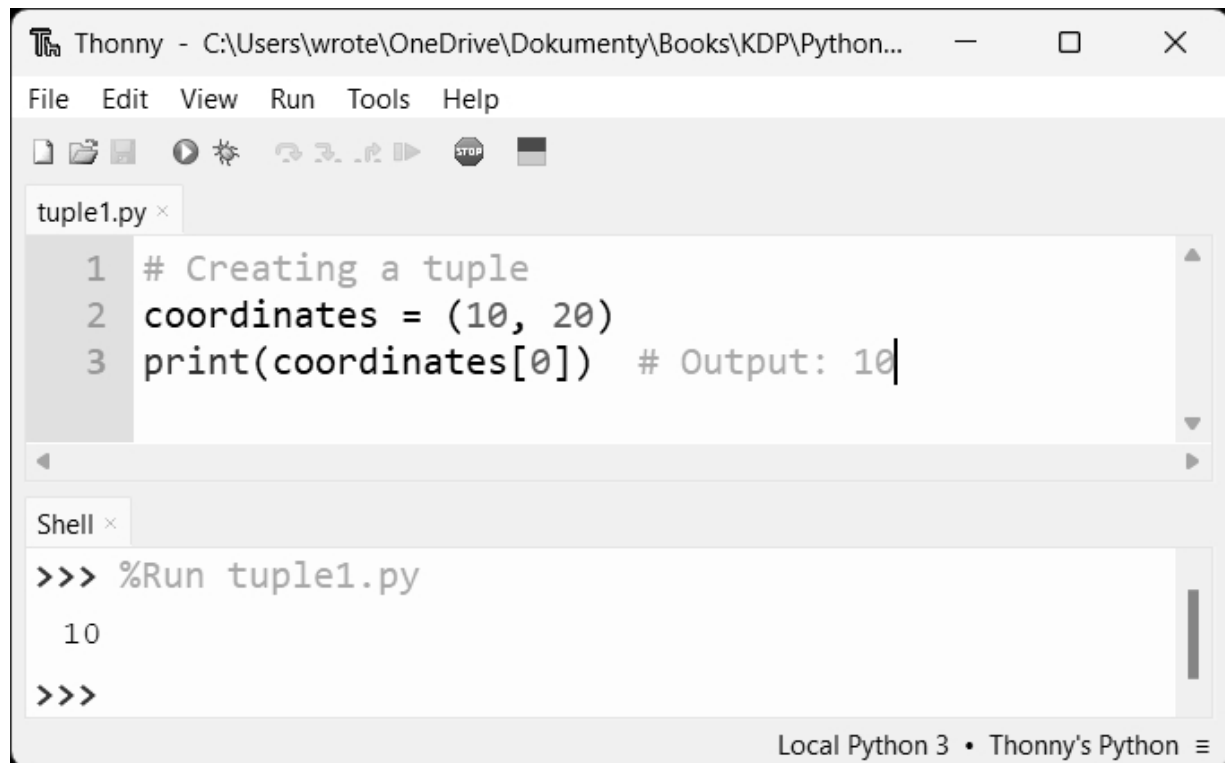
The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

Key Features of Tuples:

- Tuples are immutable, so their values cannot be changed after creation.
- Like lists, tuples maintain the order of items.
- Tuples are often used for fixed sets of related data, such as coordinates or points.

Accessing Data in Tuples: Just like lists, you can access tuple elements using their index:

```
print(coordinates[0]) # Output: 10
```

The image shows a screenshot of the Thonny Python IDE. The window title is "Thonny - C:\Users\wrote\OneDrive\Dokumenty\Books\KDP\Python...". The menu bar includes "File", "Edit", "View", "Run", "Tools", and "Help". Below the menu bar is a toolbar with icons for file operations and execution. The main editor area shows a file named "tuple1.py" with the following code:

```
1 # Creating a tuple
2 coordinates = (10, 20)
3 print(coordinates[0]) # Output: 10
```

The bottom panel, labeled "Shell", shows the execution of the code:

```
>>> %Run tuple1.py
10
>>>
```

The status bar at the bottom right indicates "Local Python 3 • Thonny's Python".

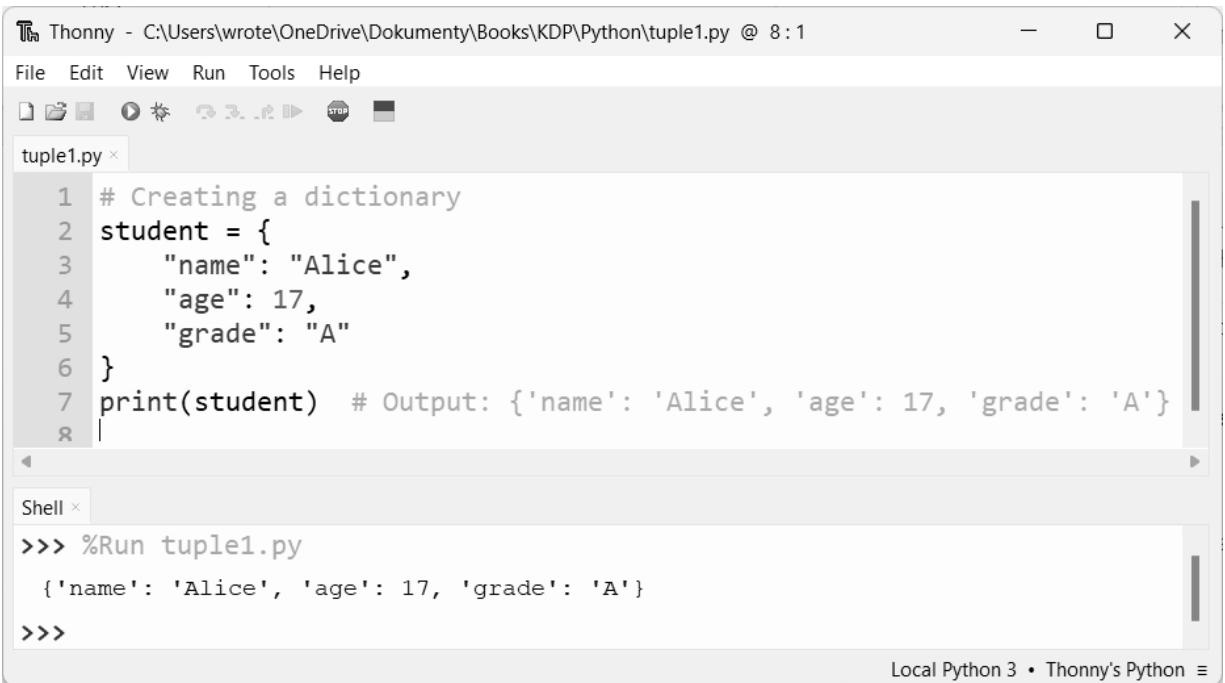
Why Use Tuples? Tuples are useful when you want to ensure that certain data remains constant throughout your program. For example, if you're working with GPS coordinates or configuration settings that shouldn't change, tuples are a great choice.

3. Dictionaries: Mapping Data with Key-Value Pairs

A **dictionary** is a collection of key-value pairs. Each key in a dictionary is unique and maps to a specific value. Think of it like a real-world dictionary, where each word (key) maps to its definition (value).

Here's how to create a dictionary:

```
# Creating a dictionary
student = {
    "name": "Alice",
    "age": 17,
    "grade": "A"
}
print(student) # Output: {'name': 'Alice', 'age': 17, 'grade': 'A'}
```



The screenshot shows the Thonny Python IDE interface. The top window, titled 'tuple1.py', contains the following Python code:

```
1 # Creating a dictionary
2 student = {
3     "name": "Alice",
4     "age": 17,
5     "grade": "A"
6 }
7 print(student) # Output: {'name': 'Alice', 'age': 17, 'grade': 'A'}
8
```

The bottom window, titled 'Shell', shows the execution of the code:

```
>>> %Run tuple1.py
{'name': 'Alice', 'age': 17, 'grade': 'A'}
>>>
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

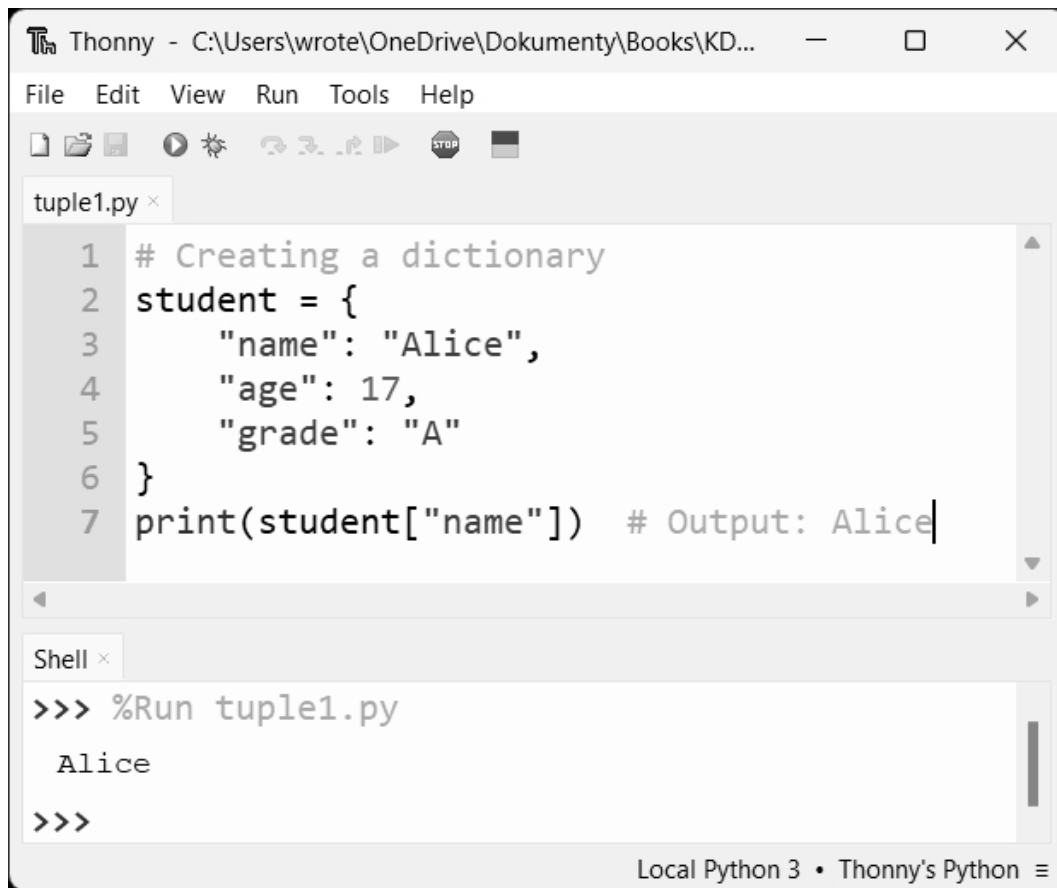
Key Features of Dictionaries:

- Dictionaries allow you to store data in pairs (keys and values), making it easy to look up information.
- Keys must be unique, and they are used to access the corresponding values.
- Dictionaries are **mutable**, so you can change, add, or remove key-value pairs.

Basic Dictionary Operations:

- **Accessing values:** You can access the value associated with a key by using the key in square brackets.

```
print(student["name"]) # Output: Alice
```



The screenshot shows the Thonny Python IDE interface. The main editor window displays a Python script named `tuple1.py`. The script creates a dictionary `student` with keys `"name"`, `"age"`, and `"grade"`, and values `"Alice"`, `17`, and `"A"` respectively. It then prints the value of `student["name"]`, which is `"Alice"`. The output is shown in the Shell window below the editor.

```
1 # Creating a dictionary
2 student = {
3     "name": "Alice",
4     "age": 17,
5     "grade": "A"
6 }
7 print(student["name"]) # Output: Alice
```

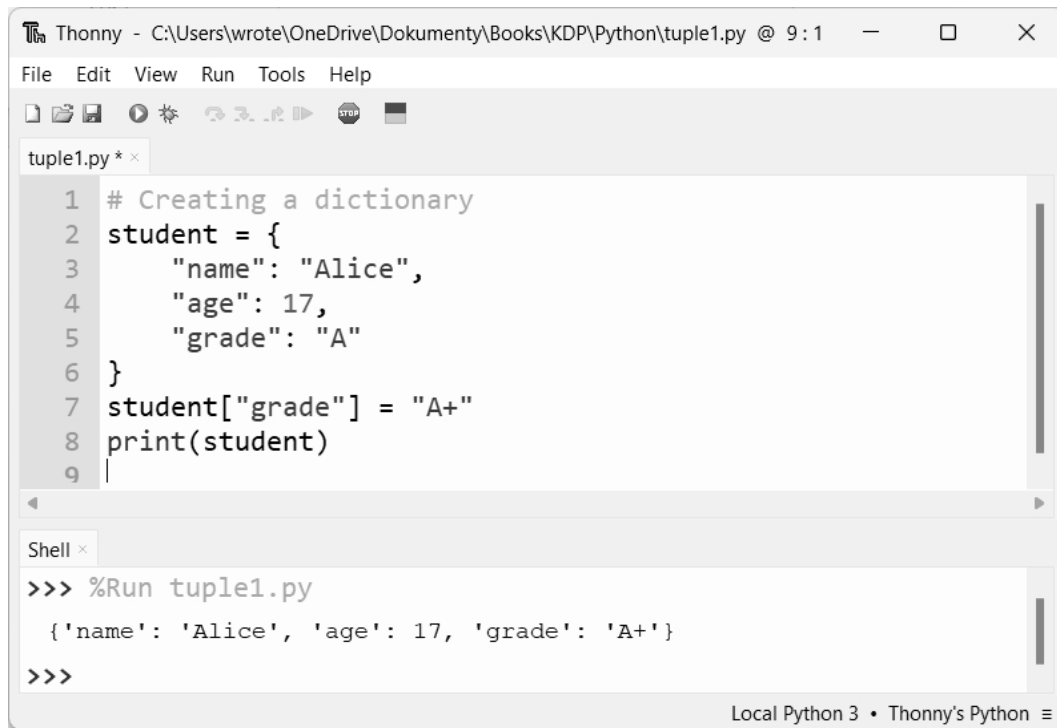
Shell

```
>>> %Run tuple1.py
Alice
>>>
```

Local Python 3 • Thonny's Python

- **Adding and modifying items:** You can add a new key-value pair or modify an existing one by assigning a value to a key.

```
student["grade"] = "A+"
print(student)
```



The screenshot shows the Thonny Python IDE interface. The top window, titled 'tuple1.py *', contains the following Python code:

```
1 # Creating a dictionary
2 student = {
3     "name": "Alice",
4     "age": 17,
5     "grade": "A"
6 }
7 student["grade"] = "A+"
8 print(student)
9
```

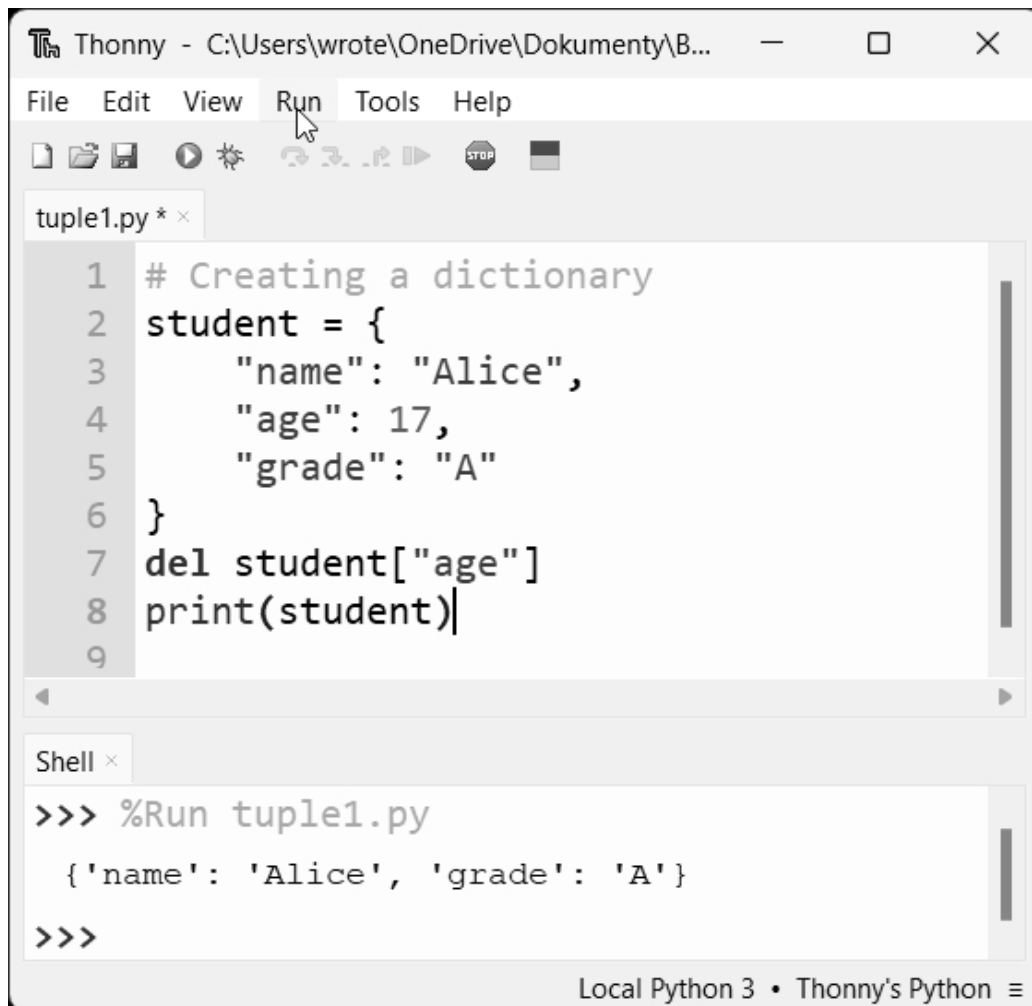
Below the code editor is a 'Shell' window showing the execution of the script:

```
>>> %Run tuple1.py
{'name': 'Alice', 'age': 17, 'grade': 'A+'}
>>>
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

- **Removing items:** Use the `del` statement to remove a key-value pair from a dictionary.

```
del student["age"]
print(student) # Output: {'name': 'Alice', 'grade': 'A+'}
```



The screenshot shows the Thonny Python IDE interface. The main editor window displays a Python script named `tuple1.py` with the following code:

```
1 # Creating a dictionary
2 student = {
3     "name": "Alice",
4     "age": 17,
5     "grade": "A"
6 }
7 del student["age"]
8 print(student)
9
```

Below the editor is a Shell window showing the execution of the script:

```
>>> %Run tuple1.py
{'name': 'Alice', 'grade': 'A'}
>>>
```

The status bar at the bottom indicates "Local Python 3 • Thonny's Python".

Example Problem: Storing and Accessing Student Information

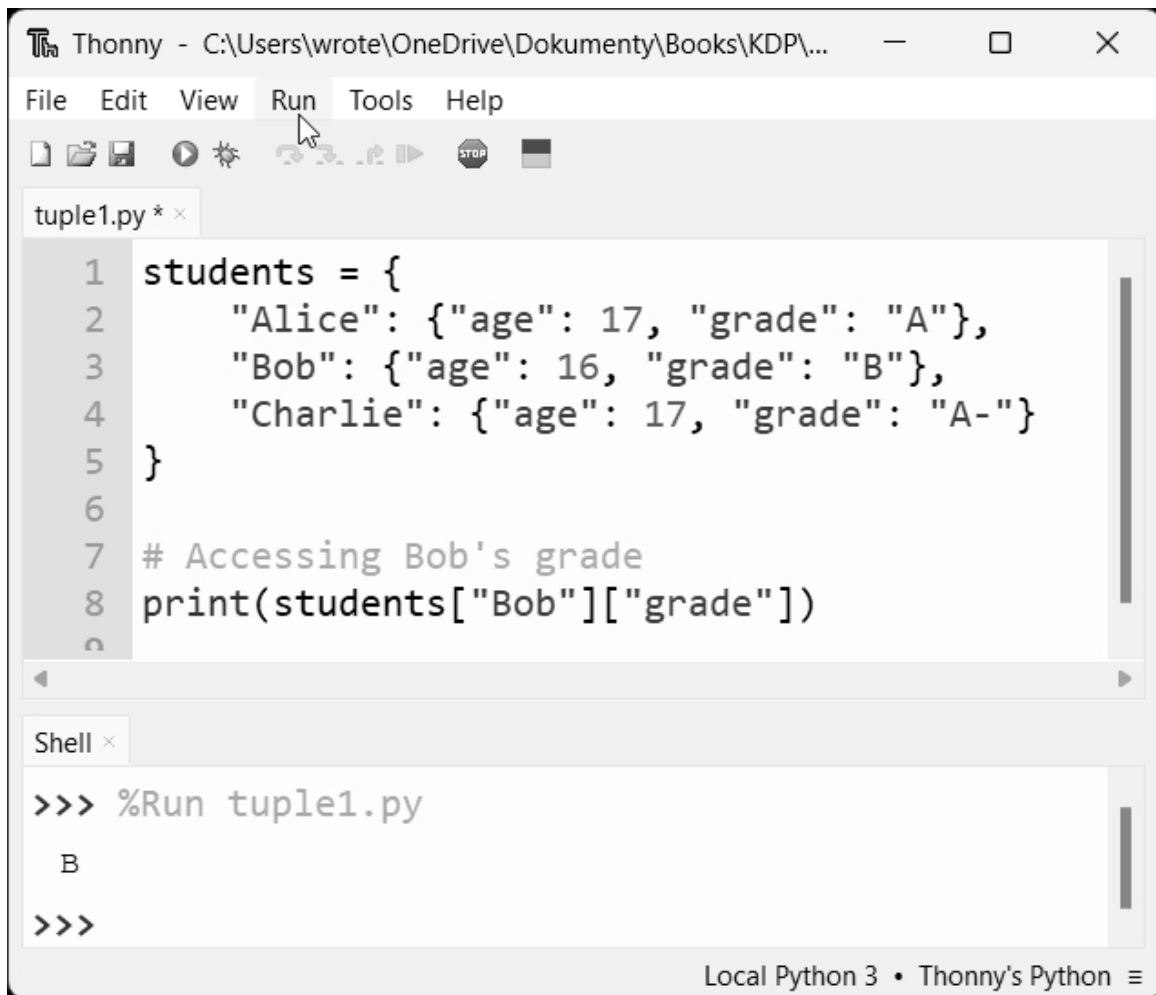
Let's say you want to store the details of several students in a class. A dictionary is perfect for this:

```
students = {
    "Alice": {"age": 17, "grade": "A"},
    "Bob": {"age": 16, "grade": "B"},
    "Charlie": {"age": 17, "grade": "A-"}
}
```

Accessing Bob's grade

```
print(students["Bob"]["grade"]) # Output: B
```

Here, we use a nested dictionary to store information about multiple students. We can easily access specific details using the keys.



The screenshot shows the Thonny Python IDE interface. The title bar indicates the file path: C:\Users\wrote\OneDrive\Dokumenty\Books\KDP\... The menu bar includes File, Edit, View, Run, Tools, and Help. The toolbar contains icons for opening files, saving, running, and stopping. The main editor window, titled 'tuple1.py *', contains the following Python code:

```
1 students = {
2     "Alice": {"age": 17, "grade": "A"},
3     "Bob": {"age": 16, "grade": "B"},
4     "Charlie": {"age": 17, "grade": "A-"}
5 }
6
7 # Accessing Bob's grade
8 print(students["Bob"]["grade"])
```

Below the editor is a 'Shell' window showing the execution of the script:

```
>>> %Run tuple1.py
B
>>>
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

4. Choosing the Right Data Structure

How do you decide whether to use a list, tuple, or dictionary? Here are some general guidelines:

- Use a **list** when you need to store a collection of items that may change (e.g., a list of books, tasks, or scores).
- Use a **tuple** when you want a collection of items that shouldn't be changed (e.g., fixed coordinates, configuration values).
- Use a **dictionary** when you need to associate values with unique keys, making it easy to look up and retrieve specific information (e.g., storing information about students, products, or settings).

Summary

Lists, tuples, and dictionaries are essential tools for solving a variety of problems in Python. They allow you to organize, store, and access data efficiently, depending on the task at hand. Understanding how and when to use each of these data structures will help you become a more effective problem-solver, whether you're handling simple lists or more complex collections of information.

Keep practicing with these powerful tools, and soon you'll be able to tackle any data-handling challenge with confidence!

Counting, Sorting, and Organizing Data

In Python, working with data often involves tasks like **counting**, **sorting**, and **organizing**. Whether you're managing a list of scores, organizing a collection of names, or tracking how often certain words appear, Python provides powerful tools to make these tasks simple and efficient. In this section, you'll learn how to easily count occurrences, sort data, and organize information using lists, tuples, and dictionaries.

1. Counting Data in Lists and Dictionaries

Often, you need to count how many times an item appears in a list or how many occurrences of a value exist in a dataset. Python offers simple ways to do this.

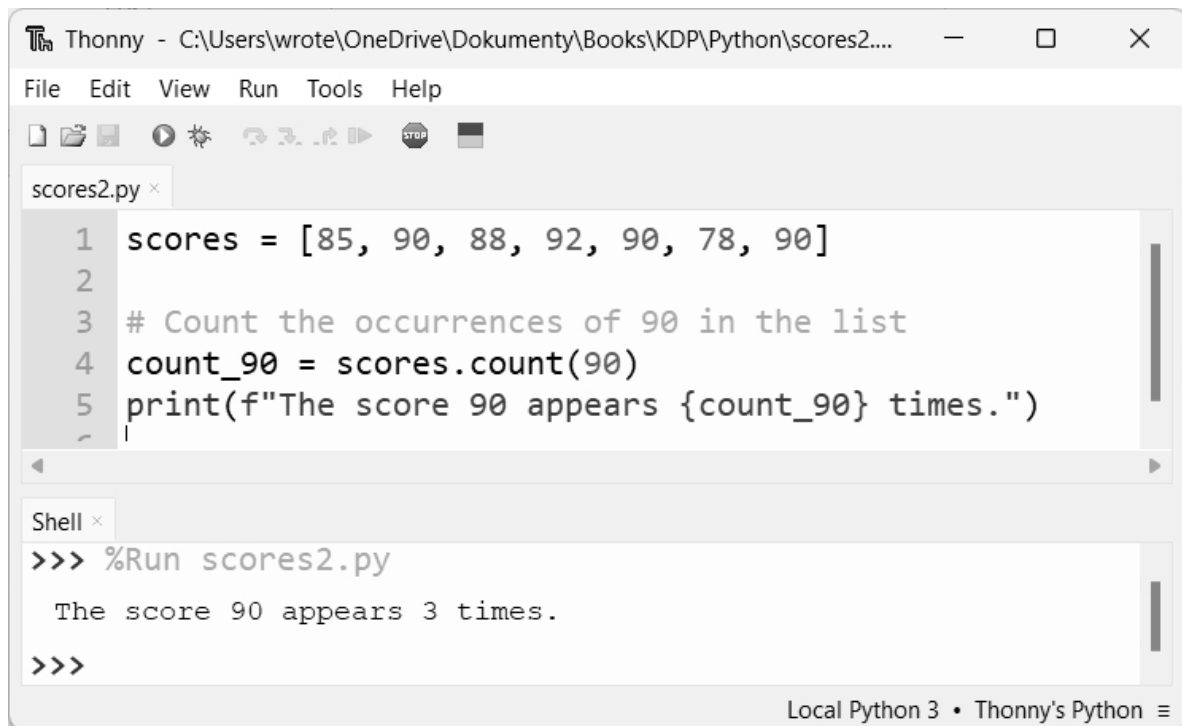
Counting in Lists

Let's say you have a list of test scores, and you want to count how many students scored 90:

```
scores = [85, 90, 88, 92, 90, 78, 90]

# Count the occurrences of 90 in the list
count_90 = scores.count(90)

print(f"The score 90 appears {count_90} times.")
```

The image shows a screenshot of the Thonny Python IDE. The main window displays a Python script named 'scores2.py' with the following code:

```
1 scores = [85, 90, 88, 92, 90, 78, 90]
2
3 # Count the occurrences of 90 in the list
4 count_90 = scores.count(90)
5 print(f"The score 90 appears {count_90} times.")
```

The script is executed, and the output is shown in the 'Shell' window at the bottom:

```
>>> %Run scores2.py
The score 90 appears 3 times.
>>>
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

Here, the `count()` method counts how many times the value 90 appears in the list. This is useful for tracking occurrences in data, such as finding how often a particular number or item is repeated.

Counting with Dictionaries

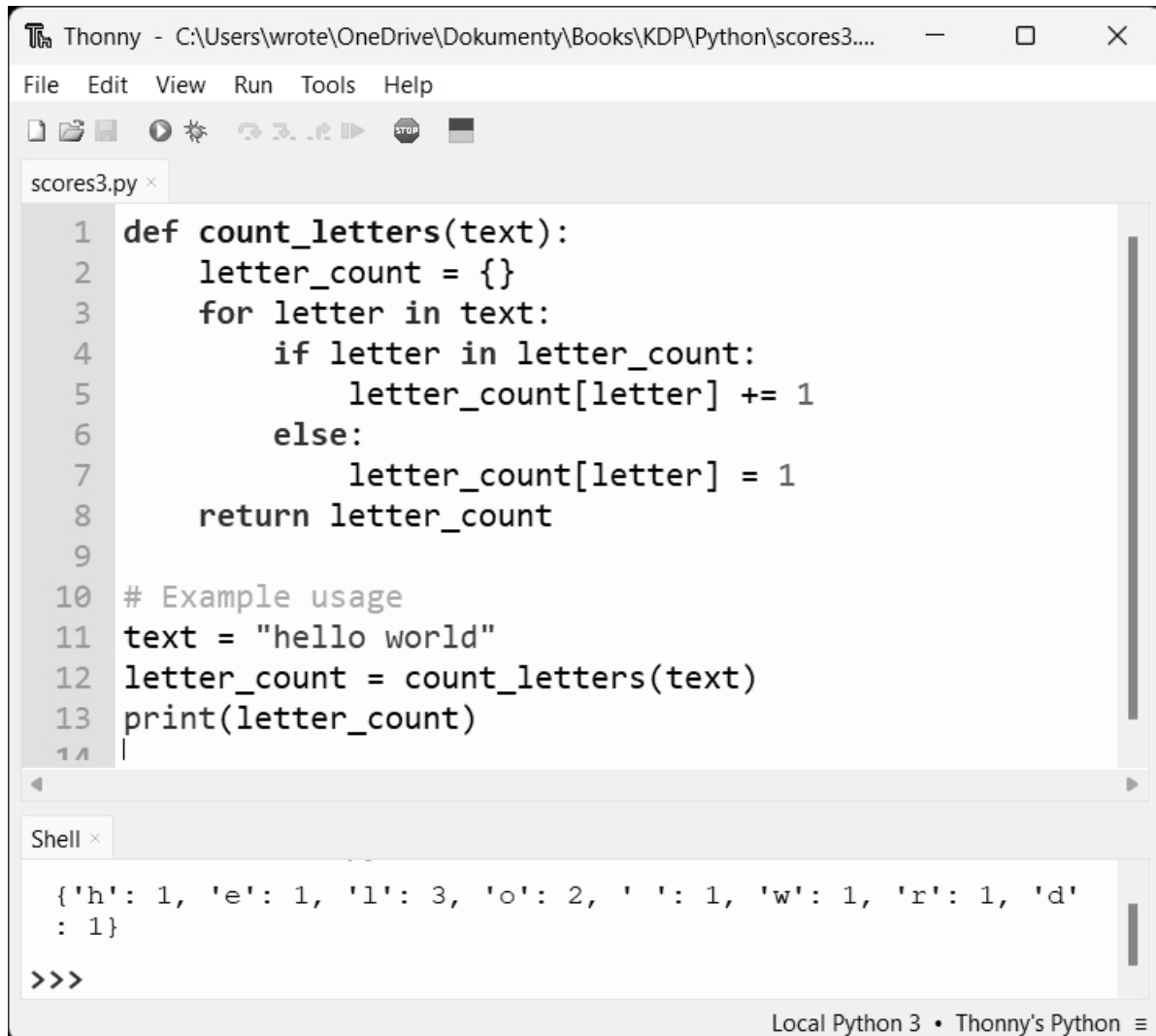
Dictionaries are even more powerful for counting because they allow you to store items as **keys** and their counts as **values**. Let's say you want to count how often each letter appears in a string:

```
def count_letters(text):
    letter_count = {}
    for letter in text:
        if letter in letter_count:
            letter_count[letter] += 1
        else:
            letter_count[letter] = 1
    return letter_count

# Example usage
text = "hello world"
letter_count = count_letters(text)
```

```
print(letter_count)
```

Output:



The screenshot shows the Thonny Python IDE interface. The top window, titled 'scores3.py', contains the following Python code:

```
1 def count_letters(text):
2     letter_count = {}
3     for letter in text:
4         if letter in letter_count:
5             letter_count[letter] += 1
6         else:
7             letter_count[letter] = 1
8     return letter_count
9
10 # Example usage
11 text = "hello world"
12 letter_count = count_letters(text)
13 print(letter_count)
14
```

The bottom window, titled 'Shell', displays the output of the script:

```
{'h': 1, 'e': 1, 'l': 3, 'o': 2, ' ': 1, 'w': 1, 'r': 1, 'd': 1}
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

In this example, we loop through each letter in the string. If the letter is already in the dictionary, we increase its count by 1. Otherwise, we add the letter to the dictionary with an initial count of 1.

2. Sorting Data in Python

Sorting is a common task when working with lists and dictionaries, especially when you want to organize data in a specific order, like arranging names alphabetically or scores from highest to lowest.

Sorting a List

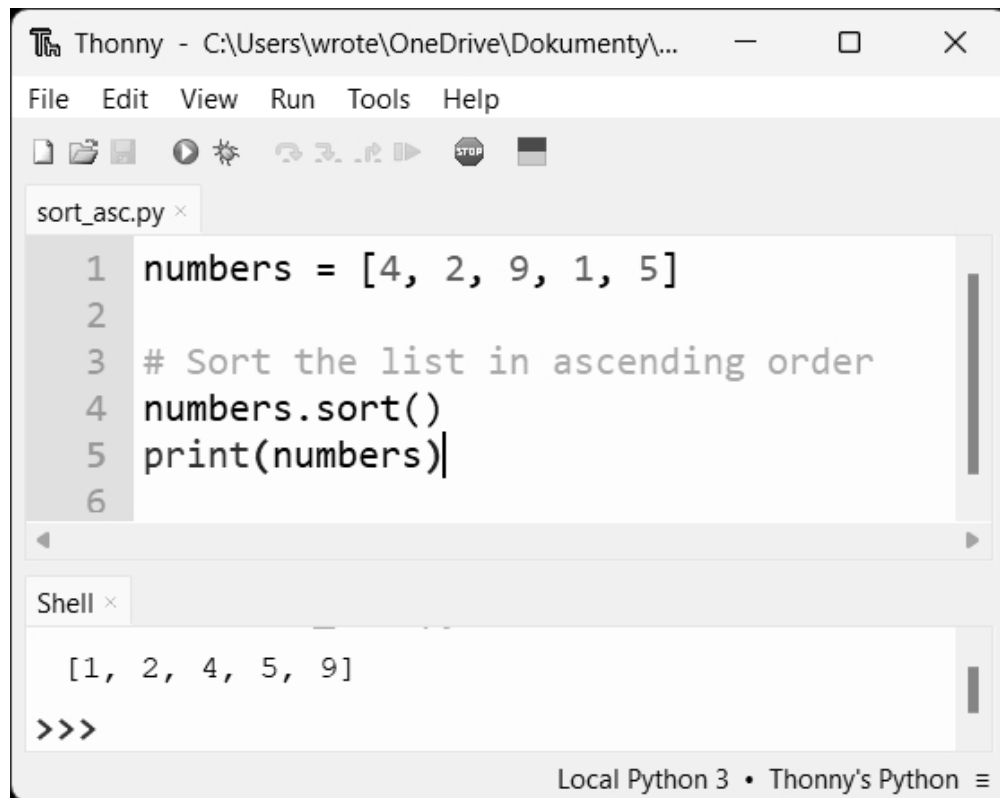
You can sort lists easily using Python's built-in `sort()` method. By default, this will sort the list in ascending order:

```
numbers = [4, 2, 9, 1, 5]
```

```
# Sort the list in ascending order
```

```
numbers.sort()
```

```
print(numbers)
```



The screenshot shows the Thonny Python IDE interface. The main editor window displays a Python script named `sort_asc.py` with the following code:

```
1 numbers = [4, 2, 9, 1, 5]
2
3 # Sort the list in ascending order
4 numbers.sort()
5 print(numbers)
6
```

Below the editor is a Shell window showing the output of the script:

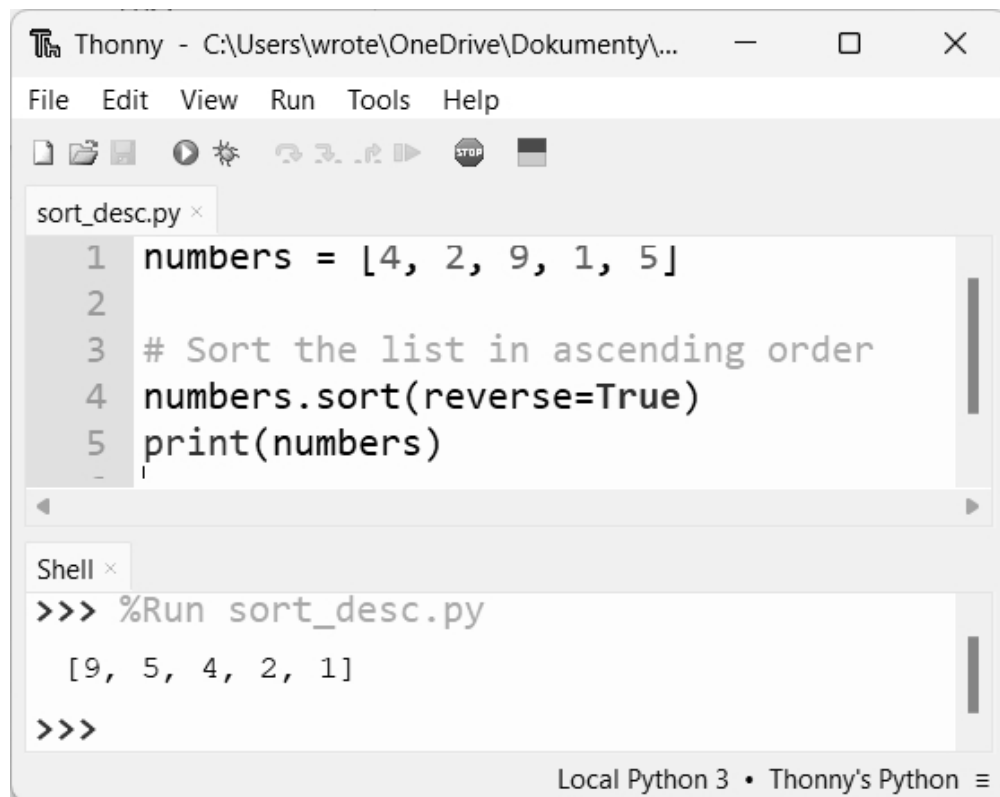
```
[1, 2, 4, 5, 9]
>>>
```

The status bar at the bottom indicates "Local Python 3 • Thonny's Python".

If you want to sort in descending order, you can add the `reverse=True` argument:

```
numbers.sort(reverse=True)
```

```
print(numbers)
```



The screenshot shows the Thonny Python IDE interface. The top window, titled 'sort_desc.py', contains the following Python code:

```
1 numbers = [4, 2, 9, 1, 5]
2
3 # Sort the list in ascending order
4 numbers.sort(reverse=True)
5 print(numbers)
```

The bottom window, titled 'Shell', shows the execution of the script:

```
>>> %Run sort_desc.py
[9, 5, 4, 2, 1]
>>>
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

Sorting a List of Strings

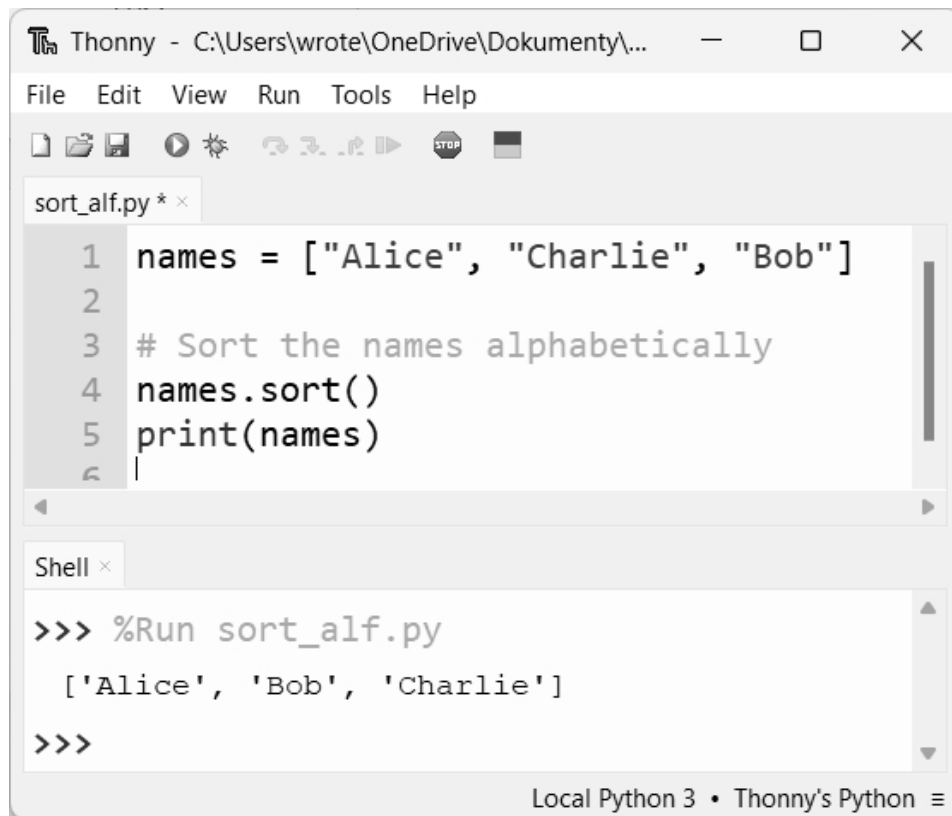
You can also sort lists of strings alphabetically:

```
names = ["Alice", "Charlie", "Bob"]
```

```
# Sort the names alphabetically
```

```
names.sort()
```

```
print(names)
```



The screenshot shows the Thonny Python IDE interface. The top window, titled 'sort_alf.py *', contains the following Python code:

```
1 names = ["Alice", "Charlie", "Bob"]
2
3 # Sort the names alphabetically
4 names.sort()
5 print(names)
```

Below the code editor is a 'Shell' window showing the execution of the script:

```
>>> %Run sort_alf.py
['Alice', 'Bob', 'Charlie']
>>>
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

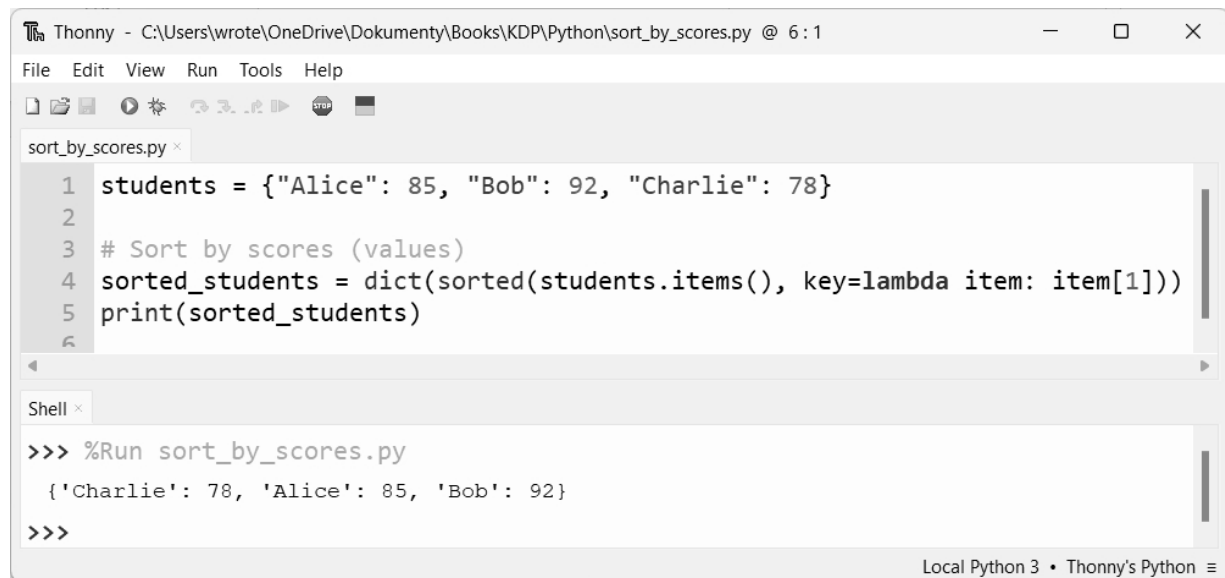
Sorting Dictionaries

To sort dictionaries, you need to decide whether to sort by **keys** or **values**. Python's `sorted()` function can help with this.

Let's say you have a dictionary of students and their scores, and you want to sort it by score:

```
students = {"Alice": 85, "Bob": 92, "Charlie": 78}

# Sort by scores (values)
sorted_students = dict(sorted(students.items(), key=lambda item: item[1]))
print(sorted_students)
```

The screenshot shows the Thonny Python IDE interface. The top window, titled 'sort_by_scores.py', contains the following Python code:

```
1 students = {"Alice": 85, "Bob": 92, "Charlie": 78}
2
3 # Sort by scores (values)
4 sorted_students = dict(sorted(students.items(), key=lambda item: item[1]))
5 print(sorted_students)
6
```

The bottom window, titled 'Shell', shows the execution output:

```
>>> %Run sort_by_scores.py
{'Charlie': 78, 'Alice': 85, 'Bob': 92}
>>>
```

The status bar at the bottom right indicates 'Local Python 3 • Thonny's Python'.

In this example, the `sorted()` function sorts the dictionary by the values (scores). The lambda function tells Python to sort using the second element of each key-value pair (the score).

3. Organizing Data in Python

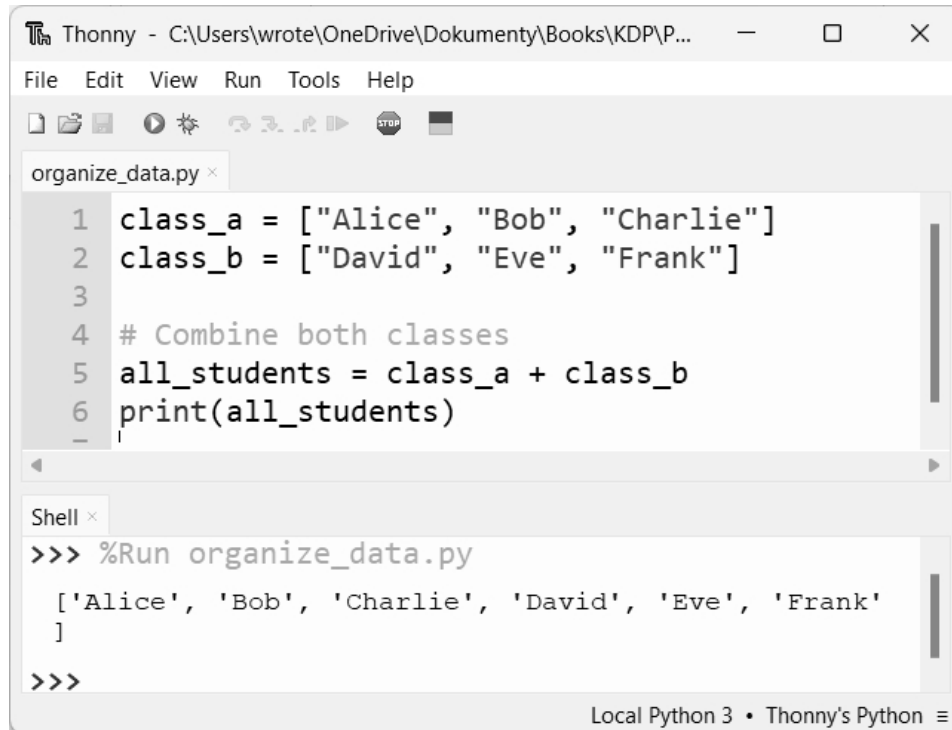
When you're dealing with large sets of data, organizing it effectively is essential. Python provides several tools to help you structure and manage your data.

Using Lists to Organize Data

Let's say you're organizing a group of students into separate classes. You can use lists to store each class:

```
class_a = ["Alice", "Bob", "Charlie"]
class_b = ["David", "Eve", "Frank"]

# Combine both classes
all_students = class_a + class_b
print(all_students)
```

The image shows a screenshot of the Thonny Python IDE. The main window displays a file named 'organize_data.py' with the following Python code:

```
1 class_a = ["Alice", "Bob", "Charlie"]
2 class_b = ["David", "Eve", "Frank"]
3
4 # Combine both classes
5 all_students = class_a + class_b
6 print(all_students)
```

The bottom panel, labeled 'Shell', shows the execution of the script with the command '>>> %Run organize_data.py' and the output:

```
['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank']
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

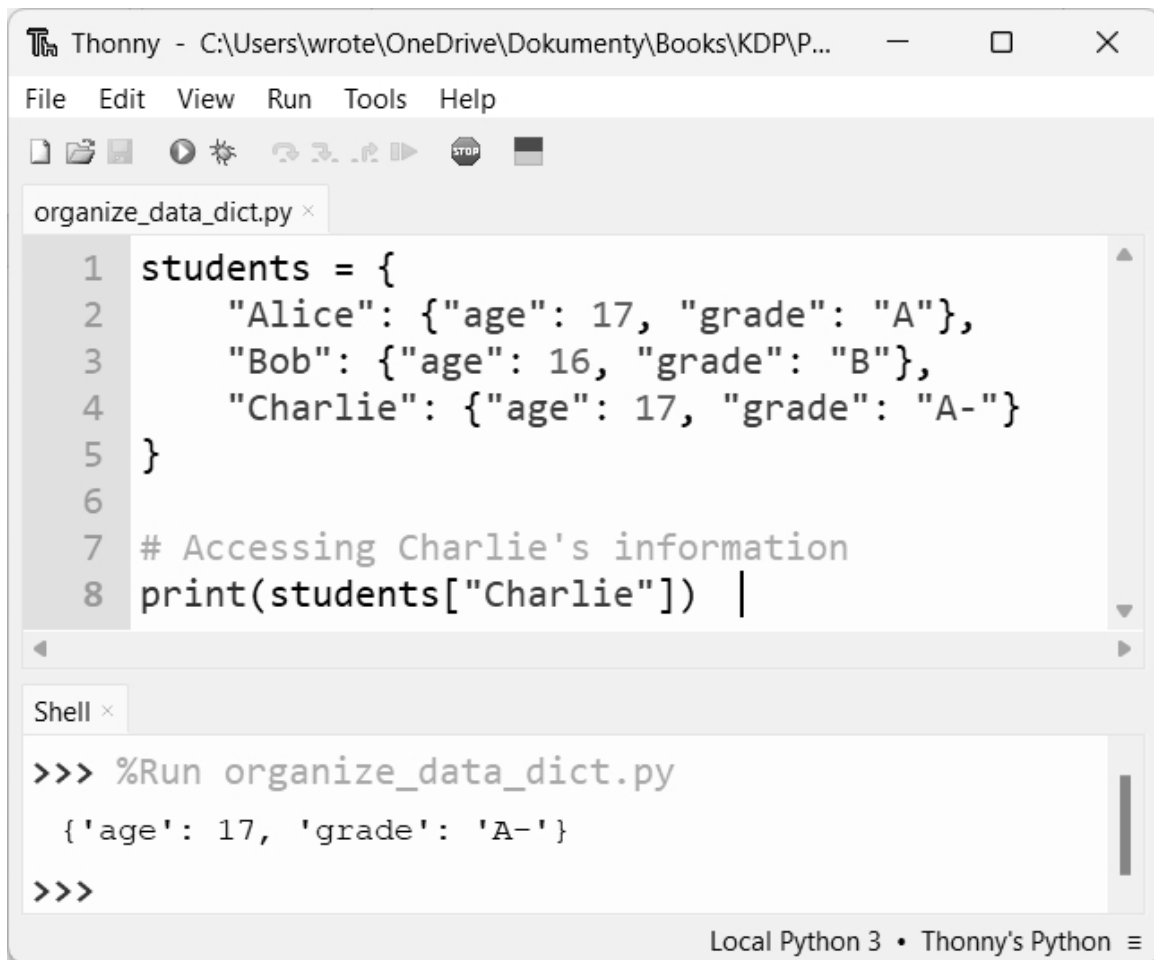
By combining the lists with the + operator, you can merge data from different categories or groups.

Using Dictionaries to Organize Data

Dictionaries are ideal for organizing data with labels or categories. For example, you can use a dictionary to store student information, like their name, age, and grade:

```
students = {
    "Alice": {"age": 17, "grade": "A"},
    "Bob": {"age": 16, "grade": "B"},
    "Charlie": {"age": 17, "grade": "A-"}
}
```

```
# Accessing Charlie's information
print(students["Charlie"])
```

The screenshot shows the Thonny Python IDE interface. The top window, titled 'organize_data_dict.py', contains the following Python code:

```
1 students = {  
2     "Alice": {"age": 17, "grade": "A"},  
3     "Bob": {"age": 16, "grade": "B"},  
4     "Charlie": {"age": 17, "grade": "A-"}  
5 }  
6  
7 # Accessing Charlie's information  
8 print(students["Charlie"]) |
```

The bottom window, titled 'Shell', shows the execution output:

```
>>> %Run organize_data_dict.py  
{'age': 17, 'grade': 'A-'}  
>>>
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

In this case, the dictionary allows you to organize related data for each student in a structured way.

4. Practical Example: Organizing and Sorting Contact Information

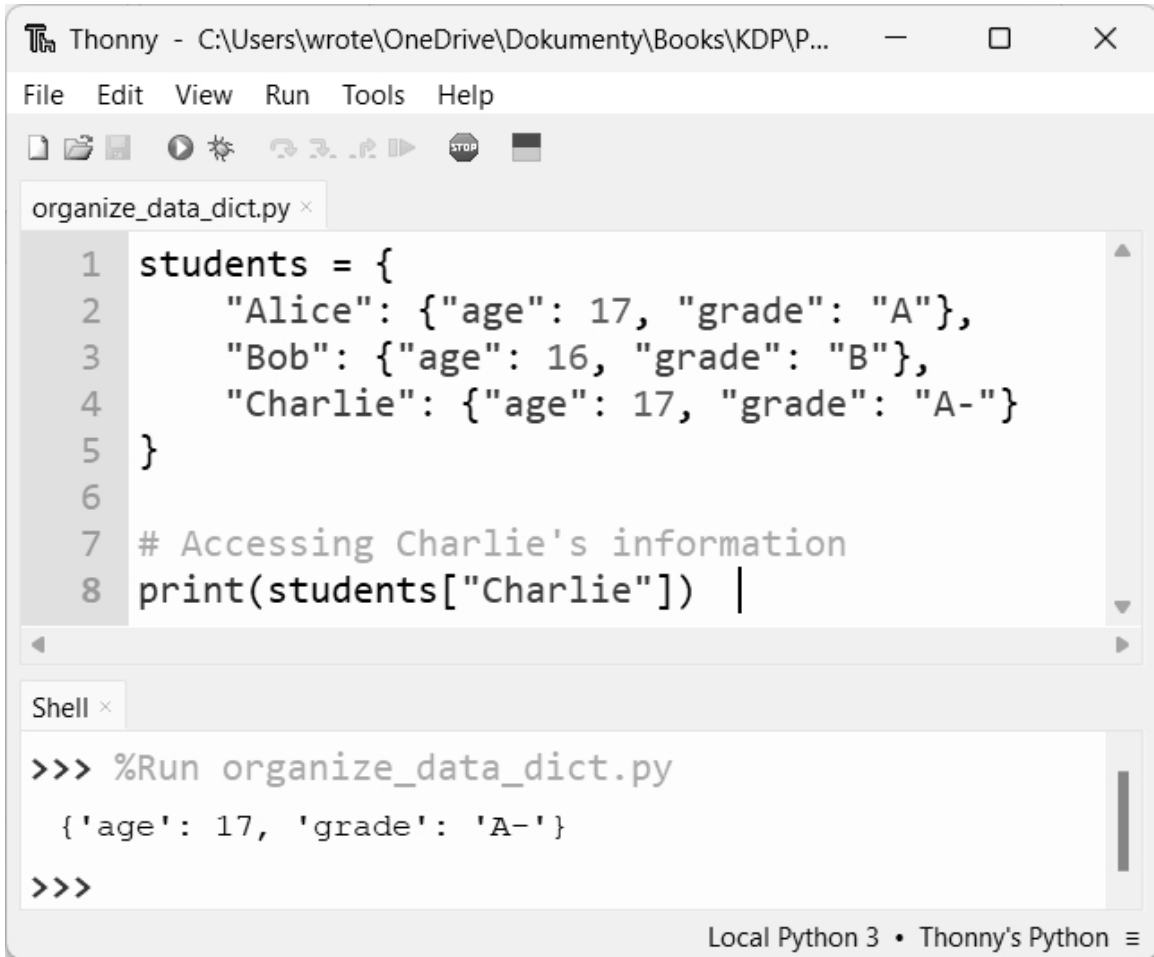
Let's bring everything together by solving a real-world problem. Imagine you are managing contact information for your friends, and you want to organize their names and phone numbers alphabetically.

Here's how you can do that using a dictionary:

```
# Create a dictionary of contacts  
contacts = {  
    "Alice": "555-1234",  
    "Charlie": "555-5678",  
    "Bob": "555-8765"  
}
```

```
# Sort the contacts alphabetically by name
sorted_contacts = dict(sorted(contacts.items()))
print("Sorted Contacts:", sorted_contacts)
```

Output:

The image shows a screenshot of the Thonny Python IDE. The main window displays a Python script named 'organize_data_dict.py'. The script defines a dictionary 'students' with three entries: 'Alice' (age 17, grade 'A'), 'Bob' (age 16, grade 'B'), and 'Charlie' (age 17, grade 'A-'). It then prints the information for 'Charlie'. Below the script, the 'Shell' window shows the command '%Run organize_data_dict.py' and the output {'age': 17, 'grade': 'A-'}. The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

In this example, we use `sorted()` to sort the contact names alphabetically, which helps in organizing the data in a neat and readable way.

5. Why Counting, Sorting, and Organizing Data Is Important

- **Counting:** Helps you track occurrences and analyze data, such as counting how many times certain items appear in a list.
- **Sorting:** Organizes data into meaningful orders, making it easier to find and analyze. For example, sorting scores from highest to lowest can quickly show you top performers.

- **Organizing:** Helps you structure your data efficiently, making it easier to access, modify, and manage as your programs and datasets grow larger.

Summary

Counting, sorting, and organizing data are fundamental tasks in programming and data management. Whether you're analyzing test scores, sorting lists of names, or organizing large amounts of information, Python provides a variety of tools to help you handle data efficiently. Mastering these techniques will allow you to tackle more complex projects with confidence, making you a more effective problem solver in the world of programming. Keep practicing, and soon these data-handling skills will become second nature!

Application: Managing a Simple Contact Book with Python

A **contact book** is a perfect project to apply your skills in managing data using Python's lists and dictionaries. Whether you want to store names, phone numbers, or email addresses, Python makes it easy to organize and retrieve this information efficiently. In this section, we'll walk through how to create a simple contact book that lets you store, search, and update contact information.

By the end of this project, you'll have a fully functioning contact management system — a real-world tool you can build on and customize!

1. Defining the Contact Book Structure

To start, we'll need a way to store the contact information. A **dictionary** is the perfect tool for this. In the contact book, the **name** will be the **key**, and the **phone number** (or other contact details) will be the **value**. Here's how you can define the basic structure of the contact book:

```
# Dictionary to store contacts
contacts = {}
```

At this point, our contact book is an empty dictionary. We'll add functionality to **add** contacts, **view** them, **update** information, and **delete** contacts as needed.

2. Adding a New Contact

We'll start by writing a function to add new contacts. The user will input a name and a phone number, and we'll store this information in the dictionary.

```
def add_contact(name, phone):
    if name in contacts:
        print(f'{name} is already in your contact book.')
    else:
        contacts[name] = phone
        print(f'{name} has been added with phone number {phone}.')
```

Explanation:

- We first check if the contact already exists using if name in contacts. If it does, we notify the user.
- If the contact doesn't exist, we add it to the dictionary with the name as the key and the phone number as the value.

Example Usage:

```
add_contact("Alice", "555-1234")
```

```
add_contact("Bob", "555-5678")
```

Output:

Alice has been added with phone number 555-1234.

Bob has been added with phone number 555-5678.

3. Viewing All Contacts

Next, we'll add functionality to view all the contacts stored in the contact book. This will loop through the dictionary and display the name and phone number of each contact.

```
def view_contacts():  
    if not contacts:  
        print("Your contact book is empty.")  
    else:  
        for name, phone in contacts.items():  
            print(f"Name: {name}, Phone: {phone}")
```

Explanation:

- We check if the contacts dictionary is empty. If it is, we notify the user that no contacts exist.
- If there are contacts, we loop through the dictionary using contacts.items() to display each contact's name and phone number.

Example Usage:

```
view_contacts()
```

Output:

Name: Alice, Phone: 555-1234

Name: Bob, Phone: 555-5678

4. Searching for a Contact

To make it easy to find a contact, we'll add a search function. This function will take a name as input and return the corresponding phone number if the contact exists.

```
def search_contact(name):  
    if name in contacts:  
        print(f'Phone number for {name}: {contacts[name]}')  
    else:  
        print(f'{name} is not found in your contact book.')
```

Explanation:

- The function checks if the given name exists in the contacts dictionary.
- If the name is found, it displays the phone number. If not, it notifies the user that the contact is not in the book.

Example Usage:

```
search_contact("Alice")
```

Output:

Phone number for Alice: 555-1234

5. Updating a Contact

If a contact's phone number changes, we'll need a way to update their information. Here's a function that allows you to update a contact's phone number:

```
def update_contact(name, new_phone):  
    if name in contacts:  
        contacts[name] = new_phone  
        print(f'{name}'s phone number has been updated to {new_phone}.')  
    else:
```

```
print(f'{name} is not found in your contact book.')
```

Explanation:

- If the contact exists, the function updates the phone number with the new value.
- If the contact doesn't exist, it notifies the user that the contact isn't in the book.

Example Usage:

```
update_contact("Alice", "555-9999")
```

Output:

Alice's phone number has been updated to 555-9999.

6. Deleting a Contact

Sometimes, you may want to remove a contact. We can write a function to delete a contact from the dictionary:

```
def delete_contact(name):  
    if name in contacts:  
        del contacts[name]  
        print(f'{name} has been deleted from your contact book.')  
    else:  
        print(f'{name} is not found in your contact book.')
```

Explanation:

- If the contact exists, the function uses del to remove it from the dictionary.
- If the contact doesn't exist, it notifies the user.

Example Usage:

```
delete_contact("Bob")
```

Output:

Bob has been deleted from your contact book.

7. Putting It All Together: A Simple Contact Book System

Now that we have functions to add, view, search, update, and delete contacts, let's combine them into a simple menu-driven system that allows the user to manage their contact book:

```
# Dictionary to store contacts
contacts = {}

def add_contact(name, phone):
    if name in contacts:
        print(f'{name} is already in your contact book.')
    else:
        contacts[name] = phone
        print(f'{name} has been added with phone number {phone}.')

def view_contacts():
    if not contacts:
        print("Your contact book is empty.")
    else:
        for name, phone in contacts.items():
            print(f'Name: {name}, Phone: {phone}')

def search_contact(name):
    if name in contacts:
        print(f'Phone number for {name}: {contacts[name]}')
    else:
        print(f'{name} is not found in your contact book.')

def update_contact(name, new_phone):
    if name in contacts:
        contacts[name] = new_phone
        print(f'{name}'s phone number has been updated to {new_phone}.')
    else:
        print(f'{name} is not found in your contact book.')

def delete_contact(name):
    if name in contacts:
        del contacts[name]
```



```

        print(f"{name} has been deleted from your contact book.")
    else:
        print(f"{name} is not found in your contact book.")

def menu():
    while True:
        print("\n--- Contact Book Menu ---")
        print("1. Add contact")
        print("2. View all contacts")
        print("3. Search contact")
        print("4. Update contact")
        print("5. Delete contact")
        print("6. Quit")

        choice = input("Choose an option: ")

        if choice == "1":
            name = input("Enter the contact's name: ")
            phone = input("Enter the contact's phone number: ")
            add_contact(name, phone)
        elif choice == "2":
            view_contacts()
        elif choice == "3":
            name = input("Enter the name to search: ")
            search_contact(name)
        elif choice == "4":
            name = input("Enter the name to update: ")
            new_phone = input("Enter the new phone number: ")
            update_contact(name, new_phone)
        elif choice == "5":
            name = input("Enter the name to delete: ")
            delete_contact(name)
        elif choice == "6":
            print("Exiting Contact Book. Goodbye!")
            break

```

```
else:  
    print("Invalid choice. Please try again.")
```

```
# Run the contact book menu  
menu()
```

Explanation:

- The menu() function provides a simple interface where the user can choose an option to add, view, search, update, or delete contacts.
- The user can exit the program by selecting the "Quit" option.

Example Output:

```
--- Contact Book Menu ---
```

1. Add contact
2. View all contacts
3. Search contact
4. Update contact
5. Delete contact
6. Quit

```
Choose an option: 2
```

```
Your contact book is empty.
```

```
--- Contact Book Menu ---
```

1. Add contact
2. View all contacts
3. Search contact
4. Update contact
5. Delete contact
6. Quit

```
Choose an option: 1
```

```
Enter the contact's name: Alice
```

```
Enter the contact's phone number: 111-222
```

```
Alice has been added with phone number 111-222.
```

--- Contact Book Menu ---

1. Add contact
2. View all contacts
3. Search contact
4. Update contact
5. Delete contact
6. Quit

Choose an option: 2

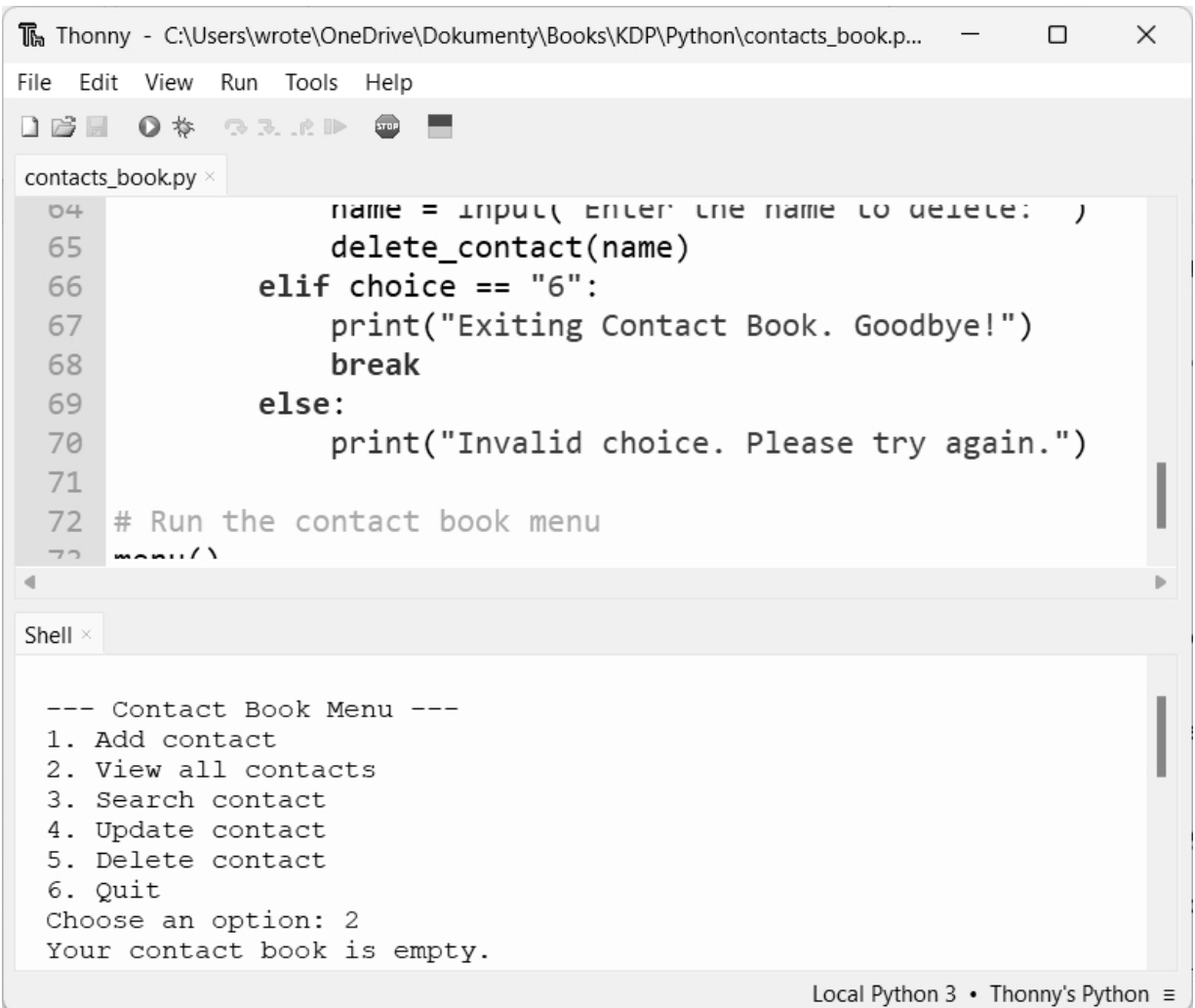
Name: Alice, Phone: 111-222

--- Contact Book Menu ---

1. Add contact
2. View all contacts
3. Search contact
4. Update contact
5. Delete contact
6. Quit

Choose an option: 6

Exiting Contact Book. Goodbye!



```
Thonny - C:\Users\wrote\OneDrive\Dokumenty\Books\KDP\Python\contacts_book.p...
File Edit View Run Tools Help
contacts_book.py x
64         name = input( Enter the name to delete: )
65         delete_contact(name)
66     elif choice == "6":
67         print("Exiting Contact Book. Goodbye!")
68         break
69     else:
70         print("Invalid choice. Please try again.")
71
72 # Run the contact book menu
73 menu()
```

```
Shell x
--- Contact Book Menu ---
1. Add contact
2. View all contacts
3. Search contact
4. Update contact
5. Delete contact
6. Quit
Choose an option: 2
Your contact book is empty.
```

Local Python 3 • Thonny's Python

8. Why Build a Contact Book?

Building a contact book in Python is a fantastic way to practice data handling with dictionaries. This project helps you:

- Understand how to store and retrieve data using key-value pairs.
- Build functions that interact with users and handle data dynamically.
- Develop an interactive program that you can expand with more features.

You can customize this contact book further by adding features like email addresses, sorting contacts alphabetically, or even saving the contact information to a file so it persists between program runs.

Summary

Managing a simple contact book with Python shows the practical power of using dictionaries to store and organize data efficiently. With the ability to add, view, search, update, and delete contacts, you've built a real-world application that's both useful and easy to manage. As you continue to practice and build more projects, you'll find these skills invaluable for handling larger datasets and more complex problems.

Chapter 7: File Handling: Reading, Writing, and Managing Files

Reading from and Writing to Text Files

In many real-world applications, programs need to work with data stored in **files**. Whether you're reading data from a file or saving information for future use, file handling is an essential skill in programming. Python makes it easy to **read** from and **write** to text files, enabling you to store data permanently, process large datasets, or even log events and results.

In this section, we'll explore how to work with text files in Python, focusing on practical, easy-to-understand examples that show how reading and writing files can solve real problems.

1. Opening a File

Before you can read from or write to a file, you need to **open** it. Python provides the `open()` function to handle this. When you open a file, you specify:

- The **filename** (the name of the file you want to work with).
- The **mode** (whether you want to read from, write to, or append to the file).

Here's the basic syntax for opening a file:

```
file = open("example.txt", "r") # Opens the file in read mode
```

The "r" mode is for **reading**. Other common modes include:

- "w": **Write mode** (creates a new file or overwrites an existing one).

- **"a": Append mode** (adds to the end of an existing file without deleting its content).

2. Reading from a Text File

To read the contents of a file, you can use several methods depending on what you need:

- **read():** Reads the entire content of the file as a single string.
- **readline():** Reads one line at a time.
- **readlines():** Reads all the lines and returns them as a list of strings.

Here's how to read the contents of a file:

```
# Open the file in read mode
file = open("example.txt", "r")

# Read the entire content of the file
content = file.read()

# Print the content
print(content)

# Close the file after reading
file.close()
```

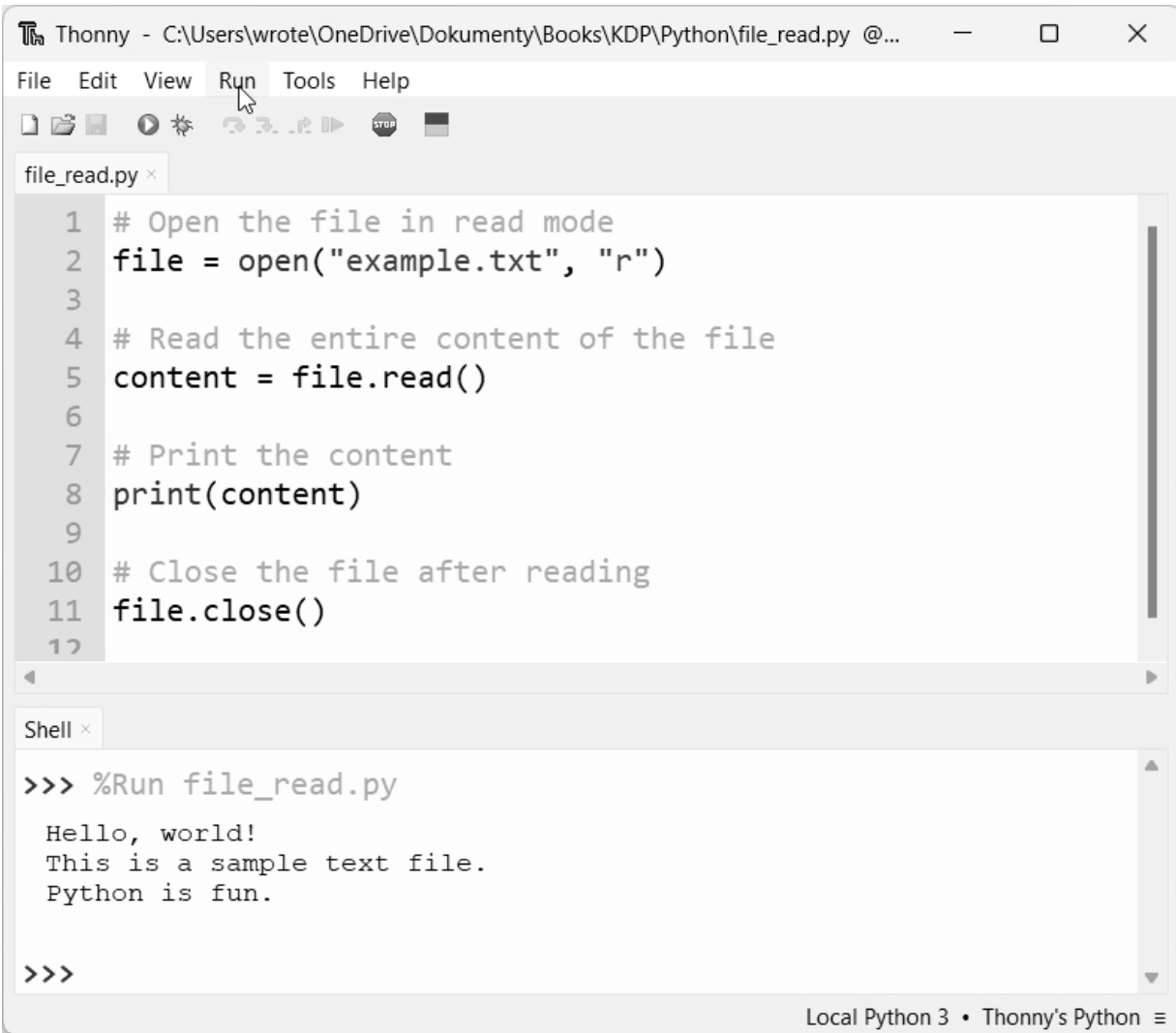
Explanation:

- `open("example.txt", "r")` opens the file in read mode.
- `file.read()` reads the entire file as a single string.
- After working with the file, you should always use `file.close()` to free up resources.

Example File Content (example.txt):

```
Hello, world!
This is a sample text file.
Python is fun.
```

Output:



The screenshot shows the Thonny Python IDE interface. The top window, titled 'file_read.py', contains the following Python code:

```
1 # Open the file in read mode
2 file = open("example.txt", "r")
3
4 # Read the entire content of the file
5 content = file.read()
6
7 # Print the content
8 print(content)
9
10 # Close the file after reading
11 file.close()
12
```

The bottom window, titled 'Shell', shows the output of running the script:

```
>>> %Run file_read.py
Hello, world!
This is a sample text file.
Python is fun.
>>>
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

Working with files in Python is an essential part of building real-world applications. Whether you need to read data from a file, save user information, or create logs, Python's file handling features make it easy to manage text files efficiently. By mastering reading and writing to files, you'll unlock the ability to store data permanently and process information from external sources, making your programs more versatile and powerful.

Keep practicing, and soon you'll be able to handle files like a pro, solving more complex data problems with ease!

Solving Real Problems with Data from Files

1. Reading Data from a Text File

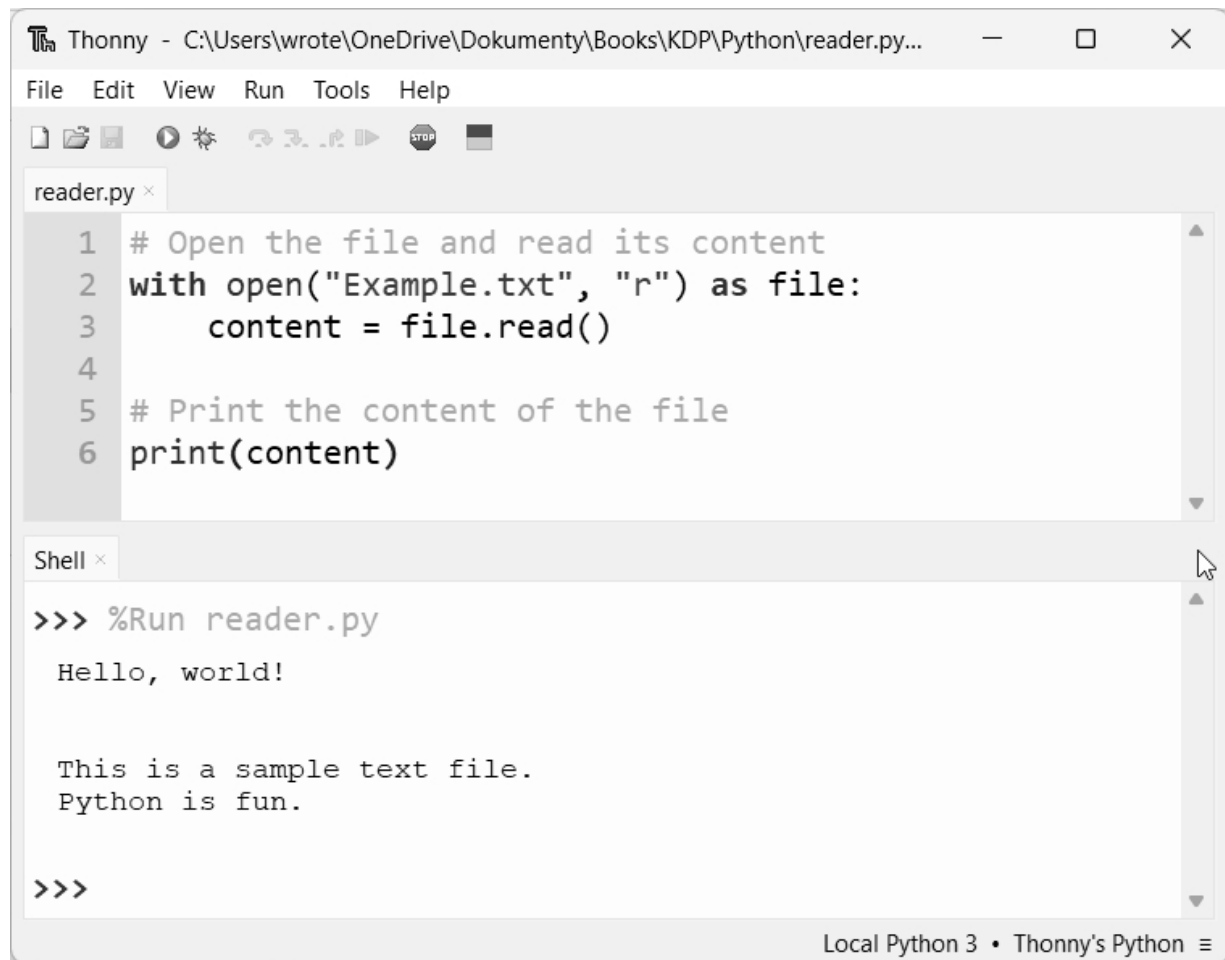
Here's a simple program to read and display the contents of a text file. This example assumes you have a file called example.txt.

Program to Read a File:

```
# Open the file and read its content
with open("example.txt", "r") as file:
    content = file.read()

# Print the content of the file
print(content)
```

This will read the entire content of example.txt and print it. Make sure the file is in the same folder as your Python script.



The screenshot shows the Thonny Python IDE interface. The top window, titled 'reader.py', contains the following Python code:

```
1 # Open the file and read its content
2 with open("Example.txt", "r") as file:
3     content = file.read()
4
5 # Print the content of the file
6 print(content)
```

The bottom window, titled 'Shell', shows the execution output:

```
>>> %Run reader.py
Hello, world!

This is a sample text file.
Python is fun.

>>>
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

2. Writing Data to a Text File

If you want to create a new file or write some text into an existing file, use the following program:

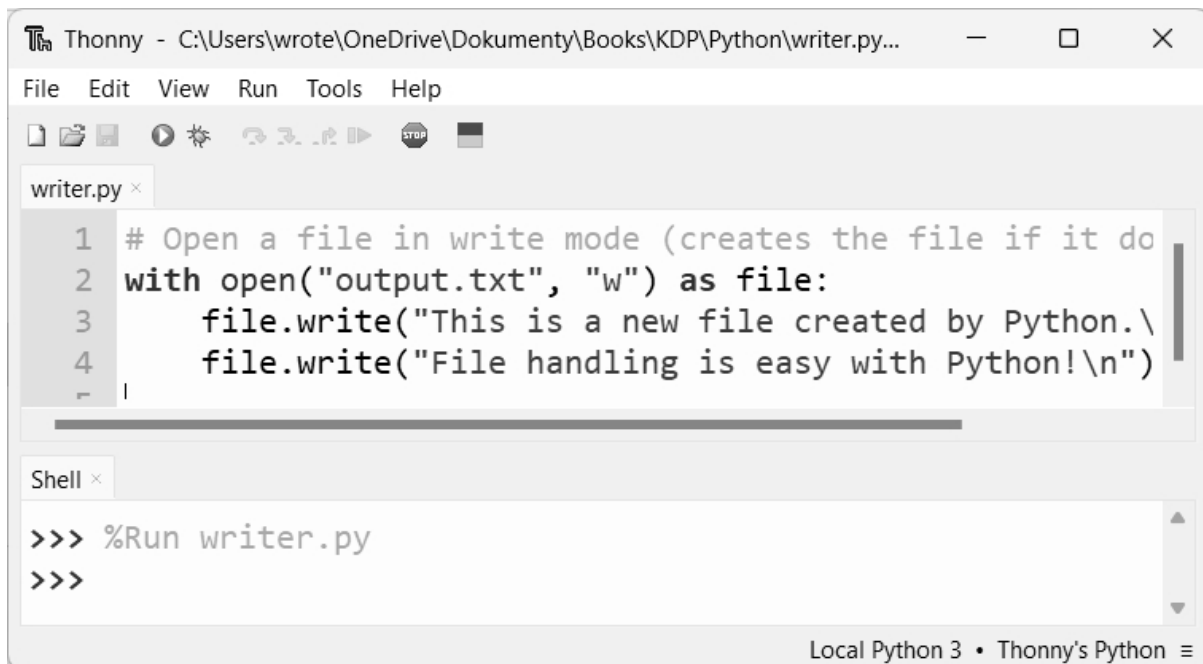
Program to Write to a File:

Open a file in write mode (creates the file if it doesn't exist)

with open("output.txt", "w") as file:

file.write("This is a new file created by Python.\n")

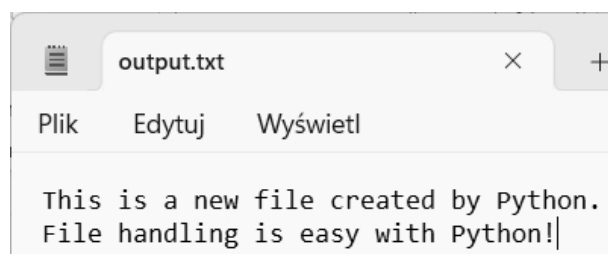
file.write("File handling is easy with Python!\n")



```
1 # Open a file in write mode (creates the file if it do
2 with open("output.txt", "w") as file:
3     file.write("This is a new file created by Python.\
4     file.write("File handling is easy with Python!\n")
|

Shell x
>>> %Run writer.py
>>>
```

This program creates a new file called output.txt and writes two lines of text into it.



3. Reading Data from a CSV File

If you have a CSV file (data.csv) with simple data like this:

Example CSV File (data.csv):

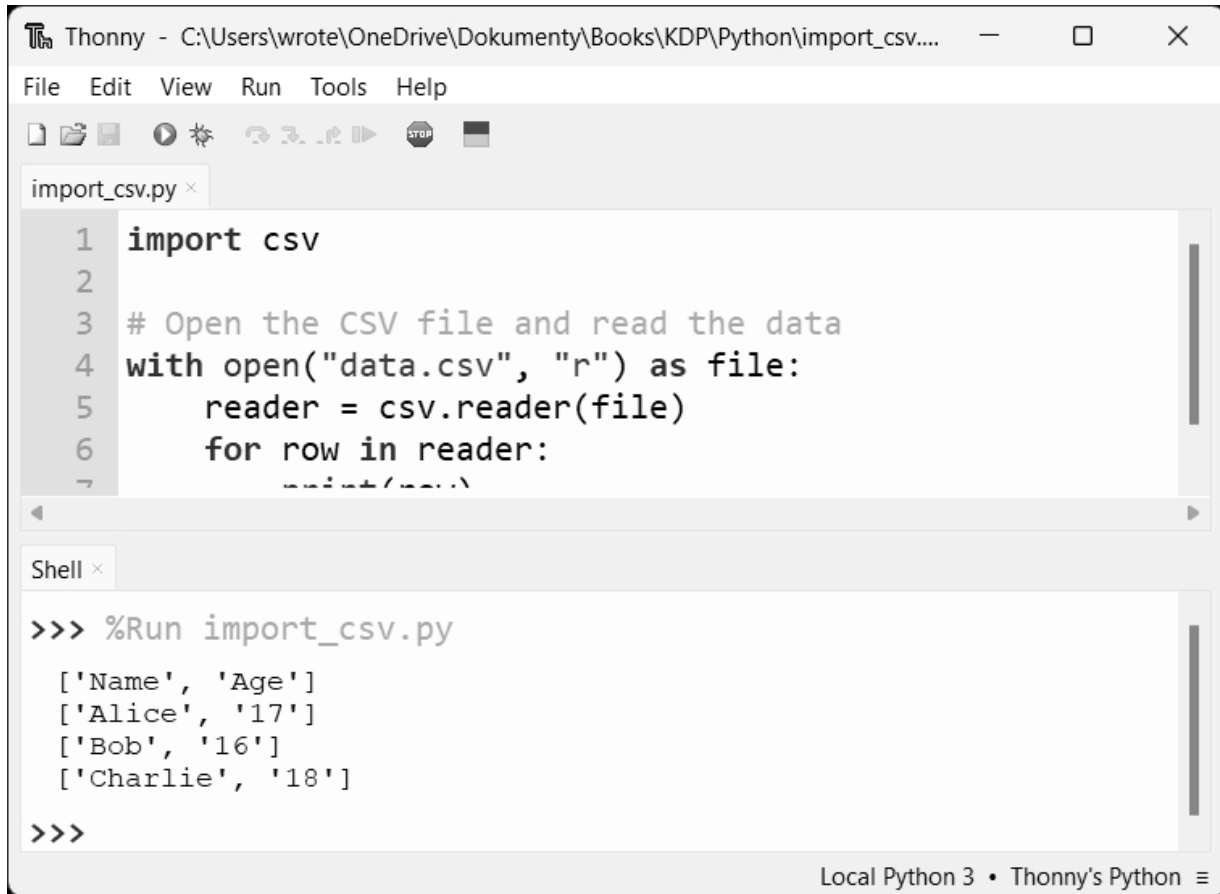
A1		Name, Age	
	A	B	C
1	Name, Age		
2	Alice, 17		
3	Bob, 16		
4	Charlie, 18		

Here's a simple program to read this data and print it:

Program to Read a CSV File:

```
import csv
# Open the CSV file and read the data
with open("data.csv", "r") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

This program will print each row of the CSV file as a list of values.

The screenshot shows the Thonny Python IDE interface. The top window, titled 'import_csv.py', contains the following Python code:

```
1 import csv
2
3 # Open the CSV file and read the data
4 with open("data.csv", "r") as file:
5     reader = csv.reader(file)
6     for row in reader:
7         print(row)
```

The bottom window, titled 'Shell', shows the execution of the script with the following output:

```
>>> %Run import_csv.py
['Name', 'Age']
['Alice', '17']
['Bob', '16']
['Charlie', '18']
>>>
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

4. Saving Processed Data to a File

If you want to write processed data to a file, such as saving the names of students who are older than 16, use this program:

Program to Save Processed Data:

```
# List of student names and ages
students = [("Alice", 17), ("Bob", 16), ("Charlie", 18)]

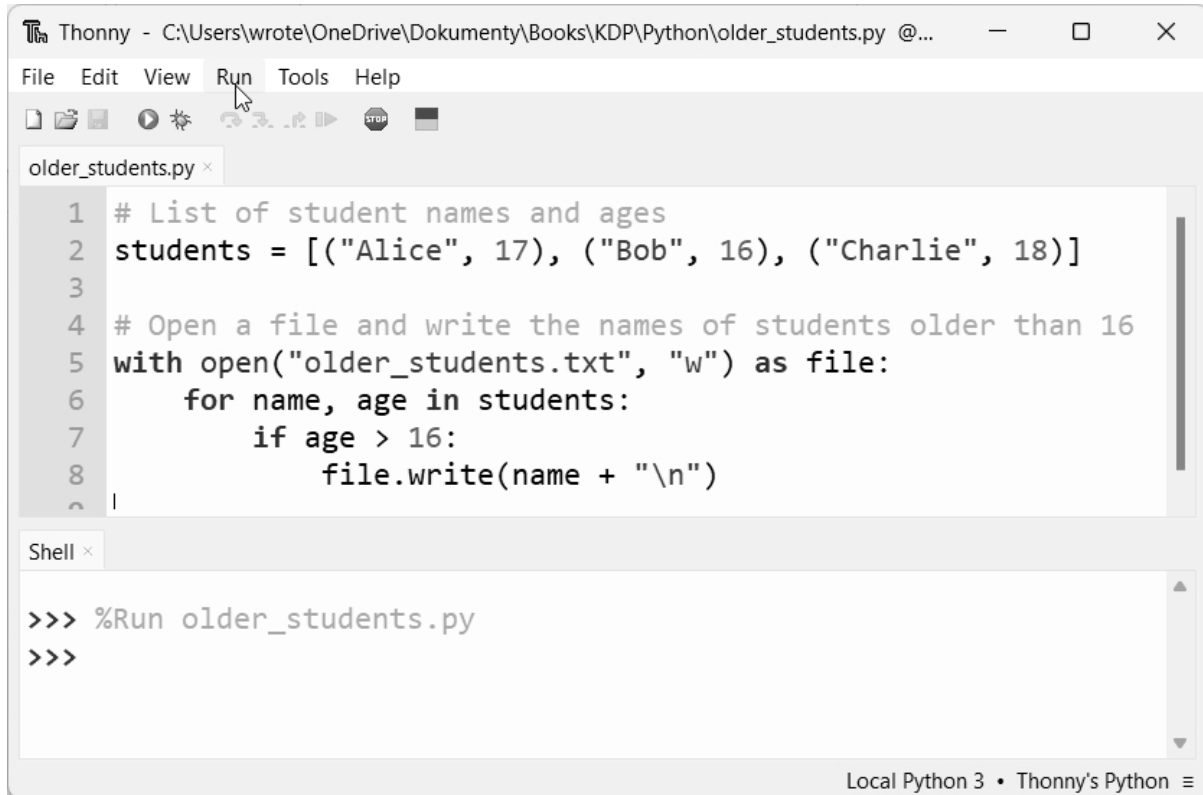
# Open a file and write the names of students older than 16
```

```
with open("older_students.txt", "w") as file:
```

```
    for name, age in students:
```

```
        if age > 16:
```

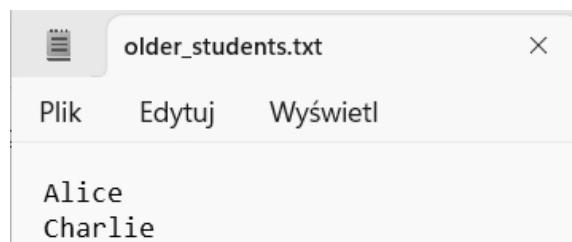
```
            file.write(name + "\n")
```



The screenshot shows the Thonny IDE interface. The top menu bar includes File, Edit, View, Run, Tools, and Help. Below the menu is a toolbar with icons for file operations and running code. The main editor window displays a Python script named `older_students.py`. The script defines a list of students and writes the names of those older than 16 to a file. The Shell window at the bottom shows the command `>>> %Run older_students.py` being executed, followed by a prompt `>>>`. The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

```
Thonny - C:\Users\wrote\OneDrive\Dokumenty\Books\KDP\Python\older_students.py @...
File Edit View Run Tools Help
older_students.py x
1 # List of student names and ages
2 students = [("Alice", 17), ("Bob", 16), ("Charlie", 18)]
3
4 # Open a file and write the names of students older than 16
5 with open("older_students.txt", "w") as file:
6     for name, age in students:
7         if age > 16:
8             file.write(name + "\n")
9
Shell x
>>> %Run older_students.py
>>>
Local Python 3 • Thonny's Python
```

This program writes the names of students older than 16 to a new file called `older_students.txt`.



5. Reading and Writing Data with a Simple Example

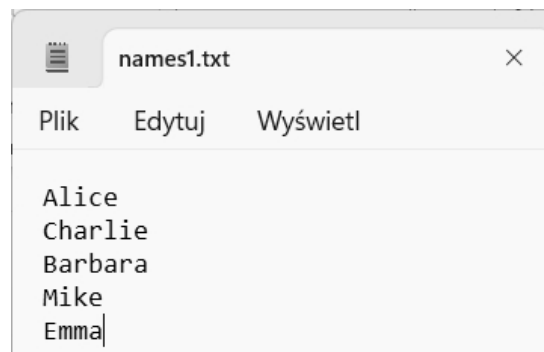
Let's combine reading and writing files with a practical example. Imagine you have a file `names.txt` with a list of names, and you want to create a new file that lists only names starting with the letter "A".

Program to Filter Names:

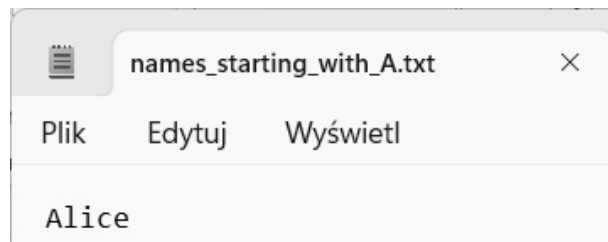
```
# Open the file with names
with open("names1.txt", "r") as file:
    names = file.readlines()

# Open a new file and write only names starting with "A"
with open("names_starting_with_A.txt", "w") as file:
    for name in names:
        if name.strip().startswith("A"):
            file.write(name)
```

This program reads names from names.txt,



checks if they start with "A", and writes those names into names_starting_with_A.txt.



These examples should be easy to follow and copy directly into your Python editor. Each program is simple and focused on one task. You can run them in your editor, see the results, and start learning file handling step by step!

Building a Simple Log System for Classroom Activities

In a classroom setting, keeping track of daily activities, attendance, or student progress can be a challenging task. By building a **simple log system** using Python, you can easily automate this process. This system will allow teachers or students to log activities, review past logs, and manage classroom events efficiently. In this section, we'll explore how to create a basic log system that saves data to a file and retrieves it whenever needed.

By the end of this section, you'll see how file handling can be applied to real-world situations in the classroom, helping to simplify tasks and keep track of important information.

1. Understanding the Purpose of a Log System

A log system is like a **journal** or **record book** that keeps track of events. In a classroom, this could be used for:

- **Logging daily activities:** Recording lessons, tasks, or events that occurred during the day.
- **Tracking attendance:** Marking which students were present or absent.
- **Saving notes or comments:** Recording important information for future reference.

Python's file handling capabilities make it easy to build a simple system that can save and retrieve logs from a file, allowing you to keep a permanent record of what happens in class.

2. Setting Up the Log System

The first step is to design how the log system will work:

- Users will be able to **add new logs** (such as activities or attendance).

- Users can **view previous logs** to see what was recorded on earlier days.
- All logs will be saved in a text file so that the data persists even after the program ends.

We'll start by defining the file where the logs will be stored:

```
LOG_FILE = "classroom_log.txt"
```

This file will be used to store all classroom activity logs.

3. Writing Logs to a File

When an activity or event occurs, we want to **record it** in the log file. This means taking input from the user (like a description of the activity) and appending it to the file, so that each log entry is stored in a new line.

Here's how you can write a log entry to the file:

```
def add_log_entry():  
    activity = input("Enter the activity or event to log: ")  
  
    with open(LOG_FILE, "a") as file:  
        file.write(activity + "\n")  
  
    print("Log entry added successfully.")
```

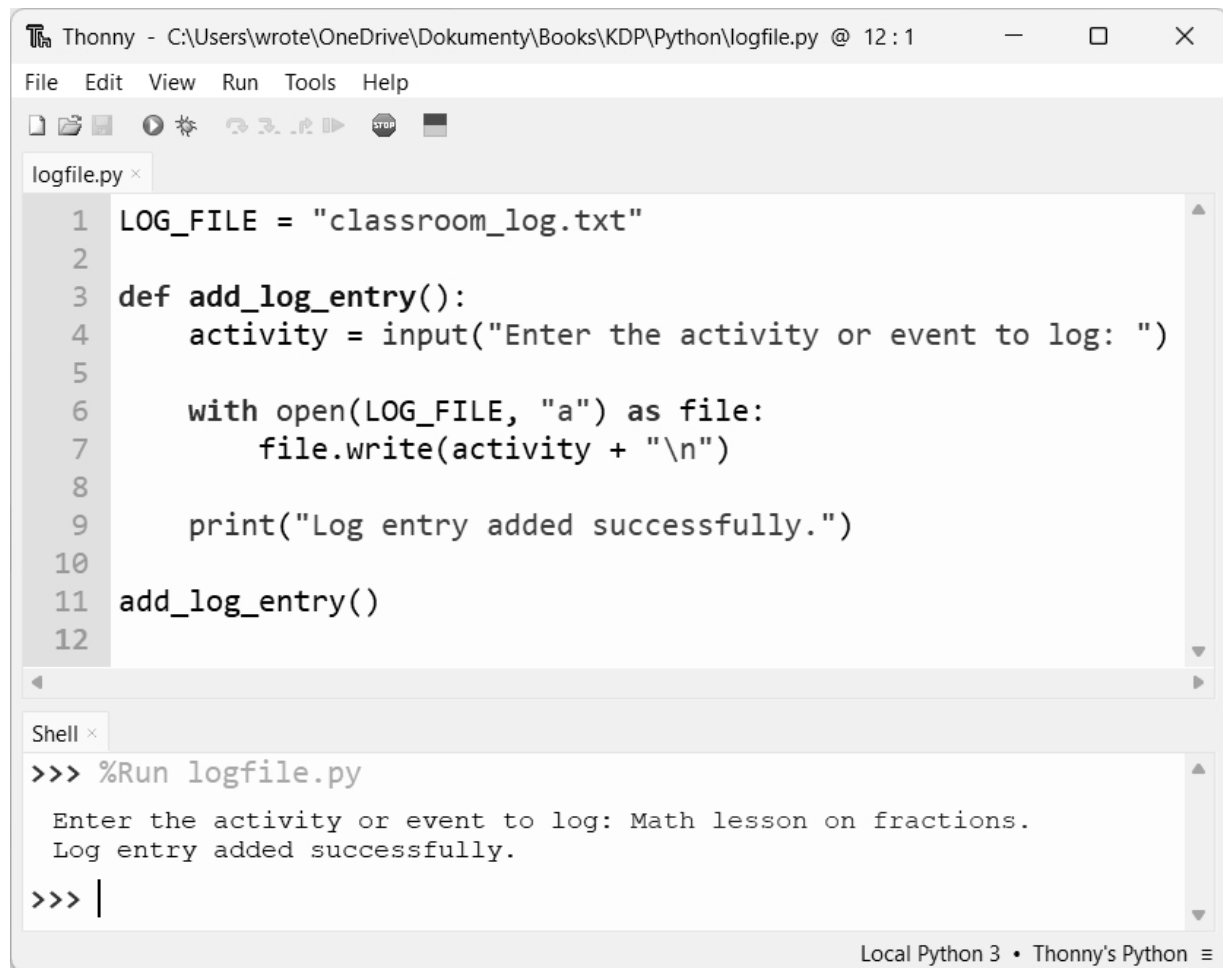
Explanation:

- The user is asked to input the activity or event they want to log.
- The log entry is appended to the file using the "a" (append) mode, so it is added to the end of the file without overwriting previous entries.
- After writing the log entry, we notify the user that it was added successfully.

Example Usage:

```
add_log_entry()
```

Output:



The screenshot shows the Thonny Python IDE interface. The top window, titled 'logfile.py', contains the following Python code:

```
1 LOG_FILE = "classroom_log.txt"
2
3 def add_log_entry():
4     activity = input("Enter the activity or event to log: ")
5
6     with open(LOG_FILE, "a") as file:
7         file.write(activity + "\n")
8
9     print("Log entry added successfully.")
10
11 add_log_entry()
12
```

The bottom window, titled 'Shell', shows the execution of the script:

```
>>> %Run logfile.py
Enter the activity or event to log: Math lesson on fractions.
Log entry added successfully.
>>> |
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

4. Reading and Displaying Logs

To view what has been logged, we need a way to **read** the file and display the contents to the user. This will allow teachers or students to review what activities or events were logged on previous days.

Here's how to read and display the logs:

```
def view_logs():
    try:
        with open(LOG_FILE, "r") as file:
            logs = file.readlines()

        if not logs:
            print("No logs available.")
        else:
            print("Classroom Activity Logs:")
```

```
for log in logs:
    print(log.strip()) # Remove any extra newline characters
except FileNotFoundError:
    print("No log file found. Start by adding a log entry.")
```

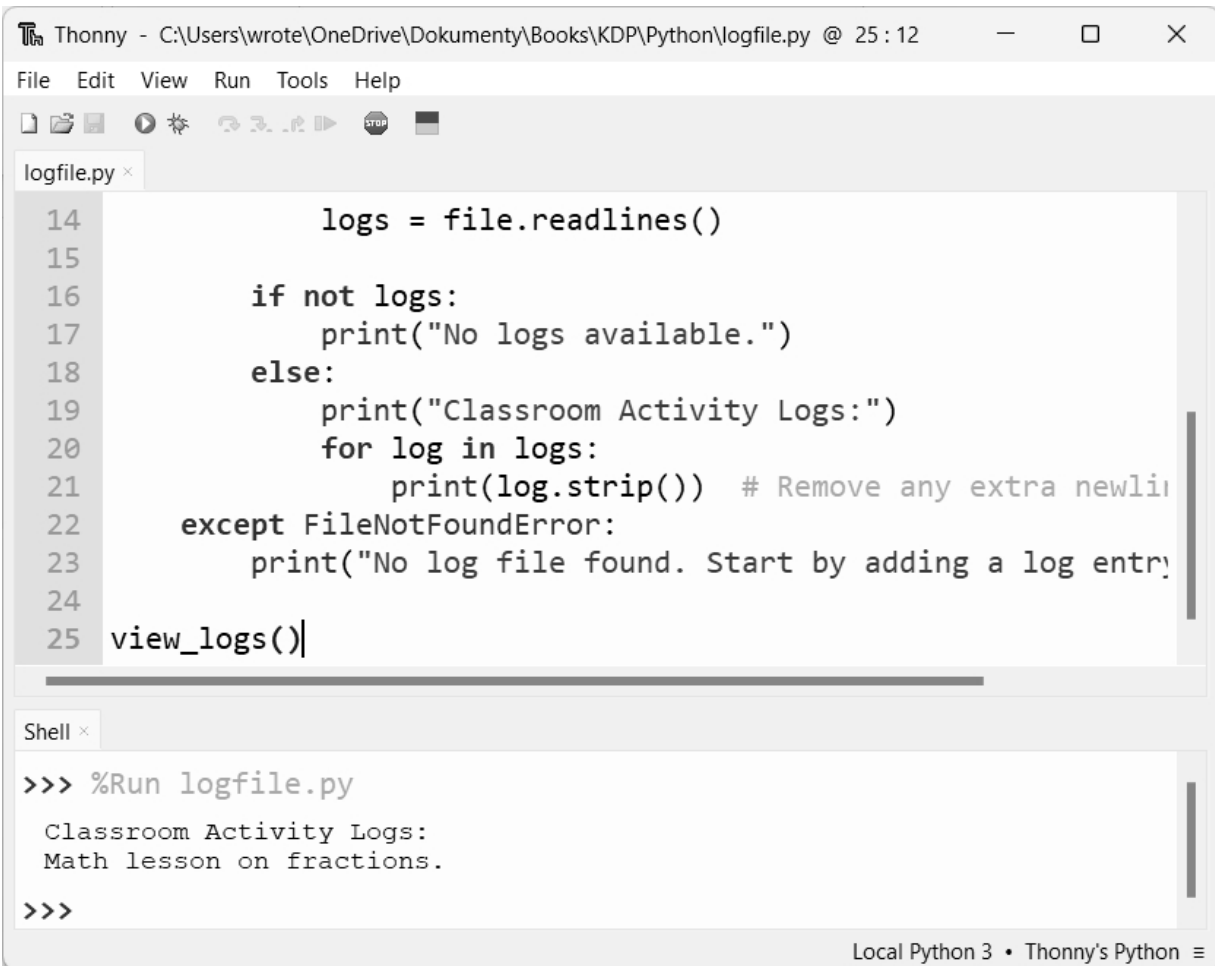
Explanation:

- We open the file in **read mode** ("r") and use readlines() to get all the entries as a list of strings.
- Each log entry is printed out. If the log file is empty or doesn't exist yet, the program informs the user.
- The FileNotFoundError is handled in case there are no logs yet, prompting the user to start logging.

Example Usage:

```
view_logs()
```

Output:



The screenshot shows the Thonny Python IDE interface. The top window, titled 'logfile.py', contains the following Python code:

```
14         logs = file.readlines()
15
16     if not logs:
17         print("No logs available.")
18     else:
19         print("Classroom Activity Logs:")
20         for log in logs:
21             print(log.strip()) # Remove any extra newlines
22 except FileNotFoundError:
23     print("No log file found. Start by adding a log entry")
24
25 view_logs()
```

The bottom window, titled 'Shell', shows the execution of the script:

```
>>> %Run logfile.py
Classroom Activity Logs:
Math lesson on fractions.
>>>
```

The status bar at the bottom right indicates 'Local Python 3 • Thonny's Python'.

5. Creating a Simple Menu System

To make the log system user-friendly, we can add a **menu** that allows users to choose whether they want to add a log, view logs, or quit the program. Here's how we can build a simple menu interface:

```
def menu():
    while True:
        print("\n--- Classroom Log Menu ---")
        print("1. Add a log entry")
        print("2. View log entries")
        print("3. Quit")

        choice = input("Choose an option: ")

        if choice == "1":
            add_log_entry()
```

```
elif choice == "2":  
    view_logs()  
elif choice == "3":  
    print("Exiting log system. Goodbye!")  
    break  
else:  
    print("Invalid choice. Please try again.")
```

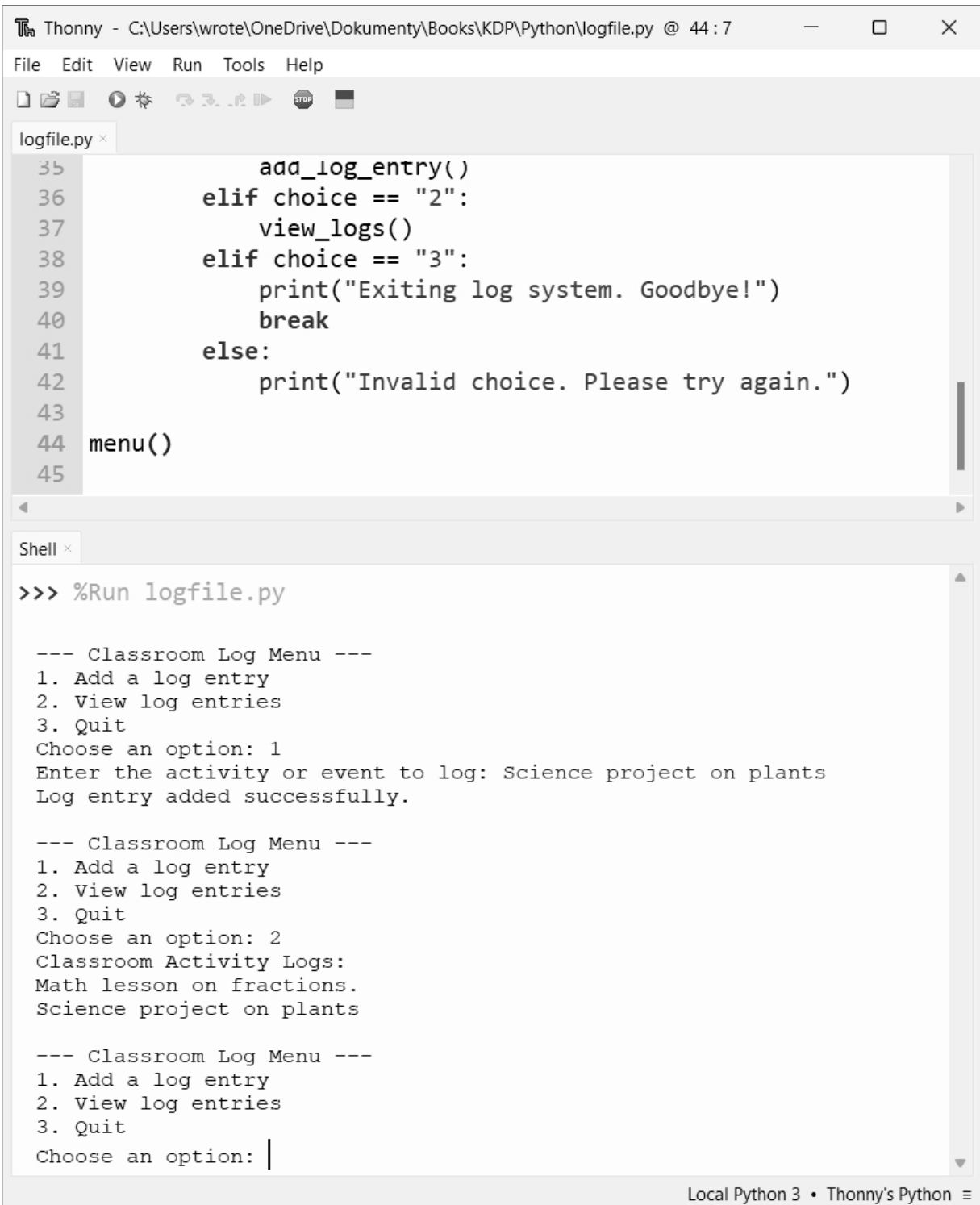
Explanation:

- The menu presents three options: adding a log entry, viewing logs, or quitting.
- Based on the user's input, the program calls the appropriate function (add_log_entry() or view_logs()).
- The program continues running until the user chooses to quit.

Example Usage:

```
menu()
```

Output:



The screenshot shows the Thonny Python IDE interface. The top window, titled 'logfile.py', contains the following Python code:

```
35         add_log_entry()
36     elif choice == "2":
37         view_logs()
38     elif choice == "3":
39         print("Exiting log system. Goodbye!")
40         break
41     else:
42         print("Invalid choice. Please try again.")
43
44 menu()
45
```

The bottom window, titled 'Shell', shows the execution of the script:

```
>>> %Run logfile.py

--- Classroom Log Menu ---
1. Add a log entry
2. View log entries
3. Quit
Choose an option: 1
Enter the activity or event to log: Science project on plants
Log entry added successfully.

--- Classroom Log Menu ---
1. Add a log entry
2. View log entries
3. Quit
Choose an option: 2
Classroom Activity Logs:
Math lesson on fractions.
Science project on plants

--- Classroom Log Menu ---
1. Add a log entry
2. View log entries
3. Quit
Choose an option: |
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

6. Enhancing the Log System

Now that we have the basic functionality in place, here are some ideas for enhancing the log system:

- **Add timestamps:** Record the date and time when each log entry is made, so you can track when activities occurred.

```
from datetime import datetime
```

```
def add_log_entry():
```

```
    activity = input("Enter the activity or event to log: ")
```

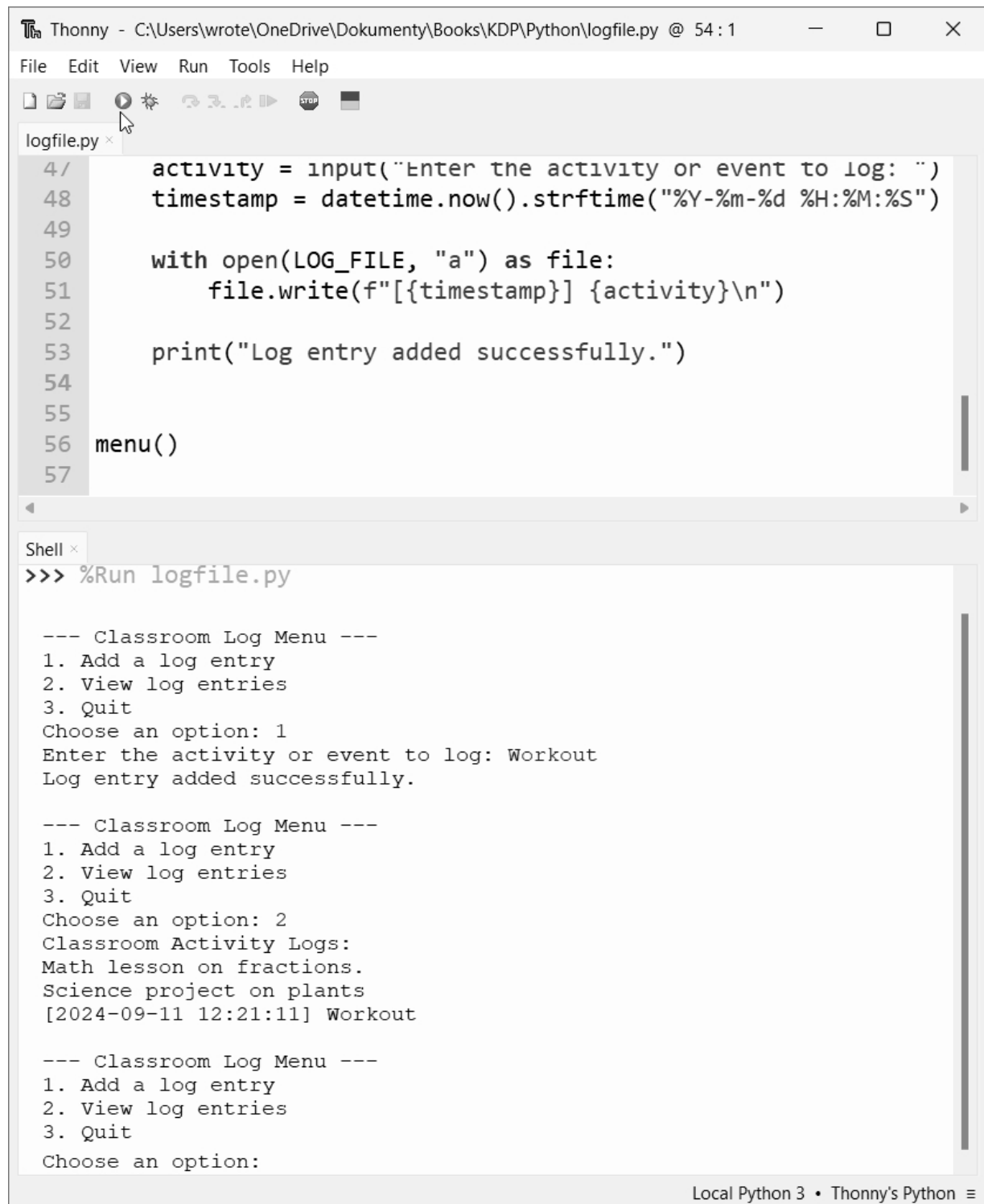
```
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
```

```
    with open(LOG_FILE, "a") as file:
```

```
        file.write(f"[{timestamp}] {activity}\n")
```

```
    print("Log entry added successfully.")
```

This version of the `add_log_entry()` function adds a timestamp to each log, making it easy to see when activities took place.



The screenshot shows the Thonny IDE interface. The top window, titled 'logfile.py', contains the following Python code:

```
47     activity = input("Enter the activity or event to log: ")
48     timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
49
50     with open(LOG_FILE, "a") as file:
51         file.write(f"[{timestamp}] {activity}\n")
52
53     print("Log entry added successfully.")
54
55
56 menu()
57
```

The bottom window, titled 'Shell', shows the execution output:

```
>>> %Run logfile.py

--- Classroom Log Menu ---
1. Add a log entry
2. View log entries
3. Quit
Choose an option: 1
Enter the activity or event to log: Workout
Log entry added successfully.

--- Classroom Log Menu ---
1. Add a log entry
2. View log entries
3. Quit
Choose an option: 2
Classroom Activity Logs:
Math lesson on fractions.
Science project on plants
[2024-09-11 12:21:11] Workout

--- Classroom Log Menu ---
1. Add a log entry
2. View log entries
3. Quit
Choose an option:
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

- **Filter logs by keyword:** Allow users to search through logs for specific events or keywords.

```
def search_logs(keyword):
```

```

try:
    with open(LOG_FILE, "r") as file:
        logs = file.readlines()

    print(f'Logs containing '{keyword}':')
    for log in logs:
        if keyword.lower() in log.lower():
            print(log.strip())
except FileNotFoundError:
    print("No log file found.")

```

This function allows users to search the logs for a specific word or phrase, making it easier to find important activities.

7. Why Build a Classroom Log System?

Building a log system has many practical uses in education:

- **Tracking progress:** Teachers can log daily activities and monitor student progress over time.
- **Attendance records:** A log system can also be used to track which students attended specific classes or events.
- **Simplifying communication:** Logs can be reviewed by both teachers and students to stay updated on classroom activities, homework, or projects.

This log system is just the beginning. You can expand it to handle more complex needs, like organizing logs by date, exporting logs to other formats (such as CSV), or integrating it into larger school management systems.

Summary

A simple log system can significantly enhance classroom management by providing an easy way to track activities and events. With Python's file handling capabilities, you can build a tool that logs, stores, and retrieves data efficiently. Whether you use it for logging daily lessons, tracking attendance, or keeping records of classroom activities, this system simplifies everyday tasks and makes it easy to stay organized.

By practicing with this project, you'll develop the skills needed to build more complex systems in the future. Keep experimenting, and soon you'll be creating even more powerful tools to solve real-world problems in education and beyond!

Note! Pay attention to how the program was built. First, the basic module was launched. While it was running, subsequent modules were built. It was like building a brick wall.

A good piece of advice: from the first days of learning, collect your programs and give them names that will allow for quick searching. Thanks to this, with each program you write, you will have more ready-made "bricks".

Part III: Intermediate Python: Working with Libraries

Chapter 8: Problem Solving with NumPy and Pandas

Introduction to Data Structures in NumPy and Pandas

As you dive deeper into Python, you'll find that handling large datasets efficiently becomes crucial. Whether you're analyzing school grades, scientific data, or financial information, Python libraries like **NumPy** and **Pandas** make this much easier. These libraries provide powerful tools for working with data in ways that are both fast and easy to understand. In this chapter, we'll introduce you to the essential **data structures** in NumPy and Pandas that will help you manage and manipulate data with ease.

By the end of this section, you'll understand how these data structures work and how to use them to solve real-world data problems.

1. Why Use NumPy and Pandas?

Before we dive into the specific data structures, let's understand why **NumPy** and **Pandas** are so important:

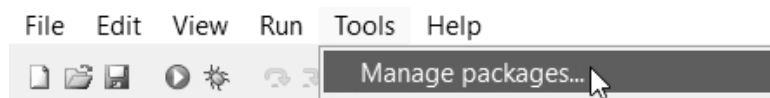
- **NumPy** is short for "Numerical Python" and is the go-to library for handling numerical data efficiently. It provides support for large, multi-dimensional arrays and matrices, along with a variety of mathematical functions.
- **Pandas** is built on top of NumPy and is designed specifically for data analysis. It simplifies handling **tabular data** (think of data in rows and columns, like a spreadsheet) and provides tools to easily manipulate and analyze it.

Both libraries are widely used in data science, research, and any application where working with data is required.

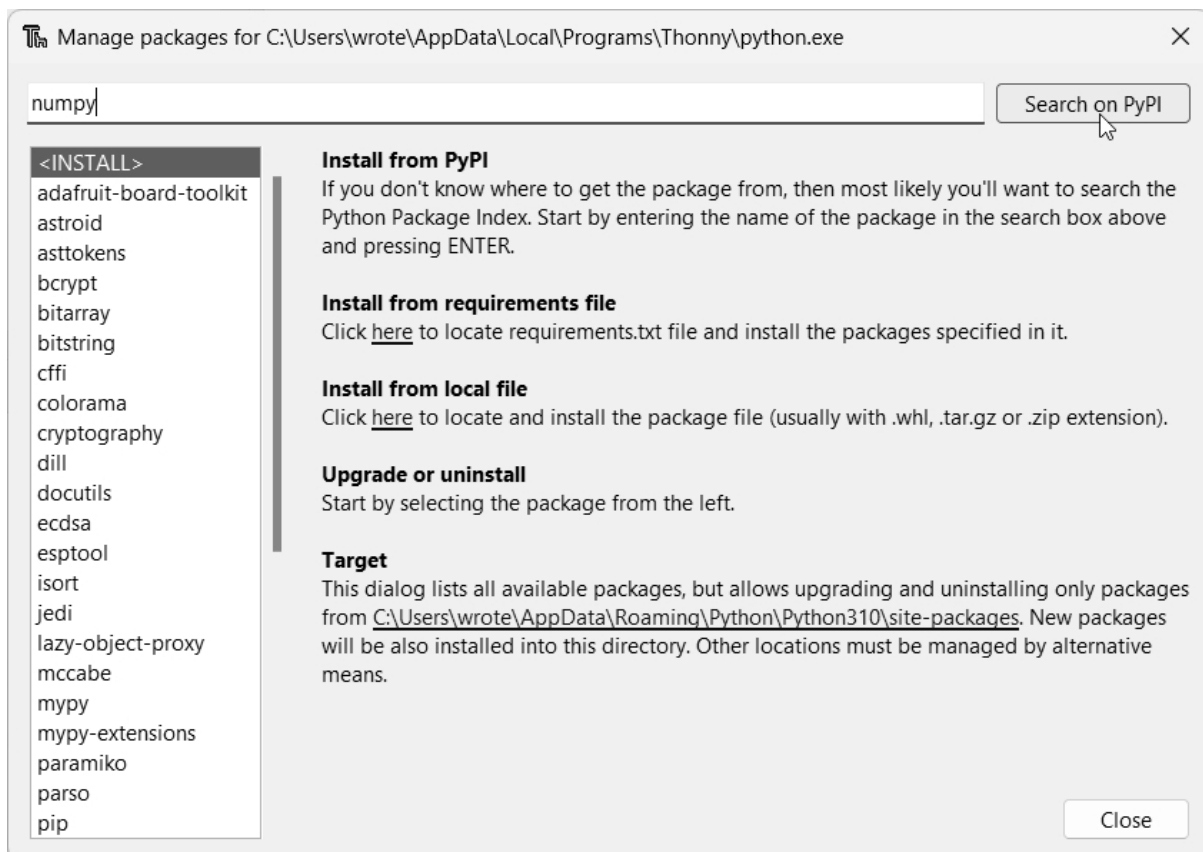
2. NumPy: The Power of Arrays

The core data structure in **NumPy** is the **array**, which is a grid-like structure that can hold many items, typically numbers, all of the same type. Arrays are more efficient than Python lists for numerical operations and make mathematical computations faster.

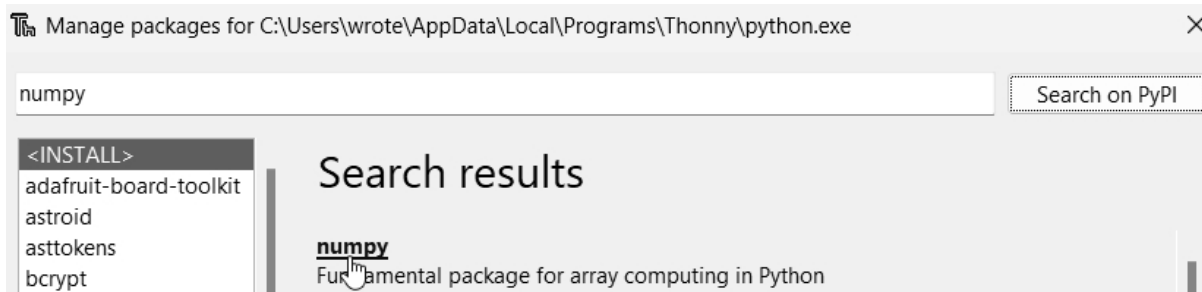
Notice! To install the numpy library choose **Tools, Manage packages** in Thonny.



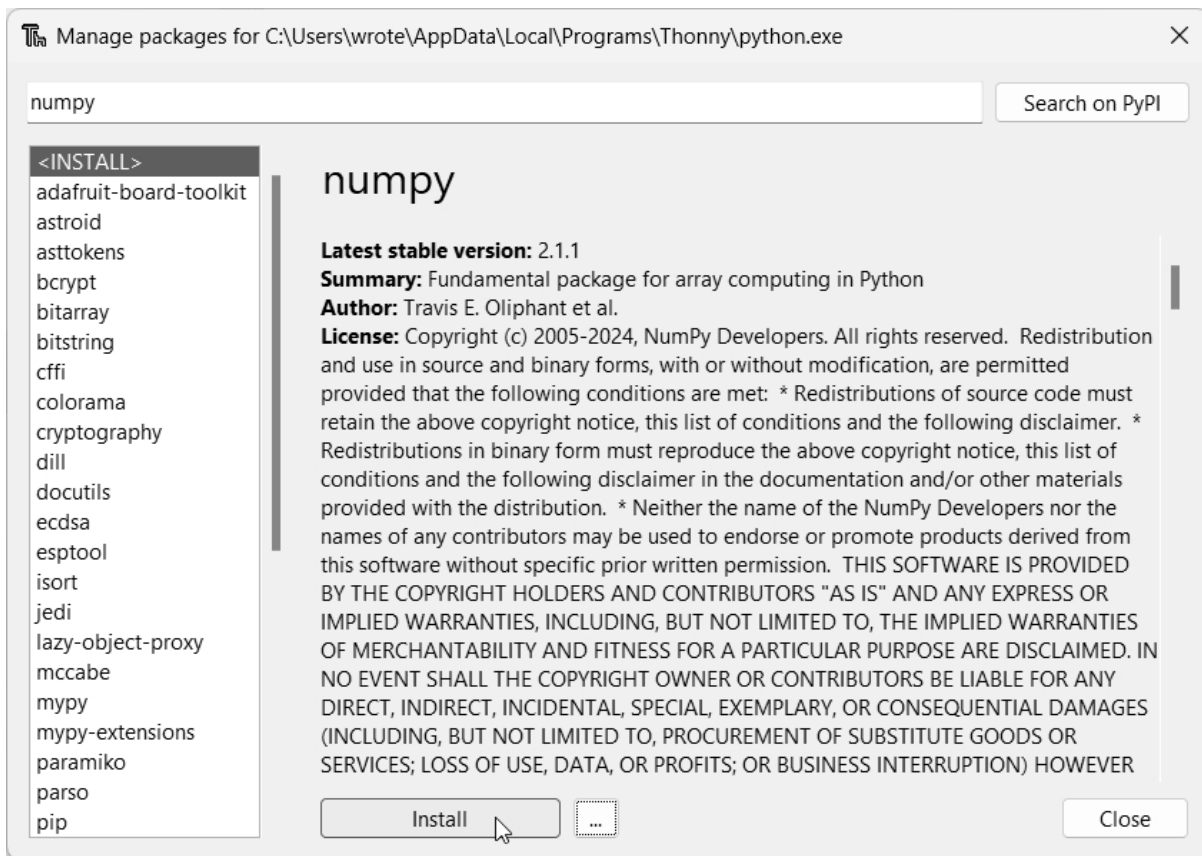
On the left side of the window you can see a list of installed libraries. Click at the top of the window. Type the name of the library you want to install (for example numpy). Click the **Search on PyPI** button.



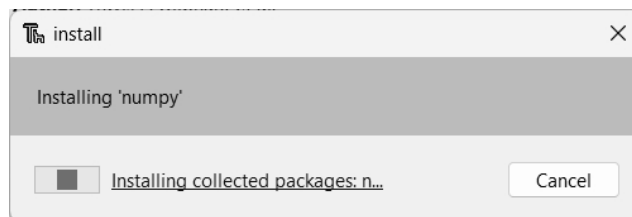
In the search results, select the fundamental package for array computing.



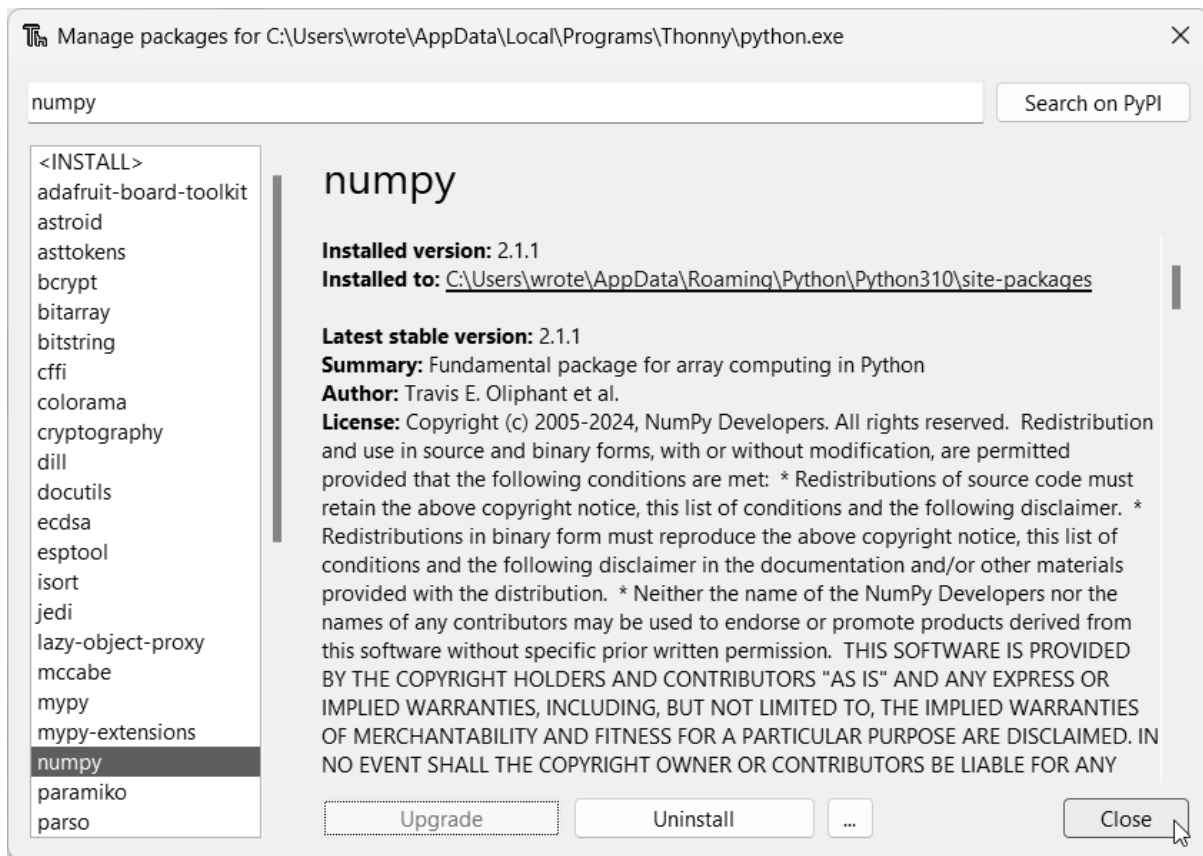
Read the description. If this is the library you want to install, click **Install**.



Wait a while...



When the name of the installed library appears on the left side of the window, close it by clicking the **Close** button.

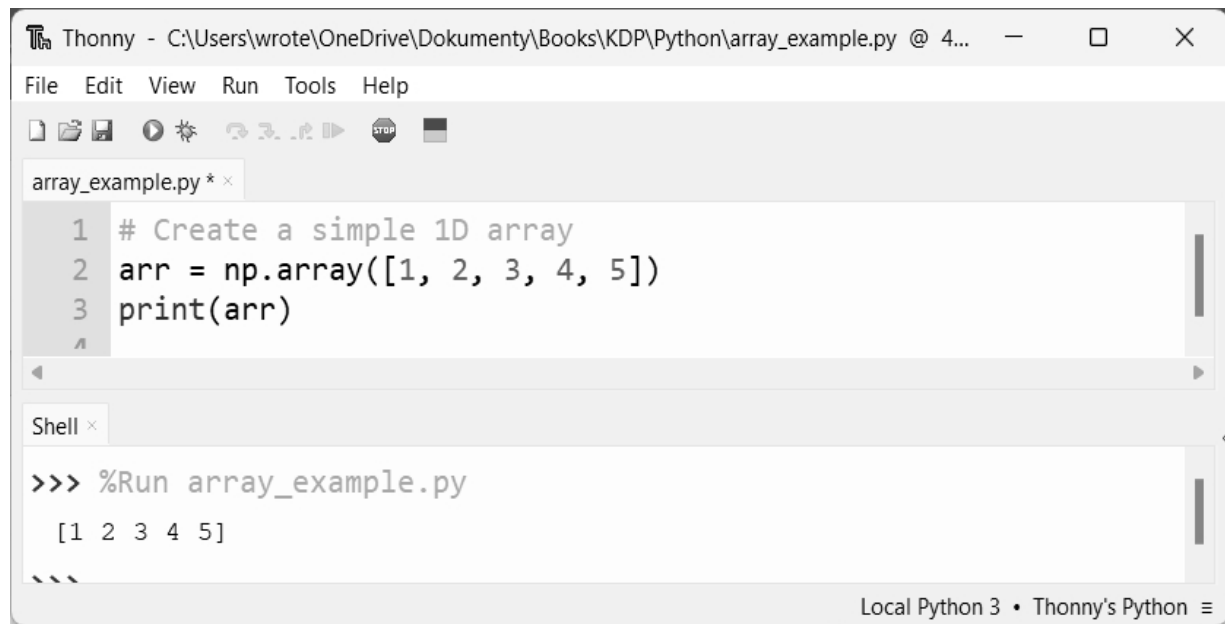


Creating a NumPy Array

Now, you can create an array like this:

```
import numpy as np  
  
# Create a simple 1D array  
arr = np.array([1, 2, 3, 4, 5])  
print(arr)
```

Output:

The image shows a screenshot of the Thonny Python IDE. The window title is "Thonny - C:\Users\wrote\OneDrive\Dokumenty\Books\KDP\Python\array_example.py @ 4...". The menu bar includes "File", "Edit", "View", "Run", "Tools", and "Help". Below the menu bar is a toolbar with icons for file operations and execution. The main editor area shows a file named "array_example.py" with the following code:

```
1 # Create a simple 1D array
2 arr = np.array([1, 2, 3, 4, 5])
3 print(arr)
```

The bottom panel, labeled "Shell", shows the command prompt with the command ">>> %Run array_example.py" and the output "[1 2 3 4 5]". The status bar at the bottom right indicates "Local Python 3 • Thonny's Python".

Advantages of NumPy Arrays

- **Speed:** NumPy arrays are much faster than Python lists when performing operations like addition, multiplication, or other mathematical functions.
- **Multidimensional:** You can create arrays with more than one dimension, making NumPy perfect for working with tables, matrices, or grids of data.

Example: Creating a 2D Array (Matrix)

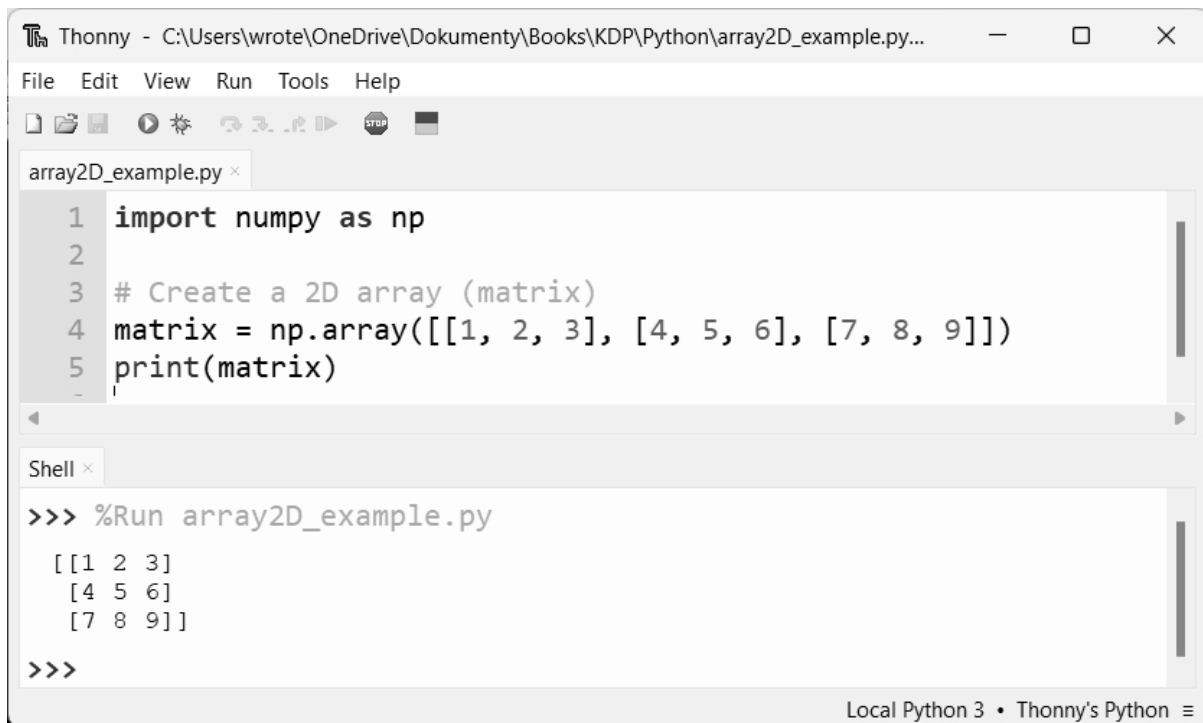
```
import numpy as np
```

```
# Create a 2D array (matrix)
```

```
matrix = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
```

```
print(matrix)
```

Output:

The image shows a screenshot of the Thonny Python IDE. The window title is "Thonny - C:\Users\wrote\OneDrive\Dokumenty\Books\KDP\Python\array2D_example.py...". The menu bar includes "File", "Edit", "View", "Run", "Tools", and "Help". Below the menu bar is a toolbar with icons for file operations and execution. The main editor area shows a file named "array2D_example.py" with the following Python code:

```
1 import numpy as np
2
3 # Create a 2D array (matrix)
4 matrix = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
5 print(matrix)
```

The bottom panel, titled "Shell", shows the execution of the script with the command ">>> %Run array2D_example.py" and the output:

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
>>>
```

The status bar at the bottom right indicates "Local Python 3 • Thonny's Python".

This 2D array is like a grid or a matrix that can be used for more complex calculations.

Basic Operations with NumPy Arrays

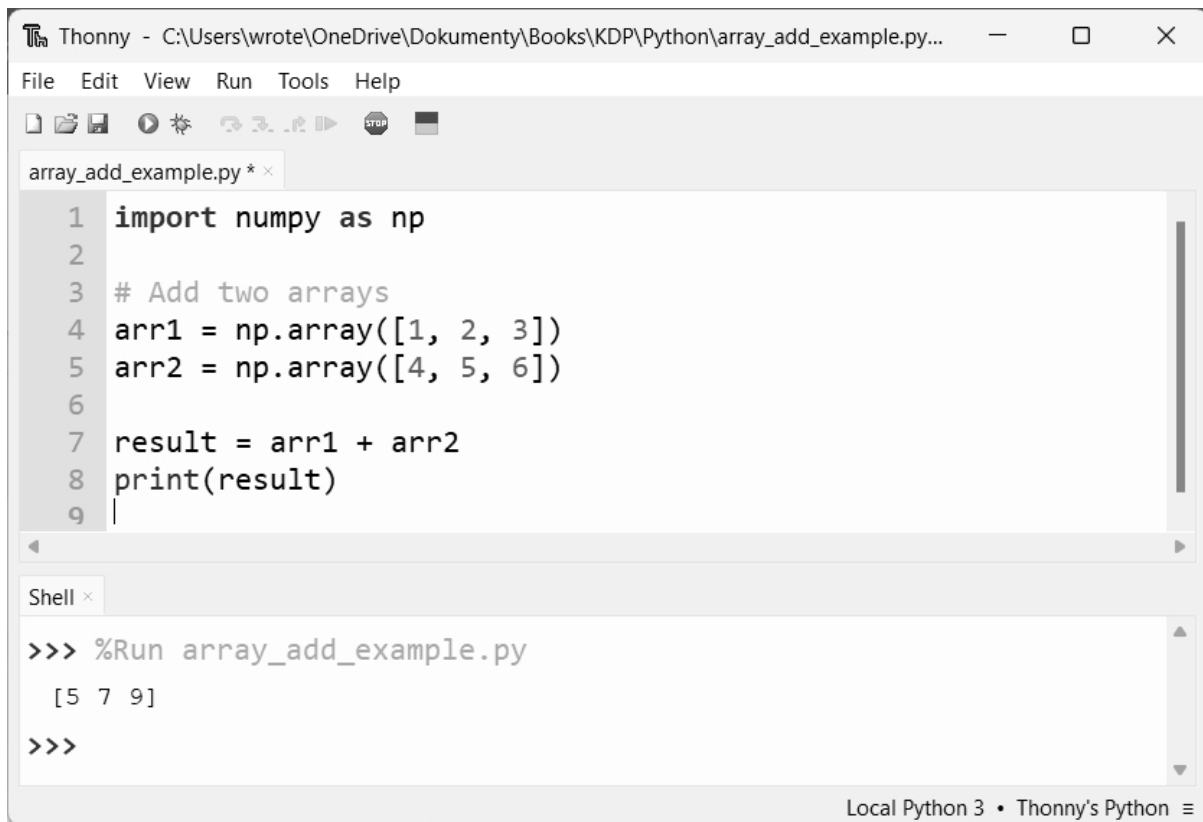
NumPy makes it easy to perform mathematical operations on arrays. You can add, subtract, multiply, or divide arrays element-wise:

```
import numpy as np

# Add two arrays
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])

result = arr1 + arr2
print(result)
```

Output:



The screenshot shows the Thonny IDE interface. The top window, titled 'array_add_example.py * x', contains the following Python code:

```
1 import numpy as np
2
3 # Add two arrays
4 arr1 = np.array([1, 2, 3])
5 arr2 = np.array([4, 5, 6])
6
7 result = arr1 + arr2
8 print(result)
9 |
```

The bottom window, titled 'Shell x', shows the execution output:

```
>>> %Run array_add_example.py
[5 7 9]
>>>
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

You can also perform operations like finding the **mean**, **sum**, or **standard deviation** of an array:

```
import numpy as np
# Calculate the mean of the array
arr = np.array([1, 2, 3])
mean_value = np.mean(arr)
print(f"Mean: {mean_value}")
```

Thonny - C:\Users\wrote\OneDrive\Dokumenty\Books\KDP\Python\mean_example.py @...

File Edit View Run Tools Help

mean_example.py ×

```
1 import numpy as np
2 # Calculate the mean of the array
3 arr = np.array([1, 2, 3])
4 mean_value = np.mean(arr)
5 print(f"Mean: {mean_value}")
6 |
```

Shell ×

```
>>> %Run mean_example.py
Mean: 2.0
>>>
```

Local Python 3 • Thonny's Python

Thonny - C:\Users\wrote\OneDrive\Dokumenty\Books\KDP\Python\sum_example.py @ 6...

File Edit View Run Tools Help

sum_example.py ×

```
1 import numpy as np
2 # Calculate the mean of the array
3 arr = np.array([1, 2, 3])
4 sum_value = np.sum(arr)
5 print(f"Sum: {sum_value}")
6 |
```

Shell ×

```
>>> %Run sum_example.py
Sum: 6
>>>
```

Local Python 3 • Thonny's Python

The screenshot shows the Thonny Python IDE interface. The title bar indicates the file path: `C:\Users\wrote\OneDrive\Dokumenty\Books\KDP\Python\standard_deviation_ex...`. The menu bar includes `File`, `Edit`, `View`, `Run`, `Tools`, and `Help`. Below the menu is a toolbar with icons for file operations and execution. The main editor window displays a Python script in `standard_deviation_example.py` with the following code:

```
1 import numpy as np
2 # Calculate the mean of the array
3 arr = np.array([1, 2, 3])
4 std_value = np.std(arr)
5 print(f"Standard deviation: {std_value}")
```

Below the editor is a `Shell` window showing the execution output:

```
>>> %Run standard_deviation_example.py
Standard deviation: 0.816496580927726
>>>
```

The status bar at the bottom right shows `Local Python 3 • Thonny's Python`.

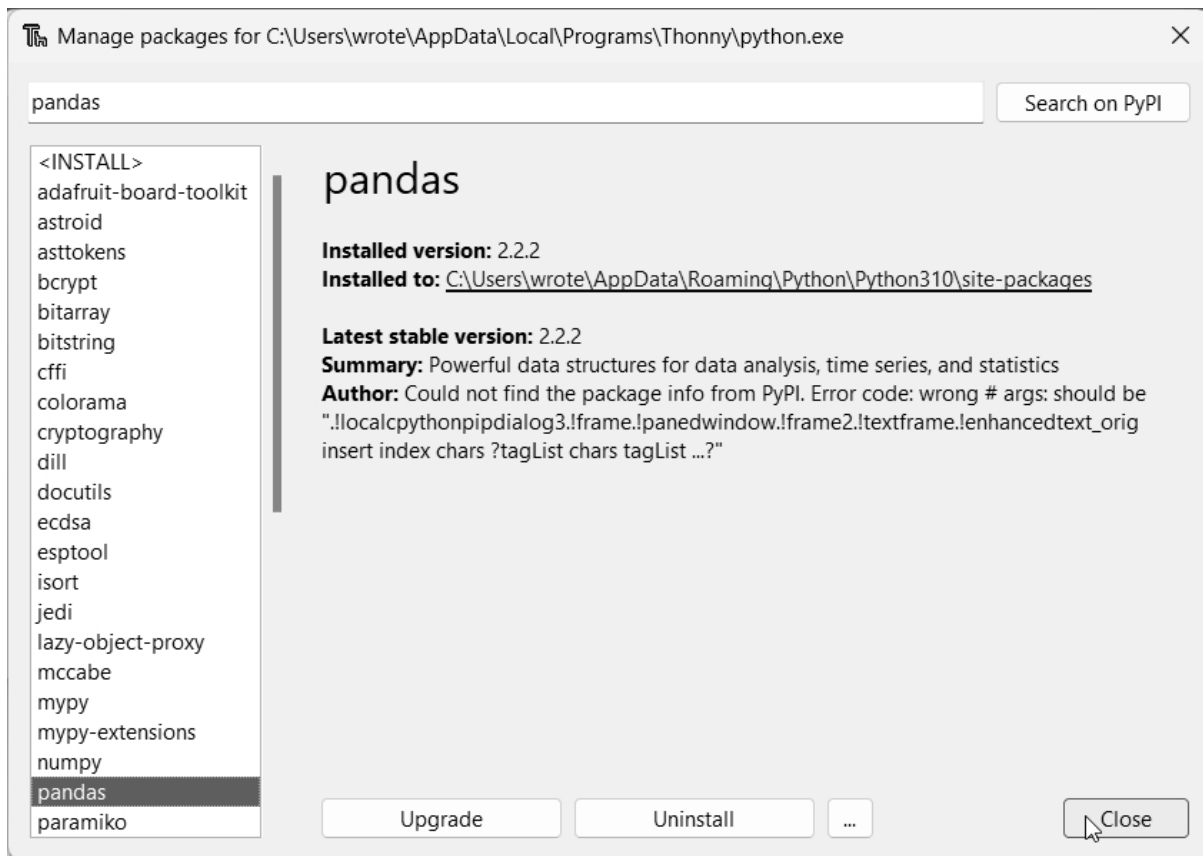
NumPy arrays are perfect for numerical and matrix-based problems, especially when you need to handle large datasets quickly and efficiently.

3. Pandas: Working with DataFrames

While NumPy is great for numerical data, **Pandas** is the go-to tool when working with structured data, such as tables or spreadsheets. The main data structure in Pandas is the **DataFrame**, which you can think of as a table with rows and columns, similar to an Excel spreadsheet.

Creating a Pandas DataFrame

To use Pandas, you need to install it first:



Now, let's create a simple DataFrame:

```
import pandas as pd
```

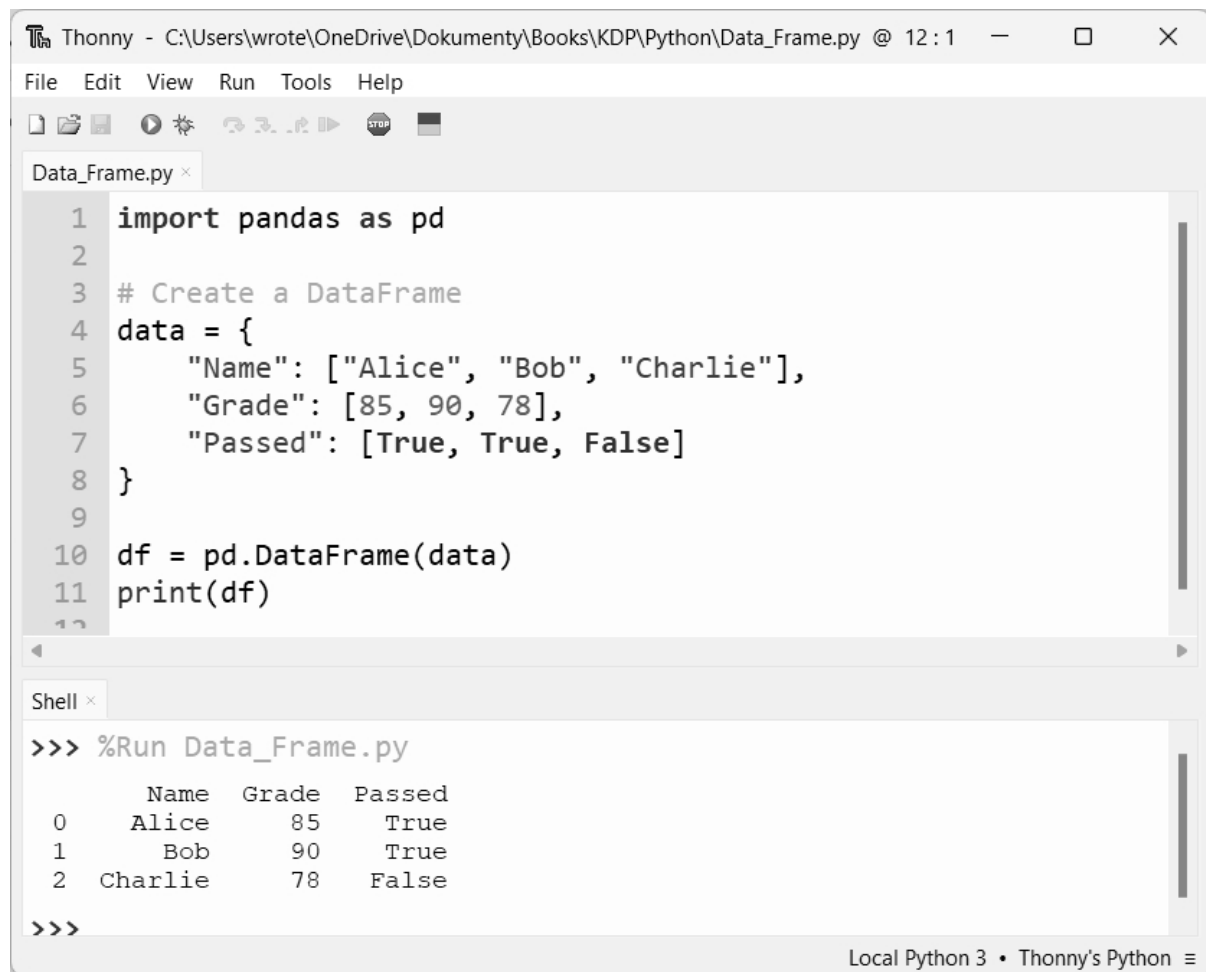
```
# Create a DataFrame
```

```
data = {  
    "Name": ["Alice", "Bob", "Charlie"],  
    "Grade": [85, 90, 78],  
    "Passed": [True, True, False]  
}
```

```
df = pd.DataFrame(data)
```

```
print(df)
```

Output:



The screenshot shows the Thonny Python IDE interface. The top window, titled 'Data_Frame.py', contains the following Python code:

```
1 import pandas as pd
2
3 # Create a DataFrame
4 data = {
5     "Name": ["Alice", "Bob", "Charlie"],
6     "Grade": [85, 90, 78],
7     "Passed": [True, True, False]
8 }
9
10 df = pd.DataFrame(data)
11 print(df)
```

The bottom window, titled 'Shell', shows the execution of the script:

```
>>> %Run Data_Frame.py
      Name  Grade  Passed
0    Alice     85     True
1     Bob     90     True
2  Charlie     78    False
>>>
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

Advantages of DataFrames

- **Labeled Data:** Each column in a DataFrame has a label, making it easier to refer to specific columns or rows.
- **Handling Missing Data:** Pandas handles missing data elegantly, filling in or removing gaps in datasets.
- **Data Analysis Tools:** Pandas provides powerful tools to analyze, filter, and manipulate data.

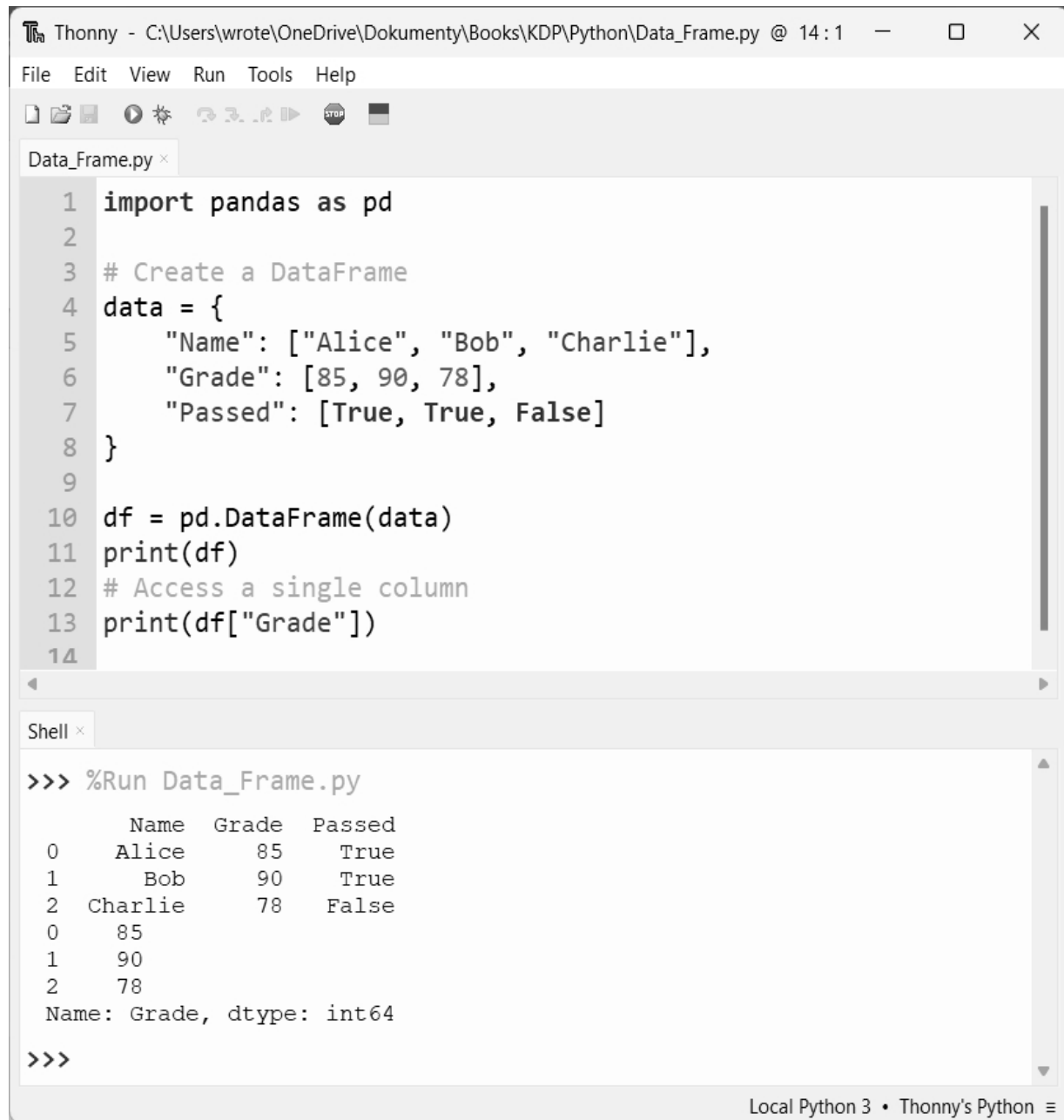
Accessing Data in a DataFrame

You can access specific columns or rows in a DataFrame easily. For example, to access the Grade column:

```
# Access a single column
```

```
print(df["Grade"])
```

Output:



The screenshot shows the Thonny IDE interface. The top window, titled 'Data_Frame.py', contains the following Python code:

```
1 import pandas as pd
2
3 # Create a DataFrame
4 data = {
5     "Name": ["Alice", "Bob", "Charlie"],
6     "Grade": [85, 90, 78],
7     "Passed": [True, True, False]
8 }
9
10 df = pd.DataFrame(data)
11 print(df)
12 # Access a single column
13 print(df["Grade"])
14
```

The bottom window, titled 'Shell', shows the output of running the script:

```
>>> %Run Data_Frame.py

   Name  Grade  Passed
0  Alice     85    True
1   Bob     90    True
2 Charlie     78   False
0     85
1     90
2     78
Name: Grade, dtype: int64

>>>
```

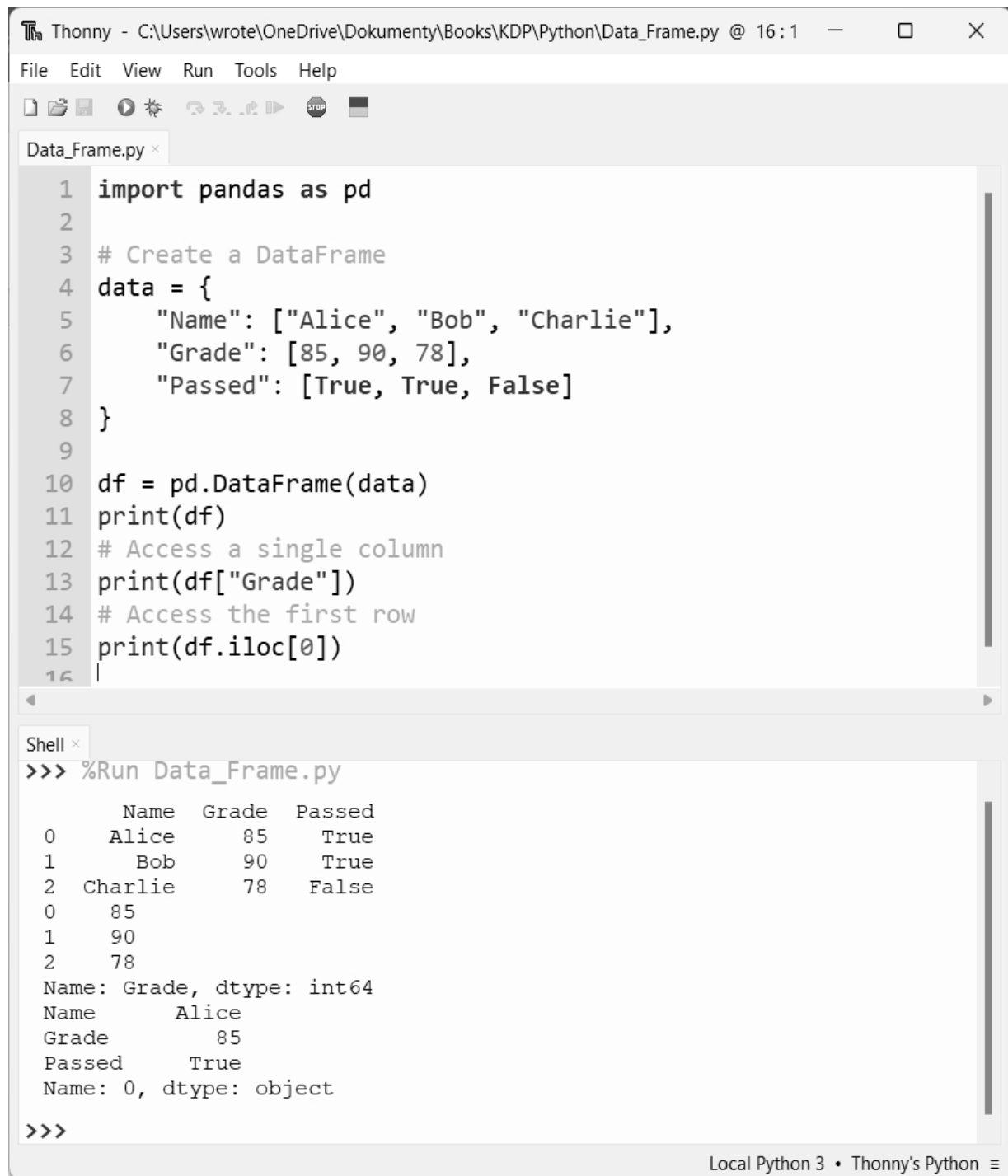
The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

You can also access a specific row by its index:

```
# Access the first row
```

```
print(df.iloc[0])
```

Output:



The screenshot shows the Thonny IDE interface. The top window, titled 'Data_Frame.py', contains the following Python code:

```
1 import pandas as pd
2
3 # Create a DataFrame
4 data = {
5     "Name": ["Alice", "Bob", "Charlie"],
6     "Grade": [85, 90, 78],
7     "Passed": [True, True, False]
8 }
9
10 df = pd.DataFrame(data)
11 print(df)
12 # Access a single column
13 print(df["Grade"])
14 # Access the first row
15 print(df.iloc[0])
16
```

The bottom window, titled 'Shell', shows the output of running the script:

```
>>> %Run Data_Frame.py
      Name  Grade  Passed
0   Alice     85    True
1    Bob     90    True
2 Charlie     78   False
0      85
1      90
2      78
Name: Grade, dtype: int64
Name      Alice
Grade      85
Passed     True
Name: 0, dtype: object
>>>
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

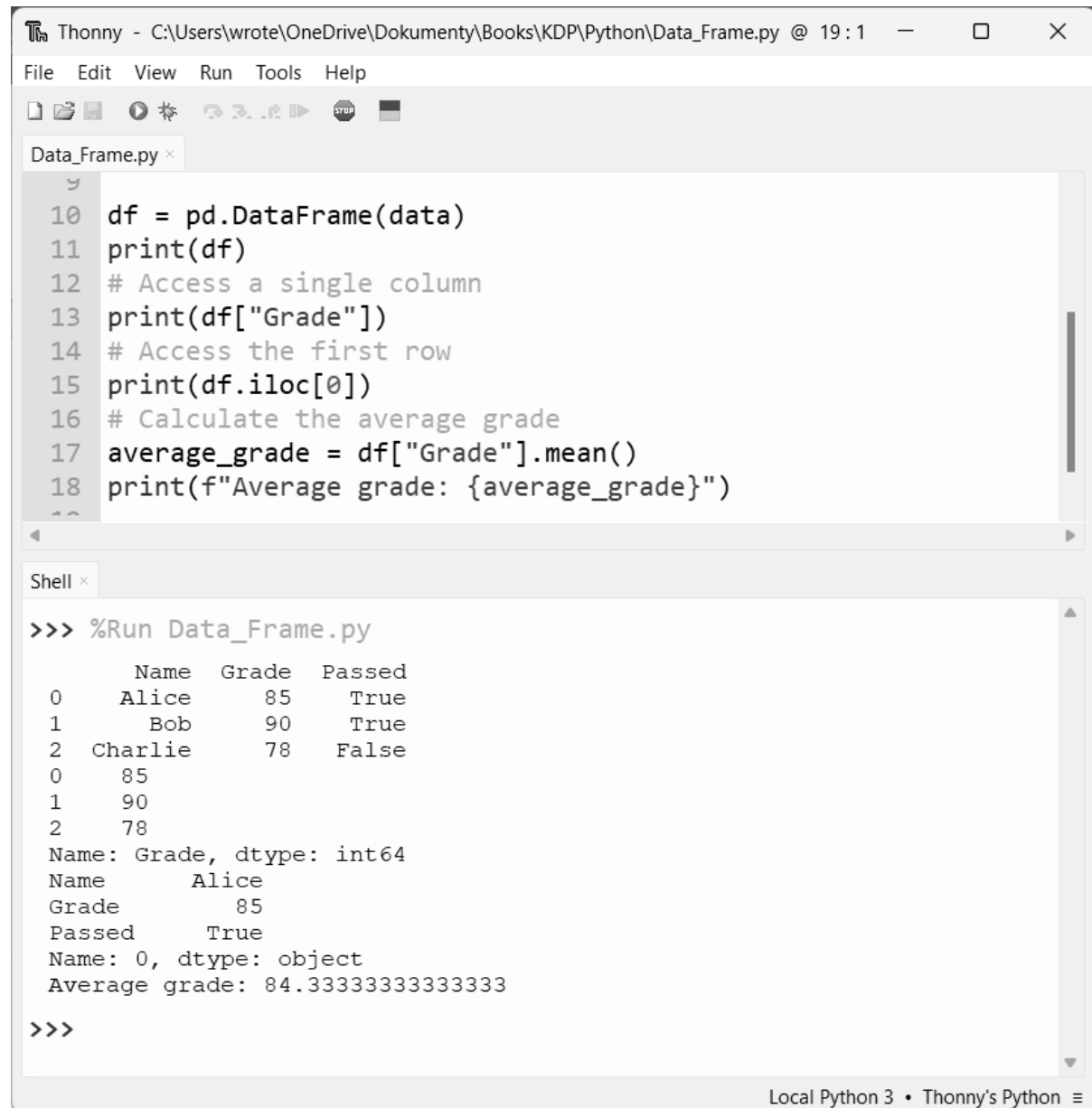
Basic Operations with DataFrames

Pandas allows you to perform quick calculations on the data within a DataFrame:

```
# Calculate the average grade
```

```
average_grade = df["Grade"].mean()
```

```
print(f"Average grade: {average_grade}")
```



The screenshot shows the Thonny Python IDE interface. The top window, titled 'Data_Frame.py', contains the following Python code:

```
10 df = pd.DataFrame(data)
11 print(df)
12 # Access a single column
13 print(df["Grade"])
14 # Access the first row
15 print(df.iloc[0])
16 # Calculate the average grade
17 average_grade = df["Grade"].mean()
18 print(f"Average grade: {average_grade}")
```

The bottom window, titled 'Shell', shows the output of running the script:

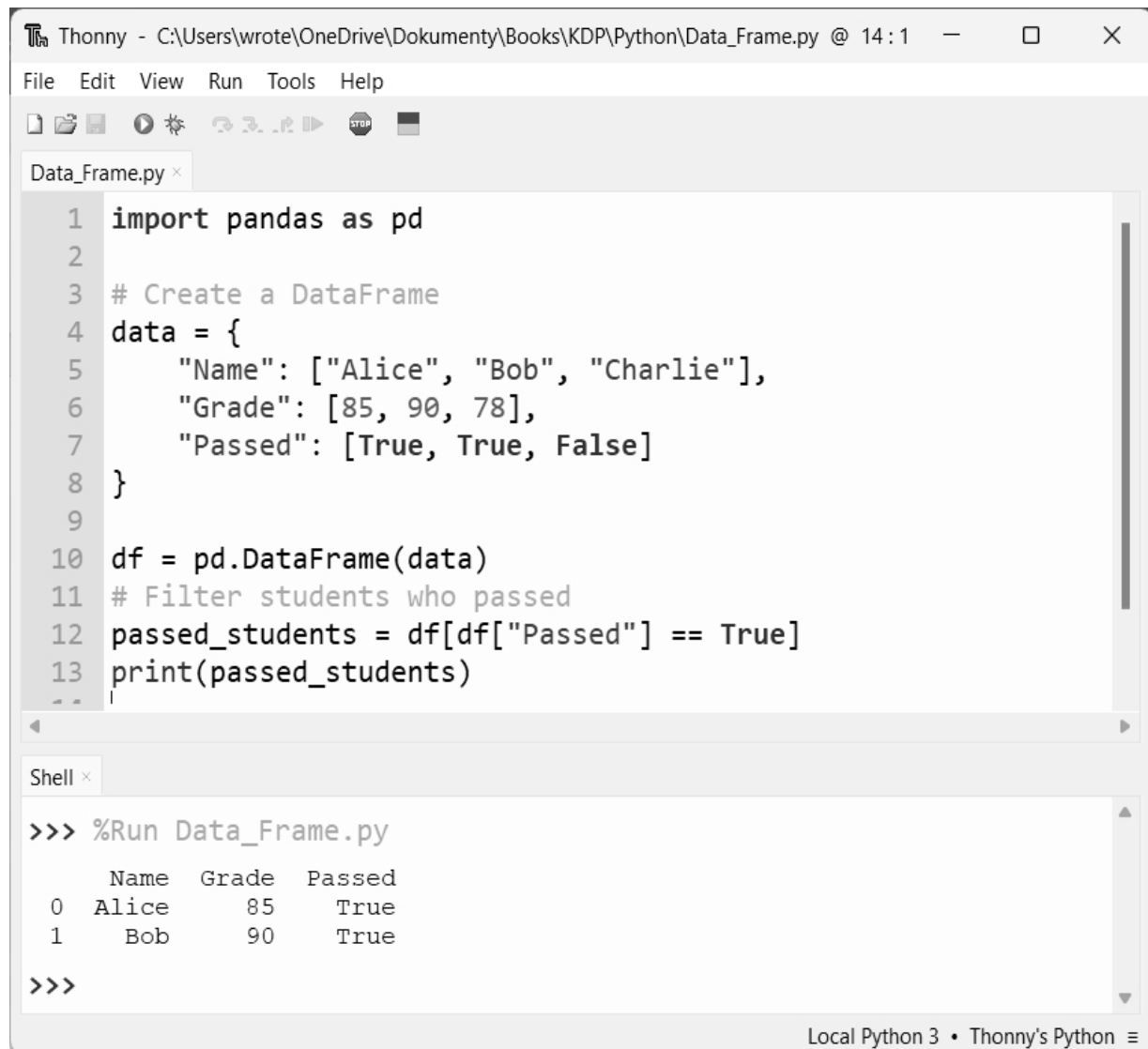
```
>>> %Run Data_Frame.py
      Name  Grade  Passed
0   Alice     85    True
1    Bob     90    True
2  Charlie     78   False
0      85
1      90
2      78
Name: Grade, dtype: int64
Name      Alice
Grade      85
Passed    True
Name: 0, dtype: object
Average grade: 84.33333333333333
>>>
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

You can also filter data based on certain conditions. For example, if you want to find students who passed:

```
# Filter students who passed
passed_students = df[df["Passed"] == True]
print(passed_students)
```

Output:



The screenshot shows the Thonny Python IDE interface. The top window, titled 'Data_Frame.py', contains the following Python code:

```
1 import pandas as pd
2
3 # Create a DataFrame
4 data = {
5     "Name": ["Alice", "Bob", "Charlie"],
6     "Grade": [85, 90, 78],
7     "Passed": [True, True, False]
8 }
9
10 df = pd.DataFrame(data)
11 # Filter students who passed
12 passed_students = df[df["Passed"] == True]
13 print(passed_students)
```

Below the code editor is a 'Shell' window showing the execution output:

```
>>> %Run Data_Frame.py
      Name  Grade  Passed
0  Alice     85     True
1   Bob     90     True
>>>
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

4. Combining NumPy and Pandas

NumPy and Pandas are often used together in data analysis. For example, you might use NumPy to perform advanced numerical computations on a dataset and then use Pandas to organize and display the results.

Let's say you have a list of test scores and want to calculate some statistics (like mean and standard deviation) using NumPy, then store the results in a Pandas DataFrame for easier viewing:

```
import numpy as np
```

```
import pandas as pd
```

```
# Test scores
```

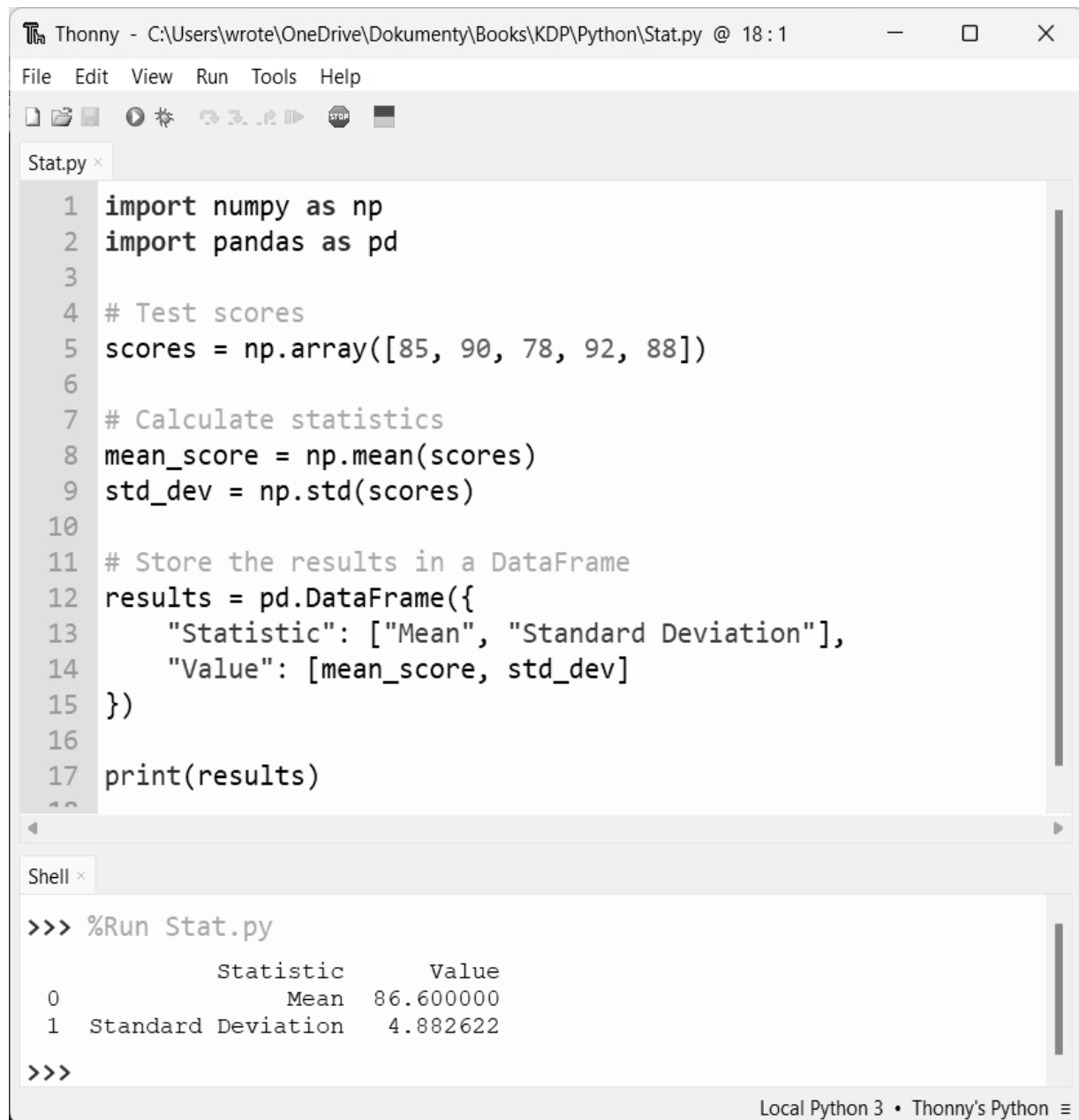
```
scores = np.array([85, 90, 78, 92, 88])

# Calculate statistics
mean_score = np.mean(scores)
std_dev = np.std(scores)

# Store the results in a DataFrame
results = pd.DataFrame({
    "Statistic": ["Mean", "Standard Deviation"],
    "Value": [mean_score, std_dev]
})

print(results)
```

Output:



The screenshot shows the Thonny Python IDE interface. The top window, titled 'Stat.py', contains the following Python code:

```
1 import numpy as np
2 import pandas as pd
3
4 # Test scores
5 scores = np.array([85, 90, 78, 92, 88])
6
7 # Calculate statistics
8 mean_score = np.mean(scores)
9 std_dev = np.std(scores)
10
11 # Store the results in a DataFrame
12 results = pd.DataFrame({
13     "Statistic": ["Mean", "Standard Deviation"],
14     "Value": [mean_score, std_dev]
15 })
16
17 print(results)
```

The bottom window, titled 'Shell', shows the execution of the script:

```
>>> %Run Stat.py
```

	Statistic	Value
0	Mean	86.600000
1	Standard Deviation	4.882622

```
>>>
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

This example demonstrates how NumPy handles the computations, while Pandas helps you organize and present the data in a meaningful way.

5. Why Data Structures Matter in Problem Solving

Understanding the data structures in NumPy and Pandas allows you to:

- **Manage large datasets** effectively.
- **Perform quick calculations** on data with ease.

- **Organize and analyze data** in a structured format, whether it's numerical data in arrays or tabular data in DataFrames.

These skills are essential for tackling real-world data problems, from analyzing student test scores to processing large datasets in fields like science, finance, and research.

Summary

The data structures in **NumPy** and **Pandas** provide the foundation for efficient data handling and analysis in Python. Whether you're working with large numerical arrays or complex tables of information, these libraries make it easy to manipulate, analyze, and draw insights from your data. With NumPy arrays for numerical operations and Pandas DataFrames for organizing structured data, you now have the tools to solve real-world problems more effectively.

Keep practicing with these libraries, and soon you'll be handling and analyzing data like a pro!

Using Pandas to Solve Real-World Data Problems

Pandas is one of the most powerful libraries in Python for handling and analyzing data. In the real world, data often comes in the form of tables or spreadsheets, and this is where Pandas shines. Whether you're working with school grades, business reports, or scientific data, Pandas provides the tools to manage, manipulate, and analyze this data efficiently.

In this section, we'll explore how you can use Pandas to solve **real-world data problems**. By the end of this chapter, you'll see how Pandas can transform raw data into valuable insights, making it an essential tool for students, teachers, and professionals alike.

1. The Power of Pandas for Data Analysis

Before diving into examples, let's quickly review why **Pandas** is such a valuable tool:

- **DataFrames:** The core structure in Pandas is the **DataFrame**, which allows you to store and manipulate tabular data (think of a spreadsheet or table).
- **Ease of Use:** With Pandas, you can quickly load data from files (like CSV or Excel), filter it, calculate statistics, and create summaries — all with just a few lines of code.
- **Handling Missing Data:** Real-world data is often incomplete or messy, and Pandas provides built-in functions to deal with missing or invalid data.

2. Loading and Inspecting Data

In real-world scenarios, data is often stored in files like CSVs or Excel spreadsheets. Pandas makes it easy to load this data into a **DataFrame** and start working with it.

Example: Loading a CSV File

Let's say you have a CSV file that contains student grades for several subjects. Here's how you can load it into a Pandas DataFrame:

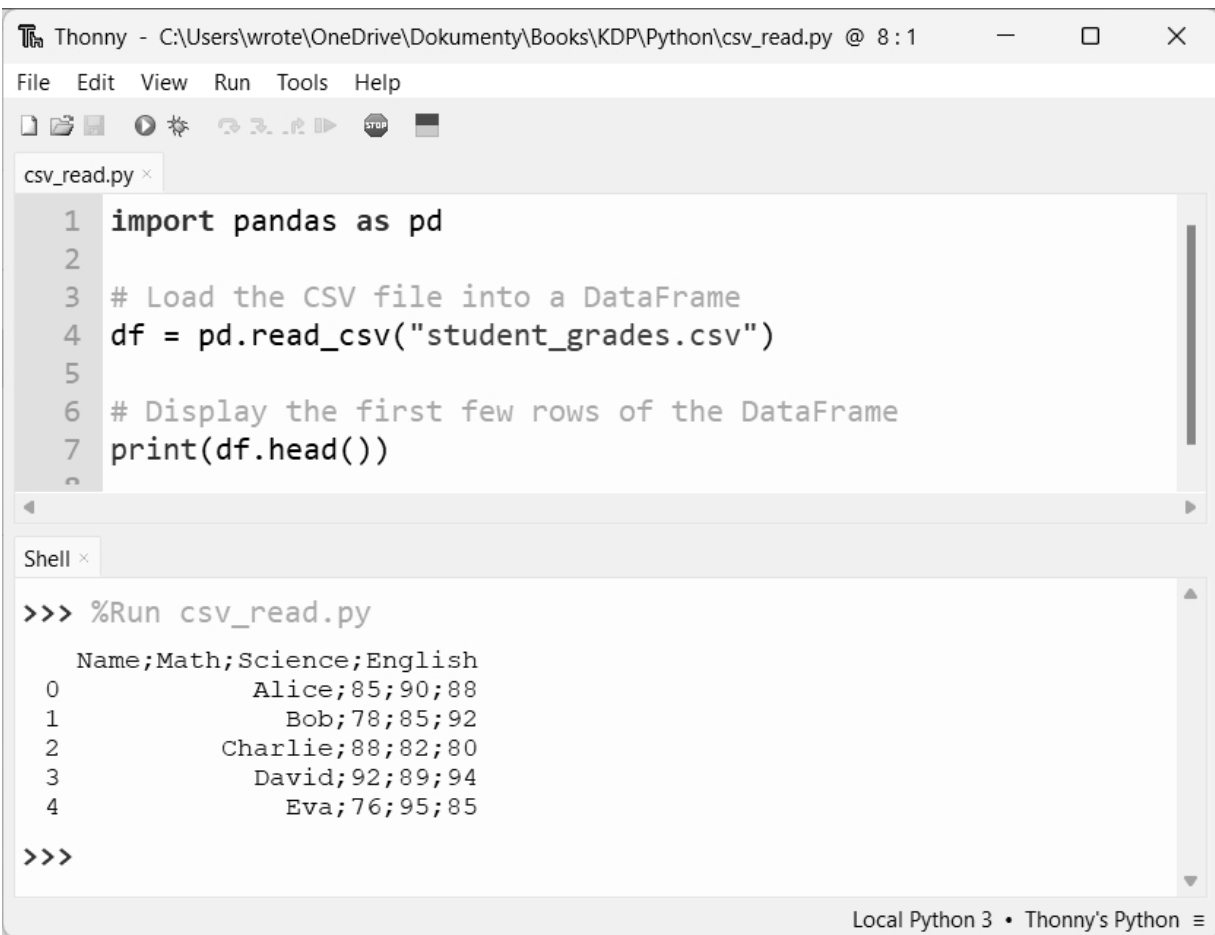
```
import pandas as pd
```

```
# Load the CSV file into a DataFrame
```

```
df = pd.read_csv("student_grades.csv")
```

```
# Display the first few rows of the DataFrame
```

```
print(df.head())
```



The screenshot shows the Thonny Python IDE interface. The top window, titled 'Thonny - C:\Users\wrote\OneDrive\Dokumenty\Books\KDP\Python\csv_read.py @ 8:1', contains a Python script. The script imports pandas as pd, loads a CSV file named 'student_grades.csv' into a DataFrame named df, and prints the first five rows of the DataFrame using df.head(). The bottom window, titled 'Shell', shows the output of running the script. It displays a table with five rows of student data, including names and grades in Math, Science, and English.

```
Thonny - C:\Users\wrote\OneDrive\Dokumenty\Books\KDP\Python\csv_read.py @ 8:1
File Edit View Run Tools Help
csv_read.py x
1 import pandas as pd
2
3 # Load the CSV file into a DataFrame
4 df = pd.read_csv("student_grades.csv")
5
6 # Display the first few rows of the DataFrame
7 print(df.head())
8

Shell x
>>> %Run csv_read.py
      Name;Math;Science;English
0      Alice;85;90;88
1      Bob;78;85;92
2      Charlie;88;82;80
3      David;92;89;94
4      Eva;76;95;85
>>>

Local Python 3 • Thonny's Python
```

Explanation:

- `pd.read_csv()` reads the CSV file and converts it into a DataFrame.

- `df.head()` displays the first 5 rows of the DataFrame, giving you a quick overview of the data.

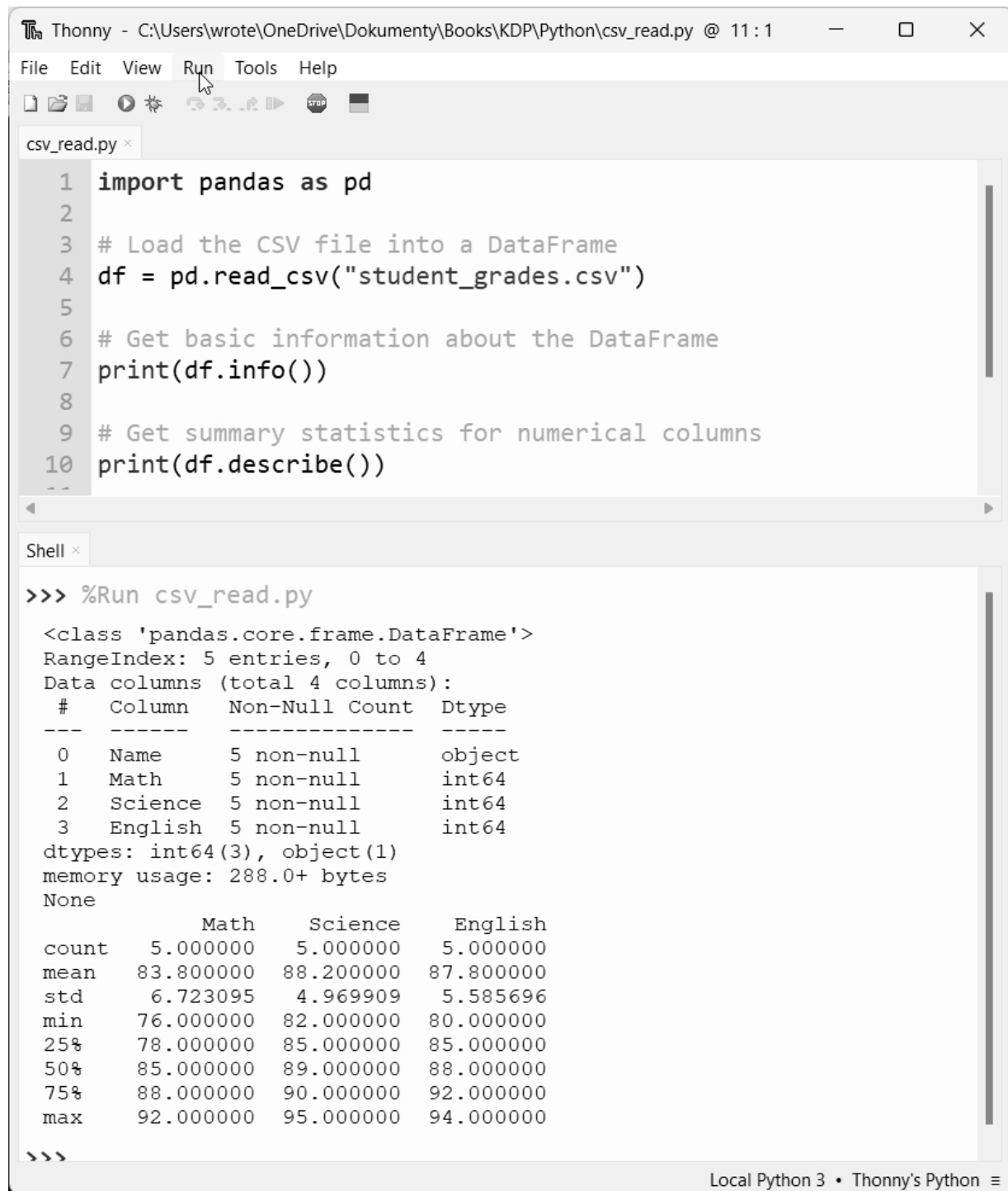
Example Data (student_grades.csv):

	A	B	C	D
1	Name	Math	Science	English
2	Alice	85	90	88
3	Bob	78	85	92
4	Charlie	88	82	80
5	David	92	89	94
6	Eva	76	95	85

After loading the data, you can easily inspect and understand its structure. Pandas provides various methods to explore the data, such as checking the number of rows and columns, identifying data types, and viewing a summary of the statistics.

```
# Get basic information about the DataFrame
print(df.info())

# Get summary statistics for numerical columns
print(df.describe())
```

The image shows a screenshot of the Thonny Python IDE. The main window displays a Python script named 'csv_read.py' with the following code:

```
1 import pandas as pd
2
3 # Load the CSV file into a DataFrame
4 df = pd.read_csv("student_grades.csv")
5
6 # Get basic information about the DataFrame
7 print(df.info())
8
9 # Get summary statistics for numerical columns
10 print(df.describe())
```

The 'Run' button in the top toolbar is highlighted. Below the code editor is a 'Shell' window showing the output of the script execution. The output starts with the command prompt '>>> %Run csv_read.py' followed by the DataFrame's information and summary statistics.

```
>>> %Run csv_read.py
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 4 columns):
#   Column   Non-Null Count  Dtype
---  ---
0   Name     5 non-null      object
1   Math     5 non-null      int64
2   Science  5 non-null      int64
3   English  5 non-null      int64
dtypes: int64(3), object(1)
memory usage: 288.0+ bytes
None
```

Below this, a summary statistics table is displayed for the numerical columns (Math, Science, English).

	Math	Science	English
count	5.000000	5.000000	5.000000
mean	83.800000	88.200000	87.800000
std	6.723095	4.969909	5.585696
min	76.000000	82.000000	80.000000
25%	78.000000	85.000000	85.000000
50%	85.000000	89.000000	88.000000
75%	88.000000	90.000000	92.000000
max	92.000000	95.000000	94.000000

The shell window ends with '>>>'. The bottom status bar indicates 'Local Python 3 • Thonny's Python'.

Note! The result of the program execution depends on the characters used to separate columns in the .csv file. I used a comma.

3. Filtering and Sorting Data

Once your data is loaded, you can start filtering and sorting it to find valuable insights. For example, you might want to know which students

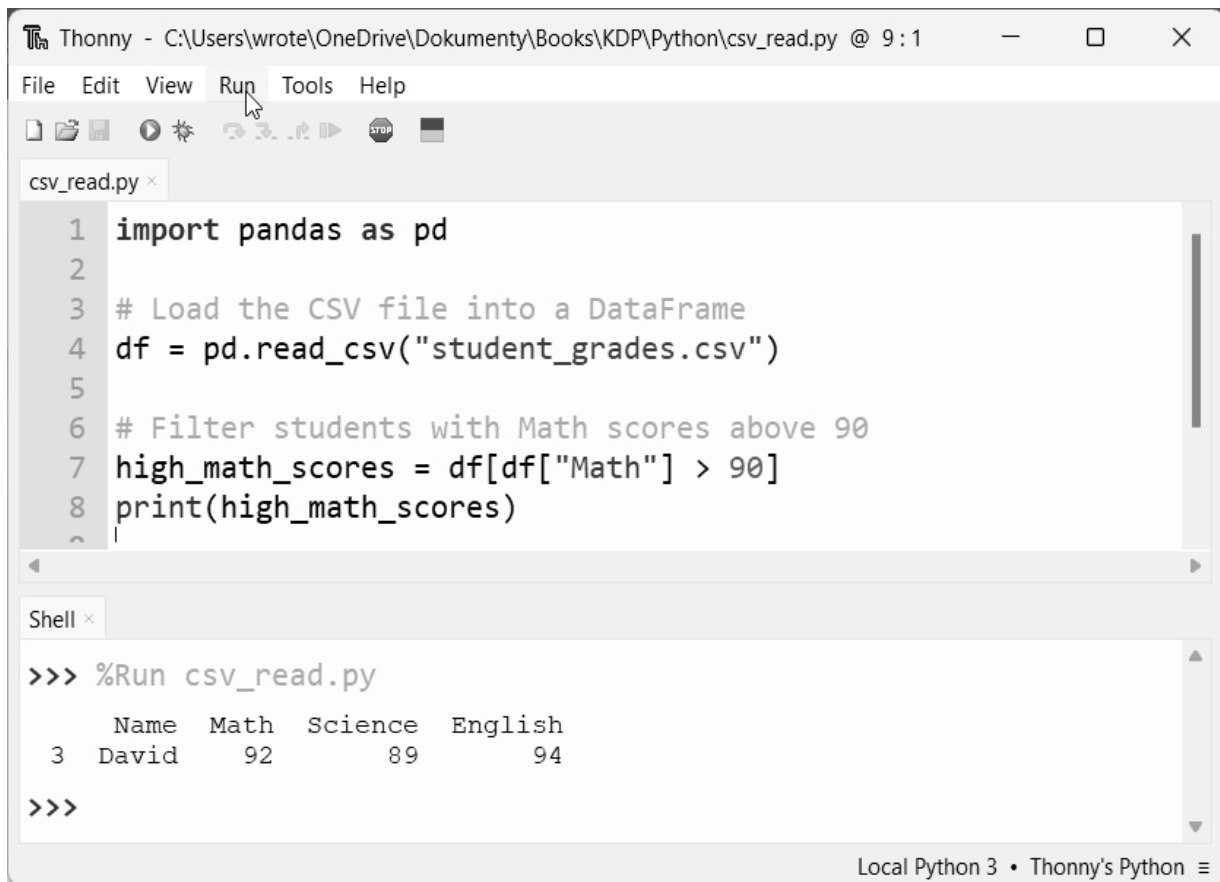
scored above 90 in Math or sort the data by Science grades.

Filtering Data

If you want to find all students who scored more than 90 in Math, you can filter the DataFrame like this:

```
# Filter students with Math scores above 90
high_math_scores = df[df["Math"] > 90]
print(high_math_scores)
```

Output:

The screenshot shows the Thonny Python IDE interface. The top window, titled 'Thonny - C:\Users\wrote\OneDrive\Dokumenty\Books\KDP\Python\csv_read.py @ 9:1', contains a Python script. The script imports pandas as pd, loads a CSV file named 'student_grades.csv' into a DataFrame df, filters for students with Math scores above 90, and prints the resulting DataFrame. The bottom window, titled 'Shell', shows the output of running the script. It displays the command prompt prompt '>>>>' followed by the command '%Run csv_read.py'. The output is a DataFrame with columns 'Name', 'Math', 'Science', and 'English', showing a single row for 'David' with Math score 92, Science score 89, and English score 94. The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

Here, the condition `df["Math"] > 90` creates a filter, and Pandas returns a new DataFrame containing only the rows where the condition is met.

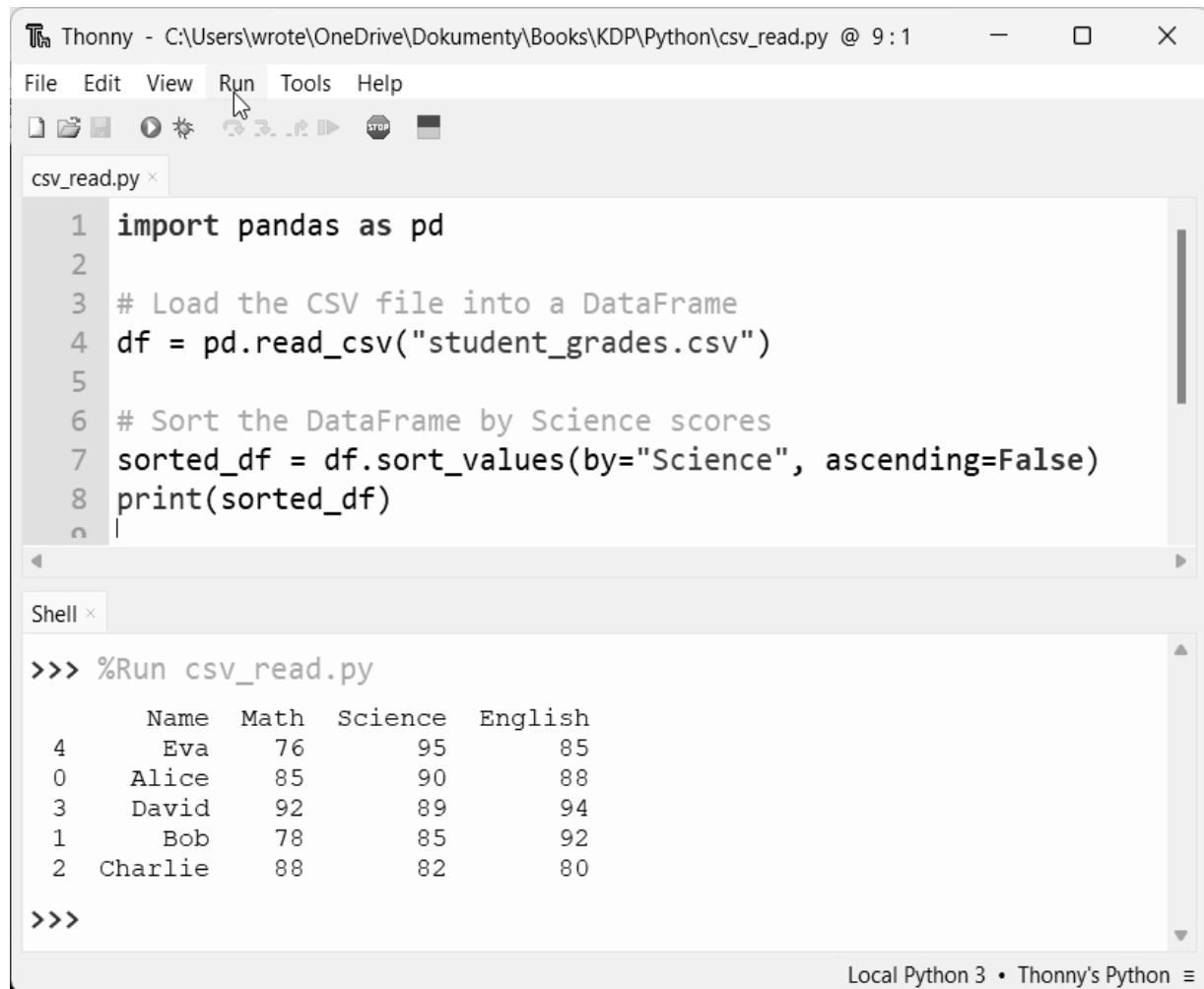
Sorting Data

To sort the data by Science scores in descending order (highest scores first):

```
# Sort the DataFrame by Science scores
sorted_df = df.sort_values(by="Science", ascending=False)
```

```
print(sorted_df)
```

Output:



The screenshot shows the Thonny Python IDE interface. The top window, titled 'Thonny - C:\Users\wrote\OneDrive\Dokumenty\Books\KDP\Python\csv_read.py @ 9:1', contains a Python script. The script imports pandas as 'pd', loads a CSV file 'student_grades.csv' into a DataFrame 'df', sorts it by 'Science' scores in descending order, and prints the result. The 'Run' button is highlighted. Below the script is a 'Shell' window showing the execution output. The output displays a table of student grades sorted by Science scores. The table has columns: Name, Math, Science, and English. The rows are: 4 Eva (Math: 76, Science: 95, English: 85), 0 Alice (Math: 85, Science: 90, English: 88), 3 David (Math: 92, Science: 89, English: 94), 1 Bob (Math: 78, Science: 85, English: 92), and 2 Charlie (Math: 88, Science: 82, English: 80). The shell prompt is '>>>'.

```
1 import pandas as pd
2
3 # Load the CSV file into a DataFrame
4 df = pd.read_csv("student_grades.csv")
5
6 # Sort the DataFrame by Science scores
7 sorted_df = df.sort_values(by="Science", ascending=False)
8 print(sorted_df)
```

```
>>> %Run csv_read.py
```

	Name	Math	Science	English
4	Eva	76	95	85
0	Alice	85	90	88
3	David	92	89	94
1	Bob	78	85	92
2	Charlie	88	82	80

```
>>>
```

Local Python 3 • Thonny's Python

This command sorts the DataFrame by the **Science** column, with the highest scores appearing at the top.

4. Handling Missing Data

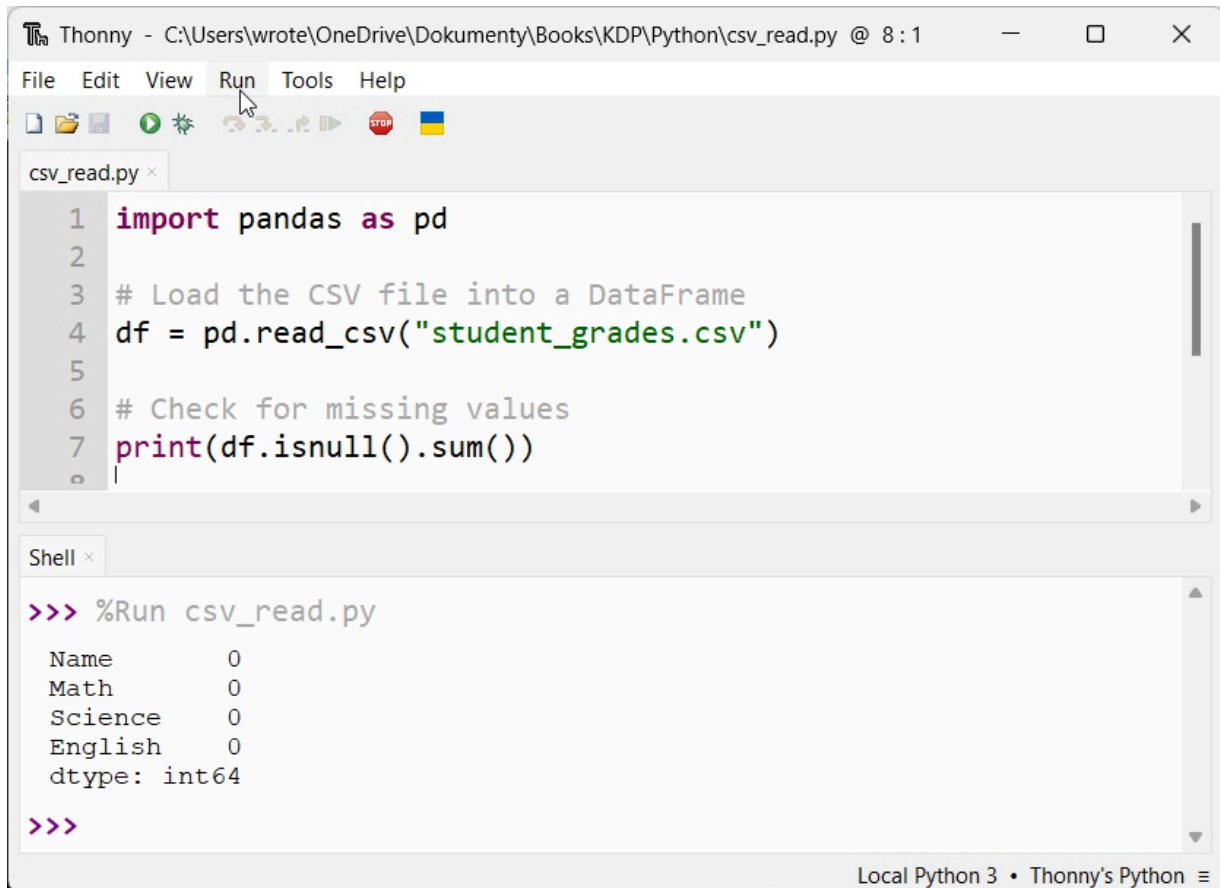
In the real world, data is often incomplete, meaning some values might be missing. Pandas makes it easy to identify and handle these missing values.

Example: Checking for Missing Data

You can check for missing data in the DataFrame using the `isnull()` function:

```
# Check for missing values
print(df.isnull().sum())
```

This will show you how many missing values are in each column. If there are any, you can decide how to handle them.



The screenshot shows the Thonny Python IDE interface. The main editor window displays a Python script named `csv_read.py` with the following code:

```
1 import pandas as pd
2
3 # Load the CSV file into a DataFrame
4 df = pd.read_csv("student_grades.csv")
5
6 # Check for missing values
7 print(df.isnull().sum())
```

Below the editor is a Shell window showing the output of running the script:

```
>>> %Run csv_read.py
Name      0
Math      0
Science   0
English   0
dtype: int64
>>>
```

The status bar at the bottom indicates "Local Python 3 • Thonny's Python".

Filling or Dropping Missing Data

There are several ways to handle missing data:

- **Filling missing values** with a default value (like the average of the column).
- **Dropping rows** that contain missing values.

Here's how you can fill missing values with the average of the column:

```
# Fill missing Math scores with the average Math score
df["Math"].fillna(df["Math"].mean(), inplace=True)
```

Alternatively, you can drop any rows that have missing values:

```
# Drop rows with missing values
df.dropna(inplace=True)
```

5. Grouping and Summarizing Data

Grouping data by certain criteria and summarizing it is a powerful way to analyze trends or get insights.

Example: Grouping Data by Categories

Let's say you want to group students based on their performance in Math. You can categorize them as "High", "Medium", or "Low" performers and count how many students fall into each category.

```
# Define a function to categorize students
def categorize_math(score):
    if score >= 90:
        return "High"
    elif score >= 80:
        return "Medium"
    else:
        return "Low"

# Apply the function to the Math column
df["Math Category"] = df["Math"].apply(categorize_math)

# Group by the new Math Category and count students
grouped_data = df.groupby("Math Category").size()
print(grouped_data)
```

Output:

The screenshot shows the Thonny Python IDE interface. The top window, titled 'csv_read.py *', contains the following Python code:

```
1 import pandas as pd
2
3 # Load the CSV file into a DataFrame
4 df = pd.read_csv("student_grades.csv")
5
6 # Define a function to categorize students
7 def categorize_math(score):
8     if score >= 90:
9         return "High"
10    elif score >= 80:
11        return "Medium"
12    else:
13        return "Low"
14
15 # Apply the function to the Math column
16 df["Math Category"] = df["Math"].apply(categorize_math)
17
18 # Group by the new Math Category and count students
19 grouped_data = df.groupby("Math Category").size()
20 print(grouped_data)
```

The bottom window, titled 'Shell', shows the output of running the script:

```
>>> %Run csv_read.py
Math Category
High      1
Low       2
Medium    2
dtype: int64
>>>
```

The status bar at the bottom right indicates 'Local Python 3 • Thonny's Python'.

This grouping allows you to see how students are distributed across different performance levels.

6. Real-World Application: Calculating Averages and Grades

One common task in schools is calculating the overall average for each student and determining if they passed or failed based on their average grade. Here's how you can do that with Pandas:

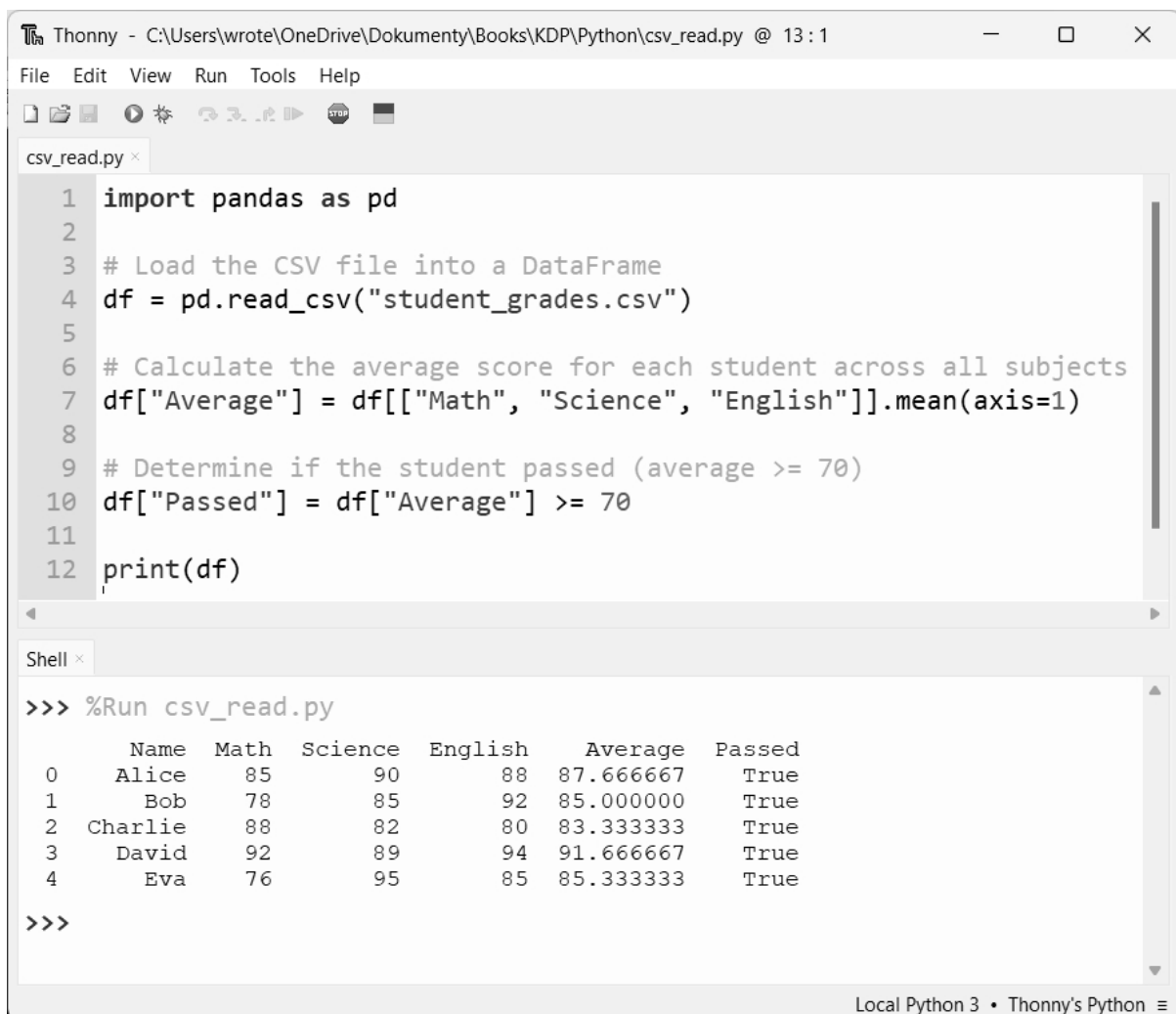
Example: Calculating the Average Score for Each Student

```
# Calculate the average score for each student across all subjects
df["Average"] = df[["Math", "Science", "English"]].mean(axis=1)

# Determine if the student passed (average >= 70)
df["Passed"] = df["Average"] >= 70

print(df)
```

Output:



The screenshot shows a Thonny Python IDE window with a file named 'csv_read.py' open. The script contains the following code:

```
1 import pandas as pd
2
3 # Load the CSV file into a DataFrame
4 df = pd.read_csv("student_grades.csv")
5
6 # Calculate the average score for each student across all subjects
7 df["Average"] = df[["Math", "Science", "English"]].mean(axis=1)
8
9 # Determine if the student passed (average >= 70)
10 df["Passed"] = df["Average"] >= 70
11
12 print(df)
```

Below the code editor is a shell window showing the output of the script:

```
>>> %Run csv_read.py
```

	Name	Math	Science	English	Average	Passed
0	Alice	85	90	88	87.666667	True
1	Bob	78	85	92	85.000000	True
2	Charlie	88	82	80	83.333333	True
3	David	92	89	94	91.666667	True
4	Eva	76	95	85	85.333333	True

The shell window ends with three prompt characters: >>>

Here, we've calculated the average grade for each student across Math, Science, and English, and then determined whether they passed based on

whether their average was 70 or higher.

7. Why Pandas Is Essential for Real-World Data Problems

Pandas allows you to:

- **Handle large datasets** with ease, even if they contain missing or messy data.
- **Perform complex operations** such as filtering, sorting, and grouping data in just a few lines of code.
- **Analyze and visualize data** in a way that provides clear insights, whether you're working with student grades, business reports, or scientific data.

With Pandas, you can solve real-world problems quickly and efficiently, transforming raw data into actionable information.

Summary

Using **Pandas** to solve real-world data problems opens up countless possibilities for handling and analyzing data. From loading and inspecting data to filtering, sorting, and grouping, Pandas provides all the tools you need to extract meaningful insights from your datasets. By mastering these techniques, you'll be able to tackle a wide range of problems, making Pandas an essential skill for students, teachers, and anyone working with data.

Keep practicing with real-world datasets, and soon you'll be using Pandas to solve complex problems with confidence!

Chapter 9: Visualizing Solutions with Matplotlib

Creating Simple Charts and Graphs to Understand Data

Visualizing data is one of the most powerful ways to understand patterns, trends, and relationships. In Python, **Matplotlib** is the go-to library for creating simple charts and graphs. Whether you're trying to see how a class performed on an exam or track temperature changes over time, visual representations make data easier to grasp.

In this section, we'll explore how to use Matplotlib to create simple charts and graphs that can help you better understand your data. By the end of this chapter, you'll know how to make effective visualizations that provide insights at a glance, even if you're just starting with Python.

1. Why Visualize Data?

Before diving into how to create charts and graphs, let's understand **why** visualizing data is important:

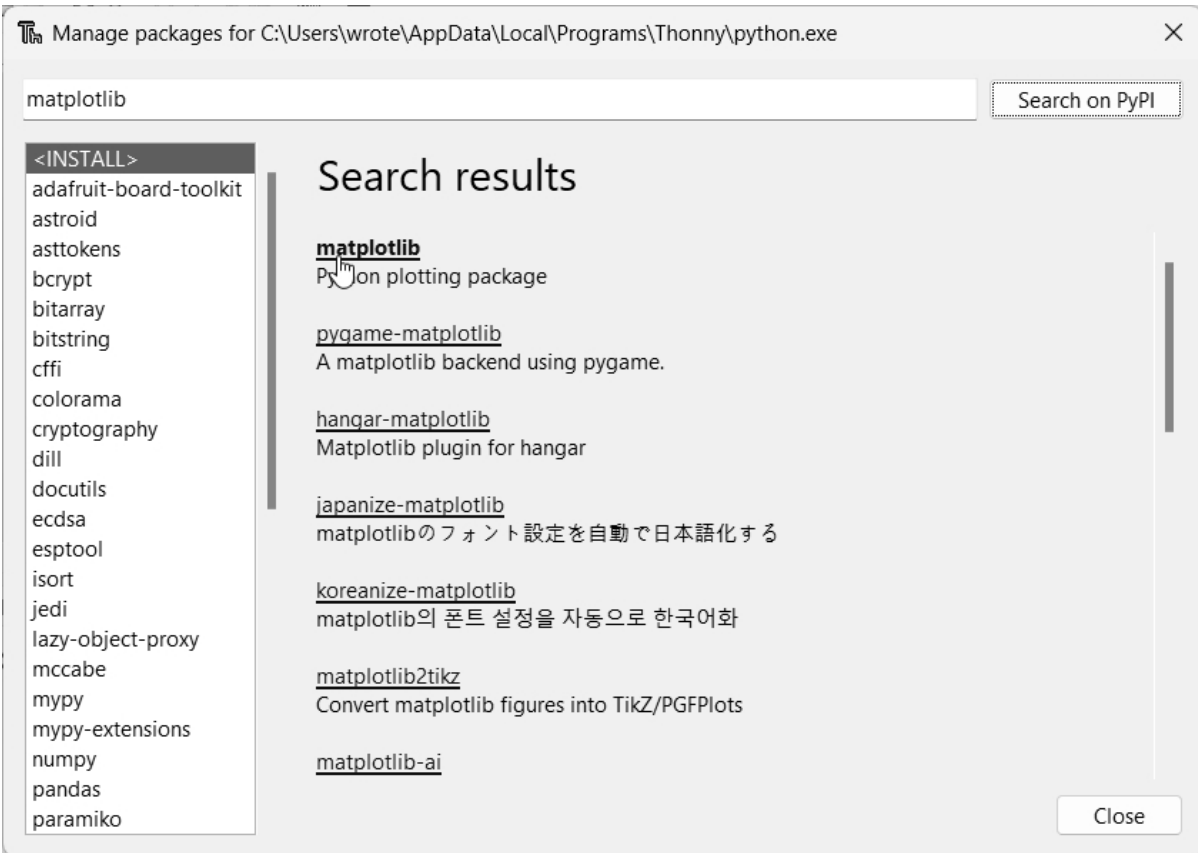
- **Clarifies complex information:** Raw data can be overwhelming, but a chart can summarize it into something easy to digest.
- **Identifies trends and patterns:** Visuals like line graphs help you spot trends over time, while bar charts make comparisons clearer.
- **Communicates insights:** Charts and graphs are great for sharing results with others in a clear, effective way.

2. Getting Started with Matplotlib

The first step in creating charts is to install and import the Matplotlib library. You can install it using pip if you haven't already:

```
pip install matplotlib
```

or using Thonny:



Next, import it into your Python script:

```
import matplotlib.pyplot as plt
```

The **pyplot** module in Matplotlib provides a wide range of functions to create different types of charts, like line plots, bar charts, and pie charts.

3. Creating a Simple Line Chart

A **line chart** is useful for visualizing data that changes over time, such as temperature, grades, or attendance. Let's create a simple line chart that shows student grades over several tests.

Example: Plotting Student Grades

```
import matplotlib.pyplot as plt

# Data for the chart
tests = [1, 2, 3, 4, 5]
grades = [85, 90, 88, 92, 87]

# Create the line chart
plt.plot(tests, grades, marker='o')

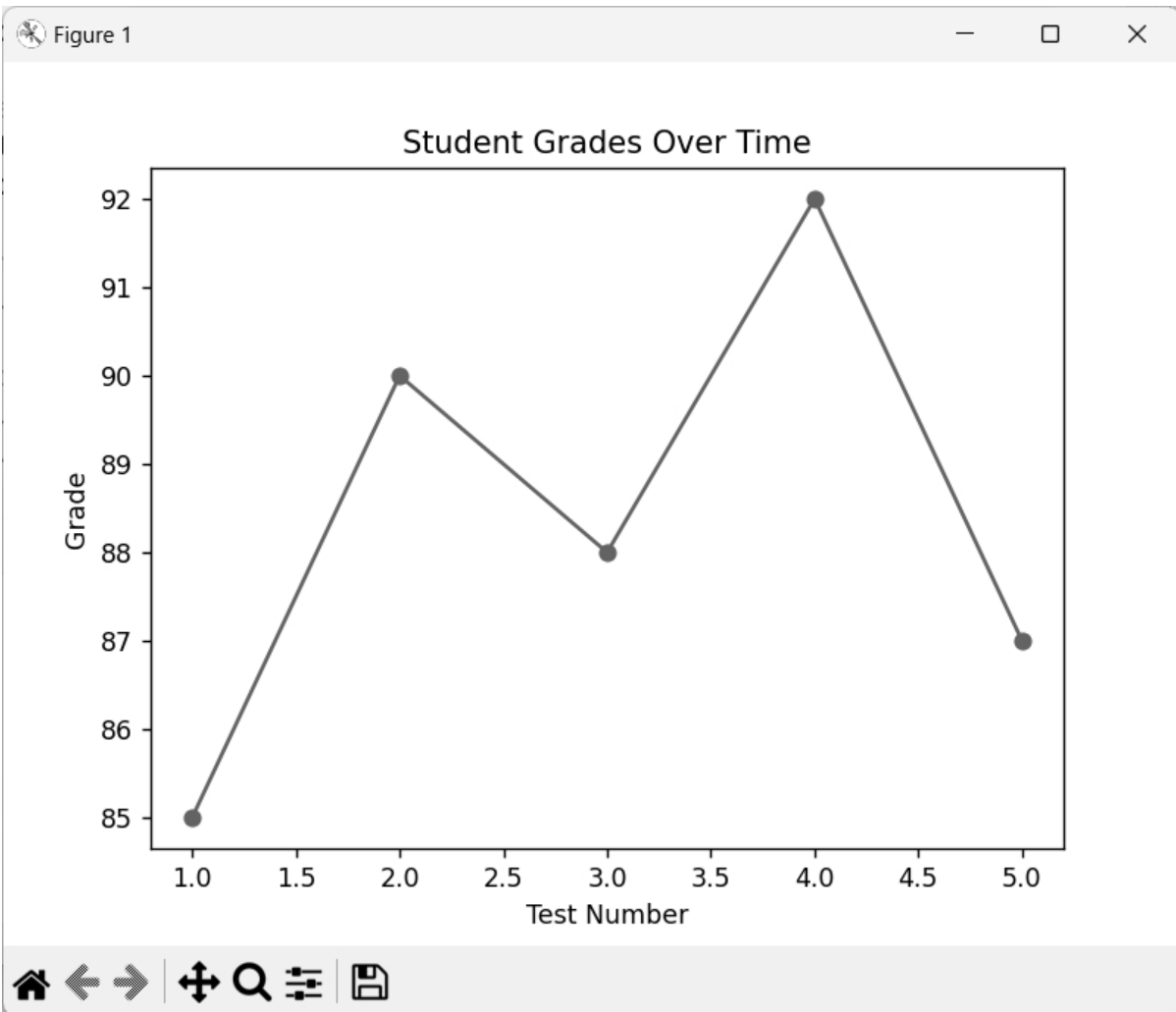
# Add labels and title
plt.title("Student Grades Over Time")
plt.xlabel("Test Number")
plt.ylabel("Grade")

# Show the chart
plt.show()
```

Explanation:

- **plt.plot()** creates a line chart with **tests** on the x-axis and **grades** on the y-axis.
- **marker='o'** adds a circle marker to each data point for clarity.
- **plt.title()**, **plt.xlabel()**, and **plt.ylabel()** add labels and a title to the chart.
- **plt.show()** displays the chart in a window.

Output: A simple line chart showing the student's grades for five tests.



4. Creating a Bar Chart

Bar charts are great for comparing different categories or groups. For example, you might want to compare the number of students who passed and failed a test.

Example: Comparing Passed and Failed Students

```
import matplotlib.pyplot as plt

# Data for the chart
categories = ["Passed", "Failed"]
student_counts = [25, 5]

# Create the bar chart
plt.bar(categories, student_counts, color=['green', 'red'])
```

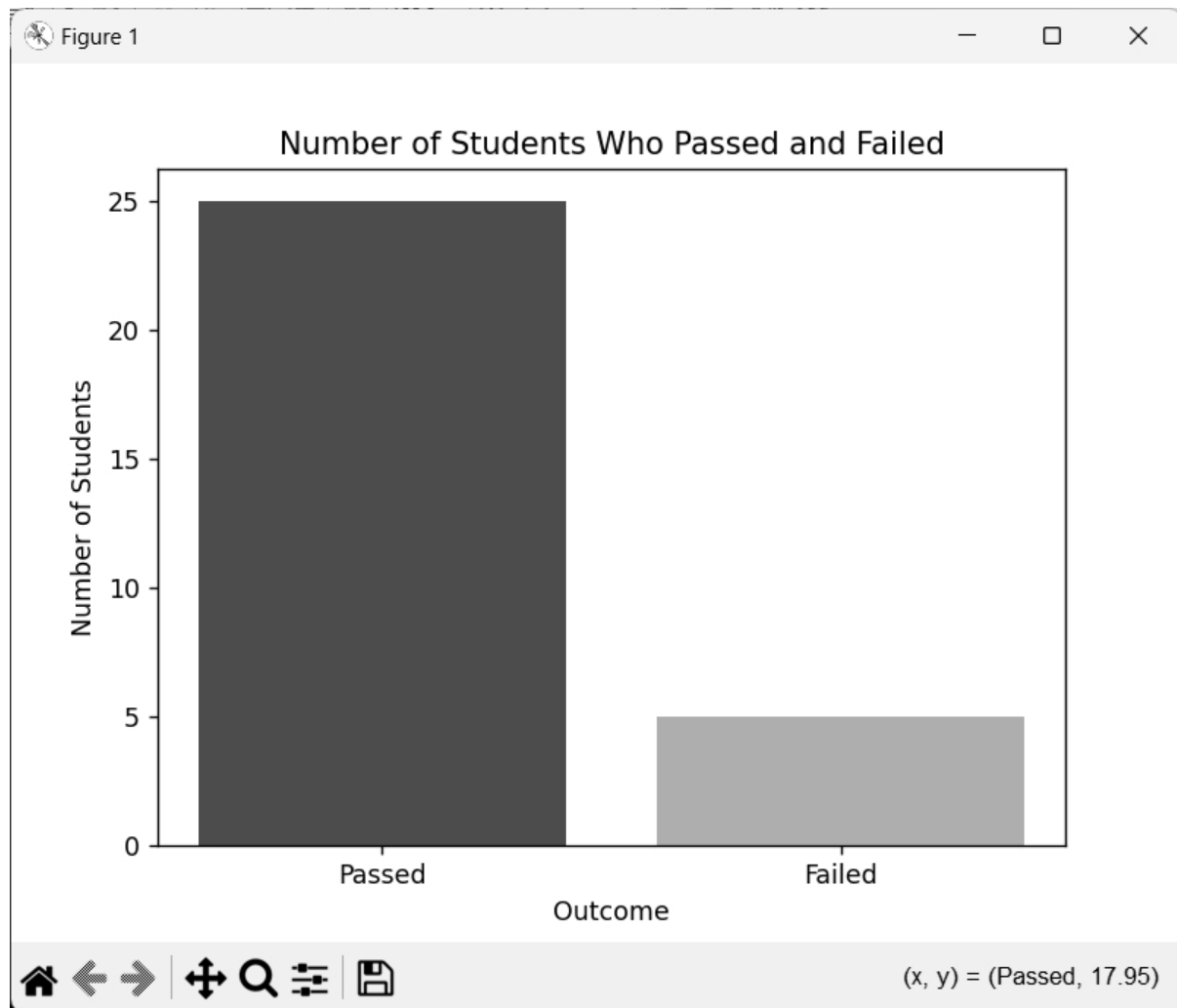
```
# Add labels and title
plt.title("Number of Students Who Passed and Failed")
plt.xlabel("Outcome")
plt.ylabel("Number of Students")

# Show the chart
plt.show()
```

Explanation:

- **plt.bar()** creates a bar chart with the categories (Passed and Failed) on the x-axis and the number of students on the y-axis.
- The **color** argument customizes the colors of the bars.

Output: A bar chart showing how many students passed and how many failed.



5. Creating a Pie Chart

A **pie chart** is useful when you want to show proportions or percentages, like how class time is divided between different subjects.

Example: Dividing Class Time Between Subjects

```
import matplotlib.pyplot as plt

# Data for the chart
subjects = ["Math", "Science", "English"]
time_spent = [3, 4, 2] # Hours spent on each subject

# Create the pie chart
plt.pie(time_spent, labels=subjects, autopct='%1.1f%%', startangle=90)

# Add a title
```

```
plt.title("Time Spent on Each Subject in a Day")
```

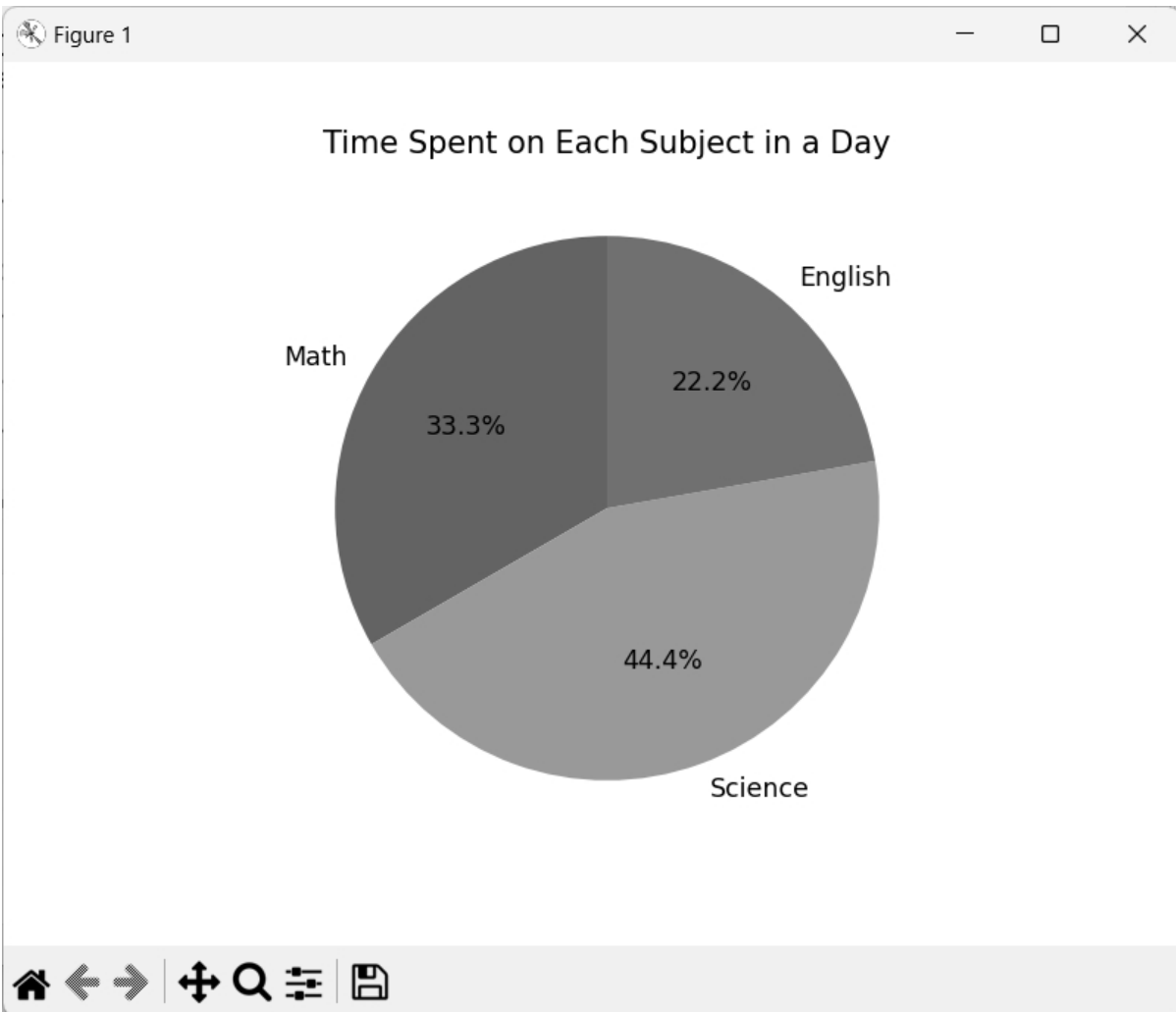
```
# Show the chart
```

```
plt.show()
```

Explanation:

- **plt.pie()** creates a pie chart with the **time_spent** values and labels them with the **subjects**.
- **autopct='%1.1f%%'** displays the percentage of each segment.
- **startangle=90** rotates the pie chart for better visibility.

Output: A pie chart showing how class time is divided between Math, Science, and English.



6. Customizing Your Charts

Matplotlib allows for a high degree of customization, so you can make your charts more informative and visually appealing.

Adding Grid Lines

Grid lines help make data points easier to read, especially in line charts:

```
import matplotlib.pyplot as plt

# Data for the chart
subjects = ["Math", "Science", "English"]
time_spent = [3, 4, 2] # Hours spent on each subject

#plt.plot(tests, grades, marker='o')
plt.title("Student Grades Over Time")
```

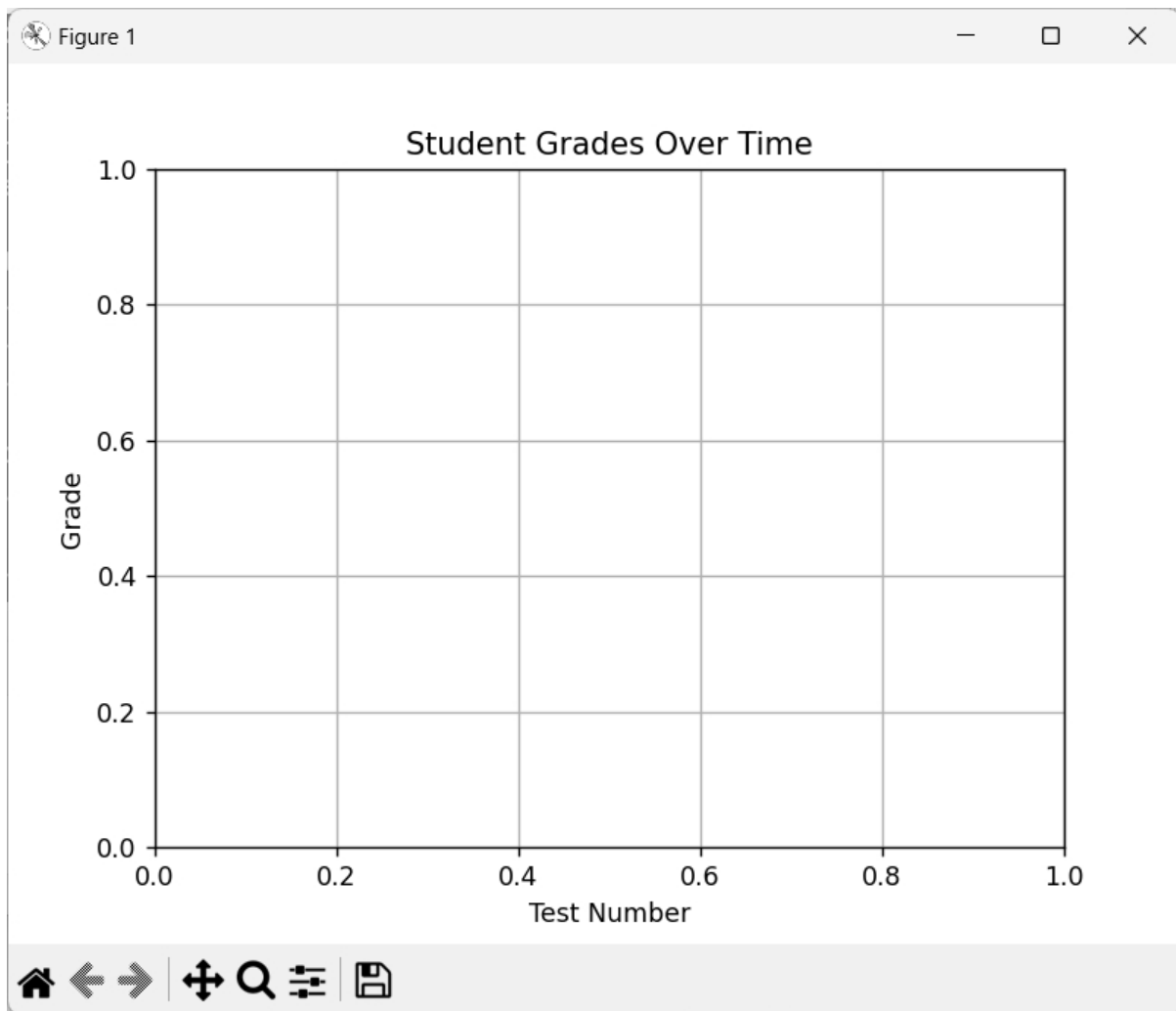
```
plt.xlabel("Test Number")
```

```
plt.ylabel("Grade")
```

```
# Add grid lines
```

```
plt.grid(True)
```

```
plt.show()
```



Adding a Legend

If you're comparing multiple data series on the same chart, adding a legend helps identify them:

```
import matplotlib.pyplot as plt
```

```
# Data for two students
```

```
grades_student1 = [85, 90, 88, 92, 87]
```



```
grades_student2 = [78, 82, 80, 86, 84]

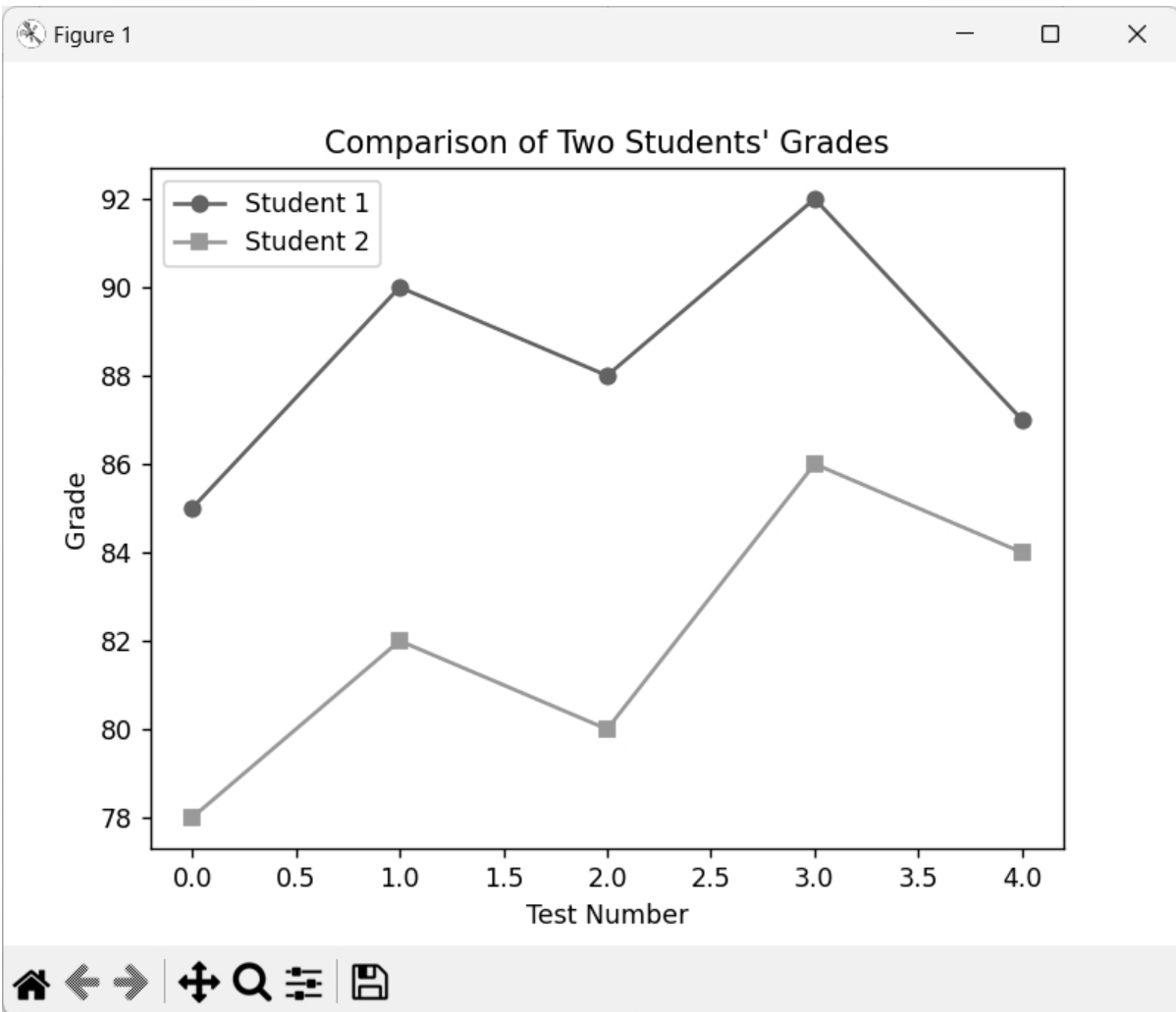
# Plot the data for both students
plt.plot(grades_student1, marker='o', label="Student 1")
plt.plot(grades_student2, marker='s', label="Student 2")

# Add labels, title, and legend
plt.title("Comparison of Two Students' Grades")
plt.xlabel("Test Number")
plt.ylabel("Grade")
plt.legend() # Add legend

plt.show()
```

Explanation:

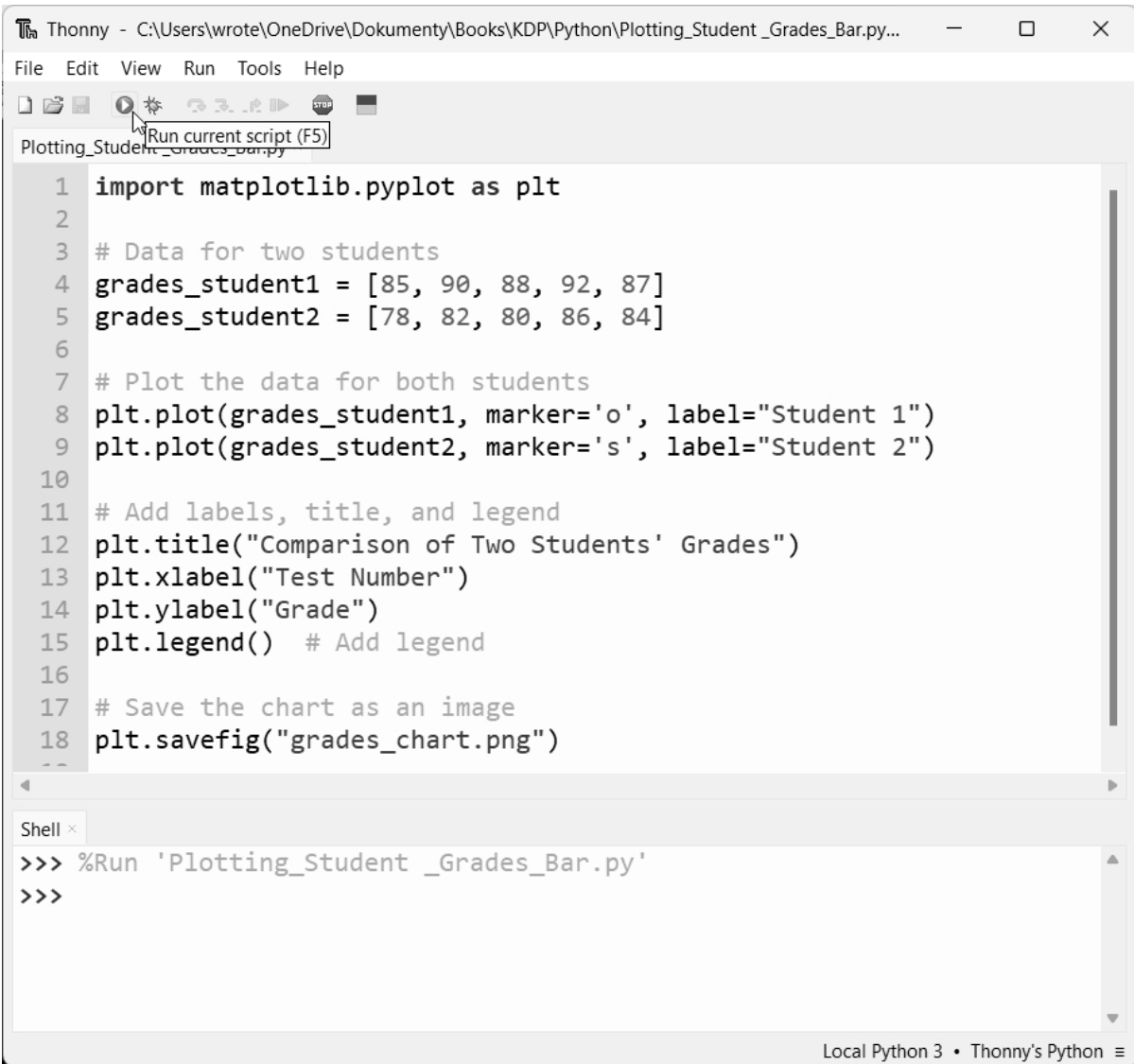
- **label="Student 1"** and **label="Student 2"** provide labels for each data series.
- **plt.legend()** displays the legend on the chart.



7. Saving Your Charts

Once you've created a chart, you might want to save it as an image to share or use in a report. Matplotlib makes this easy with the `savefig()` function:

```
# Save the chart as an image  
plt.savefig("grades_chart.png")
```



The screenshot shows the Thonny IDE interface. The top menu bar includes File, Edit, View, Run, Tools, and Help. Below the menu is a toolbar with icons for file operations and running code. The main editor window displays a Python script named 'Plotting_Student_Grades_Bar.py'. The script imports matplotlib.pyplot as plt, defines two lists of grades for two students, plots the data using plt.plot, adds a title and labels, and saves the figure as 'grades_chart.png'. The bottom panel shows the Shell window with the command '%Run \'Plotting_Student_Grades_Bar.py\'' and the output 'grades_chart.png'.

```
1 import matplotlib.pyplot as plt
2
3 # Data for two students
4 grades_student1 = [85, 90, 88, 92, 87]
5 grades_student2 = [78, 82, 80, 86, 84]
6
7 # Plot the data for both students
8 plt.plot(grades_student1, marker='o', label="Student 1")
9 plt.plot(grades_student2, marker='s', label="Student 2")
10
11 # Add labels, title, and legend
12 plt.title("Comparison of Two Students' Grades")
13 plt.xlabel("Test Number")
14 plt.ylabel("Grade")
15 plt.legend() # Add legend
16
17 # Save the chart as an image
18 plt.savefig("grades_chart.png")
```

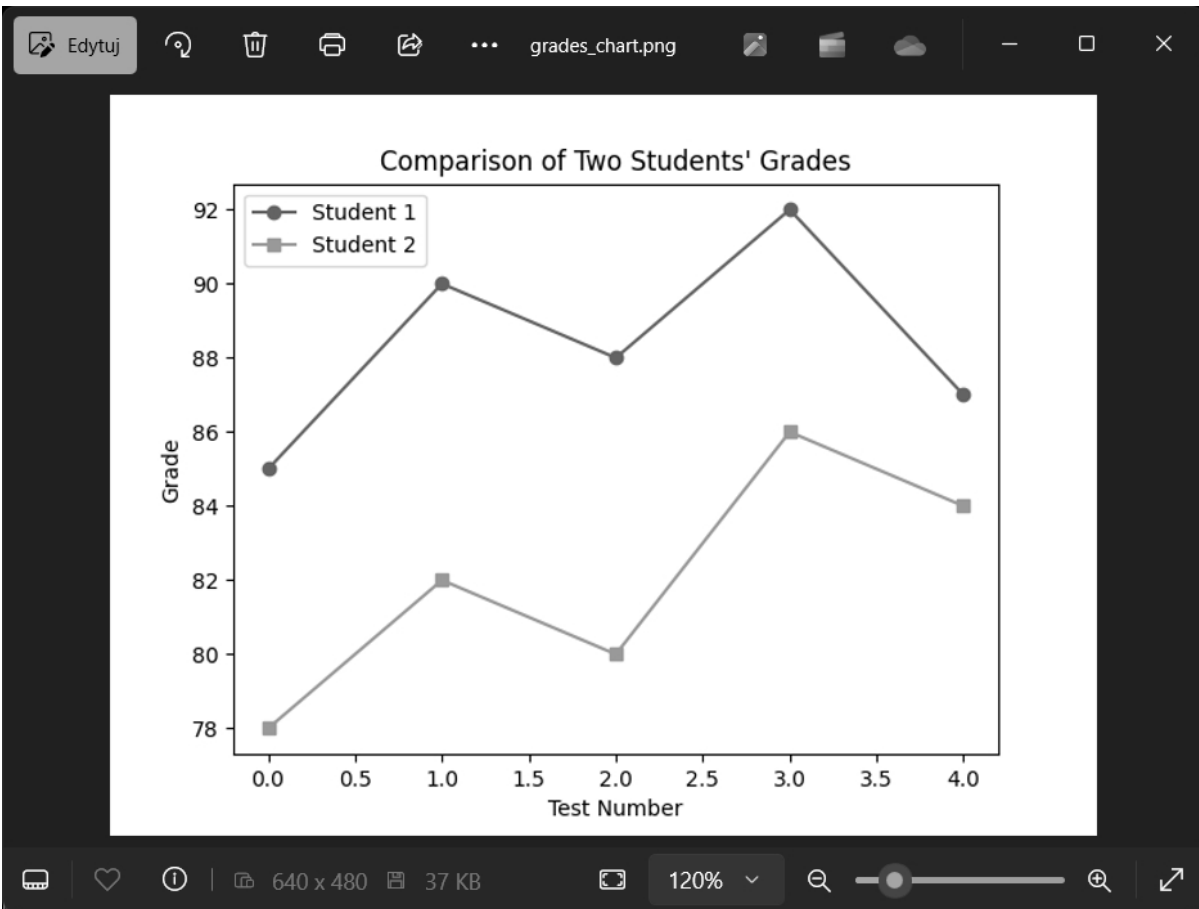
Shell x

```
>>> %Run 'Plotting_Student_Grades_Bar.py'
>>>
```

Local Python 3 • Thonny's Python

This saves the chart as a PNG file in the current working directory. You can also save it as other formats, like PDF or SVG.

 grades_chart	✓	12.09.2024 12:28	Plik PNG	38 KB
 Typ elementu: Plik PNG	?	12.09.2024 12:28	Python File	1 KB
Wymiary: 640 x 480				
 Rozmiar: 37,0 KB	?	12.09.2024 09:16	Python File	1 KB
Plotting_Student_Grades				
Stan dostępności: Dostępny na tym urządzeniu				



8. Choosing the Right Chart for Your Data

Choosing the right type of chart depends on the data you're working with and the message you want to convey:

- Use **line charts** to track changes over time (e.g., grades, temperature, or attendance).
- Use **bar charts** to compare categories (e.g., student performance or test results).
- Use **pie charts** to show proportions or percentages (e.g., time spent on subjects or classroom resources).

The right visualization can make all the difference in how well your audience understands the data.

Summary

Creating charts and graphs with Matplotlib is an excellent way to understand your data and present it in a clear, visual format. Whether you're tracking student grades, comparing test scores, or analyzing how time is spent in class, these simple visualizations can reveal patterns and insights that might not be obvious in raw numbers alone.

With just a few lines of code, you can generate powerful charts that make your data come to life. Keep experimenting with different types of charts, and soon you'll be creating visuals that not only help you understand data better but also communicate your findings effectively to others!

Application: Plotting Temperature Changes Over Time

Visualizing how temperatures change over time is a great way to understand patterns and trends in weather data. Whether you're tracking daily temperatures over the course of a week or looking at how temperatures vary across seasons, plotting this data using **Matplotlib** makes it easy to see the changes at a glance. In this section, we'll explore how to use Matplotlib to create a line chart that shows temperature changes over time.

By the end of this chapter, you'll be able to plot data like temperatures and use charts to reveal important trends, making your data more understandable and meaningful.

1. Understanding the Data

Before we start plotting, let's assume we have data representing the temperature recorded over seven days. Here's the data we'll be working with:

Day	Temperature (°C)
Monday	22
Tuesday	24
Wednesday	19
Thursday	21
Friday	23
Saturday	25
Sunday	20

This data can be easily visualized using a **line chart**, where the x-axis will represent the days of the week and the y-axis will represent the temperature in degrees Celsius.

2. Plotting the Temperature Data

To create a line chart of this temperature data, we'll use Matplotlib's `plot()` function. Here's the code to generate the chart:

```
import matplotlib.pyplot as plt

# Data for the plot
days = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"]
temperatures = [22, 24, 19, 21, 23, 25, 20]

# Create the line chart
plt.plot(days, temperatures, marker='o', linestyle='-', color='b')

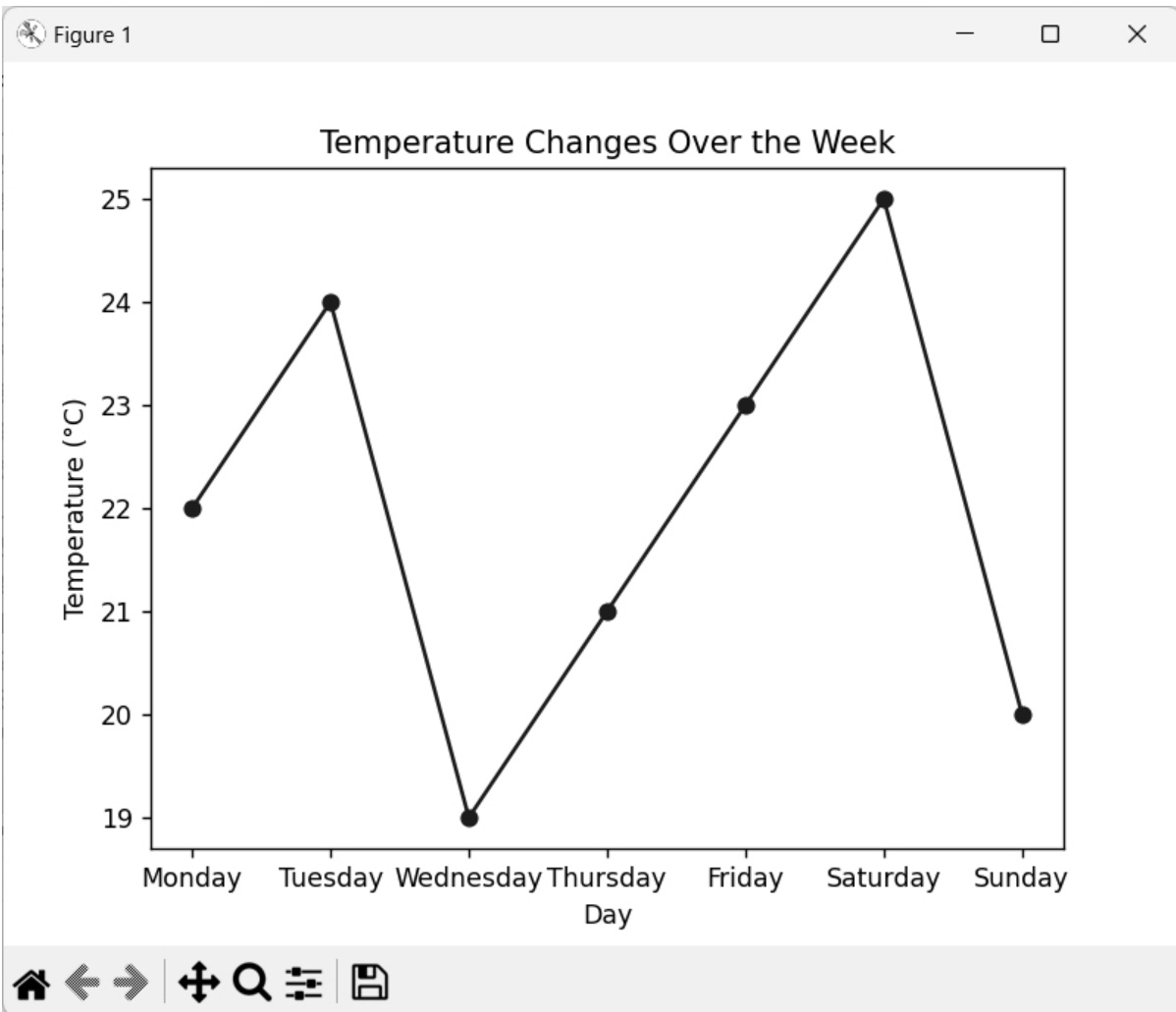
# Add titles and labels
plt.title("Temperature Changes Over the Week")
plt.xlabel("Day")
plt.ylabel("Temperature (°C)")

# Show the chart
plt.show()
```

Explanation:

- **days:** This list represents the x-axis, which contains the days of the week.
- **temperatures:** This list represents the y-axis, with the corresponding temperatures for each day.
- **plt.plot():** This function creates the line chart. We use `marker='o'` to display a dot at each data point and `color='b'` to make the line blue.
- **plt.title(), plt.xlabel(), and plt.ylabel():** These functions add a title and labels for the x and y axes.

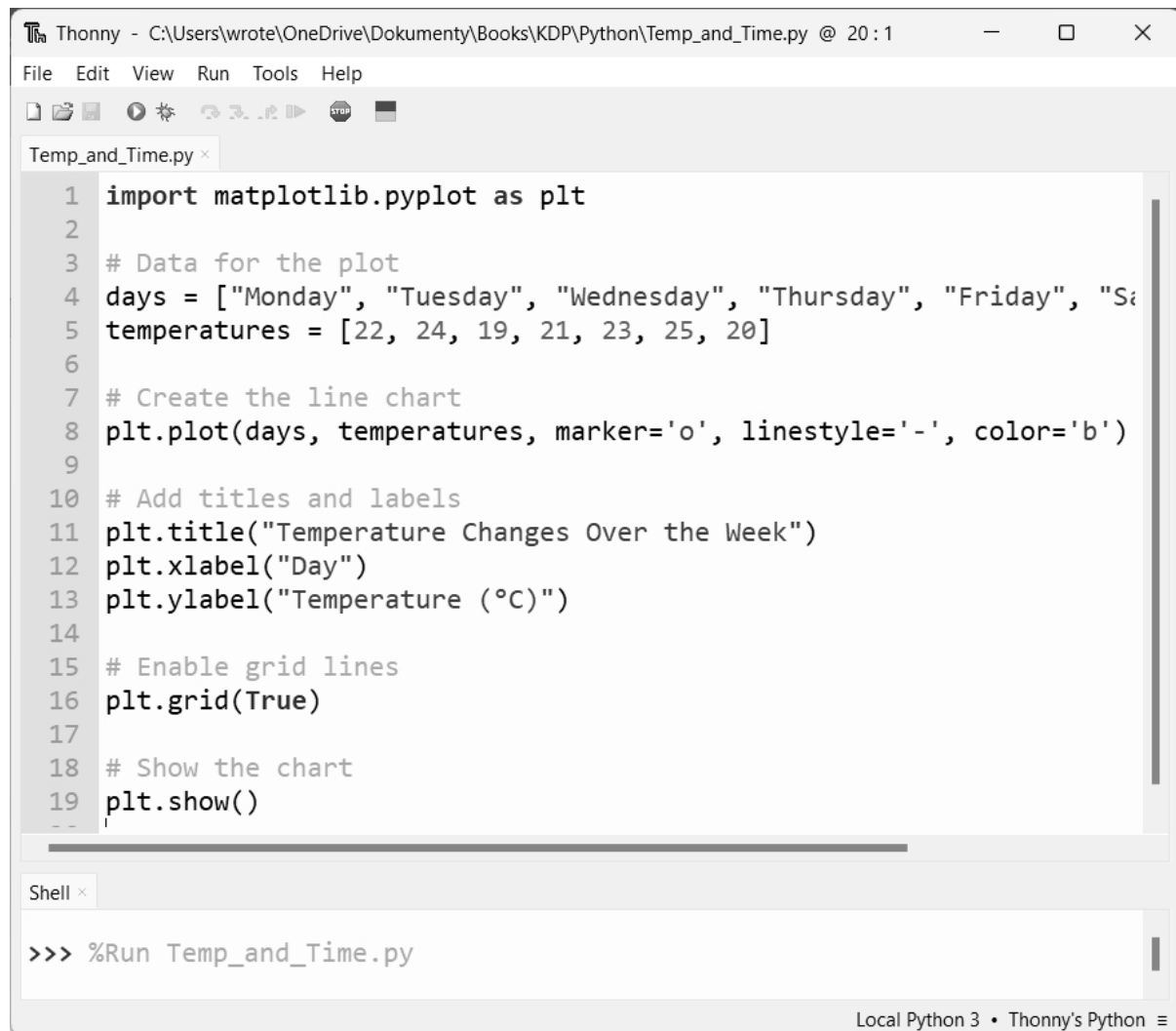
When you run this code, you'll see a line chart that visually shows how the temperature fluctuated over the course of the week.



3. Adding Grid Lines for Clarity

Grid lines can make it easier to read values from a chart, especially when you want to see the exact temperature for each day. You can add grid lines to the plot like this:

```
# Enable grid lines  
plt.grid(True)
```

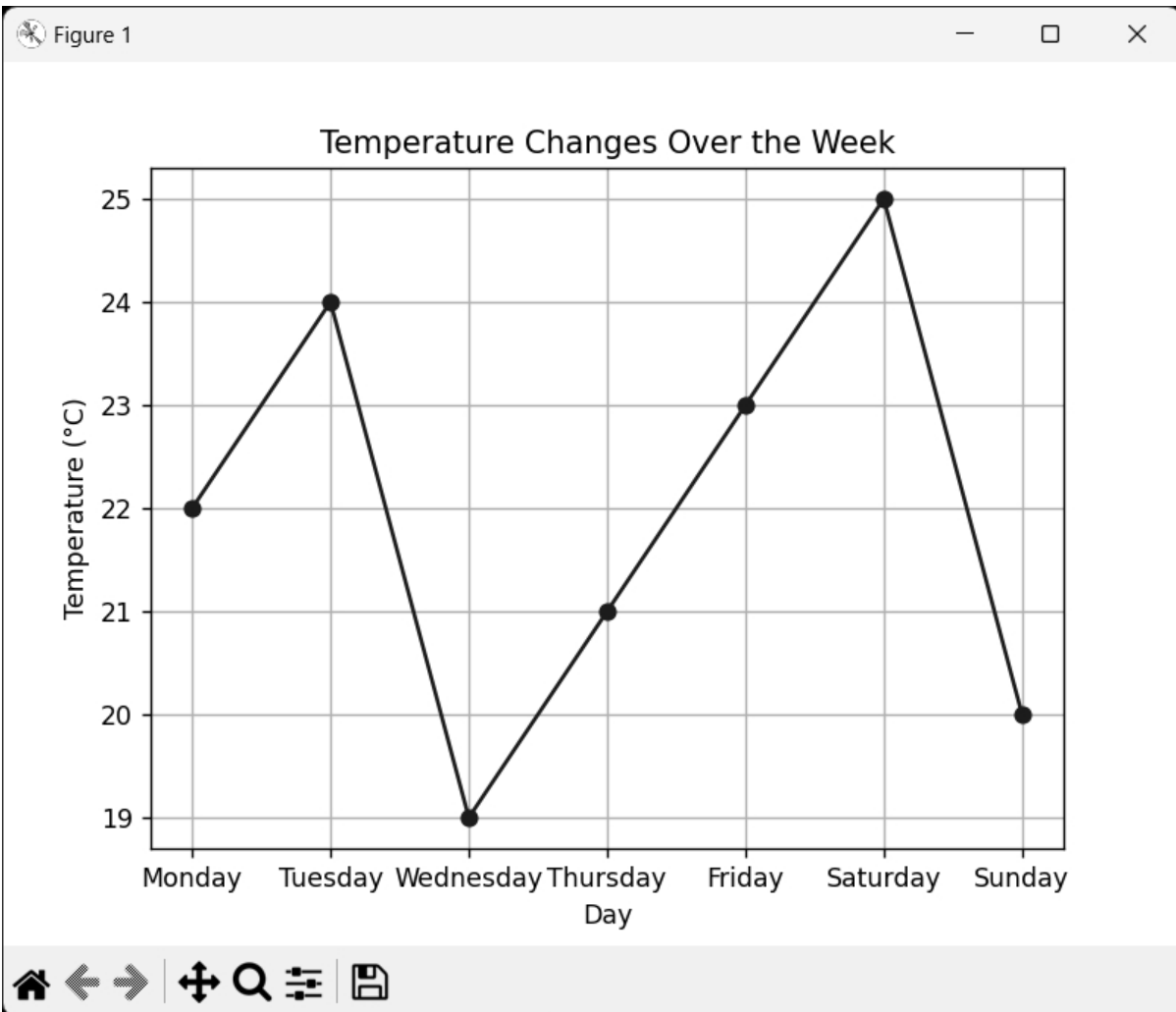
```
1 import matplotlib.pyplot as plt
2
3 # Data for the plot
4 days = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"]
5 temperatures = [22, 24, 19, 21, 23, 25, 20]
6
7 # Create the line chart
8 plt.plot(days, temperatures, marker='o', linestyle='--', color='b')
9
10 # Add titles and labels
11 plt.title("Temperature Changes Over the Week")
12 plt.xlabel("Day")
13 plt.ylabel("Temperature (°C)")
14
15 # Enable grid lines
16 plt.grid(True)
17
18 # Show the chart
19 plt.show()
```

Shell ×

```
>>> %Run Temp_and_Time.py
```

Local Python 3 • Thonny's Python

Add this line to the previous code, and it will display grid lines on the chart, making it easier to interpret the data.



4. Highlighting Temperature Peaks and Dips

Sometimes, it's useful to highlight key points in the data, like the highest and lowest temperatures recorded during the week. Here's how you can do that:

```
import matplotlib.pyplot as plt
```

```
# Data for the plot
```

```
days = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"]
```

```
temperatures = [22, 24, 19, 21, 23, 25, 20]
```

```
# Highlight the maximum and minimum temperatures
```

```
max_temp = max(temperatures)
```

```
min_temp = min(temperatures)
```

```
# Find the corresponding days
max_day = days[temperatures.index(max_temp)]
min_day = days[temperatures.index(min_temp)]

# Plot the temperatures
plt.plot(days, temperatures, marker='o', linestyle='-', color='b')

# Highlight the max and min points
plt.scatter(max_day, max_temp, color='red', label=f"Max Temp: {max_temp}°C")
plt.scatter(min_day, min_temp, color='green', label=f"Min Temp: {min_temp}°C")

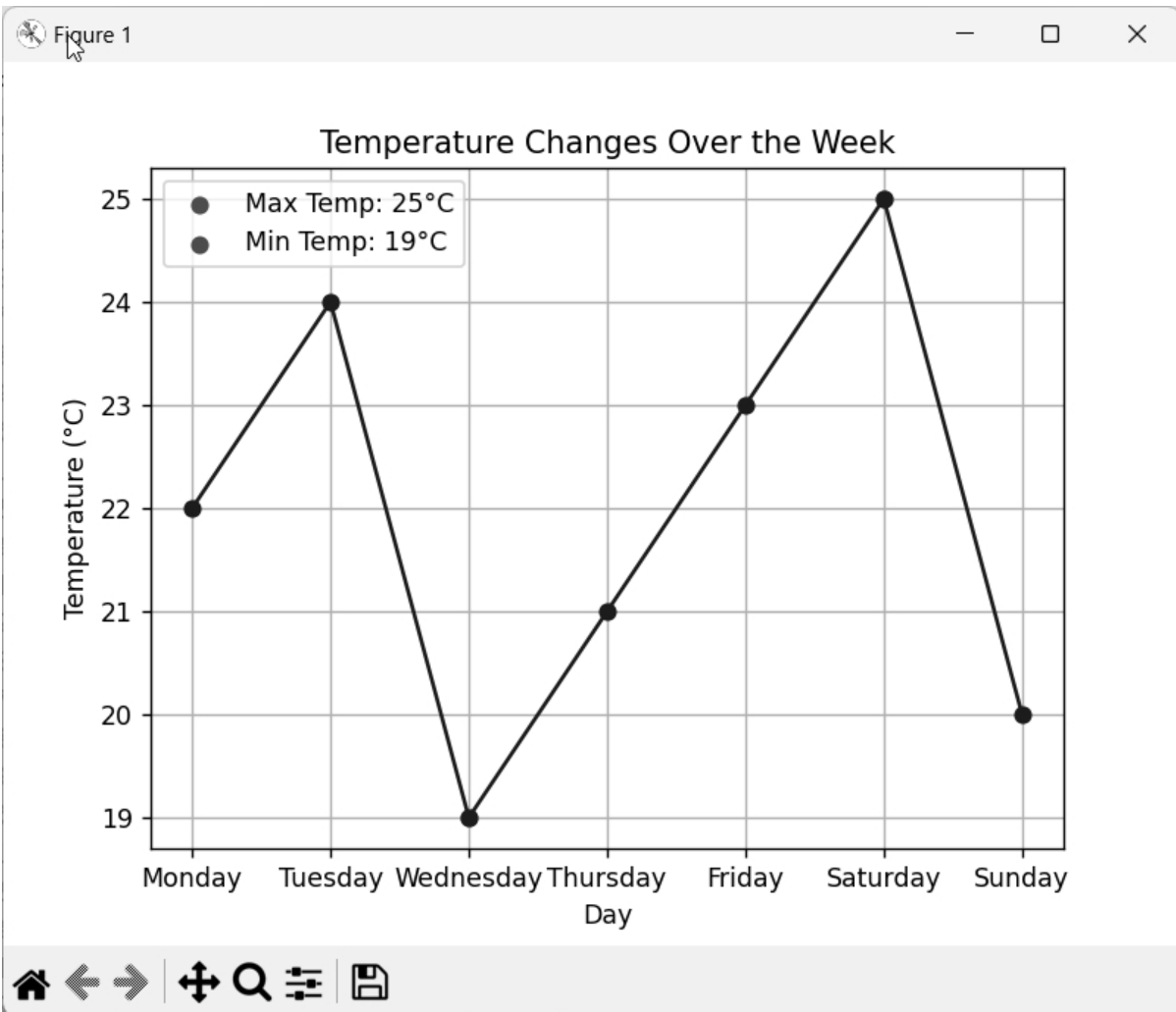
# Add titles, labels, and legend
plt.title("Temperature Changes Over the Week")
plt.xlabel("Day")
plt.ylabel("Temperature (°C)")
plt.legend()

# Show the chart
plt.grid(True)
plt.show()
```

Explanation:

- **max() and min():** These functions find the highest and lowest temperatures in the dataset.
- **plt.scatter():** This function is used to highlight the highest and lowest temperature points on the chart with different colors (red for max, green for min).
- **plt.legend():** Adds a legend to explain what the red and green points represent.

The chart will now clearly show the temperature changes over the week, with the highest temperature on **Saturday** (25°C) and the lowest on **Wednesday** (19°C) highlighted.

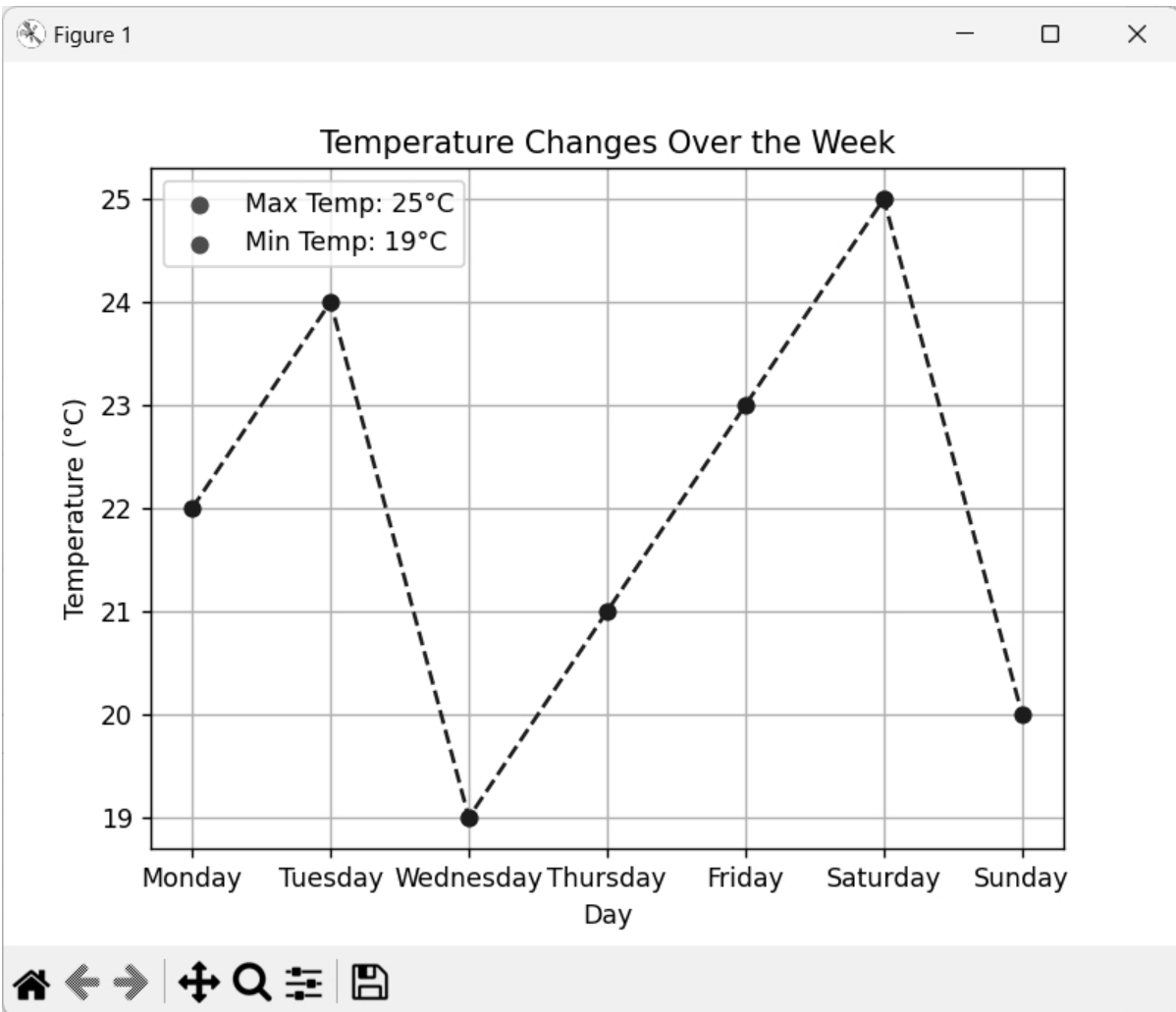


5. Customizing the Chart

Matplotlib allows you to customize charts to suit your preferences or make them more readable. Here are a few ways you can modify the temperature chart:

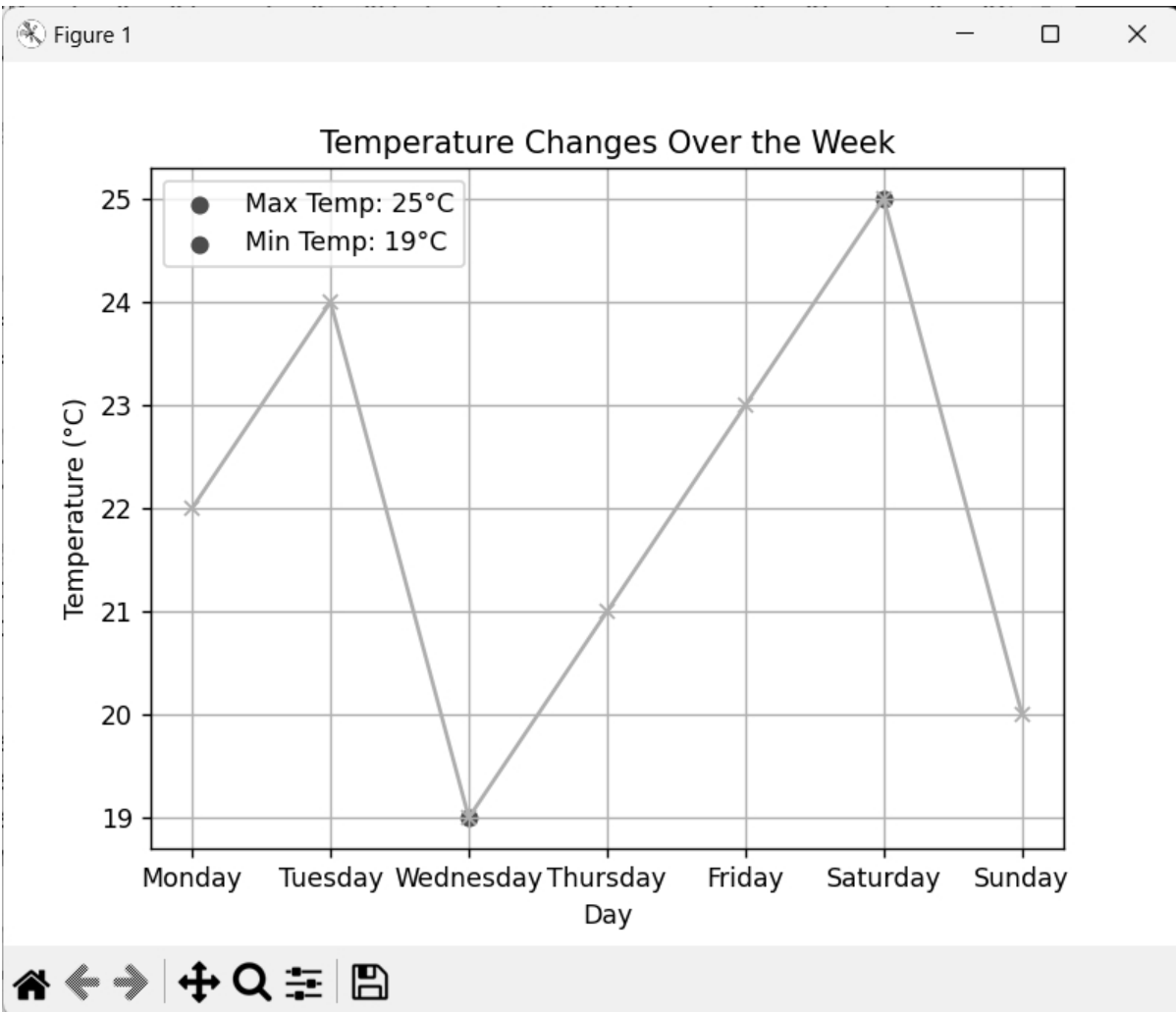
- **Changing the Line Style:** You can make the line dashed or dotted by adjusting the `linestyle` parameter.

```
plt.plot(days, temperatures, linestyle='--', marker='o', color='b') # Dashed line
```



- **Changing the Color Scheme:** You can choose different colors for the lines and markers to improve visual appeal or match a specific theme.

```
plt.plot(days, temperatures, color='orange', marker='x')
```



6. Saving the Plot as an Image

After creating the chart, you may want to save it as an image file to include it in a report or presentation. Matplotlib makes this easy:

```
# Save the plot as an image file
```

```
plt.savefig("temperature_plot.png")
```

This will save the chart as a PNG file in the current working directory. You can also save it in other formats like PDF or SVG by changing the file extension.

```
import matplotlib.pyplot as plt
```

```
# Data for the plot
```

```
days = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"]
```

```
temperatures = [22, 24, 19, 21, 23, 25, 20]
```

```
# Highlight the maximum and minimum temperatures
max_temp = max(temperatures)
min_temp = min(temperatures)

# Find the corresponding days
max_day = days[temperatures.index(max_temp)]
min_day = days[temperatures.index(min_temp)]

# Plot the temperatures
plt.plot(days, temperatures, color='orange', marker='x')

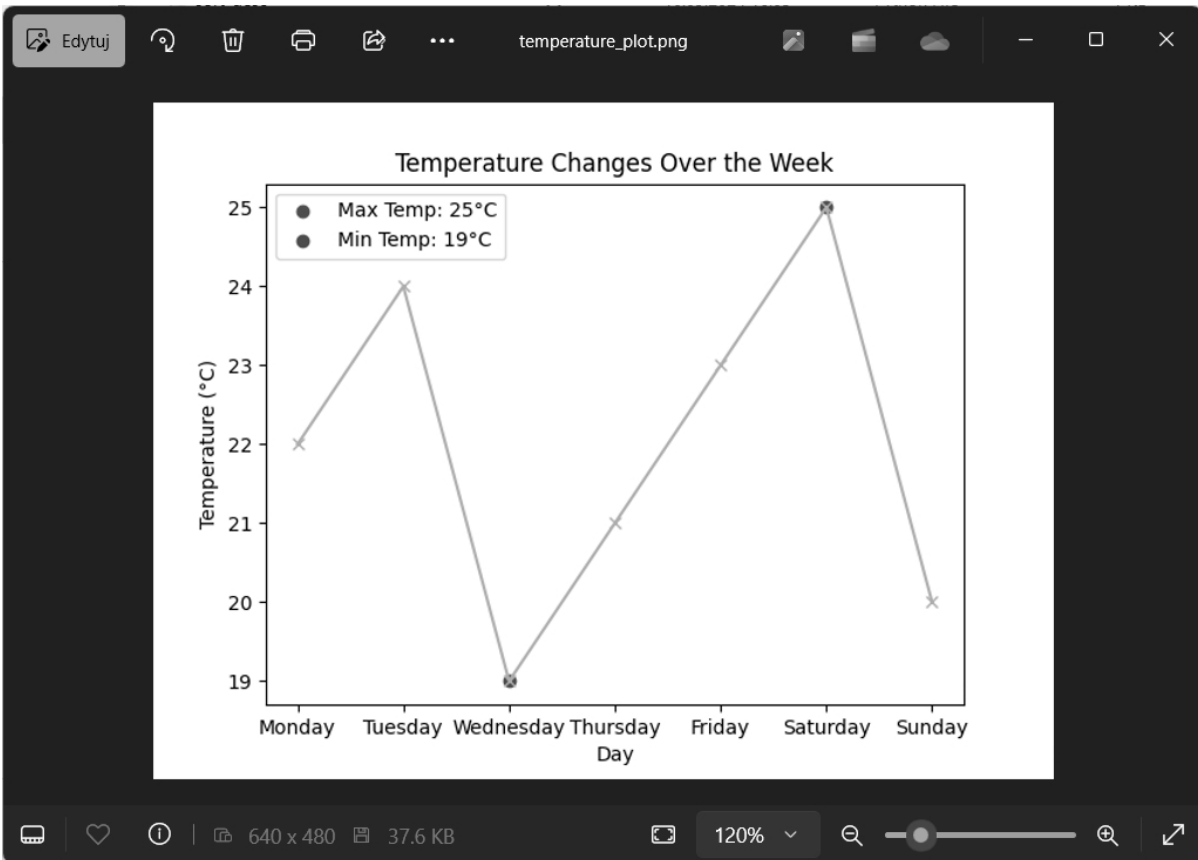
# Highlight the max and min points
plt.scatter(max_day, max_temp, color='red', label=f"Max Temp: {max_temp}°C")
plt.scatter(min_day, min_temp, color='green', label=f"Min Temp: {min_temp}°C")

# Add titles, labels, and legend
plt.title("Temperature Changes Over the Week")
plt.xlabel("Day")
plt.ylabel("Temperature (°C)")
plt.legend()

# Save the plot as an image file
plt.savefig("temperature_plot.png")

# Show the chart
plt.grid(True)
plt.show()
```

 temperature_plot		12.09.2024 12:54	Plik PNG	38 KB
 Typ elementu: Plik PNG		10.09.2024 09:08	Python File	1 KB
 Wymiary: 640 x 480		10.09.2024 15:09	Python File	1 KB
 Rozmiar: 37,6 KB				
 Stan dostępności: Dostępny na tym urządzeniu				



7. Real-World Applications of Temperature Plotting

Temperature plots like this one are used in various fields:

- **Weather Forecasting:** Tracking temperature changes over time helps meteorologists predict weather patterns and extreme weather events.
- **Environmental Studies:** Scientists use temperature data to monitor climate change, study ecosystems, or analyze seasonal trends.
- **Energy Consumption:** Companies that manage heating and cooling systems rely on temperature data to optimize energy usage and reduce costs.

In everyday life, tracking temperature changes can also help with activities like gardening, planning outdoor events, or monitoring the weather for

health reasons.

Summary

Plotting temperature changes over time is an excellent way to apply your skills with **Matplotlib**. Using just a few lines of code, you can create a line chart that visually represents how temperatures fluctuate, making it easy to spot trends like peaks and dips.

By adding grid lines, highlighting key points, and customizing the chart's appearance, you can create meaningful, informative visualizations that make data analysis more engaging. Whether you're tracking weather patterns, monitoring environmental changes, or simply practicing your Python skills, plotting data is a powerful tool for understanding and communicating trends.

Visualizing Classroom Attendance Data

Tracking and understanding classroom attendance is crucial for monitoring student engagement and performance. With **Matplotlib**, you can visualize attendance data to see trends, spot irregularities, and gain insights into class participation. In this section, we'll explore how to use Matplotlib to visualize attendance data, making it easier to interpret and analyze.

By the end of this chapter, you'll be able to create clear and informative charts that show how students are attending class over time, helping teachers and students alike stay on top of participation.

1. Understanding the Data

Let's assume we have attendance data for a group of students over the course of a week. Each day, we record whether a student was present or absent. Here's what the data might look like:

Student	Monday	Tuesday	Wednesday	Thursday	Friday
Alice	Present	Absent	Present	Present	Present
Bob	Present	Present	Absent	Present	Absent
Charlie	Absent	Present	Present	Absent	Present
David	Present	Present	Present	Present	Present
Eva	Absent	Absent	Present	Present	Present

We can use this data to create various visualizations that show how often students are present or absent during the week.

2. Creating a Bar Chart of Total Attendance

A **bar chart** is a simple way to visualize the total number of days each student attended class. First, we'll calculate how many days each student was present and plot this information.

Example: Total Attendance per Student

```
import matplotlib.pyplot as plt

# Data for the chart
students = ["Alice", "Bob", "Charlie", "David", "Eva"]
attendance = [4, 3, 3, 5, 3] # Total days each student was present

# Create the bar chart
plt.bar(students, attendance, color='skyblue')

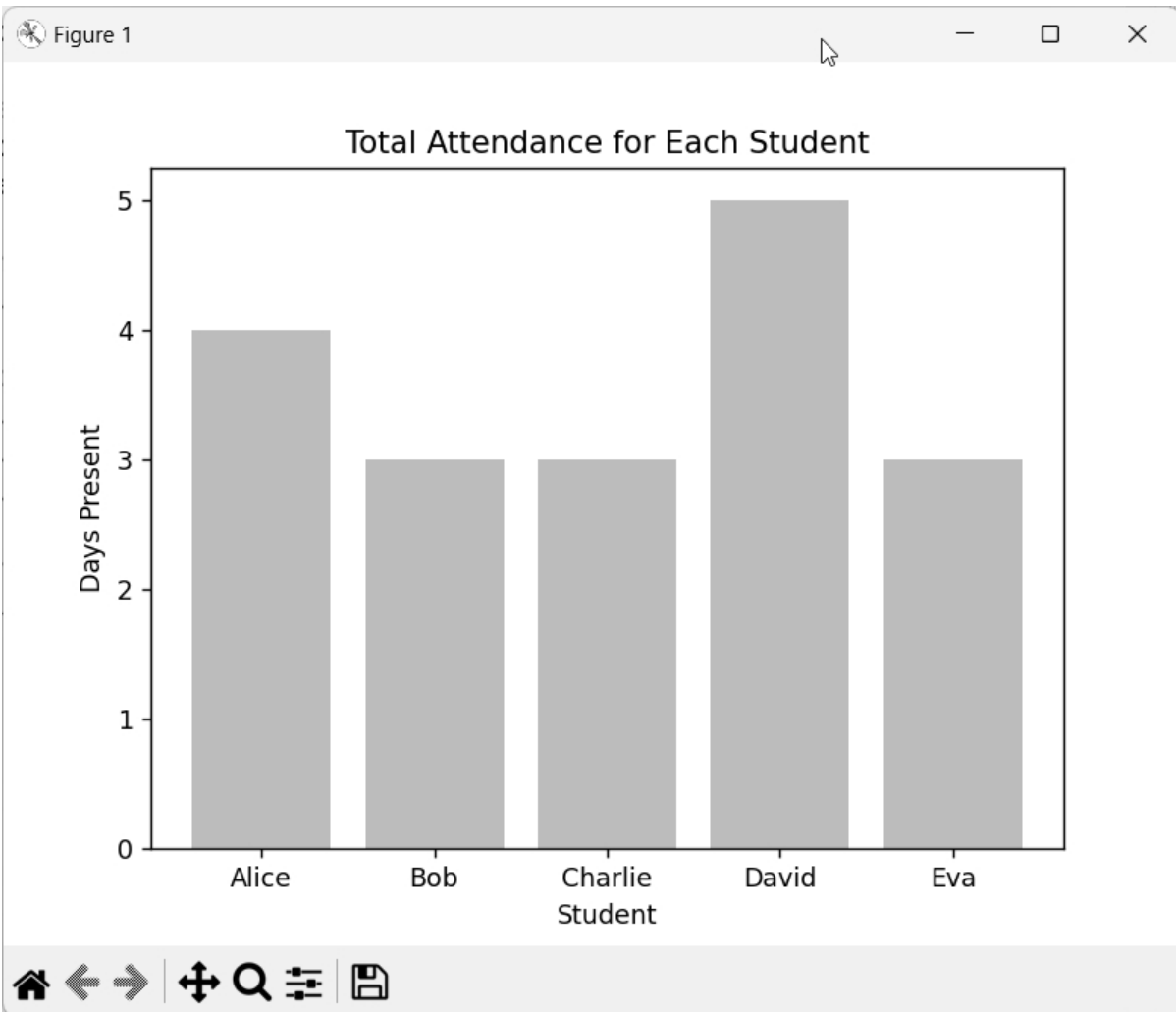
# Add labels and title
plt.title("Total Attendance for Each Student")
plt.xlabel("Student")
plt.ylabel("Days Present")

# Show the chart
plt.show()
```

Explanation:

- **students:** A list of student names.
- **attendance:** A list representing the total number of days each student was present during the week.
- **plt.bar():** Creates a bar chart, with student names on the x-axis and days present on the y-axis.
- **color='skyblue':** Adds a soft blue color to the bars for a clearer visualization.

When you run this code, the bar chart will display how many days each student attended class during the week.



3. Creating a Heatmap for Daily Attendance

A **heatmap** is another powerful tool for visualizing attendance data, especially when tracking attendance over multiple days. In a heatmap, colors represent different values—in this case, "Present" or "Absent". This provides an easy way to spot trends, such as students who frequently miss certain days.

Example: Daily Attendance Heatmap

```
import numpy as np
import matplotlib.pyplot as plt

# Data for the heatmap (1 for Present, 0 for Absent)
attendance_data = np.array([
    [1, 0, 1, 1, 1], # Alice
```

```

[1, 1, 0, 1, 0], # Bob
[0, 1, 1, 0, 1], # Charlie
[1, 1, 1, 1, 1], # David
[0, 0, 1, 1, 1] # Eva
])

# Create a heatmap
plt.imshow(attendance_data, cmap='Greens', aspect='auto')

# Add labels
plt.title("Classroom Attendance Heatmap")
plt.xlabel("Day")
plt.ylabel("Student")
plt.xticks(ticks=[0, 1, 2, 3, 4], labels=["Monday", "Tuesday", "Wednesday", "Thursday", "Friday"])
plt.yticks(ticks=[0, 1, 2, 3, 4], labels=["Alice", "Bob", "Charlie", "David", "Eva"])

# Add a color bar to show what the colors represent
plt.colorbar(label="Attendance (1 = Present, 0 = Absent)")

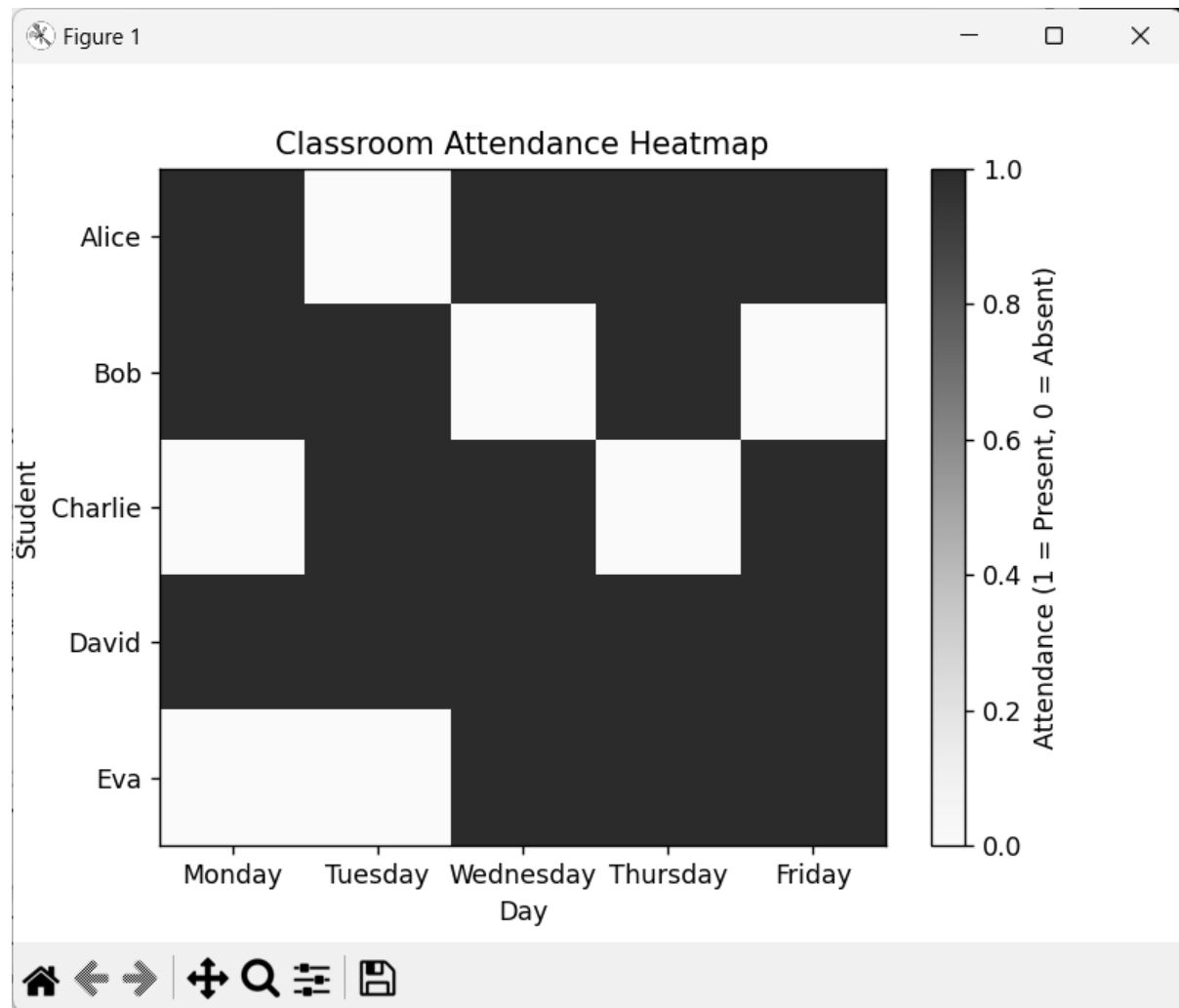
# Show the heatmap
plt.show()

```

Explanation:

- **attendance_data:** This array contains the attendance information for each student over five days. 1 represents "Present," and 0 represents "Absent."
- **plt.imshow():** This function creates a heatmap, with `cmap='Greens'` indicating a green color scale (darker green for "Present" and lighter for "Absent").
- **plt.xticks() and plt.yticks():** These functions label the x-axis (days of the week) and y-axis (student names).
- **plt.colorbar():** Adds a color bar on the side to indicate what the colors represent.

The heatmap makes it easy to see at a glance which students were absent or present on specific days, highlighting attendance patterns.



4. Tracking Attendance Trends Over Time

Sometimes it's useful to see how attendance trends change over time. A **line chart** can help you visualize the total number of students present each day during the week.

Example: Attendance Trend Line Chart

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# Data for the total number of students present each day
```

```
days = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday"]
```

```
total_present = [3, 3, 4, 4, 4] # Number of students present each day
```

```
# Create the line chart
```

```
plt.plot(days, total_present, marker='o', linestyle='-', color='blue')
```

```
# Add labels and title
```

```
plt.title("Attendance Trend Over the Week")
```

```
plt.xlabel("Day")
```

```
plt.ylabel("Number of Students Present")
```

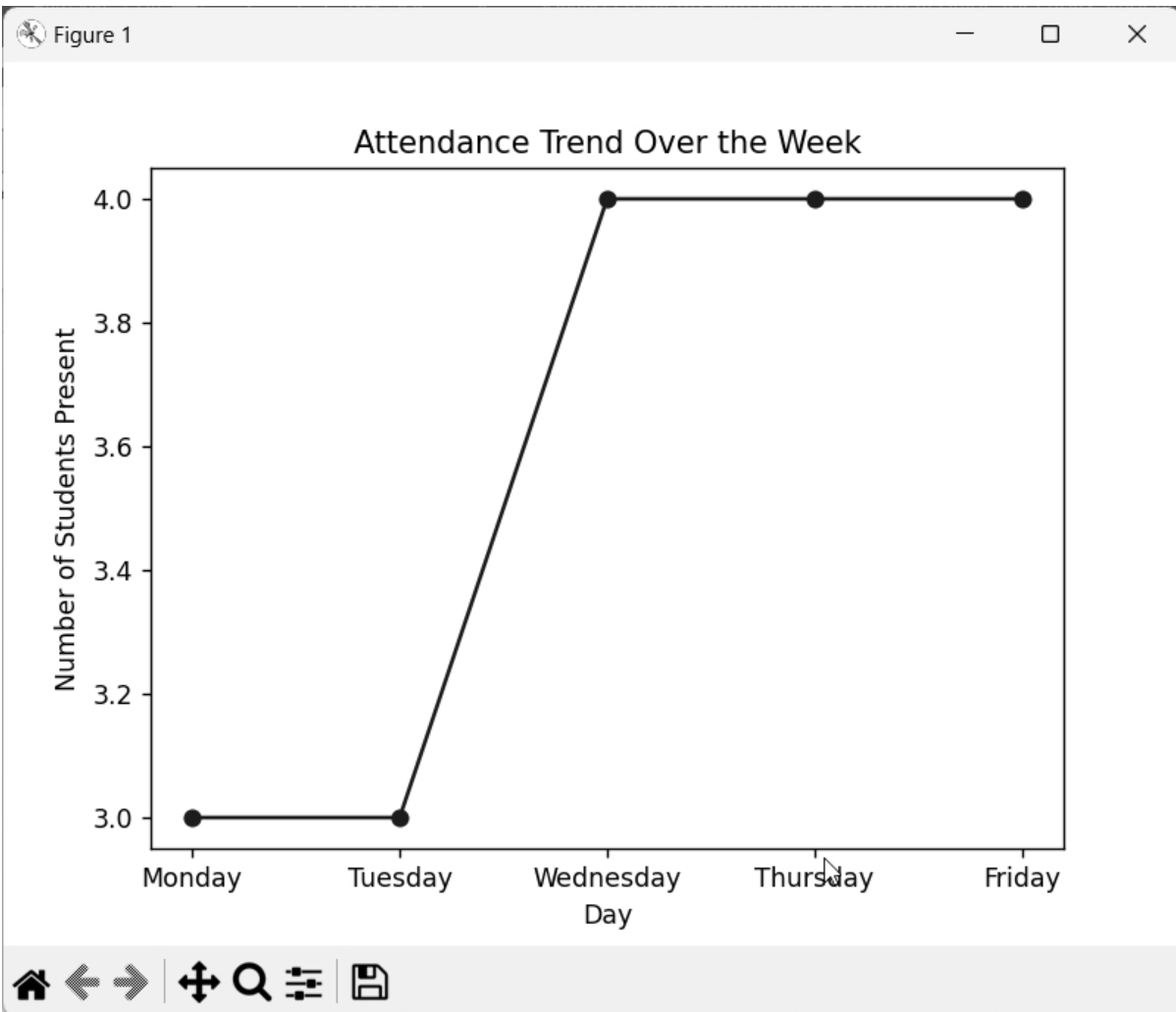
```
# Show the chart
```

```
plt.show()
```

Explanation:

- **total_present:** This list contains the total number of students who were present each day.
- **plt.plot():** Creates a line chart to visualize the attendance trend over time, with markers at each data point.

This line chart helps you see how the number of students attending class fluctuates over the week.

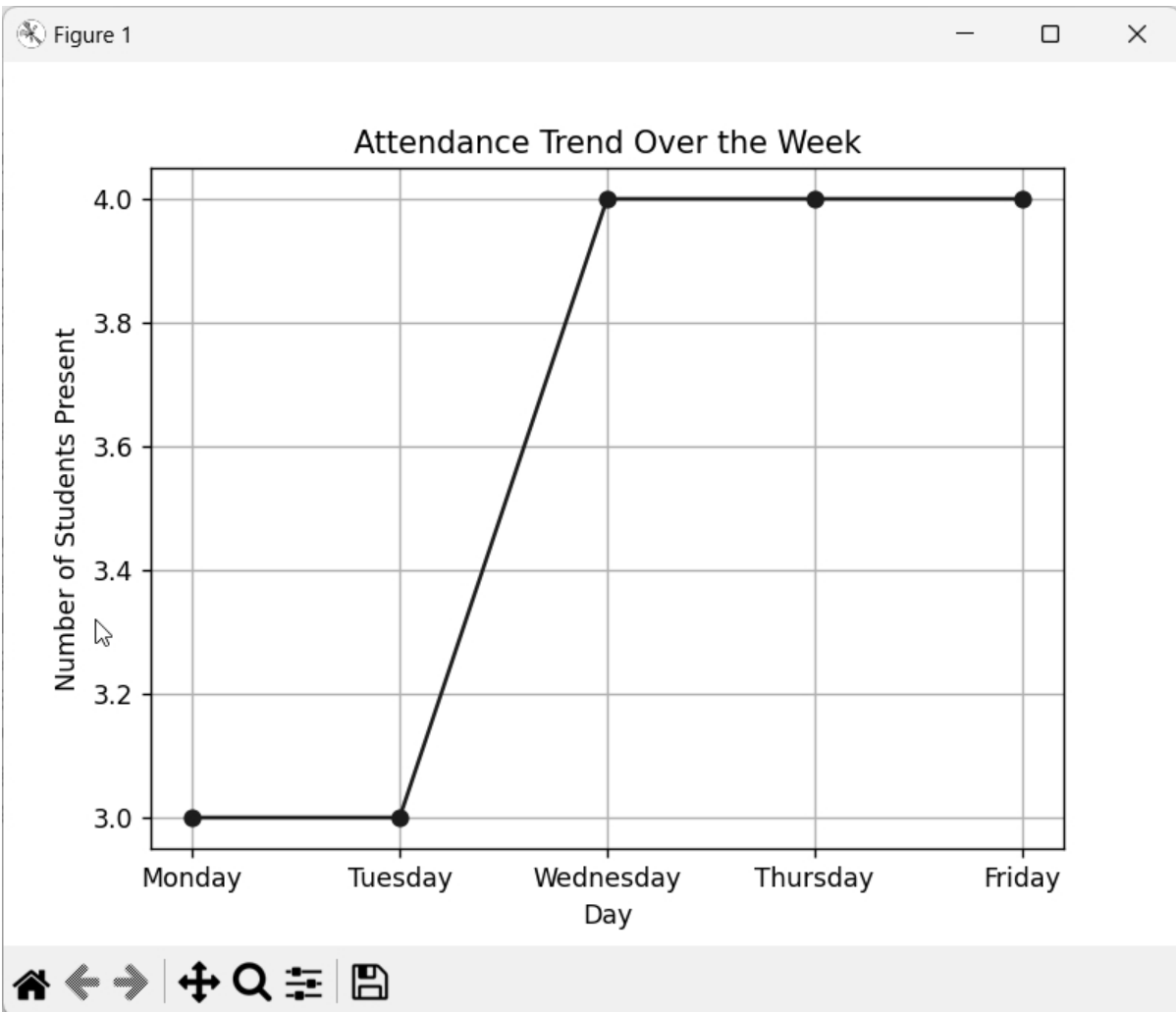


5. Customizing the Attendance Visualizations

Just like with any other Matplotlib chart, you can customize your attendance visualizations to make them more informative and visually appealing. Here are a few ways to do that:

- **Adding Grid Lines:** To make it easier to read the values from the chart:

```
plt.grid(True)
```

6. Saving and Sharing Your Visualizations

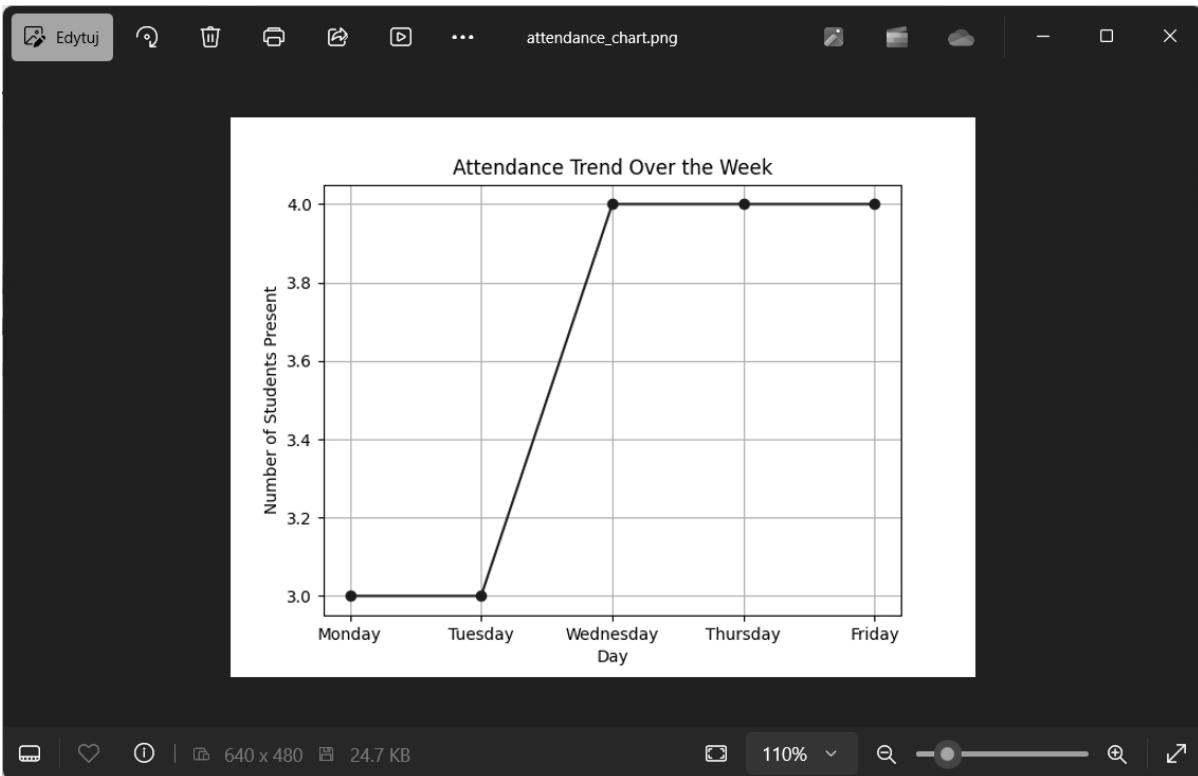
After creating your visualizations, you can save them as image files to use in reports, presentations, or classroom discussions.

Save the plot as an image file

```
plt.savefig("attendance_chart.png")
```

This will save your chart as a PNG file, making it easy to share or include in other documents.

	attendance_chart	12.09.2024 13:52	Plik PNG	25 KB
	Typ elementu: Plik PNG	12.09.2024 13:52	Python File	1 KB
	Wymiary: 640 x 480	12.09.2024 13:50	Dokument progra...	1 458 KB
	Rozmiar: 24,6 KB			
	Stan dostępności: Dostępny na tym urządzeniu			



7. Why Visualizing Attendance Data Matters

Visualizing attendance data helps in several key ways:

- **Spotting Trends:** Teachers can quickly identify patterns, such as students who frequently miss specific days or weeks.
- **Tracking Engagement:** Visuals help monitor student engagement and participation over time.
- **Improving Decision-Making:** By seeing the data in a visual format, educators can make informed decisions, such as identifying students who need additional support.

These visualizations are especially helpful in schools, where understanding attendance patterns can contribute to better student outcomes and overall classroom management.

Summary

Visualizing classroom attendance data using **Matplotlib** is a simple yet powerful way to monitor student participation and spot trends over time.

Whether you use bar charts to track total attendance, heatmaps to see daily patterns, or line charts to follow attendance trends, these visualizations make complex data easy to understand and act upon.

With a few lines of code, you can turn raw attendance data into meaningful insights, helping teachers and students stay engaged and organized. Keep practicing with different types of visualizations, and soon you'll be creating custom charts that make your data come to life!

Chapter 10: Automating Everyday Tasks with Python

Automating Repetitive Tasks with Python

One of the most exciting aspects of learning Python is the ability to **automate repetitive tasks**. Whether it's sending emails, organizing files, or collecting data from the web, Python can handle these tasks quickly and efficiently, saving you time and effort. In this chapter, we'll explore how Python can be used to automate common everyday tasks, allowing you to focus on more important activities.

By the end of this section, you'll see how automation can simplify your life and make your workflow smoother. Plus, you'll learn how to write scripts that perform tasks automatically — making Python a powerful tool not just for programming, but for practical problem-solving in daily life.

1. Why Automate Tasks?

Repetitive tasks can be time-consuming and tedious, whether you're a student trying to organize study materials or a teacher managing classroom updates. Automation helps by:

- **Saving time:** Instead of manually completing repetitive tasks, Python can do them for you in seconds.
- **Reducing errors:** Automation reduces the chance of mistakes that can occur when tasks are done manually.
- **Boosting productivity:** By automating small tasks, you free up your time for more important work or learning.

2. Setting Up for Automation

Before we jump into examples, let's understand what it means to automate tasks using Python:

- **Write scripts** that interact with files, systems, or even the web.
- **Schedule tasks** to run automatically at specific times.
- **Use Python libraries** that are designed for automation, such as `os`, `shutil`, and `smtplib`.

3. Automating File Management Tasks

One of the simplest but most useful automations you can set up with Python is organizing files. Imagine you have a folder full of documents, images, and videos, and you want to automatically move them into separate folders based on their file type.

Example: Organizing Files by Type

Here's a Python script that automatically organizes files in a folder by their file extensions:

```
import os
import shutil

# Define the folder you want to organize
folder_path = "/path/to/your/folder"

# Create subfolders for different file types
file_types = {
    "Images": [".jpg", ".png", ".gif"],
    "Documents": [".pdf", ".docx", ".txt"],
    "Videos": [".mp4", ".mov", ".avi"]
}

# Create the folders if they don't exist
for folder in file_types:
    folder_dir = os.path.join(folder_path, folder)
```

```

if not os.path.exists(folder_dir):
    os.makedirs(folder_dir)

# Move files into the appropriate folder
for filename in os.listdir(folder_path):
    file_extension = os.path.splitext(filename)[1]

    for folder, extensions in file_types.items():
        if file_extension in extensions:
            source = os.path.join(folder_path, filename)
            destination = os.path.join(folder_path, folder, filename)
            shutil.move(source, destination)

print("Files have been organized by type.")

```

Explanation:

- **os** and **shutil**: These libraries allow Python to interact with your operating system and move files around.
- **file_types**: This dictionary lists different types of files (like images, documents, and videos) and their corresponding extensions.
- **shutil.move()**: Moves each file into the appropriate folder based on its extension.

After running this script, all files in your folder will be neatly organized by type, saving you the trouble of sorting through them manually.

4. Automating Data Collection from the Web

Python can also automate tasks like **collecting data** from websites, which is especially useful for gathering information for research or personal projects. This process is called **web scraping**.

Example: text from a Website

Suppose you want to collect the text data from a website. You can use the **BeautifulSoup** library to extract text from HTML pages.

```
pip install requests beautifulsoup4
```

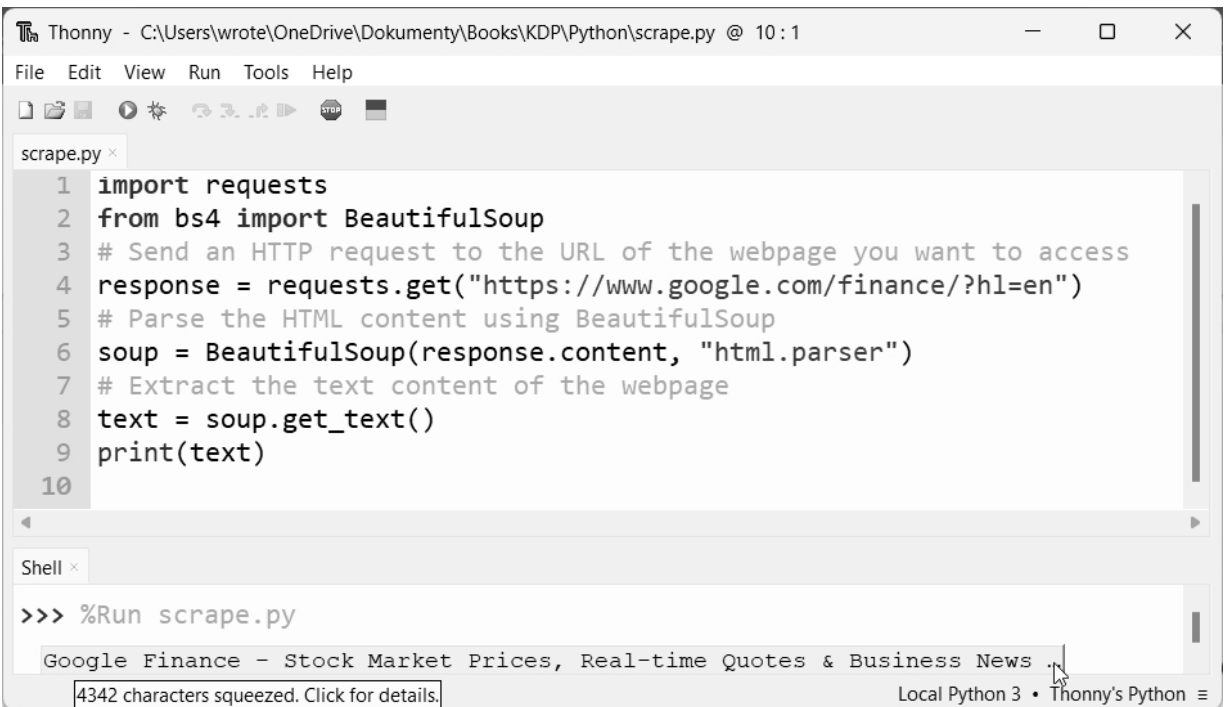
Then, use the following script to scrape headlines from a news website:

```
import requests
from bs4 import BeautifulSoup
# Send an HTTP request to the URL of the webpage you want to access
response = requests.get("https://www.example.com")
# Parse the HTML content using BeautifulSoup
soup = BeautifulSoup(response.content, "html.parser")
# Extract the text content of the webpage
text = soup.get_text()
print(text)
```

Explanation:

- **requests.get():** Retrieves the webpage content.
- **BeautifulSoup():** Parses the HTML content and makes it easy to search for specific elements (like headlines).
- **soup.get_text():** Finds all elements that match a certain pattern (in this case text).

You can modify it to scrape data from any website for research, school projects, or personal use.



The screenshot shows the Thonny IDE interface. The top window, titled 'Thonny - C:\Users\wrote\OneDrive\Dokumenty\Books\KDP\Python\scrape.py @ 10:1', contains a Python script named 'scrape.py'. The script imports 'requests' and 'BeautifulSoup' from 'bs4', sends an HTTP request to 'https://www.google.com/finance/?hl=en', parses the HTML content, and prints the extracted text. Below the script editor is a 'Shell' window showing the command '>>> %Run scrape.py' and the output: 'Google Finance - Stock Market Prices, Real-time Quotes & Business News'. A tooltip indicates '4342 characters squeezed. Click for details.' The status bar at the bottom shows 'Local Python 3 • Thonny's Python'.

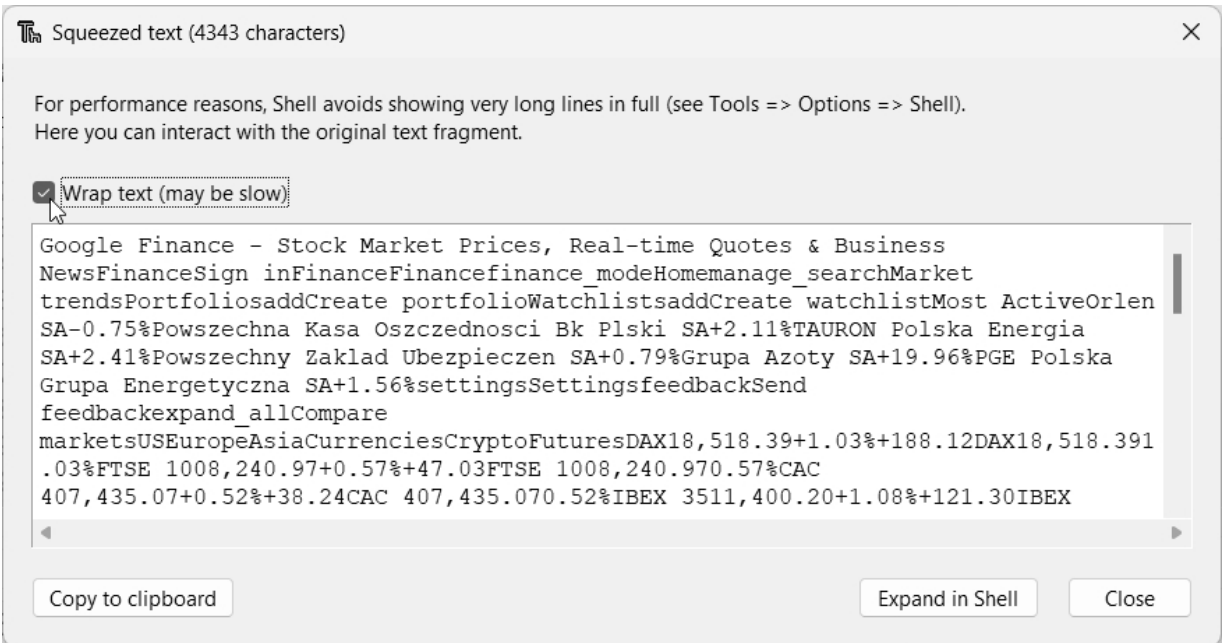
```
1 import requests
2 from bs4 import BeautifulSoup
3 # Send an HTTP request to the URL of the webpage you want to access
4 response = requests.get("https://www.google.com/finance/?hl=en")
5 # Parse the HTML content using BeautifulSoup
6 soup = BeautifulSoup(response.content, "html.parser")
7 # Extract the text content of the webpage
8 text = soup.get_text()
9 print(text)
10
```

>>> %Run scrape.py

Google Finance - Stock Market Prices, Real-time Quotes & Business News

4342 characters squeezed. Click for details.

Local Python 3 • Thonny's Python



To personalize this web scraping script, you can modify it to meet specific goals or add functionality tailored to your needs. Here are several ways to personalize the script:

5. Targeting Specific Elements (e.g., Headlines, Links)

Instead of extracting all the text on the page, you can target specific HTML elements like headlines, links, or paragraphs. For example, if you're

scraping headlines (usually inside `<h1>`, `<h2>`, or `<a>` tags), you can modify the script like this:

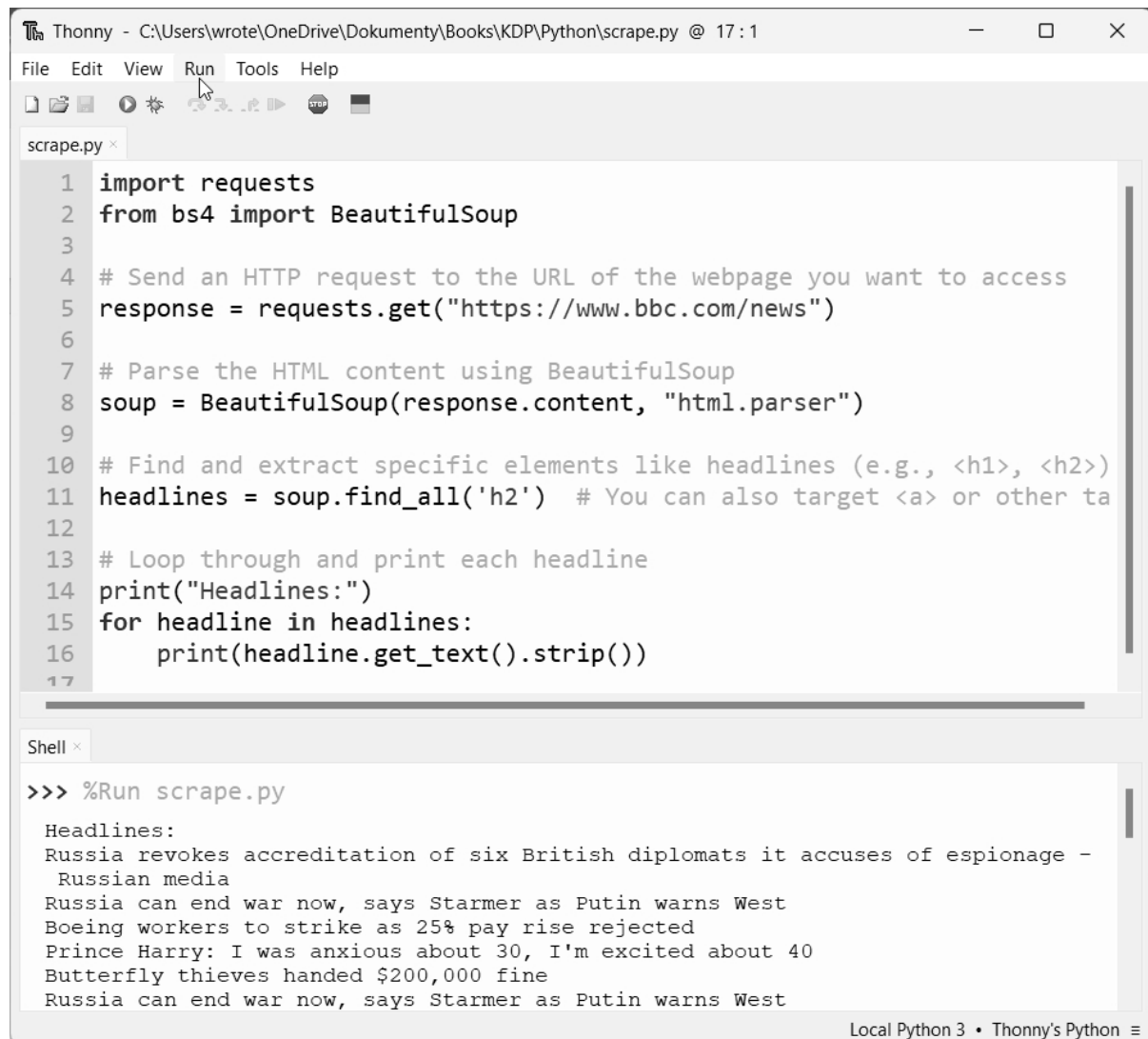
```
import requests
from bs4 import BeautifulSoup

# Send an HTTP request to the URL of the webpage you want to access
response = requests.get("https://www.bbc.com/news")

# Parse the HTML content using BeautifulSoup
soup = BeautifulSoup(response.content, "html.parser")

# Find and extract specific elements like headlines (e.g., <h1>, <h2>)
headlines = soup.find_all('h2') # You can also target <a> or other tags

# Loop through and print each headline
print("Headlines:")
for headline in headlines:
    print(headline.get_text().strip())
```



The screenshot shows the Thonny Python IDE interface. The top window, titled 'scrape.py', contains the following Python code:

```
1 import requests
2 from bs4 import BeautifulSoup
3
4 # Send an HTTP request to the URL of the webpage you want to access
5 response = requests.get("https://www.bbc.com/news")
6
7 # Parse the HTML content using BeautifulSoup
8 soup = BeautifulSoup(response.content, "html.parser")
9
10 # Find and extract specific elements like headlines (e.g., <h1>, <h2>)
11 headlines = soup.find_all('h2') # You can also target <a> or other ta
12
13 # Loop through and print each headline
14 print("Headlines:")
15 for headline in headlines:
16     print(headline.get_text().strip())
17
```

The bottom window, titled 'Shell', shows the output of running the script:

```
>>> %Run scrape.py
Headlines:
Russia revokes accreditation of six British diplomats it accuses of espionage -
Russian media
Russia can end war now, says Starmer as Putin warns West
Boeing workers to strike as 25% pay rise rejected
Prince Harry: I was anxious about 30, I'm excited about 40
Butterfly thieves handed $200,000 fine
Russia can end war now, says Starmer as Putin warns West
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

6. Saving Results to a File

Instead of just printing the results, you can save them to a text file for later use.

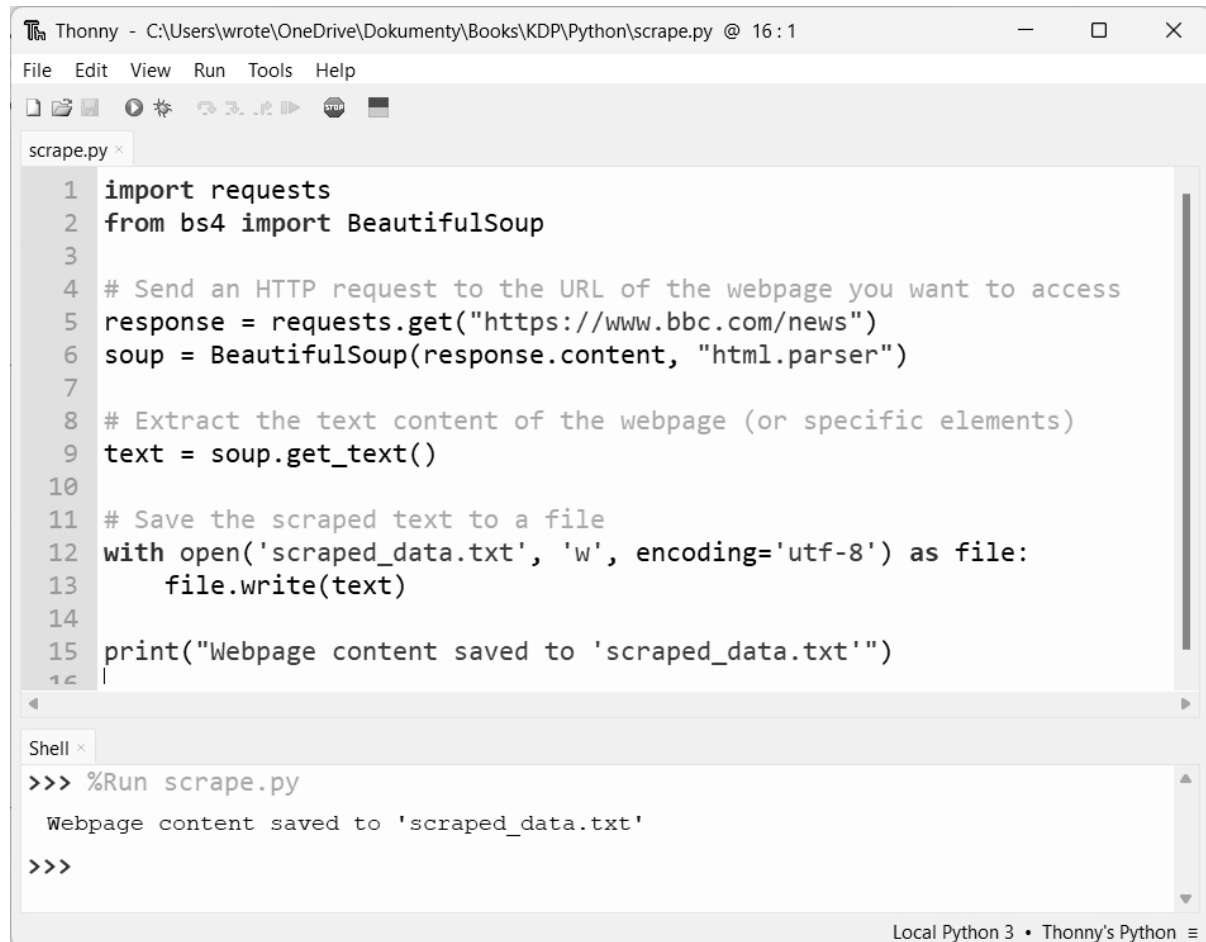
```
import requests
from bs4 import BeautifulSoup

# Send an HTTP request to the URL of the webpage you want to access
response = requests.get("https://www.example.com")
soup = BeautifulSoup(response.content, "html.parser")

# Extract the text content of the webpage (or specific elements)
text = soup.get_text()
```

```
# Save the scraped text to a file
with open('scraped_data.txt', 'w', encoding='utf-8') as file:
    file.write(text)

print("Webpage content saved to 'scraped_data.txt'")
```



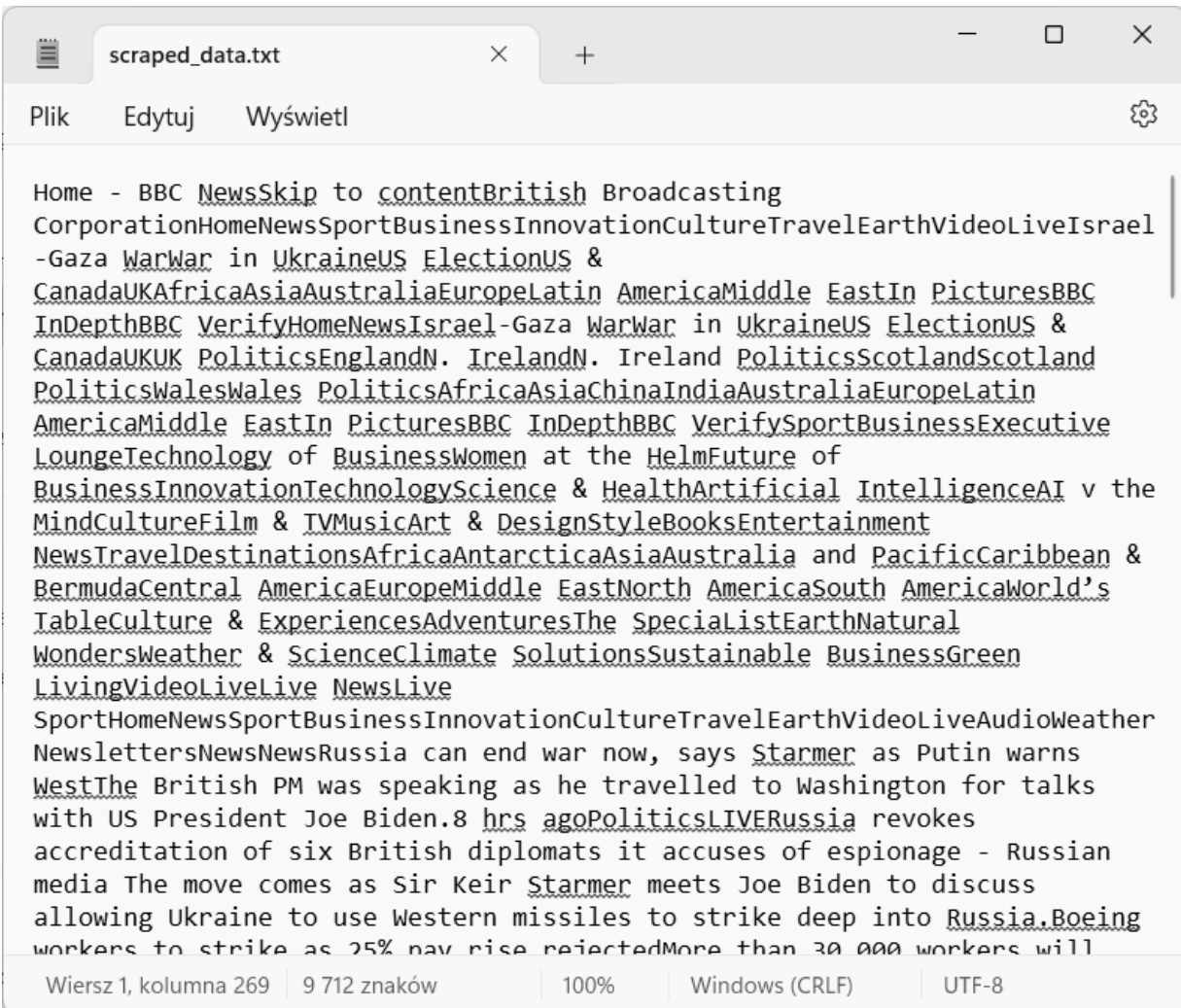
The screenshot shows the Thonny Python IDE interface. The top window, titled 'scrape.py', contains the following Python code:

```
1 import requests
2 from bs4 import BeautifulSoup
3
4 # Send an HTTP request to the URL of the webpage you want to access
5 response = requests.get("https://www.bbc.com/news")
6 soup = BeautifulSoup(response.content, "html.parser")
7
8 # Extract the text content of the webpage (or specific elements)
9 text = soup.get_text()
10
11 # Save the scraped text to a file
12 with open('scraped_data.txt', 'w', encoding='utf-8') as file:
13     file.write(text)
14
15 print("Webpage content saved to 'scraped_data.txt'")
16
```

Below the code editor is a 'Shell' window showing the execution output:

```
>>> %Run scrape.py
Webpage content saved to 'scraped_data.txt'
>>>
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.



7. Handling Errors Gracefully

You can add error handling to ensure the script doesn't crash if the webpage is down or if there's an issue with the connection.

```
import requests
from bs4 import BeautifulSoup

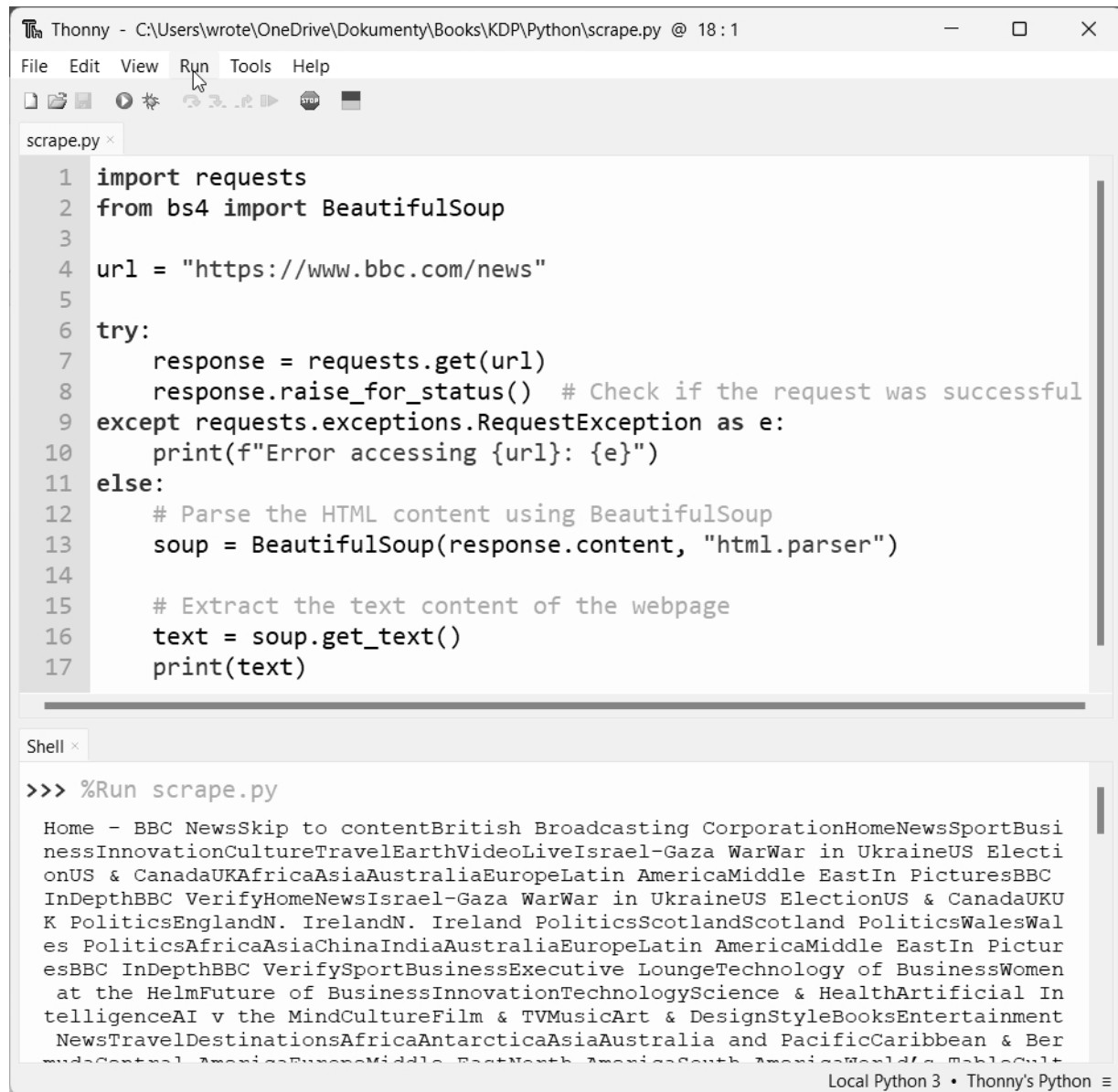
url = "https://www.example.com"

try:
    response = requests.get(url)
    response.raise_for_status() # Check if the request was successful
except requests.exceptions.RequestException as e:
    print(f"Error accessing {url}: {e}")
else:
```

```
# Parse the HTML content using BeautifulSoup
soup = BeautifulSoup(response.content, "html.parser")

# Extract the text content of the webpage
text = soup.get_text()

print(text)
```



The screenshot shows the Thonny Python IDE interface. The top window, titled 'scrape.py', contains the following Python code:

```
1 import requests
2 from bs4 import BeautifulSoup
3
4 url = "https://www.bbc.com/news"
5
6 try:
7     response = requests.get(url)
8     response.raise_for_status() # Check if the request was successful
9 except requests.exceptions.RequestException as e:
10     print(f"Error accessing {url}: {e}")
11 else:
12     # Parse the HTML content using BeautifulSoup
13     soup = BeautifulSoup(response.content, "html.parser")
14
15     # Extract the text content of the webpage
16     text = soup.get_text()
17     print(text)
```

The bottom window, titled 'Shell', shows the output of running the script:

```
>>> %Run scrape.py
Home - BBC NewsSkip to contentBritish Broadcasting CorporationHomeNewsSportBusi
nessInnovationCultureTravelEarthVideoLiveIsrael-Gaza WarWar in UkraineUS Electi
onUS & CanadaUKAfricaAsiaAustraliaEuropeLatin AmericaMiddle EastIn PicturesBBC
InDepthBBC VerifyHomeNewsIsrael-Gaza WarWar in UkraineUS ElectionUS & CanadaUKU
K PoliticsEnglandN. IrelandN. Ireland PoliticsScotlandScotland PoliticsWalesWal
es PoliticsAfricaAsiaChinaIndiaAustraliaEuropeLatin AmericaMiddle EastIn Pictur
esBBC InDepthBBC VerifySportBusinessExecutive LoungeTechnology of BusinessWomen
at the HelmFuture of BusinessInnovationTechnologyScience & HealthArtificial In
telligenceAI v the MindCultureFilm & TVMusicArt & DesignStyleBooksEntertainment
NewsTravelDestinationsAfricaAntarcticaAsiaAustralia and PacificCaribbean & Ber
mudaCentral AmericaEuropeMiddle EastNorth AmericaSouth AmericaWorld's TableCult
```

The status bar at the bottom right indicates 'Local Python 3 • Thonny's Python'.

8. Customizing the URL

You can make the URL dynamic, allowing the user to input the website they want to scrape.

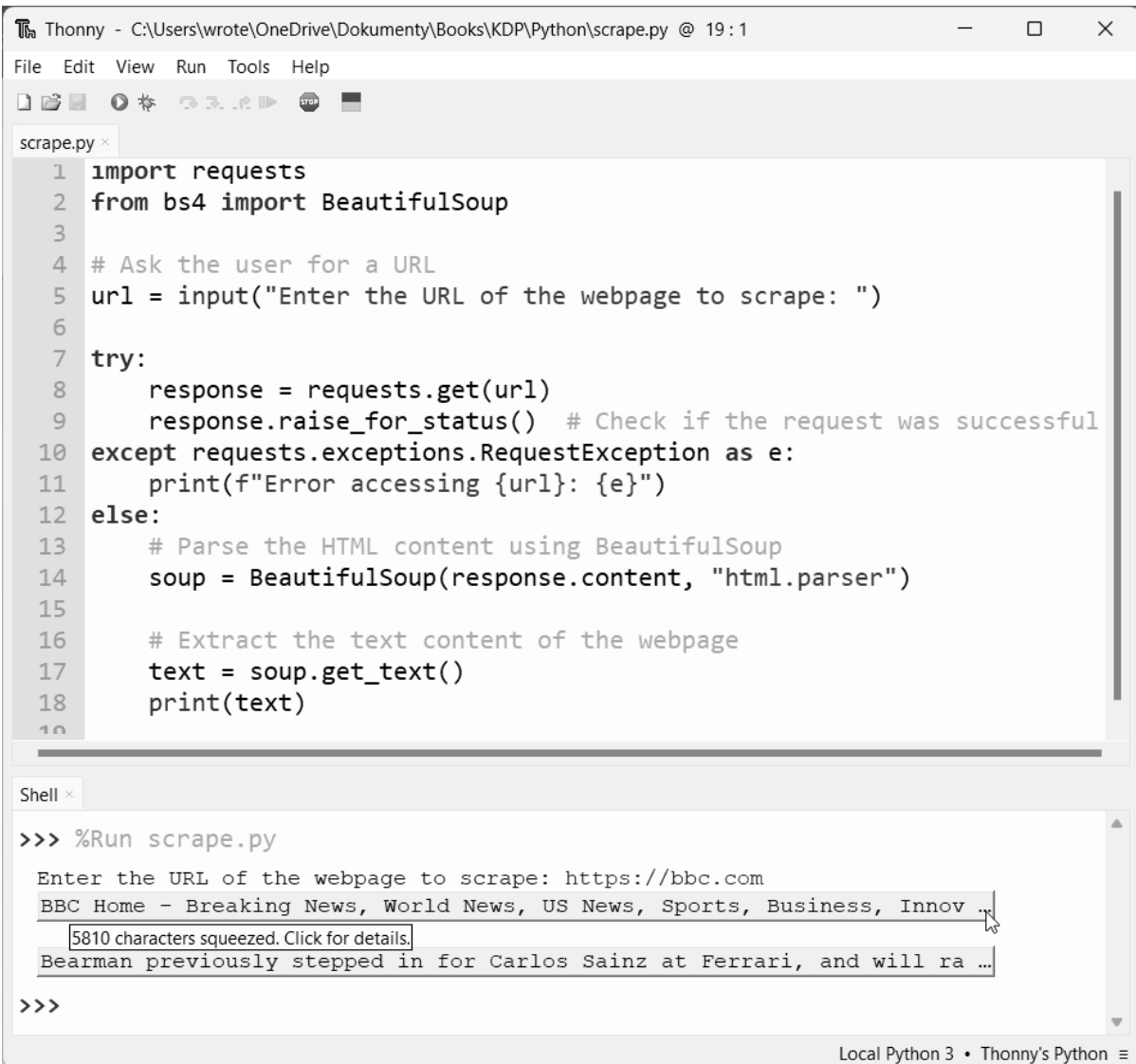
```
import requests
```

```
from bs4 import BeautifulSoup

# Ask the user for a URL
url = input("Enter the URL of the webpage to scrape: ")

try:
    response = requests.get(url)
    response.raise_for_status() # Check if the request was successful
except requests.exceptions.RequestException as e:
    print(f"Error accessing {url}: {e}")
else:
    # Parse the HTML content using BeautifulSoup
    soup = BeautifulSoup(response.content, "html.parser")

    # Extract the text content of the webpage
    text = soup.get_text()
    print(text)
```



The screenshot shows the Thonny IDE interface. The top window, titled 'scrape.py', contains a Python script that uses the 'requests' and 'BeautifulSoup' libraries to fetch and parse a webpage. The script prompts the user for a URL, attempts to fetch it, and if successful, extracts and prints the text content. The bottom window, titled 'Shell', shows the execution of the script. It displays the prompt 'Enter the URL of the webpage to scrape:', the user input 'https://bbc.com', and the resulting text output from the BBC homepage, which has been truncated to 5810 characters.

```
1 import requests
2 from bs4 import BeautifulSoup
3
4 # Ask the user for a URL
5 url = input("Enter the URL of the webpage to scrape: ")
6
7 try:
8     response = requests.get(url)
9     response.raise_for_status() # Check if the request was successful
10 except requests.exceptions.RequestException as e:
11     print(f"Error accessing {url}: {e}")
12 else:
13     # Parse the HTML content using BeautifulSoup
14     soup = BeautifulSoup(response.content, "html.parser")
15
16     # Extract the text content of the webpage
17     text = soup.get_text()
18     print(text)
```

```
>>> %Run scrape.py
Enter the URL of the webpage to scrape: https://bbc.com
BBC Home - Breaking News, World News, US News, Sports, Business, Innov
5810 characters squeezed. Click for details.
Bearman previously stepped in for Carlos Sainz at Ferrari, and will ra ...
>>>
```

9. Extracting Links or Images

If you're interested in scraping all the links or images from the webpage, you can modify the script like this:

Extracting Links:

```
import requests
from bs4 import BeautifulSoup

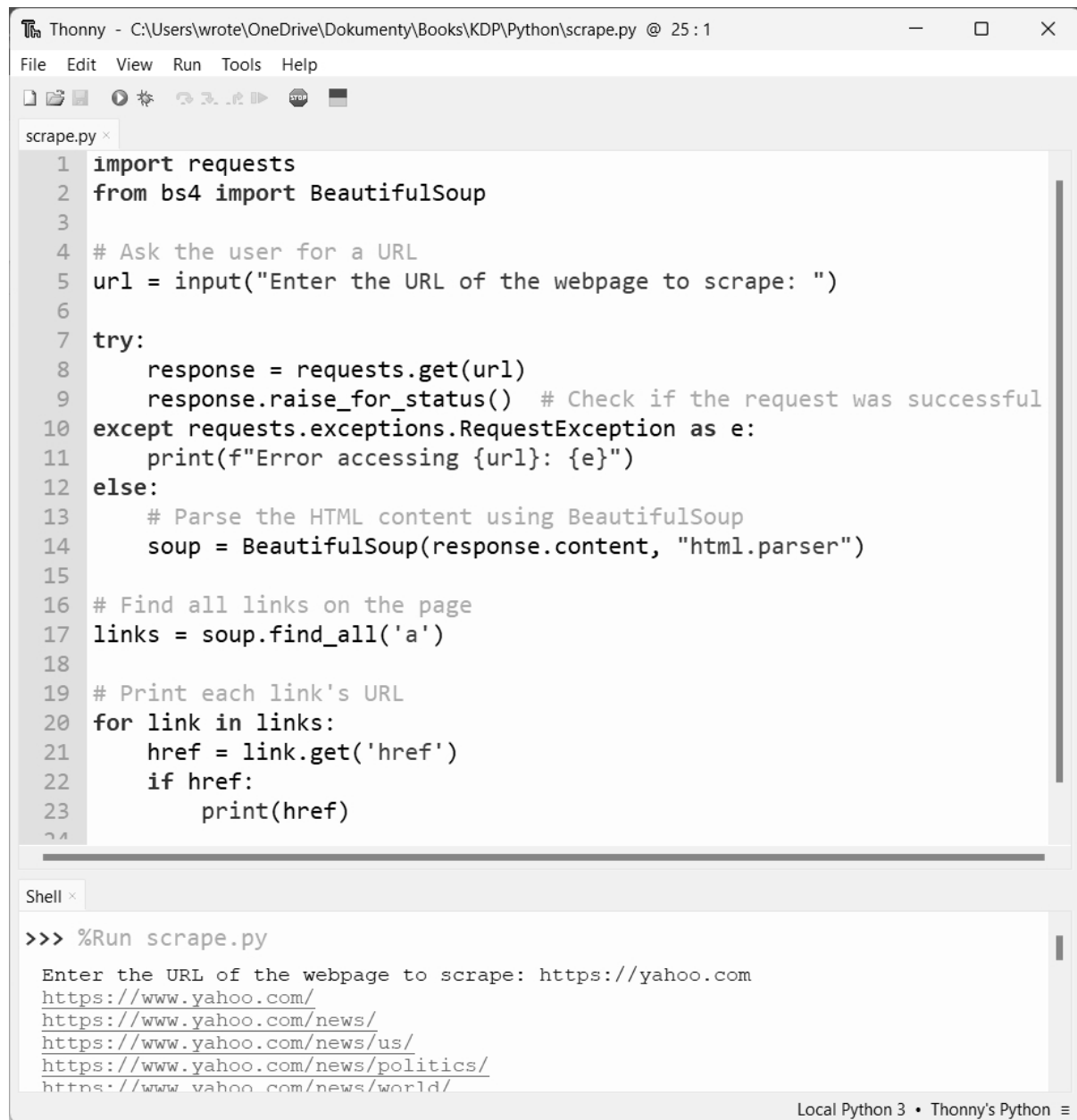
# Ask the user for a URL
url = input("Enter the URL of the webpage to scrape: ")

try:
    response = requests.get(url)
```

```
response.raise_for_status() # Check if the request was successful
except requests.exceptions.RequestException as e:
    print(f'Error accessing {url}: {e}')
else:
    # Parse the HTML content using BeautifulSoup
    soup = BeautifulSoup(response.content, "html.parser")

    # Find all links on the page
    links = soup.find_all('a')

    # Print each link's URL
    for link in links:
        href = link.get('href')
        if href:
            print(href)
```

The screenshot shows the Thonny Python IDE interface. The top window, titled 'scrape.py', contains a Python script that uses the 'requests' and 'BeautifulSoup' libraries to fetch a webpage and extract all links. The script prompts the user for a URL, attempts to fetch it, and if successful, parses the HTML to find all 'a' tags and prints their 'href' attributes. The bottom window, titled 'Shell', shows the execution of the script. It displays the prompt 'Enter the URL of the webpage to scrape:' followed by the user input 'https://yahoo.com'. Below this, the script outputs five URLs found on the page: 'https://www.yahoo.com/', 'https://www.yahoo.com/news/', 'https://www.yahoo.com/news/us/', 'https://www.yahoo.com/news/politics/', and 'https://www.yahoo.com/news/world/'.

```
1 import requests
2 from bs4 import BeautifulSoup
3
4 # Ask the user for a URL
5 url = input("Enter the URL of the webpage to scrape: ")
6
7 try:
8     response = requests.get(url)
9     response.raise_for_status() # Check if the request was successful
10 except requests.exceptions.RequestException as e:
11     print(f"Error accessing {url}: {e}")
12 else:
13     # Parse the HTML content using BeautifulSoup
14     soup = BeautifulSoup(response.content, "html.parser")
15
16 # Find all links on the page
17 links = soup.find_all('a')
18
19 # Print each link's URL
20 for link in links:
21     href = link.get('href')
22     if href:
23         print(href)
```

```
>>> %Run scrape.py
Enter the URL of the webpage to scrape: https://yahoo.com
https://www.yahoo.com/
https://www.yahoo.com/news/
https://www.yahoo.com/news/us/
https://www.yahoo.com/news/politics/
https://www.yahoo.com/news/world/
```

Extracting Images:

Find all images on the page

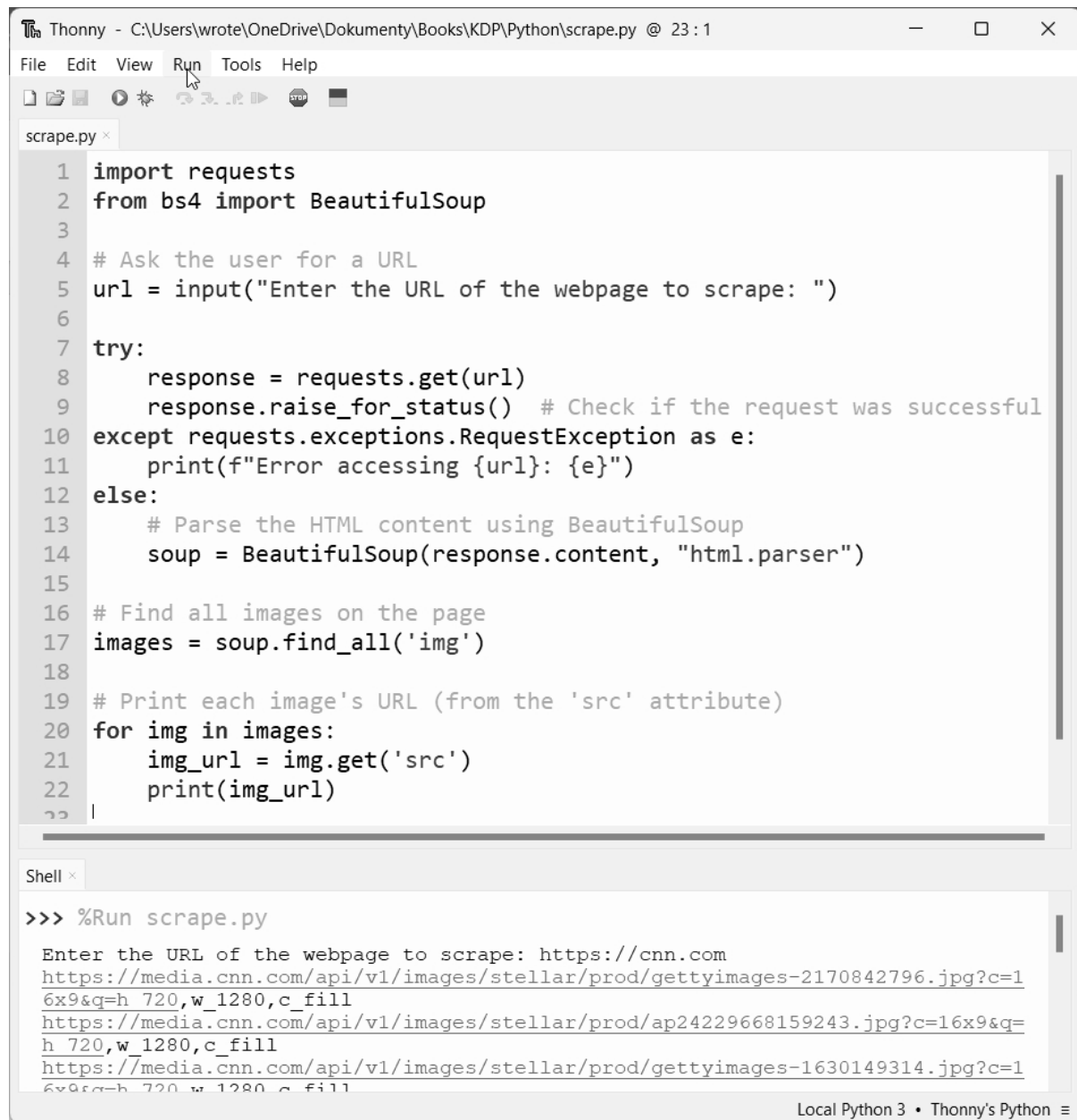
```
images = soup.find_all('img')
```

Print each image's URL (from the 'src' attribute)

```
for img in images:
```

```
    img_url = img.get('src')
```

```
    print(img_url)
```



The screenshot shows the Thonny Python IDE interface. The top menu bar includes File, Edit, View, Run, Tools, and Help. Below the menu is a toolbar with icons for file operations and running code. The main editor window displays a Python script named 'scrape.py' with the following code:

```
1 import requests
2 from bs4 import BeautifulSoup
3
4 # Ask the user for a URL
5 url = input("Enter the URL of the webpage to scrape: ")
6
7 try:
8     response = requests.get(url)
9     response.raise_for_status() # Check if the request was successful
10 except requests.exceptions.RequestException as e:
11     print(f"Error accessing {url}: {e}")
12 else:
13     # Parse the HTML content using BeautifulSoup
14     soup = BeautifulSoup(response.content, "html.parser")
15
16 # Find all images on the page
17 images = soup.find_all('img')
18
19 # Print each image's URL (from the 'src' attribute)
20 for img in images:
21     img_url = img.get('src')
22     print(img_url)
```

Below the editor is a Shell window showing the execution of the script:

```
>>> %Run scrape.py
Enter the URL of the webpage to scrape: https://cnn.com
https://media.cnn.com/api/v1/images/stellar/prod/gettyimages-2170842796.jpg?c=1
6x9&q=h_720,w_1280,c_fill
https://media.cnn.com/api/v1/images/stellar/prod/ap24229668159243.jpg?c=16x9&q=
h_720,w_1280,c_fill
https://media.cnn.com/api/v1/images/stellar/prod/gettyimages-1630149314.jpg?c=1
6x9&q=h_720,w_1280,c_fill
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

10. Adding Search Criteria

You can modify the script to allow users to input keywords to search for specific text or elements on the page.

```
import requests
```

```
from bs4 import BeautifulSoup
```

```
# Ask the user for a URL
```

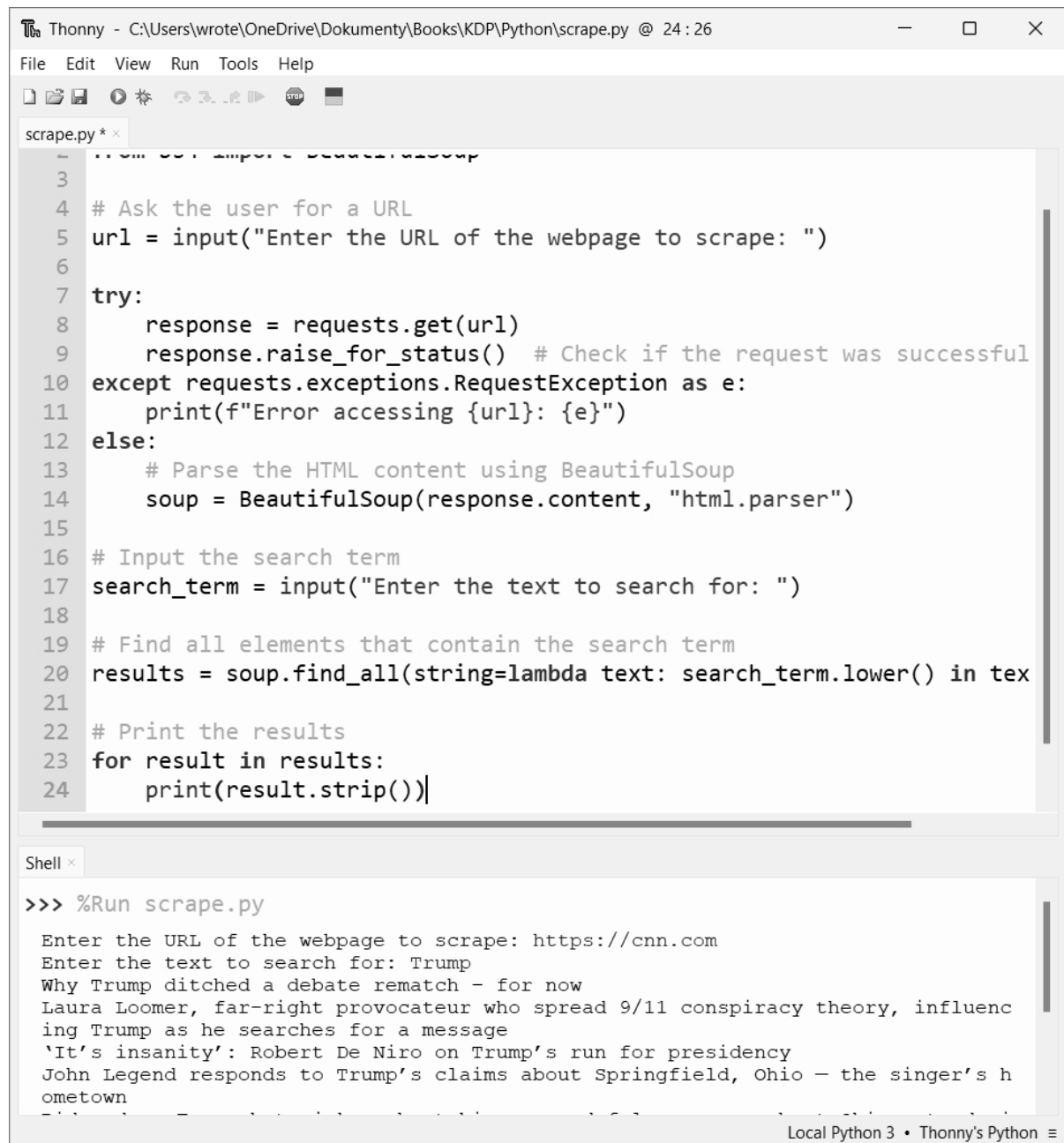
```
url = input("Enter the URL of the webpage to scrape: ")
```

```
try:
    response = requests.get(url)
    response.raise_for_status() # Check if the request was successful
except requests.exceptions.RequestException as e:
    print(f"Error accessing {url}: {e}")
else:
    # Parse the HTML content using BeautifulSoup
    soup = BeautifulSoup(response.content, "html.parser")

    # Input the search term
    search_term = input("Enter the text to search for: ")

    # Find all elements that contain the search term
    results = soup.find_all(string=lambda text: search_term.lower() in text.lower())

    # Print the results
    for result in results:
        print(result.strip())
```



The screenshot shows the Thonny IDE interface. The top window, titled 'scrape.py *', contains a Python script that uses the 'requests' and 'BeautifulSoup' libraries to fetch a webpage and search for a specific term. The script prompts the user for a URL and a search term. The bottom window, titled 'Shell', shows the execution of the script, where the user has entered 'https://cnn.com' and 'Trump', resulting in a list of search results from CNN.

```
1 from urllib import request, urlopen
2
3
4 # Ask the user for a URL
5 url = input("Enter the URL of the webpage to scrape: ")
6
7 try:
8     response = requests.get(url)
9     response.raise_for_status() # Check if the request was successful
10 except requests.exceptions.RequestException as e:
11     print(f"Error accessing {url}: {e}")
12 else:
13     # Parse the HTML content using BeautifulSoup
14     soup = BeautifulSoup(response.content, "html.parser")
15
16 # Input the search term
17 search_term = input("Enter the text to search for: ")
18
19 # Find all elements that contain the search term
20 results = soup.find_all(string=lambda text: search_term.lower() in text)
21
22 # Print the results
23 for result in results:
24     print(result.strip())
```

```
>>> %Run scrape.py
Enter the URL of the webpage to scrape: https://cnn.com
Enter the text to search for: Trump
Why Trump ditched a debate rematch - for now
Laura Loomer, far-right provocateur who spread 9/11 conspiracy theory, influencing Trump as he searches for a message
'It's insanity': Robert De Niro on Trump's run for presidency
John Legend responds to Trump's claims about Springfield, Ohio - the singer's hometown
```

Local Python 3 • Thonny's Python

11. Adding a Delay (Politeness for Web Scraping)

If you're scraping multiple pages or large websites, it's polite to add a delay between requests to avoid overwhelming the server.

```
import time
```

```
# Delay between requests (e.g., 2 seconds)
```

```
time.sleep(2)
```

Notice! By personalizing this basic web scraping script, you can tailor it to your specific needs, whether you're extracting headlines, links, images, or any other web data. You can add error handling, customize the target elements, save data to files, and even scrape multiple pages by modifying this base script.

12. Automating Email Sending

If you regularly send emails — for example, to notify students or colleagues about updates — Python can automate this process. Using the **smtplib** library, you can send emails directly from your Python script.

Example: Sending an Automated Email

Here's a simple example of how to send an email using Python:

```
import smtplib
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart

# Email account credentials
sender_email = "your_email@example.com"
receiver_email = "recipient_email@example.com"
password = "your_password"

# Create the email content
subject = "Automated Email from Python"
body = "Hello, this is an automated email sent from a Python script."

# Set up the email
msg = MIMEMultipart()
msg["From"] = sender_email
msg["To"] = receiver_email
msg["Subject"] = subject
msg.attach(MIMEText(body, "plain"))

# Connect to the email server and send the email
server = smtplib.SMTP("smtp.gmail.com", 587)
server.starttls() # Secure the connection
server.login(sender_email, password)
```

```
server.sendmail(sender_email, receiver_email, msg.as_string())
server.quit()
```

```
print("Email sent successfully!")
```

Explanation:

- **smtplib**: Python's built-in library for sending emails using the Simple Mail Transfer Protocol (SMTP).
- **MIMEText** and **MIMEMultipart**: These classes help format the email's content (subject, body, etc.).
- The script logs in to the email server and sends the email automatically.

After setting up your email account credentials, this script will send an email with the specified subject and body to the recipient.

13. Automating Task Scheduling

If you want Python to automatically run certain tasks at specific times — such as sending reports or organizing files daily — you can use the **schedule** library.

Example: Scheduling a Task to Run Every Day

Install the schedule library first:

```
pip install schedule
```

Then, use the following code to run a specific function (like sending an email or organizing files) every day at a certain time:

```
import schedule
import time

# Define the task to be automated
def send_reminder():
    print("This is your daily reminder!")

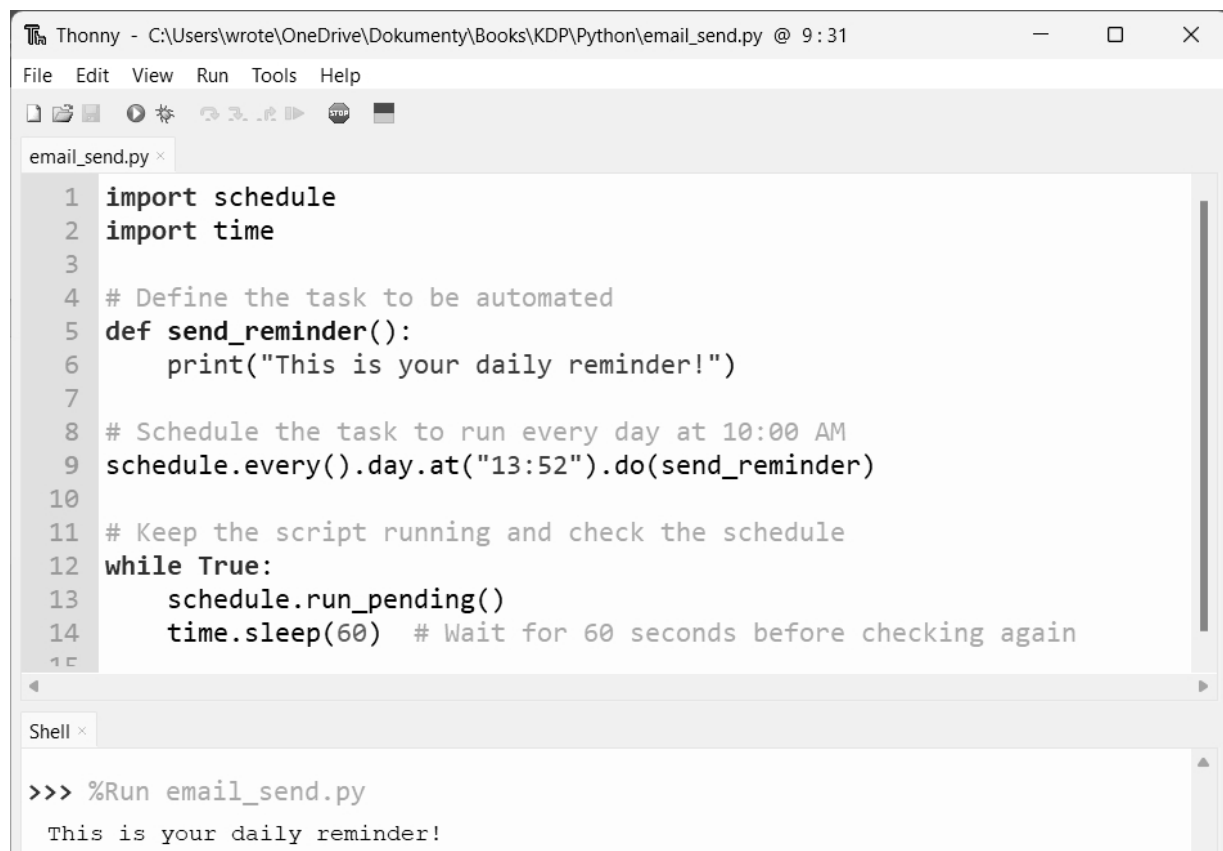
# Schedule the task to run every day at 10:00 AM
schedule.every().day.at("10:00").do(send_reminder)
```

```
# Keep the script running and check the schedule
while True:
    schedule.run_pending()
    time.sleep(60) # Wait for 60 seconds before checking again
```

Explanation:

- `schedule.every().day.at("10:00")` schedules the `send_reminder()` function to run every day at 10:00 AM.
- The script runs continuously and checks every 60 seconds to see if it's time to execute the scheduled task.

This way, you can automate tasks to run at specific intervals, such as daily or weekly, without having to manually run the script.



The screenshot shows the Thonny IDE interface. The top window, titled 'email_send.py', contains the following Python code:

```
1 import schedule
2 import time
3
4 # Define the task to be automated
5 def send_reminder():
6     print("This is your daily reminder!")
7
8 # Schedule the task to run every day at 10:00 AM
9 schedule.every().day.at("13:52").do(send_reminder)
10
11 # Keep the script running and check the schedule
12 while True:
13     schedule.run_pending()
14     time.sleep(60) # Wait for 60 seconds before checking again
15
```

The bottom window, titled 'Shell', shows the command prompt output:

```
>>> %Run email_send.py
This is your daily reminder!
```

Summary

Python makes it easy to automate repetitive tasks, saving you time and reducing the risk of errors. Whether you're organizing files, collecting data

from the web, sending emails, or scheduling tasks, Python's powerful libraries provide simple solutions to everyday problems. With a few lines of code, you can automate tasks that would otherwise take up a lot of your time, boosting your productivity and allowing you to focus on more important work.

Keep practicing, and soon you'll be able to automate a wide range of tasks, turning Python into your personal assistant for everyday life!

Application: Automating Email Sending for Class Updates

Sending class updates, announcements, or reminders via email can be time-consuming if done manually. However, Python can help you **automate email sending**, making this process fast and efficient. Whether you're a teacher sending homework reminders or a student notifying classmates about a project, you can set up a Python script to send emails automatically.

In this section, you'll learn how to use Python to send emails, personalize the content, and even automate the process for regular updates. By the end of this chapter, you'll be able to automate email communication, making it easier to manage class updates and notifications.

1. How Does Email Automation Work?

Email automation with Python involves:

- **Writing a script** that connects to an email server (like Gmail) to send messages.
- **Creating email content**, such as the subject, body, and recipient list.
- **Automating the sending process**, which can be done at regular intervals or triggered by certain events.

For this task, we'll use Python's built-in **smtplib** library to send emails, and **email.mime** to format the content. Let's go step by step.

2. Setting Up for Automated Email Sending

To send emails with Python, you'll need:

- A **sender email address** (this will be the account you use to send the emails).

- The **recipient's email address** (the person or group receiving the emails).
- The login credentials (username and password) of the sender's email account.

Example: Sending a Simple Email

Here's a basic Python script that sends an email using **smtplib**:

```
import smtplib
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart

# Email credentials
sender_email = "your_email@example.com"
receiver_email = "recipient_email@example.com"
password = "your_password"

# Create the email content
subject = "Class Update: Homework Reminder"
body = """
Hello everyone,

Just a reminder that the homework for this week is due on Friday. Please submit it through the online
portal.

Best regards,
[Your Name]
"""

# Set up the MIME object
msg = MIMEMultipart()
msg["From"] = sender_email
msg["To"] = receiver_email
msg["Subject"] = subject
msg.attach(MIMEText(body, "plain"))

# Set up the SMTP server connection and send the email
try:
```

```

server = smtplib.SMTP("smtp.gmail.com", 587)
server.starttls() # Secures the connection
server.login(sender_email, password)
server.sendmail(sender_email, receiver_email, msg.as_string())
server.quit()
print("Email sent successfully!")
except Exception as e:
    print(f"Error sending email: {e}")

```

Explanation:

- **smtplib**: This library is used to connect to the email server and send the message.
- **MIMEMultipart**: This class allows you to structure the email with a subject, body, and attachments (if needed).
- The script logs in to your email account, sends the email, and then closes the connection.

3. Customizing the Email Content

You can personalize the email content to suit different purposes, such as:

- **Homework reminders.**
- **Class announcements.**
- **Event notifications.**

Example: Sending Multiple Emails

You can send emails to multiple recipients by looping through a list of email addresses:

```

# List of recipients
recipients = ["student1@example.com", "student2@example.com", "student3@example.com"]

# Loop through each recipient and send the email
for receiver_email in recipients:
    msg["To"] = receiver_email

```

```
server.sendmail(sender_email, receiver_email, msg.as_string())
```

This script will send the same email to multiple students, saving you time from having to send emails one by one.

4. Adding Attachments to Emails

You can also include attachments, such as homework files, class notes, or event flyers, in the emails.

Example: Attaching a File to the Email

To attach a file (e.g., a PDF or Word document), use the **MIMEBase** class:

```
from email.mime.base import MIMEBase
from email import encoders

# Specify the file you want to attach
file_path = "class_schedule.pdf"

# Open the file in binary mode
with open(file_path, "rb") as attachment:
    part = MIMEBase("application", "octet-stream")
    part.set_payload(attachment.read())

# Encode the file for safe transmission
encoders.encode_base64(part)
part.add_header(
    "Content-Disposition",
    f"attachment; filename= {file_path}",
)

# Attach the file to the email
msg.attach(part)
```

This adds a file as an attachment to your email, allowing you to send class materials along with the message.

5. Scheduling Emails for Regular Updates

Python can also automate sending emails at specific times or intervals, such as sending weekly reminders or monthly class updates. You can do this by combining your email script with a scheduling library like **schedule**.

Example: Scheduling an Email to Send Every Friday

First, install the schedule library:

```
pip install schedule
```

Then, modify your script to automatically send an email every Friday at 10 AM:

```
import schedule
import time

def send_class_update():
    # Place the email sending code here
    print("Sending class update email...")

# Schedule the task for every Friday at 10 AM
schedule.every().friday.at("10:00").do(send_class_update)

# Keep the script running and checking for the scheduled time
while True:
    schedule.run_pending()
    time.sleep(60) # Check every 60 seconds
```

This script will automatically send the email every Friday at 10:00 AM without any manual intervention.

6. Enhancing Security: Using Environment Variables

It's important to **protect your email credentials** (like passwords) when using automation. One way to do this is by storing sensitive information in environment variables instead of hardcoding them into the script.

Example: Using Environment Variables

```
import os

# Use environment variables to store your email credentials securely
sender_email = os.getenv("SENDER_EMAIL")
password = os.getenv("EMAIL_PASSWORD")

# Continue with the rest of your email sending script
```

You can set environment variables in your operating system to keep your sensitive data safe.

7. Handling Errors and Improving Reliability

When automating email sending, it's important to handle errors gracefully. For example, the internet connection might be lost, or the email server might be temporarily down. You can improve your script's reliability by catching exceptions and retrying when necessary:

try:

```
server = smtplib.SMTP("smtp.gmail.com", 587)
server.starttls()
server.login(sender_email, password)
server.sendmail(sender_email, receiver_email, msg.as_string())
server.quit()
print("Email sent successfully!")
```

except smtplib.SMTPException as e:

```
print(f"Failed to send email: {e}")
```

This ensures that even if something goes wrong, your script will handle the error without crashing.

Summary

Automating email sending with Python makes it easy to stay on top of classroom communication. Whether you need to send weekly updates, homework reminders, or announcements, this powerful tool simplifies the process, saving time and effort.

By using Python's smtplib library, you can send emails with customized content, include attachments, and even schedule emails to go out at specific times. With a little practice, you'll be able to automate email communication in a way that improves efficiency and keeps your class up-to-date with important information.

Building a Simple Task Scheduler

In everyday life, there are many repetitive tasks that need to be done at specific times—whether it’s sending reminders, cleaning up files, or running reports. Manually performing these tasks can be tedious and time-consuming. This is where a **task scheduler** comes in handy. Using Python, you can create a simple task scheduler to automate these repetitive tasks and run them at regular intervals or specific times.

In this section, we’ll explore how to build a Python-based task scheduler using a straightforward approach that anyone can follow, even if you’re a beginner. By the end of this chapter, you’ll be able to set up your own automated tasks, freeing up your time and making your workflow smoother.

1. What Is a Task Scheduler?

A **task scheduler** is a tool that allows you to run specific tasks automatically at predetermined times. For example, you can:

- Schedule a daily backup of important files.
- Set up a weekly email reminder.
- Run data processing scripts every hour.

With Python, building a task scheduler is simple, especially with the help of libraries like `schedule` that manage the scheduling for you.

2. Getting Started with Python Scheduling

To build a task scheduler in Python, you’ll first need the **`schedule`** library, which makes it easy to schedule tasks at specific times. You can install it using:

```
pip install schedule
```

Once installed, you can start scheduling tasks in just a few lines of code.

3. How to Schedule a Simple Task

Let’s begin by scheduling a simple task that runs every day at a specific time. For example, we can schedule a function that sends a reminder every morning at 8:00 AM.

Example: Scheduling a Daily Reminder

```
import schedule
import time

# Define the task you want to automate
def send_reminder():
    print("This is your daily reminder!")

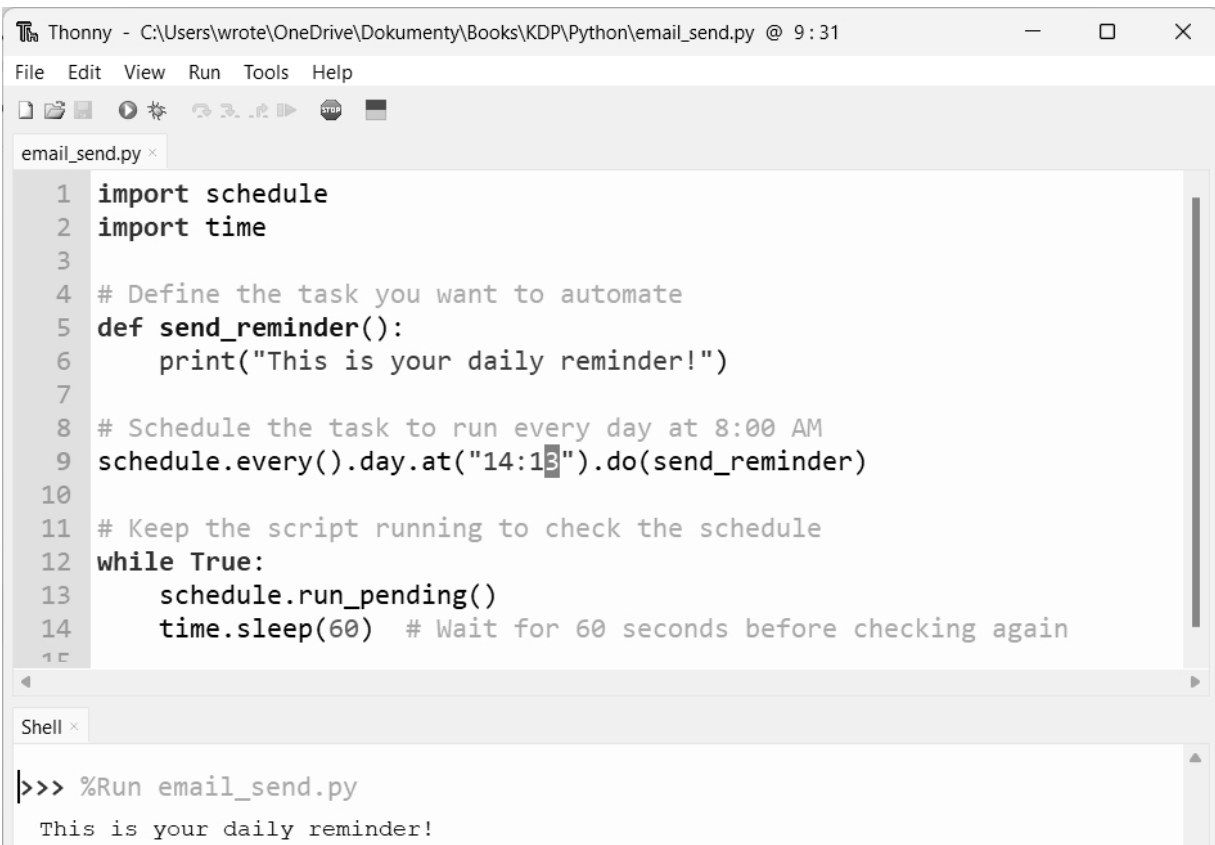
# Schedule the task to run every day at 8:00 AM
schedule.every().day.at("08:00").do(send_reminder)

# Keep the script running to check the schedule
while True:
    schedule.run_pending()
    time.sleep(60) # Wait for 60 seconds before checking again
```

Explanation:

- **schedule.every().day.at("08:00")** schedules the `send_reminder()` function to run every day at 8:00 AM.
- **schedule.run_pending()** checks whether any scheduled tasks need to be run. This function continuously runs in the background.
- **time.sleep(60)** ensures the script waits for 60 seconds before checking the schedule again.

This simple script will print "This is your daily reminder!" every day at 8:00 AM. You can replace the `send_reminder()` function with any other task you want to automate.



The screenshot shows the Thonny Python IDE interface. The title bar indicates the file path: C:\Users\wrote\OneDrive\Dokumenty\Books\KDP\Python\email_send.py @ 9:31. The menu bar includes File, Edit, View, Run, Tools, and Help. Below the menu bar is a toolbar with icons for file operations and running code. The main editor window displays the following Python code in email_send.py:

```
1 import schedule
2 import time
3
4 # Define the task you want to automate
5 def send_reminder():
6     print("This is your daily reminder!")
7
8 # Schedule the task to run every day at 8:00 AM
9 schedule.every().day.at("14:13").do(send_reminder)
10
11 # Keep the script running to check the schedule
12 while True:
13     schedule.run_pending()
14     time.sleep(60) # Wait for 60 seconds before checking again
```

Below the editor is a Shell window. It shows the command `>>> %Run email_send.py` being executed, followed by the output: `This is your daily reminder!`

4. Customizing Task Intervals

With the **schedule** library, you can schedule tasks to run at various intervals—every minute, hour, day, or even weekly. Here are some common ways to customize your schedule:

Scheduling a Task Every Hour

```
# Schedule a task to run every hour
schedule.every().hour.do(send_reminder)
```

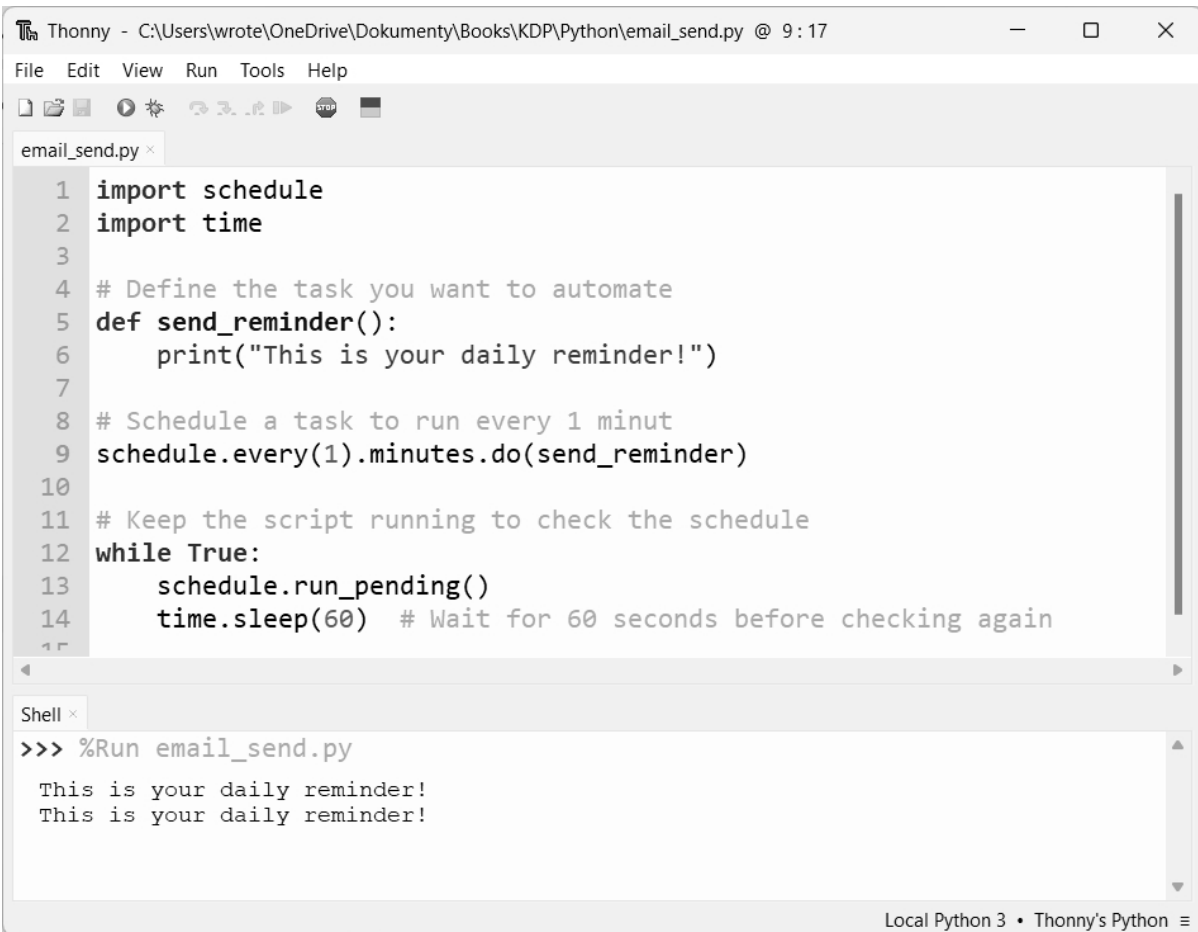
Scheduling a Task Every Monday at a Specific Time

```
# Schedule a task to run every Monday at 9:00 AM
schedule.every().monday.at("09:00").do(send_reminder)
```

Scheduling a Task Every 10 Minutes

```
# Schedule a task to run every 10 minutes
schedule.every(10).minutes.do(send_reminder)
```

These examples show how flexible the scheduling library is, allowing you to set up tasks to run at whatever frequency you need.



The screenshot shows the Thonny Python IDE interface. The top window, titled 'email_send.py', contains the following Python code:

```
1 import schedule
2 import time
3
4 # Define the task you want to automate
5 def send_reminder():
6     print("This is your daily reminder!")
7
8 # Schedule a task to run every 1 minute
9 schedule.every(1).minutes.do(send_reminder)
10
11 # Keep the script running to check the schedule
12 while True:
13     schedule.run_pending()
14     time.sleep(60) # Wait for 60 seconds before checking again
```

The bottom window, titled 'Shell', shows the output of running the script:

```
>>> %Run email_send.py
This is your daily reminder!
This is your daily reminder!
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

5. Running Multiple Tasks with Different Schedules

You're not limited to scheduling just one task. You can set up multiple tasks with different schedules in the same script.

Example: Scheduling Multiple Tasks

```
def backup_files():
    print("Backing up files...")

def clean_folder():
    print("Cleaning up temporary files...")

# Schedule multiple tasks
schedule.every().day.at("08:00").do(send_reminder) # Daily reminder at 8:00 AM
schedule.every().friday.at("18:00").do(backup_files) # Backup files every Friday at 6:00 PM
schedule.every(15).minutes.do(clean_folder) # Clean folder every 15 minutes

# Keep the script running
```

```
while True:
    schedule.run_pending()
    time.sleep(60)
```

In this example, the script:

- Sends a daily reminder at 8:00 AM.
- Runs a backup every Friday at 6:00 PM.
- Cleans up temporary files every 15 minutes.

The script continuously runs and checks if any tasks are scheduled to run at the current time.

6. Making Your Scheduler More Reliable

When scheduling tasks, it's important to handle errors gracefully, especially if something goes wrong, like losing internet connectivity or encountering a file that doesn't exist. You can wrap your tasks in try-except blocks to catch and handle errors.

Example: Adding Error Handling

```
def send_reminder():
    try:
        print("Sending reminder...")
        # Simulate sending an email or notification
        # raise Exception("Failed to send email") # Uncomment to simulate an error
    except Exception as e:
        print(f"Error: {e}")

schedule.every().day.at("08:00").do(send_reminder)
```

```
while True:
    schedule.run_pending()
    time.sleep(60)
```

Here, the `send_reminder()` function will continue running even if an error occurs, and the script will print the error message instead of crashing.

7. Practical Use Cases for a Task Scheduler

Here are some practical examples of how you can use a Python task scheduler in daily life:

- **Automating backups:** Schedule a script to back up important files at the end of each week.
- **Sending reminders:** Send emails or notifications to students or colleagues about important deadlines.
- **Cleaning up files:** Automatically delete old or temporary files from specific folders every hour or day.
- **Fetching data:** Schedule a web scraper to collect data from a website every morning or at regular intervals.

Summary

Building a simple task scheduler in Python is a powerful way to automate repetitive tasks and manage your time more efficiently. Whether you need to send reminders, back up files, or clean up folders, Python's schedule library makes it easy to set up and run these tasks at regular intervals.

By customizing your schedules, handling errors gracefully, you can automate tasks without worrying about manually starting or managing them. With these skills, you'll be able to streamline your daily routines and boost your productivity. Keep experimenting, and soon you'll have a fully automated system that takes care of everyday tasks for you!

Part IV: Problem Solving in Real-World Scenarios

Chapter 11: Games and Puzzles with Python

Building Simple Games: Rock, Paper, Scissors, and Tic-Tac-Toe

Building simple games like **Rock, Paper, Scissors** and **Tic-Tac-Toe** is a fun and effective way to learn Python. These games involve basic programming concepts such as loops, conditionals, user input, and even some simple algorithms. They are great for beginners because they are interactive and easy to understand, yet they provide valuable practice for problem-solving and thinking logically.

In this section, we'll walk through how to build both games step by step. By the end, you'll be able to create your own versions of these games, which you can play with friends or even expand into more advanced projects.

1. Building Rock, Paper, Scissors

Rock, Paper, Scissors is a classic two-player game where players each choose one of three options: rock, paper, or scissors. The rules are simple:

- Rock beats scissors.
- Scissors beats paper.
- Paper beats rock.

In this version of the game, the player competes against the computer, which randomly chooses rock, paper, or scissors.

Step-by-Step Implementation

Let's start by writing the Python code for a simple Rock, Paper, Scissors game.

```

import random

# Define the possible moves
moves = ["rock", "paper", "scissors"]

# Function to determine the winner
def determine_winner(player_move, computer_move):
    if player_move == computer_move:
        return "It's a tie!"
    elif (player_move == "rock" and computer_move == "scissors") or \
        (player_move == "scissors" and computer_move == "paper") or \
        (player_move == "paper" and computer_move == "rock"):
        return "You win!"
    else:
        return "Computer wins!"

# Main game loop
while True:
    # Get the player's move
    player_move = input("Enter rock, paper, or scissors (or 'quit' to exit): ").lower()

    # Allow the player to exit the game
    if player_move == "quit":
        break

    # Ensure the player enters a valid move
    if player_move not in moves:
        print("Invalid input, please try again.")
        continue

    # Get the computer's move
    computer_move = random.choice(moves)
    print(f"Computer chose: {computer_move}")

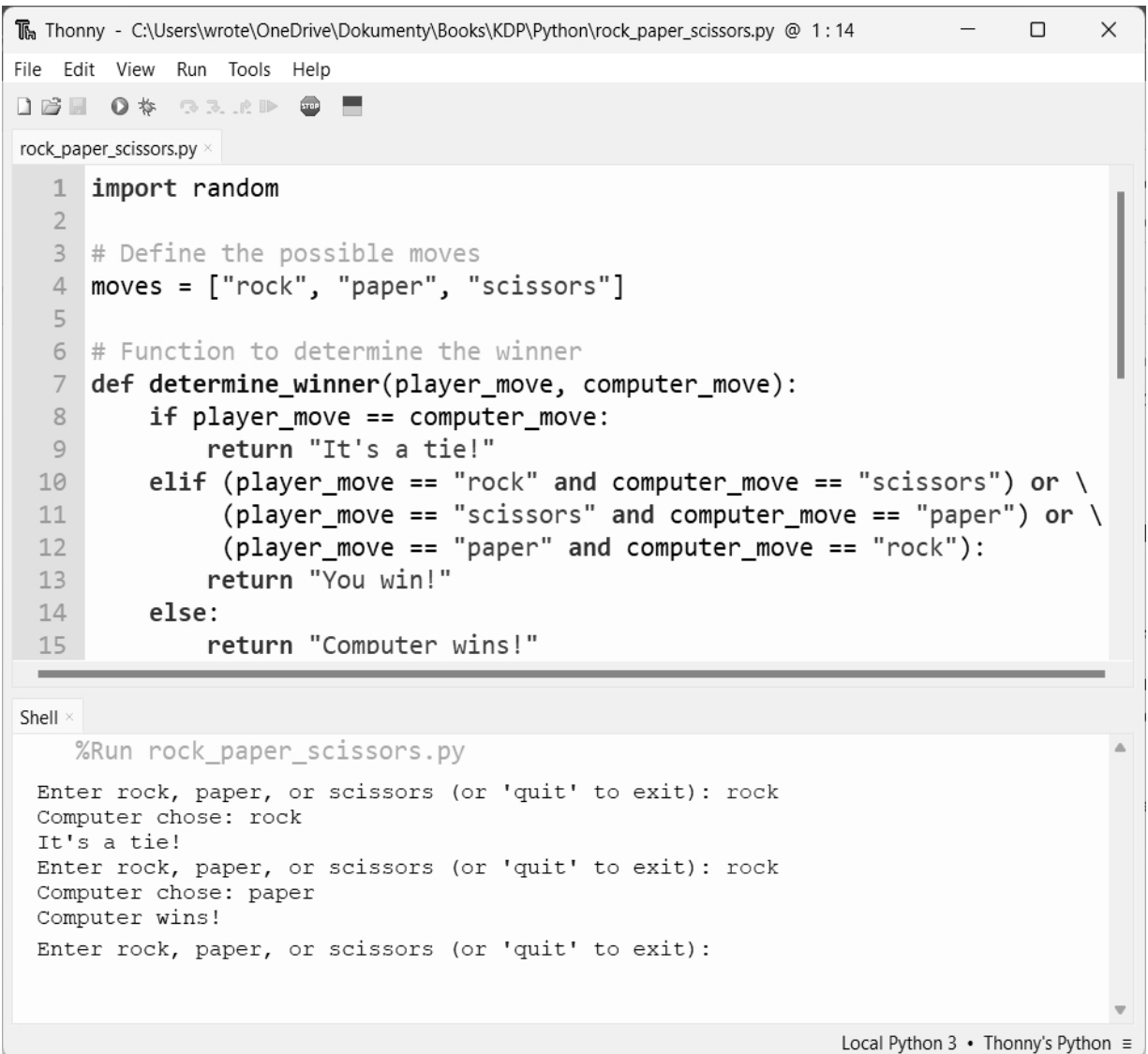
    # Determine and print the winner
    result = determine_winner(player_move, computer_move)
    print(result)

```

How It Works:

- **User Input:** The player enters their move (rock, paper, or scissors).
- **Random Computer Move:** The computer randomly chooses one of the three moves using the `random.choice()` function.
- **Determine the Winner:** The function `determine_winner()` checks the player's move against the computer's move and prints who wins, loses, or if it's a tie.
- **Game Loop:** The game continues in a loop until the player types "quit" to exit.

Output Example:



The screenshot shows the Thonny Python IDE interface. The top window, titled 'rock_paper_scissors.py', contains the following Python code:

```
1 import random
2
3 # Define the possible moves
4 moves = ["rock", "paper", "scissors"]
5
6 # Function to determine the winner
7 def determine_winner(player_move, computer_move):
8     if player_move == computer_move:
9         return "It's a tie!"
10    elif (player_move == "rock" and computer_move == "scissors") or \
11          (player_move == "scissors" and computer_move == "paper") or \
12          (player_move == "paper" and computer_move == "rock"):
13        return "You win!"
14    else:
15        return "Computer wins!"
```

The bottom window, titled 'Shell', shows the execution output:

```
%Run rock_paper_scissors.py
Enter rock, paper, or scissors (or 'quit' to exit): rock
Computer chose: rock
It's a tie!
Enter rock, paper, or scissors (or 'quit' to exit): rock
Computer chose: paper
Computer wins!
Enter rock, paper, or scissors (or 'quit' to exit):
```

The status bar at the bottom right indicates 'Local Python 3 • Thonny's Python'.

This simple game is a great way to practice conditional statements and handling user input. You can easily expand it by keeping track of scores or adding more features, like a best-of-five mode.

2. Building Tic-Tac-Toe

Tic-Tac-Toe is another classic game played on a 3x3 grid. Two players take turns placing their markers (X or O) on the grid, and the goal is to get three of their markers in a row, either horizontally, vertically, or diagonally.

In this Python version, you can play against another human or, with a few modifications, against the computer.

Step-by-Step Implementation

```

# Define the board (a list of 9 empty spaces)
board = [" " for _ in range(9)]

# Function to print the board
def print_board():
    print(f"{board[0]} | {board[1]} | {board[2]}")
    print("-" * 9)
    print(f"{board[3]} | {board[4]} | {board[5]}")
    print("-" * 9)
    print(f"{board[6]} | {board[7]} | {board[8]}")

# Function to check if a player has won
def check_winner(player):
    win_conditions = [
        [0, 1, 2], # Top row
        [3, 4, 5], # Middle row
        [6, 7, 8], # Bottom row
        [0, 3, 6], # Left column
        [1, 4, 7], # Middle column
        [2, 5, 8], # Right column
        [0, 4, 8], # Diagonal from top-left
        [2, 4, 6]  # Diagonal from top-right
    ]

    for condition in win_conditions:
        if board[condition[0]] == board[condition[1]] == board[condition[2]] == player:
            return True
    return False

# Main game loop
current_player = "X"
while True:
    print_board()

    # Get the player's move
    try:
        move = int(input(f"Player {current_player}, enter your move (1-9): ")) - 1

```

```

except ValueError:
    print("Invalid input, please enter a number between 1 and 9.")
    continue

# Check if the move is valid
if move < 0 or move > 8 or board[move] != " ":
    print("Invalid move, please try again.")
    continue

# Place the player's marker on the board
board[move] = current_player

# Check if the current player has won
if check_winner(current_player):
    print_board()
    print(f"Player {current_player} wins!")
    break

# Check for a tie (if the board is full)
if " " not in board:
    print_board()
    print("It's a tie!")
    break

# Switch to the other player
current_player = "O" if current_player == "X" else "X"

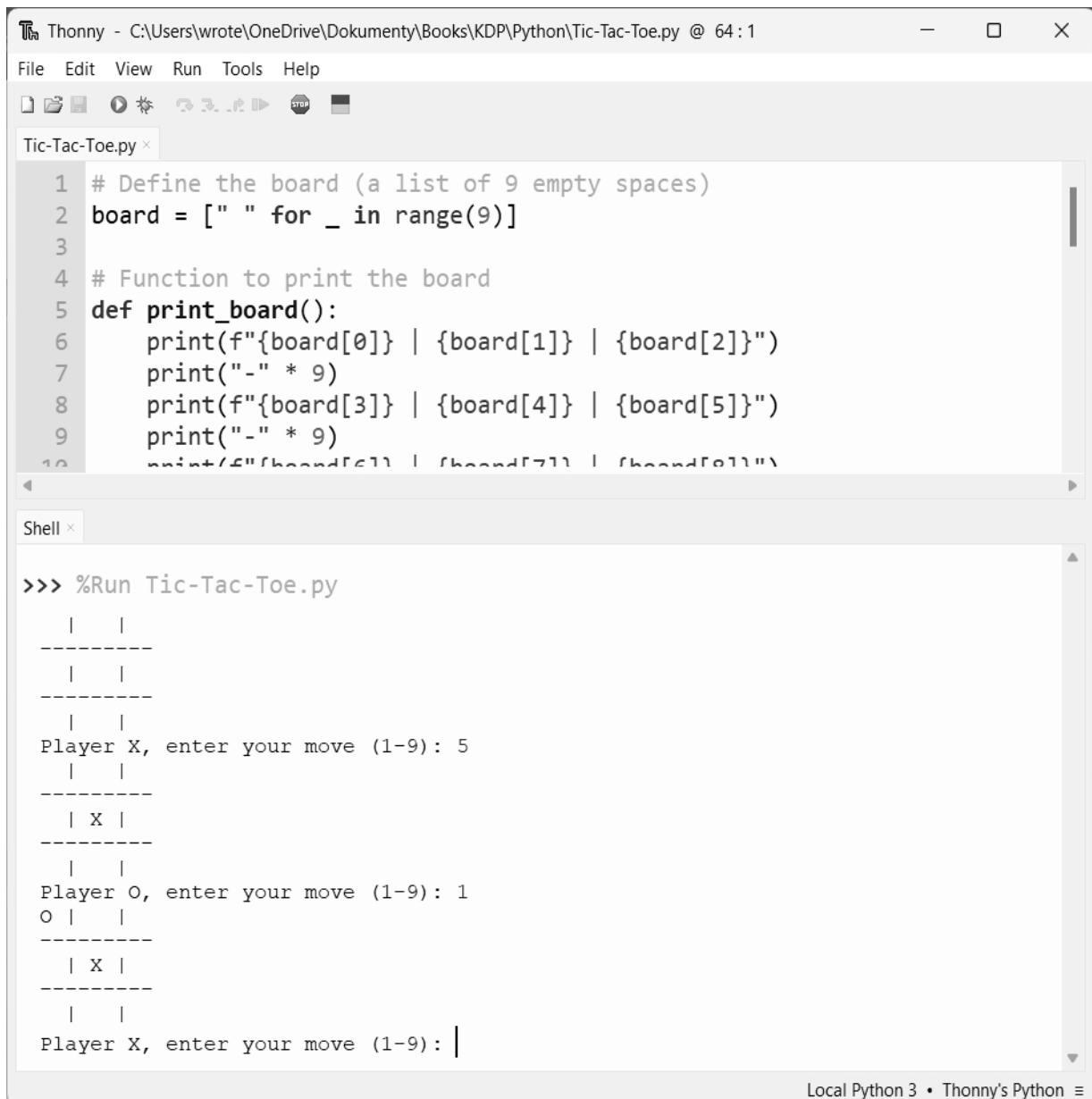
```

How It Works:

- **Board Representation:** The board is represented as a list of 9 elements, which starts out as empty spaces.
- **Printing the Board:** The `print_board()` function displays the current state of the board.
- **Player Input:** Each player enters their move by choosing a number between 1 and 9 (which corresponds to a position on the board).

- **Win Conditions:** The `check_winner()` function checks the board for winning conditions after each move.
- **Switch Players:** After each turn, the game switches between Player X and Player O.

Output Example:



The screenshot shows the Thonny IDE with a file named `Tic-Tac-Toe.py` open. The code defines a 3x3 board as a list of 9 empty spaces and a function to print the board. The Shell window shows the execution of the program, where Player X enters move 5 and Player O enters move 1. The board is displayed with dashes for empty spaces and X or O for occupied spaces.

```

1 # Define the board (a list of 9 empty spaces)
2 board = [" " for _ in range(9)]
3
4 # Function to print the board
5 def print_board():
6     print(f"{board[0]} | {board[1]} | {board[2]}")
7     print("-" * 9)
8     print(f"{board[3]} | {board[4]} | {board[5]}")
9     print("-" * 9)
10    print(f"{board[6]} | {board[7]} | {board[8]}")

```

```

>>> %Run Tic-Tac-Toe.py
  |  |
-----
  |  |
-----
  |  |
Player X, enter your move (1-9): 5
  |  |
-----
  | X |
-----
  |  |
Player O, enter your move (1-9): 1
O |  |
-----
  | X |
-----
  |  |
Player X, enter your move (1-9): |

```

The game continues until one player wins or the game ends in a tie. You can further customize this game by adding a restart option or even a simple AI opponent.

3. Why Build Simple Games?

Building simple games like Rock, Paper, Scissors and Tic-Tac-Toe is not just fun; it's a great way to learn core programming concepts:

- **User Input and Validation:** Handling input from players and ensuring it's valid.
- **Loops and Conditionals:** Repeating actions (like turns) and checking for win conditions or ties.
- **Data Structures:** Using lists to represent the game board and track moves.
- **Problem Solving:** Figuring out how to structure a game, handle the rules, and determine when someone wins.

These projects are also easy to expand. You can add more features, create a graphical user interface, or turn them into multiplayer online games as you advance in your programming journey.

Summary

Building simple games like **Rock, Paper, Scissors** and **Tic-Tac-Toe** provides a solid foundation in Python programming. These games involve many important concepts, such as loops, conditionals, and data structures, making them ideal for learning while having fun. Plus, they offer plenty of room for creativity and expansion as your skills grow.

With these basic games, you're not just learning to code — you're learning to think logically and solve problems in creative ways, all while building something you can enjoy and share with others. Keep experimenting, and soon you'll be creating even more advanced games and applications!

Solving Logical Puzzles with Python

Algorithms

Logical puzzles are a fun and engaging way to improve your problem-solving skills, and Python provides the perfect toolkit to solve them. By applying Python algorithms, you can break down even the most challenging puzzles into manageable parts. Whether it's cracking a number puzzle, solving a maze, or tackling a riddle, Python can help you arrive at the solution logically and efficiently.

In this section, we'll explore how to approach different types of puzzles using Python algorithms. These puzzles not only test your logical thinking but also help you sharpen your programming skills. By the end, you'll be able to solve puzzles in a way that's both practical and fun, making Python a valuable tool for students and puzzle enthusiasts alike.

1. Why Use Python for Logical Puzzles?

Solving puzzles with Python has many advantages:

- **Automation:** You can let Python handle repetitive tasks, such as testing multiple possibilities or performing calculations quickly.
- **Efficiency:** Python's algorithms can process large amounts of data faster than you could manually, making even complex puzzles solvable in seconds.
- **Flexibility:** Python's libraries and tools make it easy to handle different types of puzzles, from word games to number-based riddles.

Let's start by exploring how Python can be applied to specific types of logical puzzles.

2. Solving Number Puzzles with Python

Number puzzles often require you to identify patterns, calculate solutions, or try different combinations of numbers. One common number puzzle is the **Sudoku** puzzle, where you need to fill a 9x9 grid with numbers from 1 to 9, ensuring that each number appears only once in every row, column, and 3x3 subgrid.

While solving Sudoku by hand can be time-consuming, Python allows us to automate this process by following a logical algorithm.

Example: Solving a Sudoku Puzzle

Here's a simple algorithm to solve a Sudoku puzzle using backtracking, a technique that tries different possibilities and "backtracks" when it hits a dead end.

```
# A partially filled Sudoku board (0 represents an empty space)
```

```
board = [  
    [5, 3, 0, 0, 7, 0, 0, 0, 0],  
    [6, 0, 0, 1, 9, 5, 0, 0, 0],  
    [0, 9, 8, 0, 0, 0, 0, 6, 0],  
    [8, 0, 0, 0, 6, 0, 0, 0, 3],  
    [4, 0, 0, 8, 0, 3, 0, 0, 1],  
    [7, 0, 0, 0, 2, 0, 0, 0, 6],  
    [0, 6, 0, 0, 0, 0, 2, 8, 0],  
    [0, 0, 0, 4, 1, 9, 0, 0, 5],  
    [0, 0, 0, 0, 8, 0, 0, 7, 9]  
]
```

```
# Function to check if a number can be placed in a position
```

```
def is_valid(board, row, col, num):
```

```
    # Check the row
```

```
    for i in range(9):
```

```
        if board[row][i] == num:
```

```
            return False
```

```
    # Check the column
```

```
    for i in range(9):
```

```
        if board[i][col] == num:
```

```
            return False
```

```

# Check the 3x3 subgrid
start_row, start_col = 3 * (row // 3), 3 * (col // 3)
for i in range(start_row, start_row + 3):
    for j in range(start_col, start_col + 3):
        if board[i][j] == num:
            return False
    return True

# Backtracking algorithm to solve the Sudoku puzzle
def solve_sudoku(board):
    for row in range(9):
        for col in range(9):
            if board[row][col] == 0:
                for num in range(1, 10):
                    if is_valid(board, row, col, num):
                        board[row][col] = num
                        if solve_sudoku(board):
                            return True
                        board[row][col] = 0 # Backtrack
                return False
    return True

# Function to print the Sudoku board
def print_board(board):
    for row in board:
        print(" ".join(str(num) if num != 0 else '.' for num in row))

# Solve the Sudoku puzzle
if solve_sudoku(board):
    print("Sudoku solved successfully:")
    print_board(board)
else:
    print("No solution exists.")

```

Explanation:

- **Backtracking:** The algorithm tries placing numbers (1-9) in empty spots and checks if the move is valid. If it's valid, it moves on. If it hits a dead end, it backtracks to the previous move and tries another number.
- **is_valid():** This function checks if placing a number in a specific row, column, or 3x3 subgrid is allowed.
- **solve_sudoku():** The recursive function that attempts to solve the puzzle by filling empty spaces.

This algorithm works well for solving most Sudoku puzzles, demonstrating how Python can handle number puzzles logically and efficiently.

The screenshot shows the Thonny Python IDE interface. The top window, titled 'Sudoku.py', contains a Python script defining a 9x9 Sudoku board. The board is represented as a list of lists, where 0 indicates an empty cell. The script defines the board as follows:

```

1 # A partially filled Sudoku board (0 represents an empty space)
2 board = [
3     [5, 3, 0, 0, 7, 0, 0, 0, 0],
4     [6, 0, 0, 1, 9, 5, 0, 0, 0],
5     [0, 9, 8, 0, 0, 0, 0, 6, 0],
6     [8, 0, 0, 0, 6, 0, 0, 0, 3],
7     [4, 0, 0, 8, 0, 3, 0, 0, 1],
8     [7, 0, 0, 0, 2, 0, 0, 0, 6],
9     [0, 6, 0, 0, 0, 0, 2, 8, 0],
10    [0, 0, 0, 4, 1, 9, 0, 0, 5],
11    [0, 0, 0, 0, 8, 0, 0, 7, 9]]

```

The bottom window, titled 'Shell', shows the execution of the script. It displays the command prompt prompt ' >>> ' followed by the command '%Run Sudoku.py'. The output of the script is 'Sudoku solved successfully:', followed by the solved 9x9 board:

```

Sudoku solved successfully:
5 3 4 6 7 8 9 1 2
6 7 2 1 9 5 3 4 8
1 9 8 3 4 2 5 6 7
8 5 9 7 6 1 4 2 3
4 2 6 8 5 3 7 9 1
7 1 3 9 2 4 8 5 6
9 6 1 5 3 7 2 8 4
2 8 7 4 1 9 6 3 5
3 4 5 2 8 6 1 7 9

```

The status bar at the bottom right of the IDE indicates 'Local Python 3 • Thonny's Python'.

3. Solving Word Puzzles with Python

Python can also be applied to solve word-based puzzles like **anagrams**, **crossword puzzles**, or **word search** games. These puzzles often involve rearranging letters, finding hidden words, or solving clues.

Example: Finding Anagrams

An anagram is a word or phrase that is formed by rearranging the letters of another. Using Python, we can write a simple algorithm to find anagrams from a list of words.

```
from collections import defaultdict

# List of words
words = ["listen", "silent", "enlist", "inlets", "stone", "notes", "tones", "hello", "below", "elbow"]

# Function to group anagrams
def find_anagrams(words):
    anagram_groups = defaultdict(list)

    for word in words:
        # Sort the letters of the word and use it as the key
        key = ''.join(sorted(word))
        anagram_groups[key].append(word)

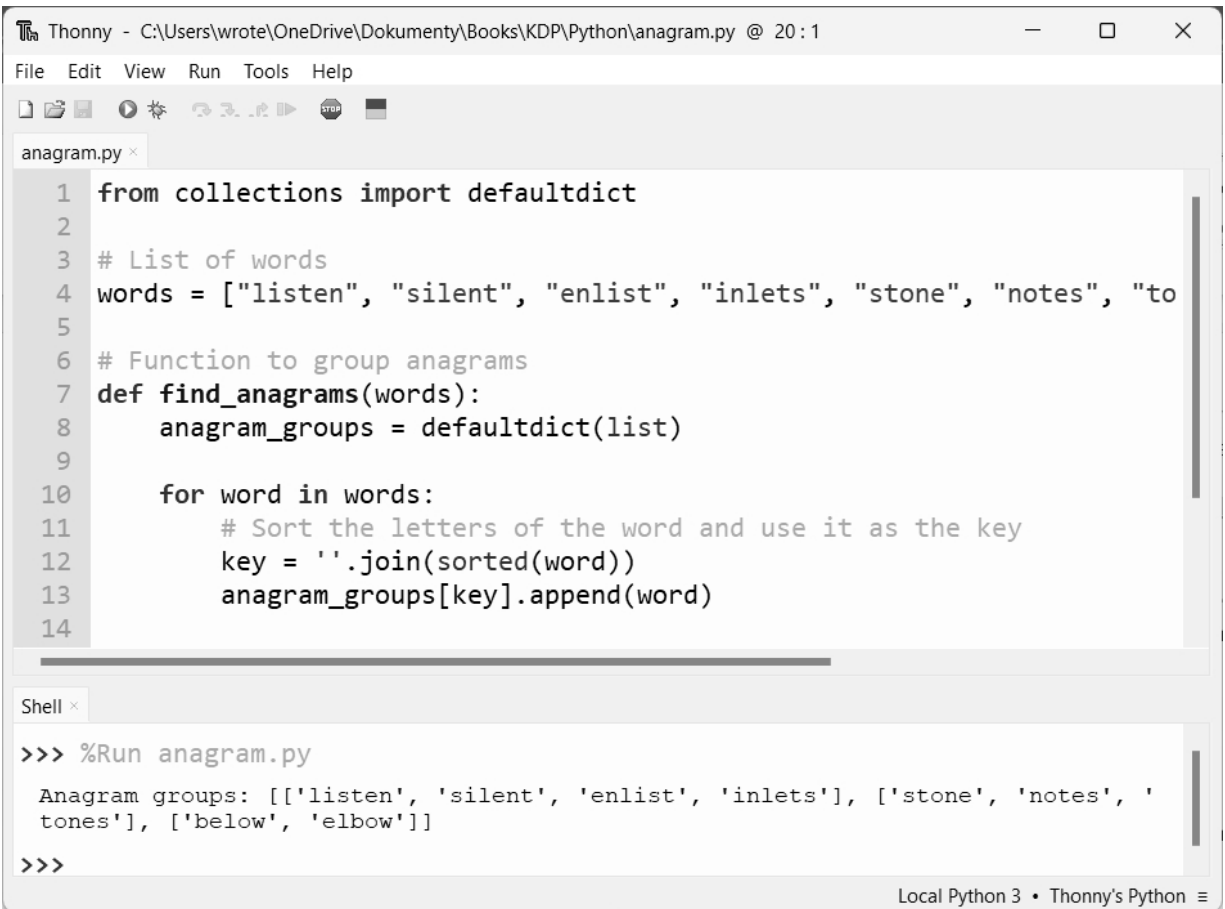
    return [group for group in anagram_groups.values() if len(group) > 1]

# Find and print the anagram groups
anagrams = find_anagrams(words)
print("Anagram groups:", anagrams)
```

Explanation:

- **Sorted Key:** For each word, we sort its letters alphabetically. Words that are anagrams will have the same sorted key.
- **Defaultdict:** We use a defaultdict to group words that have the same sorted key together.
- **Find Anagrams:** Finally, we return all groups of words that contain more than one word (i.e., the anagrams).

Output:

The image shows a screenshot of the Thonny Python IDE. The main window displays a Python script named 'anagram.py'. The script imports 'defaultdict' from the 'collections' module, defines a list of words, and implements a function 'find_anagrams' that groups words by their sorted letters. Below the script, a 'Shell' window shows the output of running the script, displaying the anagram groups. The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

```
Thonny - C:\Users\wrote\OneDrive\Dokumenty\Books\KDP\Python\anagram.py @ 20:1
File Edit View Run Tools Help
anagram.py x
1 from collections import defaultdict
2
3 # List of words
4 words = ["listen", "silent", "enlist", "inlets", "stone", "notes", "to
5
6 # Function to group anagrams
7 def find_anagrams(words):
8     anagram_groups = defaultdict(list)
9
10    for word in words:
11        # Sort the letters of the word and use it as the key
12        key = ''.join(sorted(word))
13        anagram_groups[key].append(word)
14
Shell x
>>> %Run anagram.py
Anagram groups: [['listen', 'silent', 'enlist', 'inlets'], ['stone', 'notes', '
tones'], ['below', 'elbow']]
>>>
Local Python 3 • Thonny's Python
```

This simple algorithm finds anagrams efficiently, demonstrating how Python can be used to solve word puzzles with minimal effort.

Summary

Using Python to solve logical puzzles is not only a fun exercise but also a practical way to improve your problem-solving skills. By applying algorithms like backtracking for number puzzles, sorting for word games, and logical state checks for riddles, Python makes solving puzzles an engaging and educational experience.

Whether you're tackling number challenges like Sudoku, finding anagrams, Python provides powerful tools that simplify the process and allow you to focus on thinking critically. Keep practicing with different types of puzzles, and soon you'll be solving them with confidence and creativity!

Application: A Number Guessing Game for Students

A **Number Guessing Game** is a simple and fun way to introduce students to the basics of Python programming. This game helps develop problem-solving skills while reinforcing core programming concepts such as loops, conditionals, and user input. The goal of the game is for the player to guess a randomly generated number within a certain range. Each time the player guesses, they receive feedback on whether their guess was too high, too low, or correct.

In this section, we'll walk through how to build a number guessing game step by step. This game is easy to understand, interactive, and provides immediate feedback, making it an enjoyable learning experience for students.

1. The Objective of the Game

The number guessing game works like this:

- The computer generates a random number within a predefined range (for example, 1 to 100).
- The player must guess the number, and after each guess, the computer provides feedback: "too high," "too low," or "correct."
- The game continues until the player guesses the correct number.
- Optionally, we can track the number of attempts it takes for the player to guess correctly.

2. Building the Game Step by Step

Let's begin by writing the code for the number guessing game. We will use Python's **random** module to generate the number, and the game will keep

asking the player for guesses until they find the correct answer.

Step 1: Import the Required Module

We'll use the **random** module to generate the random number for the game.

```
import random
```

Step 2: Generate a Random Number

Next, we generate a random number between 1 and 100. This number will be the one the player tries to guess.

```
# Generate a random number between 1 and 100
random_number = random.randint(1, 100)
```

Step 3: Get the Player's Guess

The player will need to enter a guess, so we'll prompt the user for input. We also need to make sure the input is a number.

```
# Get the player's guess
player_guess = int(input("Guess the number between 1 and 100: "))
```

Step 4: Provide Feedback on the Guess

Once the player enters a guess, we compare it to the random number and provide feedback: "too high," "too low," or "correct." The game will continue until the player guesses the correct number.

```
# Main game loop
while True:
    player_guess = int(input("Guess the number between 1 and 100: "))

    if player_guess < random_number:
        print("Too low! Try again.")
    elif player_guess > random_number:
        print("Too high! Try again.")
    else:
        print("Congratulations! You guessed the correct number.")
        break # End the game when the player guesses correctly
```

Step 5: Counting the Number of Guesses (Optional)

To make the game more engaging, we can track the number of attempts it takes for the player to guess the number correctly. This adds an element of

competition, as players can try to guess the number in as few attempts as possible.

```
# Initialize attempt counter
```

```
attempts = 0
```

```
# Main game loop
```

```
while True:
```

```
    player_guess = int(input("Guess the number between 1 and 100: "))
```

```
    attempts += 1 # Increment the attempt counter
```

```
    if player_guess < random_number:
```

```
        print("Too low! Try again.")
```

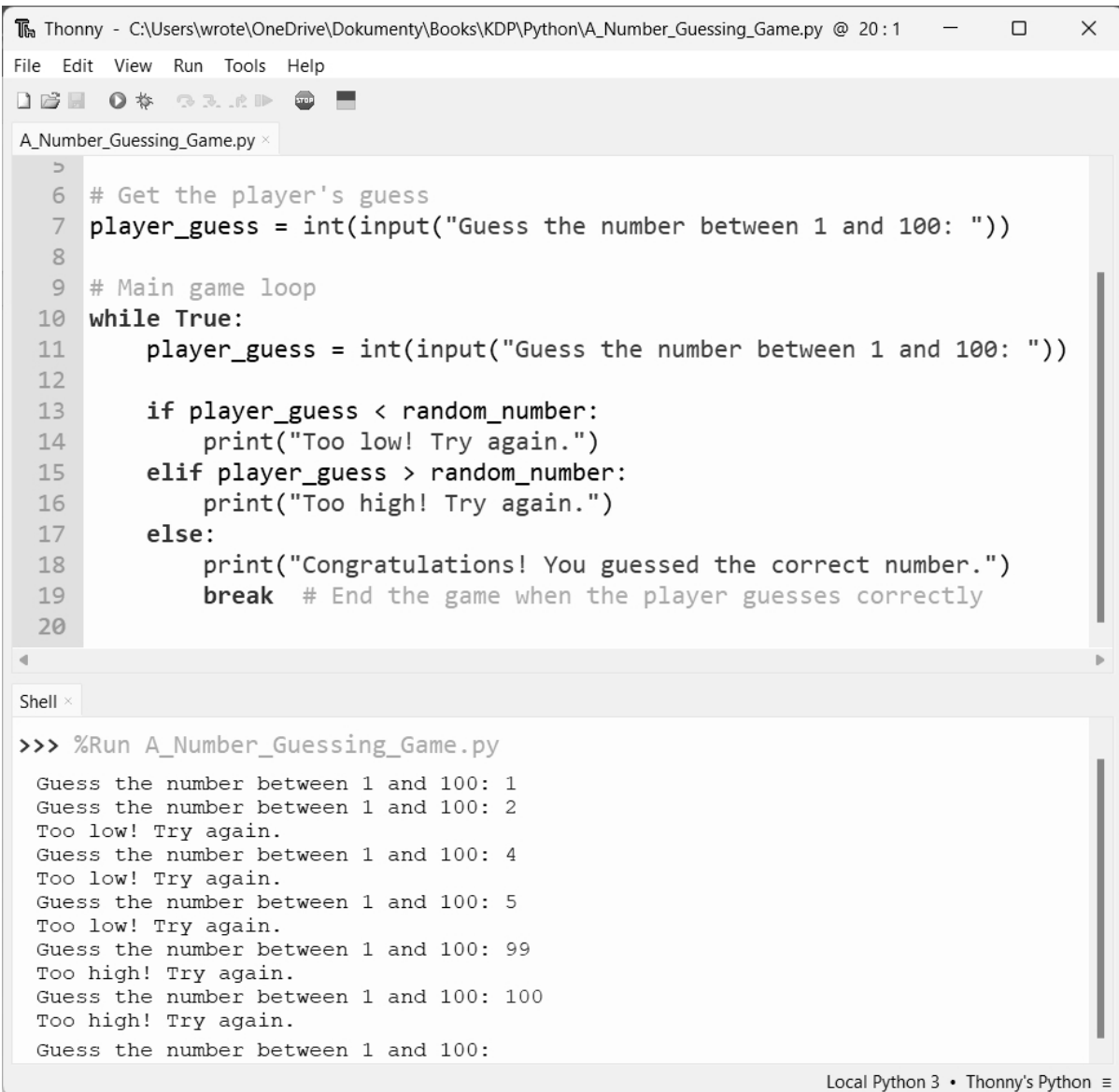
```
    elif player_guess > random_number:
```

```
        print("Too high! Try again.")
```

```
    else:
```

```
        print(f"Congratulations! You guessed the correct number in {attempts} attempts.")
```

```
        break
```



The screenshot shows the Thonny Python IDE interface. The top window displays the script `A_Number_Guessing_Game.py` with the following code:

```
5
6 # Get the player's guess
7 player_guess = int(input("Guess the number between 1 and 100: "))
8
9 # Main game loop
10 while True:
11     player_guess = int(input("Guess the number between 1 and 100: "))
12
13     if player_guess < random_number:
14         print("Too low! Try again.")
15     elif player_guess > random_number:
16         print("Too high! Try again.")
17     else:
18         print("Congratulations! You guessed the correct number.")
19         break # End the game when the player guesses correctly
20
```

The bottom window, titled "Shell", shows the output of running the script:

```
>>> %Run A_Number_Guessing_Game.py
Guess the number between 1 and 100: 1
Guess the number between 1 and 100: 2
Too low! Try again.
Guess the number between 1 and 100: 4
Too low! Try again.
Guess the number between 1 and 100: 5
Too low! Try again.
Guess the number between 1 and 100: 99
Too high! Try again.
Guess the number between 1 and 100: 100
Too high! Try again.
Guess the number between 1 and 100:
```

The status bar at the bottom indicates "Local Python 3 • Thonny's Python".

3. Adding Features to Enhance the Game

You can customize the number guessing game to make it more interesting or challenging. Here are a few ideas:

1. Limiting the Number of Guesses

To add more challenge, you can limit the number of guesses a player has. For example, the player might have only 7 attempts to guess the correct number. If they don't guess within that limit, the game ends.

```
# Limit the number of guesses
```

```
max_attempts = 7
```

```

attempts = 0

while attempts < max_attempts:
    player_guess = int(input("Guess the number between 1 and 100: "))
    attempts += 1

    if player_guess < random_number:
        print(f"Too low! You have {max_attempts - attempts} attempts left.")
    elif player_guess > random_number:
        print(f"Too high! You have {max_attempts - attempts} attempts left.")
    else:
        print(f"Congratulations! You guessed the correct number in {attempts} attempts.")
        break
else:
    print(f"Sorry, you've run out of attempts! The correct number was {random_number}.")

```

2. Adjusting the Difficulty Level

You can let the player choose a difficulty level, which adjusts the range of the random number. For example, an "easy" level might use a range of 1 to 50, while a "hard" level might use a range of 1 to 500.

```

# Let the player choose a difficulty level
difficulty = input("Choose difficulty (easy, medium, hard): ").lower()

if difficulty == "easy":
    random_number = random.randint(1, 50)
    max_attempts = 10
elif difficulty == "medium":
    random_number = random.randint(1, 100)
    max_attempts = 7
else:
    random_number = random.randint(1, 500)
    max_attempts = 5

```

3. Giving Hints

To make the game more interactive, you can give players hints after a certain number of guesses. For example, after three incorrect guesses, you

could give a hint like "The number is even" or "The number is divisible by 5."

```
if attempts == 3 and player_guess != random_number:
    if random_number % 2 == 0:
        print("Hint: The number is even.")
    else:
        print("Hint: The number is odd.")
```

4. Why a Number Guessing Game Is Valuable

This number guessing game teaches several important programming concepts in a way that's both fun and practical:

- **User Input:** The game prompts the player for input and checks whether it's valid.
- **Conditionals:** The game uses if, elif, and else statements to compare the player's guess to the random number.
- **Loops:** The game continues until the player guesses correctly, demonstrating how while loops work in Python.
- **Randomness:** The game uses Python's random module, introducing students to how random numbers can be generated in programming.

These concepts are essential for students learning Python and help them build a solid foundation for more complex programming tasks.

Summary

The **Number Guessing Game** is an excellent way for students to practice key programming skills while having fun. By engaging with concepts like loops, conditionals, and user input, students not only learn how to code but also how to think logically and solve problems efficiently.

This game is easy to build, easy to modify, and can be expanded with additional features like difficulty levels, limited guesses, or hints. It's a great starting point for students who are just beginning their Python journey, and it encourages creative thinking and problem-solving in a

playful, interactive way. Keep practicing, and soon you'll be able to create even more engaging and challenging games!

Chapter 12: Web Scraping for Problem Solving

Extracting Useful Data from Websites

The internet is full of valuable information, and with Python, you can extract data from websites to solve real-world problems. This process, known as **web scraping**, allows you to gather data directly from web pages, whether it's for research, pricing comparisons, or tracking trends. By writing a Python script, you can automate this process and extract large amounts of data quickly and efficiently.

In this section, you'll learn how to extract useful data from websites using Python in an accessible and straightforward way. We'll guide you through the process step by step, so you can understand the core concepts, see real examples, and start scraping data yourself.

1. What Is Web Scraping?

Web scraping is the process of using a program or script to extract specific data from web pages. It's like copying and pasting, but automated. You can use web scraping to:

- Collect data for a project or research.
- Track prices for products across different sites.
- Gather data like headlines, news, or reviews for analysis.

Web scraping involves identifying the parts of a website you want to extract, such as text, links, or images, and writing a Python script to do the work for you.

2. How Web Scraping Works

Web scraping uses Python's ability to interact with websites, download their HTML content, and extract specific information. The two main libraries

we'll use are:

- **requests**: To send an HTTP request to the website and get the page's HTML content.
- **BeautifulSoup**: To parse and extract data from the HTML.

Let's start with a simple example of scraping data from a website.

Example: Scraping Headings from a Website

Here's how you can scrape headings from a webpage using Python:

```
import requests
from bs4 import BeautifulSoup

# Send an HTTP request to the webpage
url = "https://example.com"
response = requests.get(url)

# Parse the HTML content using BeautifulSoup
soup = BeautifulSoup(response.content, "html.parser")

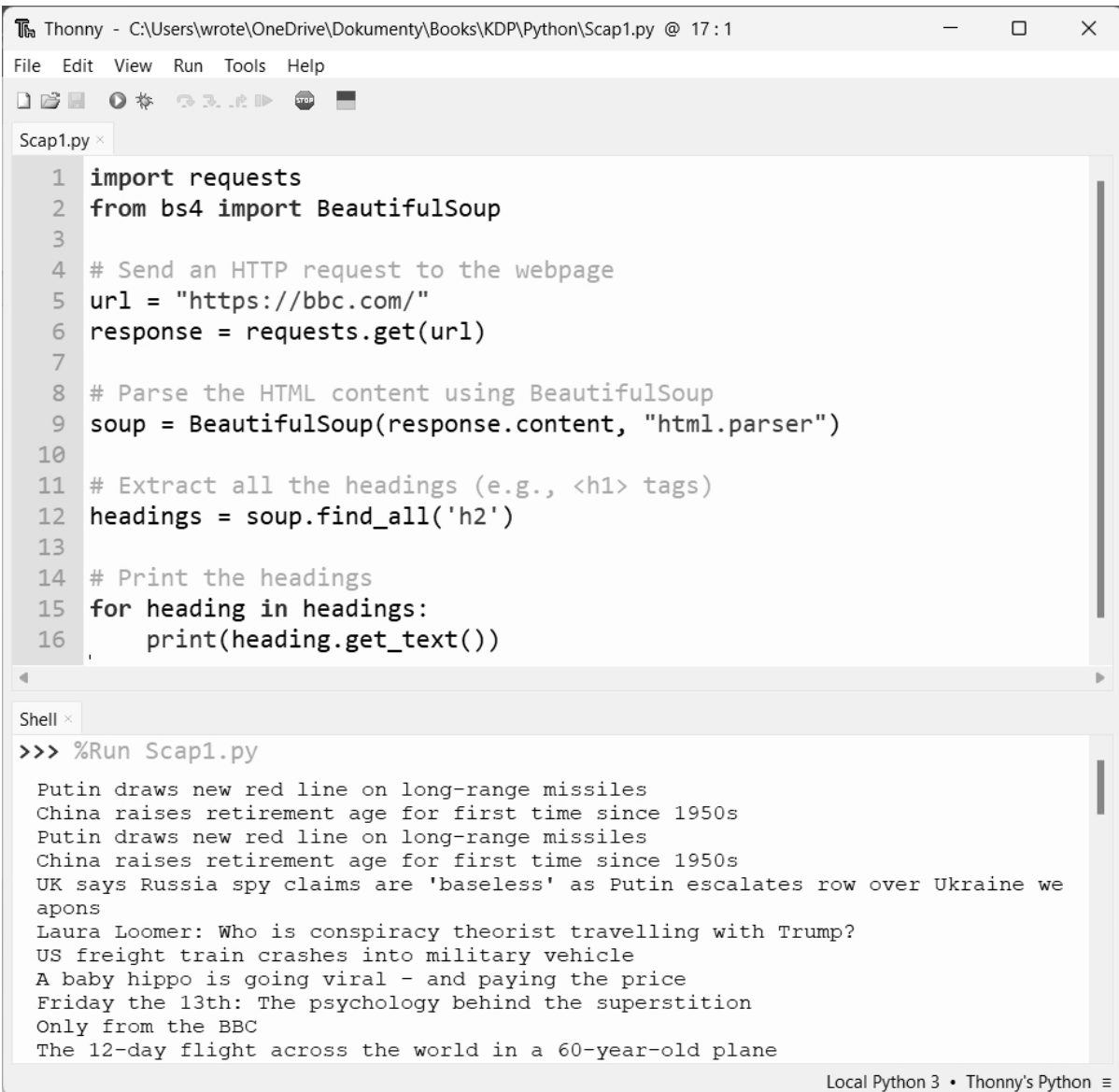
# Extract all the headings (e.g., <h1> tags)
headings = soup.find_all('h1')

# Print the headings
for heading in headings:
    print(heading.get_text())
```

Explanation:

- **requests.get()**: Sends a request to the webpage and gets its HTML content.
- **BeautifulSoup()**: Parses the HTML content and allows you to navigate through the page's structure.
- **soup.find_all('h1')**: Finds all the <h1> tags on the page (headings) and stores them in a list.
- **get_text()**: Extracts the text from each heading tag.

This basic example shows how you can extract headings from a webpage, but the same process can be applied to other types of data, such as paragraphs, links, or images.



The screenshot shows the Thonny IDE interface. The top window, titled 'Scap1.py', contains the following Python code:

```
1 import requests
2 from bs4 import BeautifulSoup
3
4 # Send an HTTP request to the webpage
5 url = "https://bbc.com/"
6 response = requests.get(url)
7
8 # Parse the HTML content using BeautifulSoup
9 soup = BeautifulSoup(response.content, "html.parser")
10
11 # Extract all the headings (e.g., <h1> tags)
12 headings = soup.find_all('h2')
13
14 # Print the headings
15 for heading in headings:
16     print(heading.get_text())
```

The bottom window, titled 'Shell', shows the output of running the script:

```
>>> %Run Scap1.py
Putin draws new red line on long-range missiles
China raises retirement age for first time since 1950s
Putin draws new red line on long-range missiles
China raises retirement age for first time since 1950s
UK says Russia spy claims are 'baseless' as Putin escalates row over Ukraine we
apons
Laura Loomer: Who is conspiracy theorist travelling with Trump?
US freight train crashes into military vehicle
A baby hippo is going viral - and paying the price
Friday the 13th: The psychology behind the superstition
Only from the BBC
The 12-day flight across the world in a 60-year-old plane
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

3. Breaking Down the Process of Web Scraping

Let's break the process of web scraping into clear, manageable steps so that you can apply these steps to any website you want to scrape.

Step 1: Inspect the Website

Before writing any code, you need to **inspect the website** to understand its structure. Every website is built using HTML, which organizes the content

using different tags like `<h1>`, `<p>`, `<a>`, and more.

To inspect a website:

1. Right-click anywhere on the webpage and select **Inspect** or **View Page Source**.
2. Look at the HTML structure to find the data you want to scrape (e.g., headings, paragraphs, product prices).
3. Identify the tags (such as `<h1>` or `<p>`) or CSS classes that contain the information you need.

Step 2: Send a Request to the Website

Use the **requests** library to send an HTTP request to the website and download its HTML content.

```
import requests

url = "https://example.com"
response = requests.get(url)

# Check if the request was successful (status code 200 means OK)
if response.status_code == 200:
    print("Website accessed successfully!")
else:
    print("Failed to access the website.")
```

This step downloads the webpage's HTML so that you can analyze it with Python.

Step 3: Parse the HTML

Once you've downloaded the HTML, use **BeautifulSoup** to parse and navigate through it.

```
from bs4 import BeautifulSoup

# Parse the HTML content
soup = BeautifulSoup(response.content, "html.parser")

# Print the HTML structure (optional, for inspection)
print(soup.prettify())
```

pretty() allows you to see the formatted HTML structure, making it easier to find the elements you want to scrape.

Step 4: Extract the Data

With the HTML parsed, you can extract the specific data you're interested in using **soup.find()** or **soup.find_all()**.

- **soup.find()**: Extracts the first occurrence of a tag (e.g., the first heading or the first paragraph).
- **soup.find_all()**: Extracts all occurrences of a tag (e.g., all the headings or all the links).

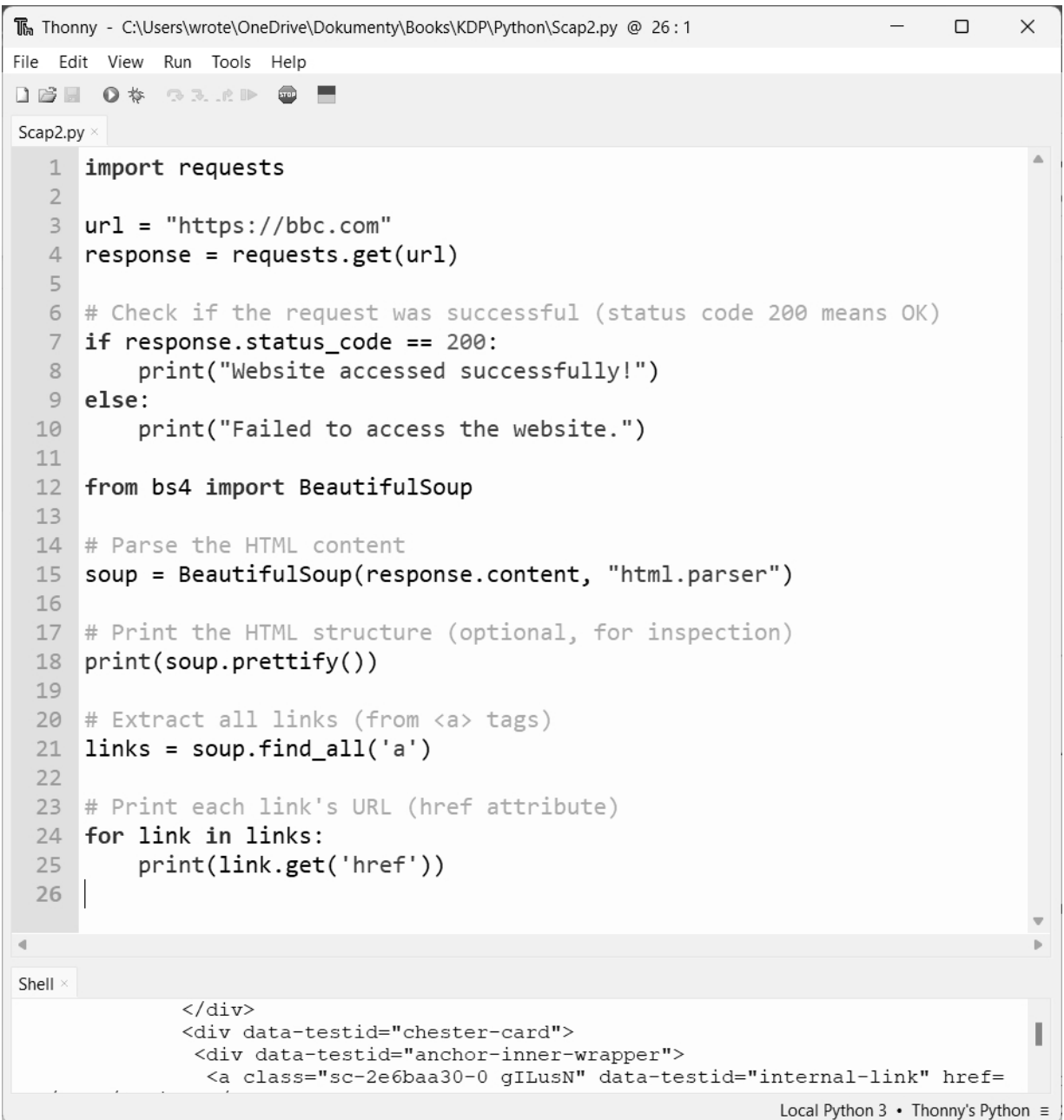
For example, to extract all links from the page:

```
# Extract all links (from <a> tags)
links = soup.find_all('a')

# Print each link's URL (href attribute)
for link in links:
    print(link.get('href'))
```

This would print out all the URLs (links) on the webpage.

And all parts:

The image shows a screenshot of the Thonny Python IDE. The main window displays a Python script named 'Scap2.py' with the following code:

```
1 import requests
2
3 url = "https://bbc.com"
4 response = requests.get(url)
5
6 # Check if the request was successful (status code 200 means OK)
7 if response.status_code == 200:
8     print("Website accessed successfully!")
9 else:
10    print("Failed to access the website.")
11
12 from bs4 import BeautifulSoup
13
14 # Parse the HTML content
15 soup = BeautifulSoup(response.content, "html.parser")
16
17 # Print the HTML structure (optional, for inspection)
18 print(soup.prettify())
19
20 # Extract all links (from <a> tags)
21 links = soup.find_all('a')
22
23 # Print each link's URL (href attribute)
24 for link in links:
25     print(link.get('href'))
26
```

The bottom panel, labeled 'Shell', shows the output of the script, which is a snippet of HTML code:

```
</div>
<div data-testid="chester-card">
  <div data-testid="anchor-inner-wrapper">
    <a class="sc-2e6baa30-0 gILusN" data-testid="internal-link" href=
```

The status bar at the bottom right indicates 'Local Python 3 • Thonny's Python'.

4. Using Web Scraping to Solve Real-World Problems

Web scraping can be used to gather data for a wide range of real-world applications. Here are a few examples:

1. Tracking Product Prices

You can scrape e-commerce websites to track prices of products over time, helping you find the best deals or monitor changes in pricing.

Example: Scraping product prices from an e-commerce site


```
product_prices = soup.find_all(class_="price")
```

```
for price in product_prices:
```

```
    print(price.get_text())
```

In this example, the class price is used to identify the product prices on the page.

2. Gathering News Headlines

You can scrape news websites to gather the latest headlines and compile them into a report or analyze trends in the news.

```
# Example: Scraping news headlines from a news site
```

```
headlines = soup.find_all('h2', class_='news-headline')
```

```
for headline in headlines:
```

```
    print(headline.get_text())
```

3. Extracting Data for Research

Researchers often scrape websites to gather large datasets, such as reviews, comments, or public records, for analysis. For example, you could scrape user reviews from a website and analyze them for sentiment (positive or negative).

5. Challenges of Web Scraping

While web scraping is a powerful tool, it comes with some challenges:

- **Dynamic Websites:** Some websites use JavaScript to load content dynamically. For these websites, you may need to use additional tools like **Selenium** or **Scrapy** to interact with the JavaScript elements.
- **Rate Limits and CAPTCHAs:** Some websites limit the number of requests you can make in a short period or use CAPTCHAs to block bots. You should always respect a website's terms of service and avoid scraping too aggressively.

6. Best Practices for Web Scraping

When scraping data from websites, it's important to follow ethical guidelines and best practices:

- **Check the website's terms of service:** Some websites do not allow scraping, while others may provide an API for data access.
- **Avoid overloading the server:** Be polite by limiting the number of requests you send in a short period.
- **Use web scraping responsibly:** Ensure you're not scraping sensitive or personal data without permission.

Here's how you can add a polite delay between requests to avoid overwhelming a server:

```
import time  
  
# Wait for 2 seconds between requests  
time.sleep(2)
```

Summary

Web scraping is a powerful tool that allows you to extract useful data from websites, turning unstructured web content into valuable information. Whether you're tracking prices, gathering news, or collecting data for research, Python's **requests** and **BeautifulSoup** libraries make it easy to automate this process and extract the information you need.

With the ability to inspect, navigate, and extract specific data from any webpage, you can solve real-world problems more efficiently and open up new possibilities for data collection and analysis. As you practice web scraping, remember to do so responsibly, following ethical guidelines and respecting the websites you scrape. Keep experimenting, and soon you'll be able to scrape data like a pro!

Application: Scraping Book Prices for a Library or Classroom

Scraping book prices is a practical way to help libraries, classrooms, or even students find the best deals when buying books. With Python, you can automate the process of gathering prices from multiple websites, saving time and making it easier to compare costs. Whether you're managing a library budget or looking for affordable textbooks, web scraping allows you to collect up-to-date pricing information efficiently.

In this section, you'll learn how to scrape book prices from different e-commerce websites using Python. We'll walk through a real-world example, explaining how to extract the relevant information and create a useful tool for price comparison.

1. Why Scrape Book Prices?

When managing a library or classroom, budgets are often tight, and finding the best deals on books can be challenging. Instead of manually checking multiple websites, web scraping allows you to:

- **Compare prices** from various online bookstores or retailers.
- **Track price changes** over time, helping you decide when to buy.
- **Build a book price database** that makes purchasing decisions more informed and efficient.

By automating this process, you can focus on more important tasks while Python does the heavy lifting.

2. How to Scrape Book Prices Step by Step

Let's build a Python script that scrapes book prices from a website. For this example, we'll assume we're scraping a popular bookstore, but you can adapt the script to other websites.

Step 1: Install Required Libraries

You'll need the **requests** and **BeautifulSoup** libraries for this task. Install them using the following commands:

```
pip install requests beautifulsoup4
```

Step 2: Identify the Target Website

The first step in web scraping is to inspect the website you want to scrape. In this case, visit the bookstore's page where book prices are listed. Look at the HTML structure to identify the tags that contain the book titles and prices. You can do this by right-clicking on the webpage and selecting **Inspect** or **View Page Source**.

For example, you might see something like this:

```
<div class="book">
  <h2 class="title">Python Programming for Beginners</h2>
  <p class="price">$29.99</p>
</div>
```

In this case, the book title is inside an `<h2>` tag with the class "title", and the price is inside a `<p>` tag with the class "price".

Step 3: Send a Request to the Website

Use the **requests** library to send a request to the website and get the HTML content of the page.

```
import requests

# Send an HTTP request to the bookstore website
url = "https://www.examplebookstore.com/books"
response = requests.get(url)

# Check if the request was successful (status code 200)
if response.status_code == 200:
    print("Successfully accessed the website.")
else:
    print("Failed to access the website.")
```

Step 4: Parse the HTML Content

Once the webpage's HTML has been retrieved, use **BeautifulSoup** to parse the content and locate the book titles and prices.

```
from bs4 import BeautifulSoup

# Parse the HTML content using BeautifulSoup
soup = BeautifulSoup(response.content, "html.parser")
```

```
# Extract book titles and prices
book_titles = soup.find_all('h2', class_='title')
book_prices = soup.find_all('p', class_='price')
```

Here, **find_all()** extracts all the elements on the page with the specified class. We're grabbing both the book titles and their corresponding prices.

Step 5: Extract and Print the Data

Now that you've located the titles and prices, you can loop through them and display the results.

```
# Print the book titles and prices
for title, price in zip(book_titles, book_prices):
    print(f'Book: {title.get_text().strip()}, Price: {price.get_text().strip()}')
```

In this code:

- **get_text()** extracts the text inside the HTML tags.
- **strip()** removes any extra whitespace or newlines.

When you run the script, you'll see output like this:

```
Book: Python Programming for Beginners, Price: $29.99
Book: Advanced Python Techniques, Price: $45.50
Book: Learn Python in 30 Days, Price: $19.99
```

Step 6: Save the Data to a CSV File

To make the scraped data more useful, you can save it to a CSV file for later reference or analysis. This allows you to build a book price database or share the information with colleagues.

```
import csv

# Open a CSV file to save the data
with open('book_prices.csv', mode='w', newline='', encoding='utf-8') as file:
    writer = csv.writer(file)
```

```
# Write the header row
writer.writerow(['Book Title', 'Price'])

# Write the book titles and prices
for title, price in zip(book_titles, book_prices):
    writer.writerow([title.get_text().strip(), price.get_text().strip()])

print("Book prices have been saved to book_prices.csv")
```

This script writes the book titles and prices to a file called `book_prices.csv`, which you can open in Excel or any spreadsheet software to view and sort the data.

3. Practical Use Cases of Scraping Book Prices

Scraping book prices is incredibly useful for:

- **Libraries:** Track book prices from different vendors to find the best deals when expanding your collection.
- **Classrooms:** Help teachers and students find affordable textbook options by comparing prices across multiple sites.
- **Bookshops:** Monitor competitors' prices to stay competitive in the market.

For example, if you are managing a school library's budget, you can set up a script that automatically scrapes book prices once a week and generates a report, allowing you to make informed purchasing decisions.

4. Customizing Your Price Scraping Tool

There are several ways to enhance your book price scraper:

- **Search by ISBN:** Instead of scraping an entire webpage, you could input a book's ISBN (International Standard Book Number) and scrape the price for that specific book across multiple sites.

```
isbn = "9780135166307"
url = f"https://www.examplebookstore.com/search?q={isbn}"
```

- **Scraping Multiple Websites:** To compare prices across different retailers, you can extend the script to scrape multiple websites and save the results in a single CSV file for comparison.
- **Tracking Price Changes:** You can build a system that scrapes book prices daily or weekly and tracks changes over time. This way, you can identify the best time to purchase a book when the price drops.

5. Handling Dynamic Websites

Some websites load content dynamically using JavaScript, making it a bit more challenging to scrape data. In such cases, you can use tools like **Selenium**, which allows you to control a web browser and extract content from dynamically loaded pages.

For simpler websites that load all content statically, the **requests** and **BeautifulSoup** libraries are sufficient.

6. Conclusion

Web scraping offers a powerful way to automate the process of comparing book prices across different websites, making it an invaluable tool for libraries, classrooms, and anyone involved in managing book purchases. By using Python, you can build a script that not only collects data efficiently but also saves you time and effort.

With a few lines of code, you can extract, display, and store book prices, enabling you to make smarter purchasing decisions and better manage your resources. Whether you're a student, teacher, or librarian, web scraping can help you stay informed and make the best use of your budget.

Ethical Considerations of Web Scraping

Web scraping is a powerful tool for gathering data from websites, but it comes with important **ethical considerations**. While scraping allows us to automate data collection and solve real-world problems, it's essential to respect the rights and terms set by website owners and ensure that our scraping practices are responsible. In this section, we'll explore how to approach web scraping ethically, discussing legal aspects, best practices, and how to balance the benefits of scraping with respect for others' digital property.

1. Understanding the Legal Landscape

Before diving into web scraping, it's crucial to understand the legal framework around it. While scraping itself is not illegal, certain actions or uses of scraped data can be against the law. Here are a few key points to consider:

- **Terms of Service (ToS):** Every website has terms of service, which often include rules about how their data can be used. Some websites explicitly prohibit scraping, while others provide APIs (Application Programming Interfaces) to access data legally. Always check a website's ToS before scraping.
- **Copyright and Data Ownership:** Website content, including text, images, and videos, may be protected by copyright law. Scraping content for personal or commercial gain without permission could violate intellectual property rights.
- **Computer Fraud and Abuse Act (CFAA):** In some countries, including the United States, laws like the CFAA prohibit unauthorized access to websites, even if the data is publicly visible. Scraping in a way that bypasses protections (such as logging in or solving CAPTCHA) could be considered illegal.

Being aware of these legal boundaries ensures that your scraping activities are not only effective but also lawful and respectful.

2. Respecting Website Policies

One of the most important ethical principles in web scraping is **respecting the website** you are scraping from. This means:

- **Checking for an API:** Some websites offer APIs that provide structured access to their data. Using an API is usually the preferred method because it's designed for third-party access and often provides a stable, legal, and ethical way to retrieve data.
- **Reading the robots.txt File:** The **robots.txt** file is a standard used by websites to guide search engine crawlers (and scrapers) on which parts of the site can and cannot be accessed. This file tells you which pages are off-limits to automated requests. Before scraping, check the website's **robots.txt** file to ensure you're scraping allowed sections.

You can access the robots.txt file by visiting the website's URL and adding /robots.txt at the end. For example:

`https://example.com/robots.txt`

The file might look like this:

User-agent: *

Disallow: /private-section/

In this case, the website owner has disallowed scraping or crawling of the "/private-section/" part of the site.

3. Avoiding Harm to Website Performance

Web scraping, when done irresponsibly, can harm the website you're targeting by overwhelming its servers with too many requests. Websites are built to handle regular human traffic, not thousands of automated requests at once. Here's how you can avoid causing harm:

- **Limit the Frequency of Requests:** Instead of sending many requests in a short period, you can add a delay between requests to avoid putting unnecessary strain on the website's servers. A delay of a few seconds between requests is a good practice.

```
import time
```

```
# Wait 2 seconds between requests to avoid overwhelming the server
```

```
time.sleep(2)
```

- **Use Rate-Limiting:** Some websites have rate limits, meaning they restrict how many requests you can make in a certain time frame. Respect these limits by building your script to work within those boundaries.
- **Respect Page Load Times:** Avoid sending requests faster than the website can handle. Websites with slow load times might get overloaded if your scraper sends too many requests too quickly.

4. Handling Sensitive or Personal Data

Some websites may contain **sensitive** or **personal data**, and scraping this type of information raises significant ethical concerns. Even if the data is publicly available, it's important to handle it responsibly:

- **Avoid Scraping Personal Information:** Data such as emails, phone numbers, addresses, or social security numbers should not be scraped without permission. Collecting personal information can violate privacy laws such as the **General Data Protection Regulation (GDPR)** in Europe or the **California Consumer Privacy Act (CCPA)** in the United States.

- **Anonymize Data:** If you are collecting public information for analysis, make sure that the data cannot be traced back to an individual, especially if you are sharing or publishing your findings.

5. Avoiding Use of Scraped Data for Harmful Purposes

The data you scrape should always be used ethically. This means avoiding activities like:

- **Reposting Content Without Permission:** Do not scrape content from a website and post it elsewhere as your own without proper credit or permission.
- **Building Unfair Advantage:** Avoid using scraped data to build unfair advantages, such as scraping competitor websites for pricing strategies or product data and using that information inappropriately.
- **Spamming:** If you're scraping contact information (such as emails), do not use that data for spam or unsolicited emails. Doing so is not only unethical but also illegal under many anti-spam laws.

6. The Importance of Transparency

If your project involves web scraping, especially in a professional or academic setting, it's important to be **transparent** about your methods. This can involve:

- **Citing Sources:** When using scraped data for research or reports, make sure to cite the websites from which you collected the data.
- **Explaining Methods:** Be clear about how the data was collected, how frequently requests were made, and what parts

of the website were accessed. This helps ensure your work is seen as legitimate and ethical.

7. When to Ask for Permission

If you're unsure whether scraping a website is allowed or ethical, it's always a good idea to **ask for permission**. Many website owners are happy to share their data, but they might prefer you use an API or follow specific rules to avoid overloading their servers.

You can reach out to the website's contact address or support team and explain your project, asking if scraping is acceptable or if they provide an API for data access.

8. Alternatives to Web Scraping

In some cases, scraping may not be the best or most ethical option. Consider these alternatives:

- **Use Public APIs:** Many websites offer public APIs, which allow you to access data legally and efficiently without scraping. APIs are usually well-documented, providing a structured way to collect the data you need.
- **Look for Open Data:** Governments, organizations, and businesses often publish **open datasets** that are free to use. These datasets are a great resource for students and researchers without needing to scrape websites.

Summary

Web scraping is a valuable tool for gathering information, but it comes with important ethical responsibilities. By respecting websites' terms of service, avoiding personal or sensitive data, and limiting the load on servers, you can use web scraping in a way that's both effective and respectful.

Remember, transparency is key—whether you're collecting data for research or building a project, always be clear about how you obtained the information and how you plan to use it. Following these ethical guidelines

ensures that web scraping remains a powerful, responsible tool for solving real-world problems.

Chapter 13: Introduction to Machine Learning (for Beginners)

Understanding Basic Concepts of Machine Learning

Machine learning is one of the most exciting and rapidly growing fields in technology today. It allows computers to learn from data and make predictions or decisions without being explicitly programmed for every task. Although machine learning can seem complex at first, the basic concepts are simple and easy to understand once you break them down. In this section, we'll introduce you to the foundational ideas behind machine learning in a way that's approachable and engaging, especially for beginners.

By the end of this chapter, you'll have a clear understanding of what machine learning is, how it works, and why it's so powerful. You'll also see how Python, with its rich libraries, makes building machine learning models more accessible than ever.

1. What Is Machine Learning?

At its core, **machine learning** (ML) is a way of teaching computers to learn patterns from data. Instead of writing a set of rules for a computer to follow, you give it data and allow it to figure out the rules by itself. For example, instead of telling a computer exactly how to recognize whether an email is spam, you can feed it examples of both spam and non-spam emails, and the computer will learn to classify emails on its own.

There are three main types of machine learning:

- **Supervised Learning:** The machine learns from **labeled data**—data that includes both the input (what you know) and the

correct output (the answer you want to predict). For example, you might provide data on students' study hours and their grades to predict future grades based on how much time a student studies.

- **Unsupervised Learning:** The machine learns from **unlabeled data**—data without clear answers. The goal here is for the computer to find patterns or groupings on its own. For example, it might group customers based on their shopping habits without being told what the groups should be.
- **Reinforcement Learning:** The machine learns by trial and error, receiving rewards or punishments based on the actions it takes. This method is often used in game playing or robotics, where the computer learns by interacting with its environment.

2. Key Concepts in Machine Learning

Now that we understand what machine learning is, let's break down a few key concepts that are central to how machine learning works.

a) Data

Machine learning relies on **data** to make predictions. Data can come in many forms—numbers, text, images, and more. For example, in a student grade prediction model, data might include hours studied, attendance, and previous grades.

The more data you have, the better the machine can learn. However, the data needs to be **clean** and **relevant** to the problem you're trying to solve. Poor-quality data can lead to poor predictions.

b) Features

Features are the individual variables or pieces of information that you use to make predictions. For example, in predicting student grades, the features could be things like:

- Hours spent studying.

- Number of classes attended.
- Homework scores.

Each feature is a part of the overall dataset, and together, the features help the machine understand what factors affect the outcome.

c) Labels

In **supervised learning**, the correct answers or outcomes are called **labels**. In the student grades example, the label would be the actual grade the student received. The machine learns by comparing its predictions to the labels and improving its guesses over time.

d) Training and Testing

To create a machine learning model, we typically split our data into two sets:

- **Training data:** This is the data the machine uses to learn. It includes both the features and the correct labels, allowing the machine to figure out how to make predictions.
- **Testing data:** This is a separate set of data that the machine hasn't seen before. It helps us see how well the model works on new, unseen data.

By using testing data, we can ensure that the machine isn't just memorizing the answers but actually learning how to generalize its predictions to new situations.

e) Model

A **model** is what the machine builds after learning from the data. It's a mathematical representation of the relationships between the features and the labels. Once trained, the model can make predictions based on new data. For example, a trained model might predict that a student who studies for 10 hours will score 85%.

3. How Machines Learn: The Process

Now let's look at the steps a machine follows to learn from data and make predictions.

Step 1: Collect Data

The first step in machine learning is to collect data. For example, if you want to predict student grades, you might collect data on how many hours students study, their class attendance, and their previous exam scores.

Step 2: Preprocess the Data

Once you have the data, you need to prepare it for the machine learning algorithm. This might involve:

- Cleaning the data (removing errors or missing values).
- Standardizing or normalizing the data (making sure all the numbers are on the same scale).
- Splitting the data into training and testing sets.

Step 3: Choose a Model

There are many different **machine learning algorithms** you can use, and choosing the right one depends on the type of problem you're trying to solve. Here are some common algorithms:

- **Linear Regression:** Used for predicting continuous values (e.g., predicting a student's exact grade).
- **Classification:** Used for predicting categories (e.g., classifying emails as spam or not spam).
- **Clustering:** Used for grouping data into similar categories (e.g., grouping students based on study habits).

Step 4: Train the Model

The machine uses the **training data** to learn the patterns between the features and the labels. During this process, the algorithm adjusts itself to improve its predictions based on the training data.

Step 5: Test the Model

Once the model is trained, it's time to test how well it performs using the **testing data**. This helps you see if the model can make accurate predictions on new data it hasn't seen before.

Step 6: Improve the Model

After testing, you can make adjustments to improve the model. This might involve adding more data, choosing a different algorithm, or tweaking the parameters of the algorithm.

4. Example: Predicting Student Grades

Let's take the example of predicting student grades based on hours studied. Here's how we might approach this problem using machine learning:

1. **Collect Data:** Gather data from students, including how many hours they studied and their final grades.
2. **Preprocess the Data:** Clean the data by removing any missing values or errors, and split it into training and testing sets.
3. **Choose a Model:** Use a **linear regression** model, which is good for predicting continuous values like grades.
4. **Train the Model:** The model learns from the training data by figuring out the relationship between study hours and grades.
5. **Test the Model:** Use the testing data to see how accurately the model predicts grades based on hours studied.
6. **Make Predictions:** Once trained, the model can predict future student grades based on how much time they spend studying.

Here's a basic Python example using **Scikit-Learn**, a popular machine learning library:

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
import numpy as np

# Example data (hours studied vs. grades)
```

```
hours_studied = np.array([2, 4, 6, 8, 10, 12]).reshape(-1, 1)
grades = np.array([50, 60, 65, 70, 85, 90])

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(hours_studied, grades, test_size=0.2,
random_state=42)

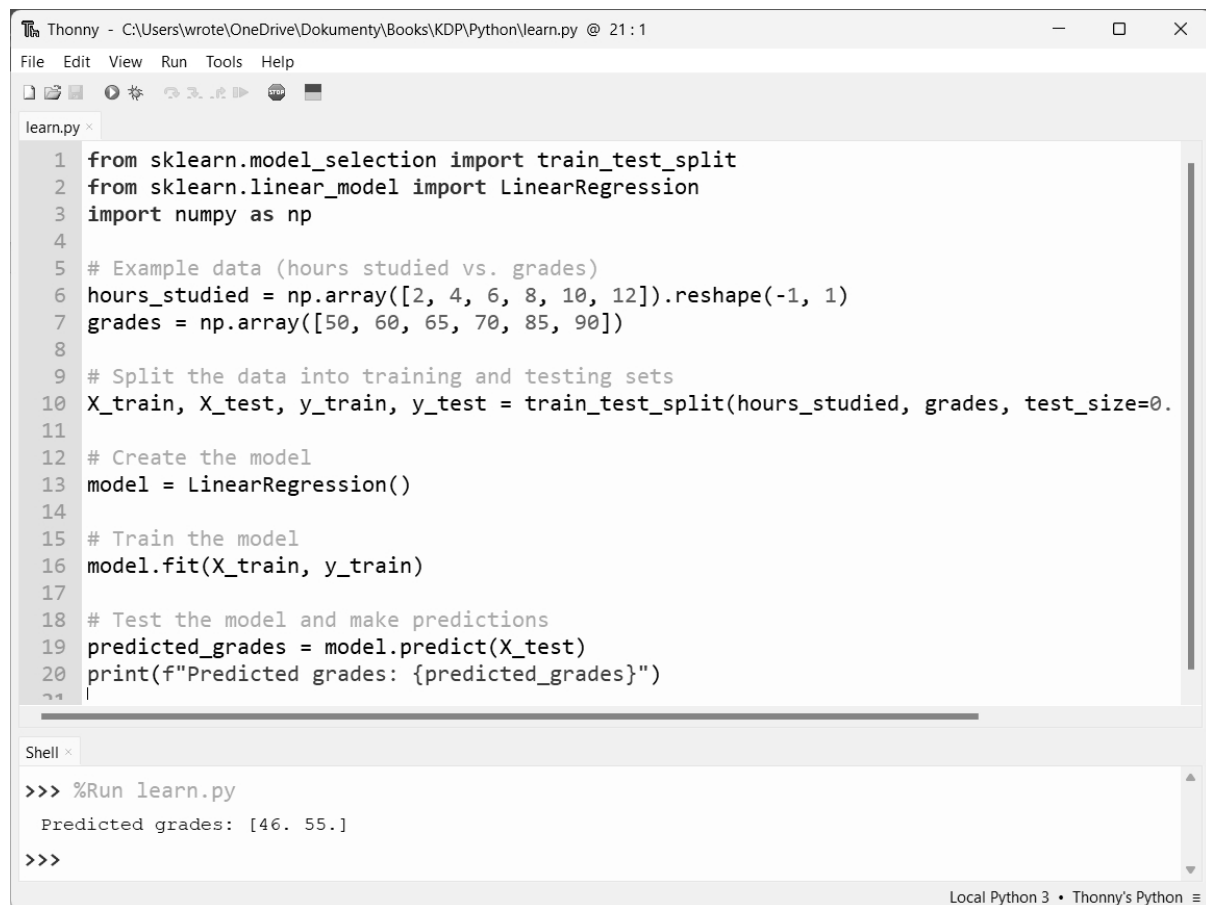
# Create the model
model = LinearRegression()

# Train the model
model.fit(X_train, y_train)

# Test the model and make predictions
predicted_grades = model.predict(X_test)
print(f"Predicted grades: {predicted_grades}")
```

In this example:

- We use **LinearRegression** to build a model that predicts grades based on hours studied.
- We split the data into training and testing sets to ensure the model can generalize well.
- After training, we use the model to predict student grades on new, unseen data.



The screenshot shows the Thonny Python IDE interface. The top window, titled 'learn.py', contains the following Python code:

```
1 from sklearn.model_selection import train_test_split
2 from sklearn.linear_model import LinearRegression
3 import numpy as np
4
5 # Example data (hours studied vs. grades)
6 hours_studied = np.array([2, 4, 6, 8, 10, 12]).reshape(-1, 1)
7 grades = np.array([50, 60, 65, 70, 85, 90])
8
9 # Split the data into training and testing sets
10 X_train, X_test, y_train, y_test = train_test_split(hours_studied, grades, test_size=0.
11
12 # Create the model
13 model = LinearRegression()
14
15 # Train the model
16 model.fit(X_train, y_train)
17
18 # Test the model and make predictions
19 predicted_grades = model.predict(X_test)
20 print(f"Predicted grades: {predicted_grades}")
```

The bottom window, titled 'Shell', shows the execution output:

```
>>> %Run learn.py
Predicted grades: [46. 55.]
>>>
```

The status bar at the bottom right indicates 'Local Python 3 • Thonny's Python'.

5. Why Machine Learning Is Powerful

Machine learning is powerful because it allows computers to:

- **Handle large amounts of data:** Machines can analyze and learn from huge datasets that would be impossible for humans to process manually.
- **Make predictions:** Once trained, a machine learning model can make predictions on new data quickly and accurately.
- **Adapt and improve:** With more data and better models, machines can continue to improve their performance over time.

Summary

Machine learning allows computers to learn from data, making it possible to solve complex problems in ways that traditional programming cannot. By understanding the basic concepts—such as data, features, models, and training—you're well on your way to building powerful, predictive models using Python. Whether predicting student grades or uncovering patterns in data, machine learning opens up endless possibilities for discovering new insights and making smarter decisions.

Using Python to Build Simple Predictive Models

Building predictive models with Python is a powerful way to solve real-world problems using data. Whether you're predicting student grades, sales trends, or even weather patterns, Python's machine learning libraries make it easy to get started, even if you're a beginner. In this section, we'll show you how to use Python to build simple predictive models, step by step, using approachable language and examples.

By the end of this chapter, you'll be able to create your own basic predictive models, understand how they work, and use them to make predictions based on data.

1. What Is a Predictive Model?

A **predictive model** is a tool that uses data to make predictions about future outcomes. For example, if you want to predict how well a student will perform on their next test based on their study habits, you can create a model that learns from past data (like study hours and test scores) to make a prediction for the future.

Predictive models are used in a wide range of areas, from forecasting stock prices to recommending movies, but at their core, they all use the same principles: learning from past data to predict future outcomes.

2. Building a Predictive Model with Python

Python is an excellent language for building predictive models because of its simplicity and the availability of powerful machine learning libraries, such as **Scikit-learn**, which provides all the tools you need to get started.

Let's walk through the process of building a simple predictive model in Python using **Scikit-learn**. We'll focus on a classic problem: predicting student grades based on how much time they spend studying.

Step 1: Install Required Libraries

To get started, you'll need to install the **Scikit-learn** library, which provides a wide range of machine learning algorithms. You can install it by running

the following command in your terminal or command prompt:

```
pip install scikit-learn
```

You'll also need **NumPy**, a library for handling numerical data, and **Pandas**, a library for working with data structures like tables:

```
pip install numpy pandas
```

Step 2: Import Libraries and Prepare the Data

Before building the model, you need data. In this case, we'll use a simple dataset that shows how many hours students studied and the grades they received on a test.

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

# Create a simple dataset with hours studied and test scores
data = {'Hours_Studied': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
        'Test_Score': [50, 55, 60, 65, 70, 75, 80, 85, 90, 95]}

# Convert the data to a Pandas DataFrame
df = pd.DataFrame(data)
```

Here, we create a small dataset that includes the number of hours studied by students and their corresponding test scores. This is the data that the model will use to learn the relationship between study time and grades.

Step 3: Split the Data into Training and Testing Sets

Once you have the data, it's important to split it into **training data** and **testing data**. The training data is used to teach the model, and the testing data is used to see how well the model can make predictions on new data.

```
# Separate the features (hours studied) from the target (test score)
X = df[['Hours_Studied']] # Features (input)
y = df['Test_Score']      # Target (output)

# Split the data into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

In this code:

- **X** represents the features (in this case, the number of hours studied).
- **y** represents the target (the test score).
- **train_test_split()** splits the data into training (80%) and testing sets (20%).

Step 4: Choose and Train a Model

Now it's time to choose a model. For this example, we'll use **linear regression**, which is a simple algorithm that tries to find a straight line that best fits the data. Linear regression is great for predicting continuous values like grades.

```
# Create a linear regression model
```

```
model = LinearRegression()
```

```
# Train the model using the training data
```

```
model.fit(X_train, y_train)
```

The **fit()** method trains the model by finding the best line that predicts test scores based on the hours studied.

Step 5: Make Predictions

Once the model is trained, we can use it to make predictions on the **testing data**—the data that the model hasn't seen yet. This will show how well the model performs.

```
# Make predictions on the testing set
```

```
y_pred = model.predict(X_test)
```

```
# Print the predictions and the actual test scores
```

```
print("Predicted test scores:", y_pred)
```

```
print("Actual test scores:", y_test.values)
```

This will output the predicted test scores based on the number of hours studied, allowing us to compare the model's predictions to the actual scores.

Step 6: Evaluate the Model

To know how good your model is, you need to evaluate it. One simple way to do this is by calculating the **mean absolute error** (MAE), which

measures how far the model's predictions are from the actual results.

```
from sklearn.metrics import mean_absolute_error
```

```
# Calculate the mean absolute error
```

```
mae = mean_absolute_error(y_test, y_pred)
```

```
print(f"Mean Absolute Error: {mae}")
```

The **mean absolute error** gives you a sense of how accurate your model is—the lower the number, the better the model's predictions.

3. Key Concepts in Building Predictive Models

Let's review some of the key concepts involved in building predictive models with Python.

a) Training the Model

When we train a model, we provide it with data so it can learn patterns. In our case, the model learns how the number of hours studied affects the test scores. The model then uses this knowledge to make predictions.

b) Testing the Model

We test the model with data it hasn't seen before to make sure it's not just memorizing the training data but actually learning how to make predictions.

c) Linear Regression

In this example, we used **linear regression**, which works well when we're trying to predict a continuous value. The model tries to find the best-fitting line through the data, making it easy to predict new values based on this relationship.

4. Why Predictive Models Are Useful

Predictive models are incredibly useful in many fields:

- **Education:** Predicting student performance based on study habits, attendance, or homework completion.
- **Business:** Forecasting sales or demand based on past trends.
- **Healthcare:** Predicting patient outcomes based on medical history and treatments.

The beauty of predictive models is that they help you make data-driven decisions. Instead of guessing or relying on intuition, you can use models to predict outcomes with confidence.

Summary

Building simple predictive models with Python is a powerful way to analyze data and make informed predictions. By using libraries like **Scikit-learn**, you can easily create models that learn from data and make accurate predictions on new data. Whether you're predicting student grades, sales trends, or any other outcome, predictive modeling gives you the tools to solve real-world problems.

With just a few steps—gathering data, splitting it into training and testing sets, choosing a model, and making predictions—you can start using Python to build your own models and unlock the potential of machine learning. Keep practicing, and soon you'll be creating even more advanced models to tackle bigger challenges!

Example: Predicting Student Grades

Based on Past Data

Predicting student grades using past data is a great way to apply machine learning in a practical, easy-to-understand way. In this example, we'll show you how to use Python to create a simple predictive model that can estimate a student's future performance based on their past study habits. This example will introduce you to how machine learning works while keeping the concepts approachable and fun.

By the end of this section, you'll see how machine learning can help us make predictions based on data, and you'll be able to build your own models using Python.

1. The Problem: Predicting Grades

Imagine you're a teacher or student trying to figure out how well a student might perform on an upcoming test. You have data on how many hours students studied in the past and the grades they received. Now, you want to build a model that can predict a student's grade based on how much time they study.

This is a classic **regression problem**, where we use one variable (hours studied) to predict another variable (test score). The more a student studies, the better their grade might be—but how strong is that relationship, and can we use it to predict future grades? That's what we'll explore in this example.

2. Steps to Build the Predictive Model

We'll break down the process of predicting student grades into the following steps:

1. **Collect and prepare the data.**
2. **Split the data into training and testing sets.**
3. **Build the model using linear regression.**
4. **Test the model and evaluate its performance.**

5. Make predictions on new data.

Step 1: Collect and Prepare the Data

First, we need data to train our model. Let's assume we have the following data showing how many hours students studied and the grades they received:

Hours Studied	Test Score
1	50
2	55
3	60
4	65
5	70
6	75
7	80
8	85
9	90
10	95

This data suggests that as students study more, their grades improve. We can now use Python to turn this data into a format that our machine learning model can understand.

```
import pandas as pd

# Create a dataset with hours studied and test scores
data = {'Hours_Studied': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
        'Test_Score': [50, 55, 60, 65, 70, 75, 80, 85, 90, 95]}

# Convert the data to a Pandas DataFrame
df = pd.DataFrame(data)
print(df)
```

Step 2: Split the Data into Training and Testing Sets

We'll divide the data into two parts: one for **training** the model and one for **testing** it. The training data is used to help the model learn, and the testing data helps us see how well the model works on new, unseen data.

```

from sklearn.model_selection import train_test_split

# Separate the features (hours studied) from the target (test scores)
X = df[['Hours_Studied']] # Input: Hours studied
y = df['Test_Score']      # Output: Test score

# Split the data (80% for training, 20% for testing)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

```

Step 3: Build the Model Using Linear Regression

We'll use **linear regression**, a simple and effective machine learning algorithm, to build our model. Linear regression works by finding the best-fitting straight line through the data that shows the relationship between hours studied and test scores.

```

from sklearn.linear_model import LinearRegression

# Create a linear regression model
model = LinearRegression()

# Train the model using the training data
model.fit(X_train, y_train)

```

Step 4: Test the Model and Evaluate Its Performance

Once the model is trained, we test it using the testing data to see how well it predicts grades. We'll also calculate the **mean absolute error (MAE)**, which tells us how far off the model's predictions are from the actual scores. The lower the MAE, the better the model is at making accurate predictions.

```

from sklearn.metrics import mean_absolute_error

# Make predictions on the testing set
y_pred = model.predict(X_test)

# Compare predictions with actual test scores
print("Predicted test scores:", y_pred)
print("Actual test scores:", y_test.values)

# Calculate the mean absolute error
mae = mean_absolute_error(y_test, y_pred)
print(f"Mean Absolute Error: {mae}")

```

Step 5: Make Predictions on New Data

Now that we have a trained model, we can use it to predict future grades based on new data. For example, if a student studies for 7.5 hours, we can predict their grade:

```
# Predict the grade for a student who studied for 7.5 hours
hours_studied = [[7.5]]
predicted_score = model.predict(hours_studied)
print(f"Predicted test score for 7.5 hours of study: {predicted_score[0]}")
```

3. Understanding the Results

Once you run the code, you'll get predictions like this:

Predicted test scores: [75.]

Actual test scores: [75]

Mean Absolute Error: 0.0

Predicted test score for 7.5 hours of study: 77.5

This means the model accurately predicted the test score for the testing data, with an error of 0, which is great! When predicting a score for a student who studied 7.5 hours, the model estimates that they would score 77.5 points.

4. Why This Example Is Useful

Predicting student grades using past data is useful because it helps:

- **Students** understand how their study habits might impact their future performance.
- **Teachers** make data-driven decisions to guide students who may need more help.
- **Parents and educators** monitor progress and provide support where needed.

By using this simple model, we can make informed predictions that help improve learning outcomes. The same techniques can be applied to many other areas, from predicting sales in a business to forecasting trends in data.

5. Exploring More Advanced Features

Once you're comfortable with this basic example, you can explore more advanced features:

- **Adding more data:** You could include other factors like attendance, homework completion, or class participation to see how they impact grades.
- **Trying different algorithms:** While linear regression works well for this example, you could explore other machine learning algorithms to see if they provide better results.
- **Visualizing the model:** You can plot the data and the model's predictions to get a visual understanding of how well it fits the data.

Summary

In this example, we used Python and machine learning to build a simple model that predicts student grades based on past study data. We explored how to prepare data, train a model, and test its performance using real-world techniques like linear regression. With just a few lines of code, you can create a model that helps make predictions and provides valuable insights for improving student performance.

This hands-on approach to machine learning makes complex topics accessible, and by practicing with more data and models, you'll soon be able to solve more advanced problems with confidence!

Part V: Advanced Projects for Students and Teachers

Chapter 14: Creating Educational Tools with Python

Building Flashcard Quizzes for Classroom Learning

Flashcard quizzes are a simple yet powerful tool for learning and retaining information. They help students actively engage with the material by challenging them to recall key concepts and facts. With Python, you can easily create digital flashcard quizzes that are interactive, fun, and customizable for any subject. Whether you're a student wanting to practice for an upcoming test or a teacher looking to create a learning tool for the classroom, this project is both accessible and rewarding.

In this section, we'll guide you through the process of building a flashcard quiz using Python. The goal is to create a program that shows a question (or term) on one side of the "flashcard" and prompts the user to provide the answer. This project will help you practice your coding skills while creating a tool that enhances learning.

1. Why Flashcard Quizzes?

Flashcards have long been used for learning because they:

- **Promote active recall:** Flashcards force you to recall an answer from memory, which strengthens learning and retention.
- **Encourage spaced repetition:** By reviewing flashcards over time, you reinforce your understanding of the material.

- **Are easy to customize:** You can create flashcards for any subject, from vocabulary and math facts to historical dates or scientific concepts.

When paired with Python, flashcards become even more powerful because you can automate the quiz process, randomize questions, track progress, and provide instant feedback.

2. Building a Flashcard Quiz in Python

Let's walk through the process of creating a flashcard quiz in Python. The basic flow of the program will be:

1. Display a question (or term).
2. Get the user's response.
3. Check if the response is correct.
4. Provide feedback.
5. Repeat for more questions.

Step 1: Define the Flashcard Data

First, we need to define the content of the flashcards. For this example, we'll create a simple flashcard quiz for learning vocabulary words, but you can easily adapt it for any subject.

A list of flashcards with questions and answers

```
flashcards = [  
    {"question": "What is the capital of France?", "answer": "Paris"},  
    {"question": "What is 5 + 7?", "answer": "12"},  
    {"question": "What is the chemical symbol for water?", "answer": "H2O"},  
    {"question": "Who wrote 'Romeo and Juliet'?", "answer": "Shakespeare"}  
]
```

In this example, each flashcard has a question and the correct answer. You can store the flashcards in a list of dictionaries, where each dictionary represents one card.

Step 2: Display a Question and Get the User's Response

Next, we'll display the question to the user and prompt them to input their answer.

```
# Function to run the flashcard quiz
def run_flashcard_quiz(flashcards):
    for card in flashcards:
        # Display the question
        print(card["question"])

        # Get the user's answer
        user_answer = input("Your answer: ").strip()

        # Check if the answer is correct
        if user_answer.lower() == card["answer"].lower():
            print("Correct!\n")
        else:
            print(f"Incorrect. The correct answer is {card['answer']}\n")

# Run the quiz
run_flashcard_quiz(flashcards)
```

Explanation:

- **Looping through flashcards:** The for loop goes through each flashcard, displaying the question and prompting the user for an answer.
- **Checking the answer:** We compare the user's answer to the correct answer, using `lower()` to ignore case differences.
- **Feedback:** The program tells the user if they were correct or incorrect and shows the correct answer if needed.

Step 3: Randomize the Questions

To make the quiz more challenging, you can randomize the order of the flashcards so that the questions appear in a different order each time the quiz is run. Python's `random.shuffle()` function can help us achieve this.

```
import random
```

```
# Shuffle the flashcards to randomize the order
random.shuffle(flashcards)
```

```
# Run the quiz with randomized flashcards
run_flashcard_quiz(flashcards)
```

Now, each time you run the quiz, the flashcards will appear in a different order, keeping the learning experience fresh and engaging.

Step 4: Track the User's Score

To make the quiz more interactive and rewarding, let's add a scoring system that tracks how many questions the user answers correctly.

```
def run_flashcard_quiz_with_score(flashcards):
    score = 0 # Initialize the score

    for card in flashcards:
        print(card["question"])
        user_answer = input("Your answer: ").strip()

        if user_answer.lower() == card["answer"].lower():
            print("Correct!\n")
            score += 1 # Increase the score for correct answers
        else:
            print(f'Incorrect. The correct answer is {card["answer"]}\n')

    # Display the final score
    print(f'You got {score} out of {len(flashcards)} correct.')

```

```
# Shuffle the flashcards and run the quiz
random.shuffle(flashcards)
run_flashcard_quiz_with_score(flashcards)
```

Now, the program will display the user's score at the end of the quiz, motivating them to improve their performance.

3. Expanding the Flashcard Quiz

Once you've built the basic flashcard quiz, you can easily expand it with additional features to make it more useful and fun. Here are a few ideas to enhance the project:

a) Add Categories

You can organize flashcards by categories (e.g., math, history, science) and allow the user to choose a category to focus on.

```
# Organize flashcards by category
```

```
flashcards = {  
    "math": [  
        {"question": "What is 5 + 7?", "answer": "12"},  
        {"question": "What is 8 x 3?", "answer": "24"}  
    ],  
    "science": [  
        {"question": "What is the chemical symbol for water?", "answer": "H2O"},  
        {"question": "What planet is known as the Red Planet?", "answer": "Mars"}  
    ]  
}
```

```
# Allow the user to choose a category
```

```
category = input("Choose a category (math/science): ").strip().lower()
```

```
if category in flashcards:
```

```
    run_flashcard_quiz(flashcards[category])
```

```
else:
```

```
    print("Invalid category.")
```

b) Keep Track of Progress Over Time

You could create a system to track the user's progress over multiple quizzes. For example, you could save the user's scores to a file and allow them to see their improvement over time.

```
# Save the user's score to a file
```

```
def save_score(score, total):
```

```
    with open('quiz_scores.txt', 'a') as file:
```

```
        file.write(f'Score: {score}/{total}\n')
```

```
# Run the quiz and save the score
```

```
run_flashcard_quiz_with_score(flashcards)
```

```
save_score(score, len(flashcards))
```

c) Add a Timer

To increase the challenge, you could add a timer to each question. If the user doesn't answer within a certain time, the program moves on to the next flashcard.

```
import time

def run_flashcard_quiz_with_timer(flashcards):
    for card in flashcards:
        print(card["question"])
        start_time = time.time()

        user_answer = input("Your answer: ").strip()

        # Check if the answer was given within 10 seconds
        if time.time() - start_time > 10:
            print("Time's up!\n")
        elif user_answer.lower() == card["answer"].lower():
            print("Correct!\n")
        else:
            print(f"Incorrect. The correct answer is {card['answer']}\n")

    random.shuffle(flashcards)
    run_flashcard_quiz_with_timer(flashcards)
```

4. Why Flashcard Quizzes Are Effective for Learning

Flashcards are an excellent tool for learning because they:

- **Engage active recall:** By forcing you to remember information, flashcards make learning more effective.
- **Support spaced repetition:** Reviewing flashcards over time helps reinforce your understanding.
- **Provide instant feedback:** Immediate feedback helps you learn from mistakes and improves retention.

By automating flashcard quizzes with Python, you can make studying more dynamic and enjoyable, whether you're preparing for exams, learning a new subject, or teaching others.

Summary

Building flashcard quizzes with Python is a fun and educational project that helps students and teachers alike. By creating interactive flashcards, you can test your knowledge, track your progress, and reinforce learning in any subject. With just a few lines of Python code, you can turn traditional flashcards into a digital tool that supports active recall and spaced repetition—two key techniques for effective learning.

From basic vocabulary quizzes to more advanced timed challenges, this project is highly customizable, and you can easily expand it with features like scoring, progress tracking, and category selection. Keep experimenting and soon you'll have a powerful learning tool that makes studying more engaging and efficient!

Application: A Python-Based Quiz Maker for Teachers

Creating quizzes is a common task for teachers, but it can be time-consuming to design and grade them manually. With Python, you can build a simple yet powerful **quiz maker** that helps teachers create, administer, and even grade quizzes automatically. This tool allows teachers to input questions and answers, shuffle the questions, and track student performance—all through Python.

In this section, we'll guide you through the process of creating a Python-based quiz maker that can be customized for any classroom subject. Whether you're a teacher looking to streamline quiz creation or a student exploring how Python can be used in education, this project will help you build a practical and useful application.

1. Why Build a Python-Based Quiz Maker?

A Python-based quiz maker is useful because it:

- **Automates quiz creation:** Teachers can easily input questions and answers, and the program generates the quiz.
- **Provides instant feedback:** Students can receive immediate results after taking the quiz, helping them learn from their mistakes.
- **Tracks progress:** The tool can store results, allowing teachers to monitor how students perform over time.

This tool can be used for any subject, and once it's built, it can be easily customized with different question sets, difficulty levels, or even timed quizzes.

2. How the Quiz Maker Will Work

The quiz maker will follow these steps:

1. **Input quiz questions and answers:** Teachers can provide questions and the correct answers.
2. **Display the quiz:** The program will present the questions to students, one by one.
3. **Get student responses:** The program will collect and store the students' answers.
4. **Grade the quiz:** At the end of the quiz, the program will grade the responses and provide feedback.
5. **Track results:** The quiz maker can save scores to help track student progress over time.

3. Building the Quiz Maker with Python

Let's walk through the steps to build a simple quiz maker. We'll start with a basic version and then add more features to improve its functionality.

Step 1: Define the Quiz Questions and Answers

First, we need to create a set of questions and their correct answers. For this example, we'll use a list of multiple-choice questions.

```
# A list of quiz questions, each with a set of answers and the correct answer
```

```
quiz = [  
    {  
        "question": "What is the capital of France?",  
        "options": ["A) Paris", "B) London", "C) Berlin", "D) Madrid"],  
        "correct": "A"  
    },  
    {  
        "question": "Who wrote 'Romeo and Juliet'?",  
        "options": ["A) Shakespeare", "B) Dickens", "C) Twain", "D) Hemingway"],  
        "correct": "A"  
    },  
    {  
        "question": "What is the chemical symbol for water?",  
        "options": ["A) CO2", "B) H2O", "C) O2", "D) NaCl"],  
        "correct": "B"  
    }  
]
```

```

    },
    {
        "question": "What is 7 + 5?",
        "options": ["A) 10", "B) 11", "C) 12", "D) 13"],
        "correct": "C"
    }
]

```

Each quiz question is represented as a dictionary, containing the question text, a list of answer options, and the correct answer.

Step 2: Display the Quiz to the Student

Next, we'll write a function that displays each question and its answer choices to the student, then collects their response.

```

def run_quiz(quiz):
    score = 0 # Initialize the score

    for i, question in enumerate(quiz):
        print(f'Question {i+1}: {question["question"]}')
        for option in question["options"]:
            print(option)

        # Get the student's answer
        answer = input("Your answer (A, B, C, D): ").strip().upper()

        # Check if the answer is correct
        if answer == question["correct"]:
            print("Correct!\n")
            score += 1 # Increase the score for correct answers
        else:
            print(f'Incorrect. The correct answer is {question["correct"]}. \n')

    # Print the final score
    print(f'Your final score: {score} out of {len(quiz)}')

```

Explanation:

- **Displaying the questions:** The function loops through each question in the quiz and displays it to the student, along with

the answer options.

- **Collecting the answer:** The student is prompted to input their answer, which is then compared to the correct answer.
- **Scoring:** The program keeps track of how many questions the student answered correctly.

Step 3: Shuffle the Quiz for Variation

To make the quiz different each time, we can randomize the order of the questions using Python's `random.shuffle()` function.

```
import random

# Shuffle the quiz to randomize the question order
random.shuffle(quiz)

# Run the quiz
run_quiz(quiz)
```

Each time the quiz is run, the questions will appear in a different order, which helps keep the quiz fresh and challenging for students.

Step 4: Save Quiz Results for Progress Tracking

To make the quiz maker more useful for teachers, we can add a feature that saves each student's results to a file. This way, teachers can track student performance over time and see how students improve with each quiz.

```
def save_results(score, total_questions):
    with open('quiz_results.txt', 'a') as file:
        file.write(f'Score: {score}/{total_questions}\n')
    print("Your results have been saved.\n")
```

```
# Run the quiz and save the results
run_quiz(quiz)
save_results(score, len(quiz))
```

After the quiz is completed, the student's score is saved to a file called `quiz_results.txt`. This file can be used to review the performance of different students or track progress over time.

4. Adding More Features

Once you've built the basic quiz maker, you can expand its functionality to include more advanced features that make it even more useful for the classroom.

a) Multiple Quiz Modes

You can offer different quiz modes, such as timed quizzes or quizzes with a difficulty level selection. For example, teachers could create quizzes for different skill levels or subjects.

```
def choose_quiz_mode():
    mode = input("Choose a quiz mode (normal/timed): ").strip().lower()
    if mode == "timed":
        run_timed_quiz(quiz)
    else:
        run_quiz(quiz)
```

b) Allow Teachers to Create Their Own Quizzes

You can create a feature that allows teachers to input their own quiz questions and answers directly into the program, making it highly customizable for different classes.

```
def create_custom_quiz():
    quiz = []
    num_questions = int(input("How many questions would you like to create? "))

    for i in range(num_questions):
        question_text = input(f"Enter the text for question {i+1}: ")
        options = [input(f"Option {letter}: ") for letter in ["A", "B", "C", "D"]]
        correct_answer = input("Which option is correct? (A, B, C, D): ").strip().upper()

        quiz.append({
            "question": question_text,
            "options": [f"{letter} {option}" for letter, option in zip("ABCD", options)],
            "correct": correct_answer
        })

    return quiz
```

Allow the teacher to create a quiz and then run it

```
custom_quiz = create_custom_quiz()
run_quiz(custom_quiz)
```

c) Add Timed Quizzes

To make quizzes more challenging, you can add a timer that limits the amount of time students have to answer each question. If the student doesn't answer in time, the quiz automatically moves to the next question.

```
import time

def run_timed_quiz(quiz):
    for question in quiz:
        print(question["question"])
        for option in question["options"]:
            print(option)

        start_time = time.time()
        answer = input("Your answer (A, B, C, D): ").strip().upper()

        if time.time() - start_time > 10:
            print("Time's up!\n")
        elif answer == question["correct"]:
            print("Correct!\n")
        else:
            print(f'Incorrect. The correct answer is {question["correct"]}\n')

    random.shuffle(quiz)
    run_timed_quiz(quiz)
```

5. Why a Quiz Maker Is Valuable

A Python-based quiz maker is valuable for several reasons:

- **Saves time:** Teachers can quickly generate quizzes without manually creating and grading them.
- **Improves learning:** Students get immediate feedback and can take quizzes multiple times to reinforce learning.

- **Tracks progress:** Teachers can monitor student performance and adjust their teaching based on quiz results.
- **Highly customizable:** Quizzes can be created for any subject and adjusted for difficulty or time constraints.

Summary

Building a Python-based quiz maker is a practical and rewarding project for both students and teachers. It automates the process of creating and grading quizzes, provides immediate feedback, and tracks student performance over time. With just a few lines of Python code, you can create customizable quizzes that suit different classroom needs, from basic multiple-choice quizzes to more advanced timed challenges.

Tracking Progress with Python

Tracking progress is essential for both students and teachers, as it allows them to monitor learning and improvement over time. Whether it's tracking quiz scores, homework completion, or study hours, Python can be a powerful tool for automating and visualizing this data. In this section, we'll explore how to create a simple system using Python to track and analyze student progress.

This project will help you practice your Python skills while building something valuable for the classroom. You'll learn how to store, update, and display student performance data in an accessible way.

1. Why Tracking Progress Is Important

For students, tracking progress helps identify areas of strength and areas that need improvement. It provides a sense of achievement and encourages a growth mindset. For teachers, progress tracking offers insights into student performance, allowing for personalized feedback and adjustments to teaching methods.

With Python, you can automate this process, making it easier to manage large amounts of data and focus on helping students succeed.

2. Building a Progress Tracking System with Python

We'll build a simple Python-based system that allows teachers to:

1. **Record quiz scores or assignments.**
2. **Store the data for each student.**
3. **Track and display each student's progress over time.**
4. **Analyze trends to provide targeted feedback.**

Step 1: Create a Data Structure to Store Student Progress

First, we need to decide how we'll store the data. A Python dictionary is a great way to organize student names and their corresponding scores. For example, each student could have a list of their quiz scores, which gets updated after each quiz.

```
# Dictionary to store students and their quiz scores
student_progress = {
    "Alice": [85, 90, 88],
    "Bob": [78, 82, 80],
    "Charlie": [92, 88, 94]
}
```

In this example, the dictionary keys are the students' names, and the values are lists containing their quiz scores.

Step 2: Add a Function to Record New Scores

Next, we'll write a function that allows a teacher to add a new quiz score for a student. This function will update the student's record with the new score.

```
# Function to add a new score for a student
def add_score(student_name, score):
    if student_name in student_progress:
        student_progress[student_name].append(score)
        print(f"Added score {score} for {student_name}.")
    else:
        print(f"Student {student_name} not found.")
```

This function checks if the student is already in the system. If they are, it appends the new score to their list of quiz scores. If not, it prints a message indicating that the student was not found.

Step 3: Display a Student's Progress

To track progress, we need a way to display a student's quiz history. Let's create a function that retrieves and prints all of a student's scores, along with their average score.

```
# Function to display a student's progress
def display_progress(student_name):
    if student_name in student_progress:
        scores = student_progress[student_name]
        average_score = sum(scores) / len(scores)
        print(f"{student_name}'s scores: {scores}")
        print(f"Average score: {average_score:.2f}")
```



```
else:
```

```
    print(f"Student {student_name} not found.")
```

This function calculates the student's average score by summing their quiz scores and dividing by the number of quizzes taken. It then prints the individual scores and the average.

Step 4: Visualize Progress Over Time

To make progress tracking more engaging and easier to understand, we can visualize the data using **matplotlib**, a Python library for plotting graphs. Let's create a line graph that shows how a student's scores have changed over time.

First, install matplotlib if you haven't already:

```
pip install matplotlib
```

Now, we can write a function to plot the student's scores.

```
import matplotlib.pyplot as plt
```

```
# Function to plot a student's progress over time
```

```
def plot_progress(student_name):
```

```
    if student_name in student_progress:
```

```
        scores = student_progress[student_name]
```

```
        plt.plot(scores, marker='o')
```

```
        plt.title(f'{student_name}'s Progress Over Time')
```

```
        plt.xlabel('Quiz Number')
```

```
        plt.ylabel('Score')
```

```
        plt.show()
```

```
    else:
```

```
        print(f"Student {student_name} not found.")
```

Explanation:

- **plt.plot()**: Creates a line graph of the student's scores.
- **marker='o'**: Adds a circle marker to each data point to make the quiz scores stand out.
- **plt.show()**: Displays the graph.

This visualization helps both students and teachers see trends in performance, such as improvement over time or areas where a student may be struggling.

Step 5: Save and Load Student Data

To ensure the data isn't lost after the program is closed, we can save the student progress data to a file and load it later. Let's use Python's **pickle** module to store the data.

```
import pickle

# Function to save student progress data to a file
def save_progress(filename):
    with open(filename, 'wb') as file:
        pickle.dump(student_progress, file)
    print("Progress saved.")

# Function to load student progress data from a file
def load_progress(filename):
    global student_progress
    with open(filename, 'rb') as file:
        student_progress = pickle.load(file)
    print("Progress loaded.")
```

The **pickle** module allows us to serialize Python objects and save them to a file. When the program is restarted, we can reload the data using `load_progress()` to continue tracking student progress.

3. Expanding the Progress Tracker

Once you have the basic progress tracker working, you can add more features to make it even more useful.

a) Track Multiple Subjects

You could extend the system to track student performance across multiple subjects, such as math, science, and history. Each student could have a separate set of scores for each subject.

```
# Dictionary to track progress in multiple subjects
student_progress = {
```

```

"Alice": {"Math": [85, 90], "Science": [88, 92]},
"Bob": {"Math": [78, 82], "Science": [80, 85]}
}

# Update functions to handle multiple subjects
def add_score(student_name, subject, score):
    if student_name in student_progress and subject in student_progress[student_name]:
        student_progress[student_name][subject].append(score)
        print(f"Added score {score} for {student_name} in {subject}.")
    else:
        print(f"Student or subject not found.")

```

b) Analyze Class-Wide Performance

In addition to tracking individual students, you could create functions that analyze the performance of the entire class, providing statistics like the average score per quiz or identifying the most challenging quiz questions.

```

# Function to calculate class average for a specific quiz
def class_average(quiz_number):
    total = 0
    count = 0
    for scores in student_progress.values():
        if len(scores) > quiz_number:
            total += scores[quiz_number]
            count += 1
    if count > 0:
        print(f"Class average for quiz {quiz_number + 1}: {total / count:.2f}")
    else:
        print("Not enough data for this quiz.")

```

4. Why Tracking Progress Is Valuable

Tracking progress benefits both students and teachers in several ways:

- **Immediate feedback:** Students can see their strengths and areas for improvement in real time, allowing them to focus on what matters most.

- **Motivation:** Watching their progress grow over time encourages students to continue putting in effort and reinforces a growth mindset.
- **Personalized teaching:** Teachers can use progress data to identify students who may need extra help or to tailor lessons based on the class's needs.

Python makes it easy to automate this process, saving time and allowing both students and teachers to focus on learning and improvement.

Summary

Tracking student progress with Python is a powerful way to monitor and improve learning outcomes. By building a simple system that records quiz scores, calculates averages, and visualizes performance over time, you can provide students with valuable feedback while saving time for teachers. With just a few lines of Python code, you can create a progress tracker that helps students stay motivated and allows teachers to personalize their instruction.

By expanding the system to include multiple subjects, saving data to files, and analyzing class-wide performance, this project becomes a flexible tool that can be adapted to any classroom environment. Keep experimenting, and soon you'll have a fully functional progress tracker that makes learning and teaching more effective!

Chapter 15: Python in Science and Engineering

Solving Physics and Chemistry Problems with Python

Python is a powerful tool for solving real-world problems in science and engineering, making it an excellent choice for students who want to explore physics and chemistry through programming. By using Python, we can automate calculations, simulate experiments, and analyze data, all of which help deepen our understanding of scientific concepts. In this section, we'll explore how Python can be used to tackle common physics and chemistry problems in a way that's accessible and engaging for students.

By the end of this chapter, you'll have learned how to apply Python to solve problems in areas such as motion, forces, energy, and chemical reactions. You'll see that even complex scientific concepts can become more manageable when you approach them with code.

1. Why Use Python for Physics and Chemistry?

Python makes it easier to solve science and engineering problems because:

- **It's simple to learn:** Python's syntax is easy to read and write, making it an ideal tool for students.
- **It handles complex calculations:** Python can perform precise calculations quickly, which is useful for solving equations and working with large data sets.
- **Visualization:** Python allows us to visualize scientific data through graphs and simulations, helping us understand

patterns and trends.

Whether you're calculating the trajectory of an object, simulating chemical reactions, or analyzing data from experiments, Python provides the tools you need to make science more interactive and fun.

2. Solving Physics Problems with Python

Physics is all about understanding the natural world through mathematical formulas and laws. Python can help us solve these equations and simulate physical processes, making abstract concepts easier to grasp. Let's start with a few examples of how Python can be used in physics.

a) Kinematic Equations: Motion and Acceleration

In physics, one of the most fundamental topics is **motion**—how objects move through space. The **kinematic equations** describe the motion of an object under constant acceleration. One common equation is:

$$v_f = v_i + at$$

Where:

- v_f = final velocity
- v_i = initial velocity
- a = acceleration
- t = time

Let's write a Python function to calculate the final velocity of an object based on its initial velocity, acceleration, and time.

```
# Function to calculate final velocity
```

```
def final_velocity(v_i, a, t):
```

```
    v_f = v_i + a * t
```

```
    return v_f
```

```
# Example: initial velocity = 10 m/s, acceleration = 2 m/s², time = 5 seconds
```

```
v_initial = 10
```

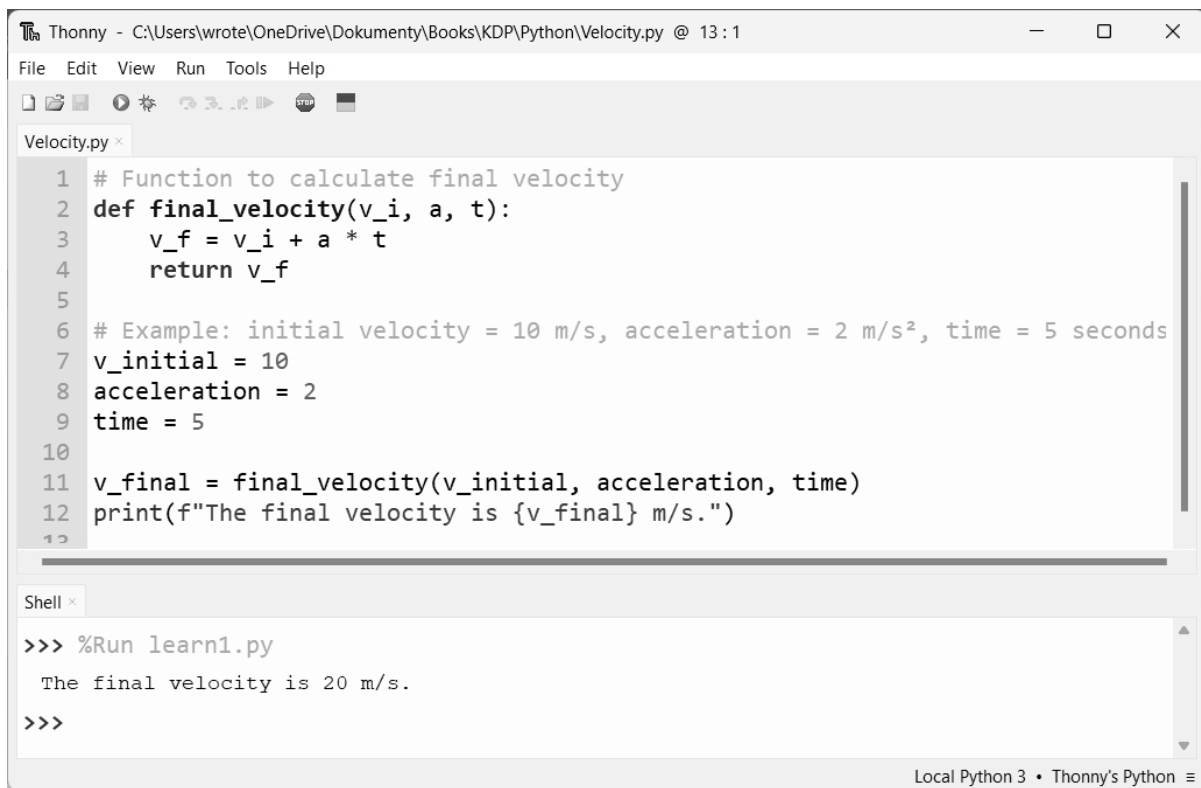
```
acceleration = 2
```

```
time = 5
```

```
v_final = final_velocity(v_initial, acceleration, time)
print(f"The final velocity is {v_final} m/s.")
```

Explanation:

- **final_velocity()** calculates the final velocity using the kinematic equation.
- In the example, we calculate the final velocity of an object starting at 10 m/s, accelerating at 2 m/s² for 5 seconds.



The screenshot shows the Thonny Python IDE interface. The top window, titled 'Velocity.py', contains the following Python code:

```
1 # Function to calculate final velocity
2 def final_velocity(v_i, a, t):
3     v_f = v_i + a * t
4     return v_f
5
6 # Example: initial velocity = 10 m/s, acceleration = 2 m/s², time = 5 seconds
7 v_initial = 10
8 acceleration = 2
9 time = 5
10
11 v_final = final_velocity(v_initial, acceleration, time)
12 print(f"The final velocity is {v_final} m/s.")
```

The bottom window, titled 'Shell', shows the execution output:

```
>>> %Run learn1.py
The final velocity is 20 m/s.
>>>
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

Projectile Motion

Another important concept in physics is **projectile motion**. When an object is thrown into the air, it follows a curved path due to the influence of gravity. We can use Python to simulate the motion of a projectile and visualize its trajectory.

The equations for projectile motion are:

- Horizontal motion: $x = v_x t$

- Vertical motion: $y = v_y t - 0.5gt$

Where:

- v_x and v_y are the horizontal and vertical components of the initial velocity.
- G is the acceleration due to gravity (approximately 9.8 m/s^2).
- t is time.

Let's simulate the motion of a projectile and plot its trajectory using Python.

```
import numpy as np
import matplotlib.pyplot as plt

# Function to simulate projectile motion
def projectile_motion(v_i, angle, g=9.8):
    angle_rad = np.radians(angle)
    v_x = v_i * np.cos(angle_rad)
    v_y = v_i * np.sin(angle_rad)

    # Time to reach the ground
    t_flight = (2 * v_y) / g

    # Time intervals
    t = np.linspace(0, t_flight, num=100)

    # Equations of motion
    x = v_x * t
    y = v_y * t - 0.5 * g * t**2

    return x, y

# Initial velocity (m/s) and angle (degrees)
v_initial = 20
launch_angle = 45

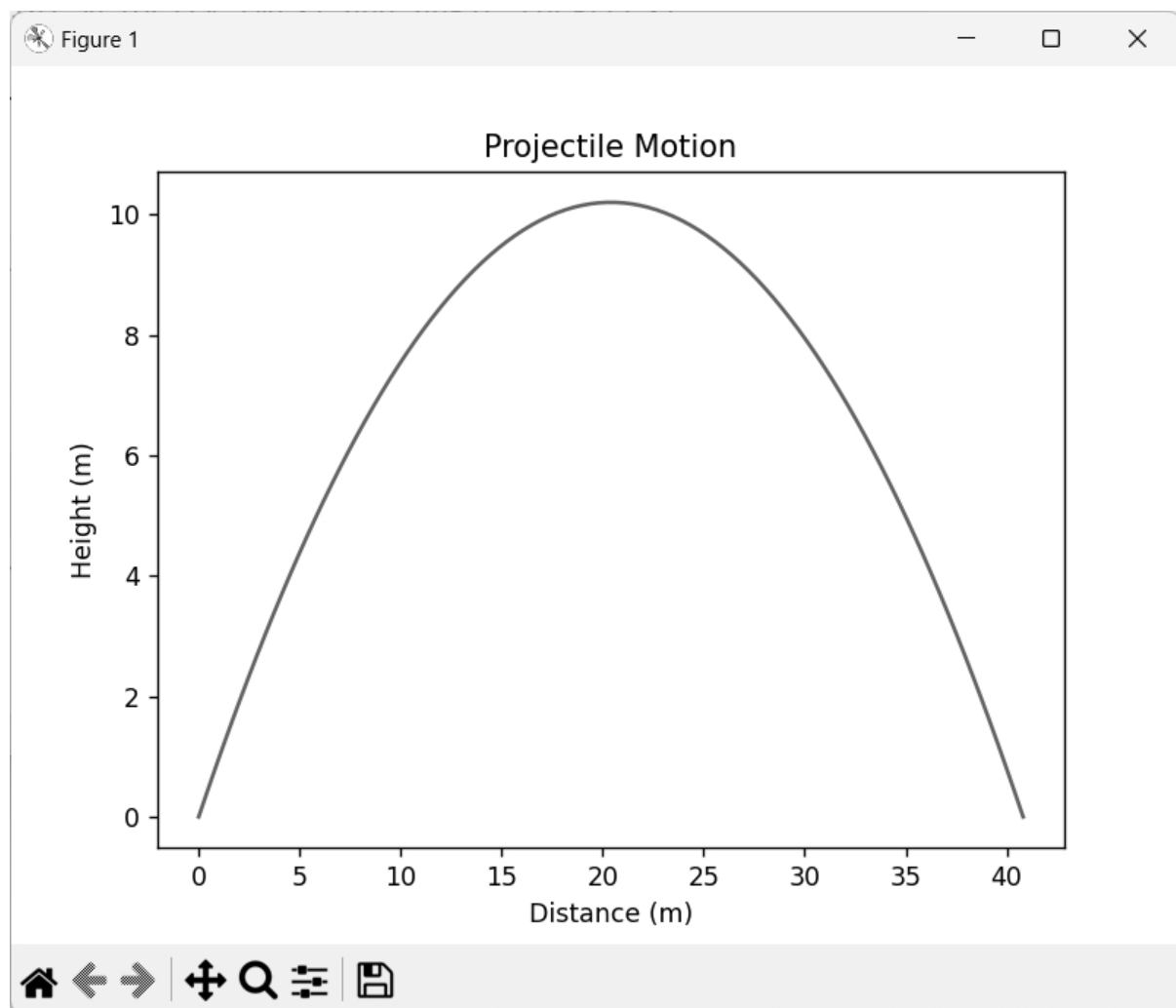
# Simulate and plot the trajectory
x, y = projectile_motion(v_initial, launch_angle)
plt.plot(x, y)
```



```
plt.title("Projectile Motion")
plt.xlabel("Distance (m)")
plt.ylabel("Height (m)")
plt.show()
```

Explanation:

- **projectile_motion()** calculates the horizontal and vertical positions of the projectile over time.
- We plot the projectile's trajectory, showing how it moves through space.
- The graph gives us a visual understanding of how factors like velocity and angle affect the motion.



c) Energy and Work

Energy is another key concept in physics. The **kinetic energy** of an object is given by:

$$KE = 0.5mv^2$$

Where:

- KE = kinetic energy
- m = mass of the object
- v = velocity

Let's write a Python function to calculate the kinetic energy of an object.

```
# Function to calculate kinetic energy
```

```
def kinetic_energy(m, v):
```

```
    KE = 0.5 * m * v**2
```

```
    return KE
```

```
# Example: mass = 5 kg, velocity = 10 m/s
```

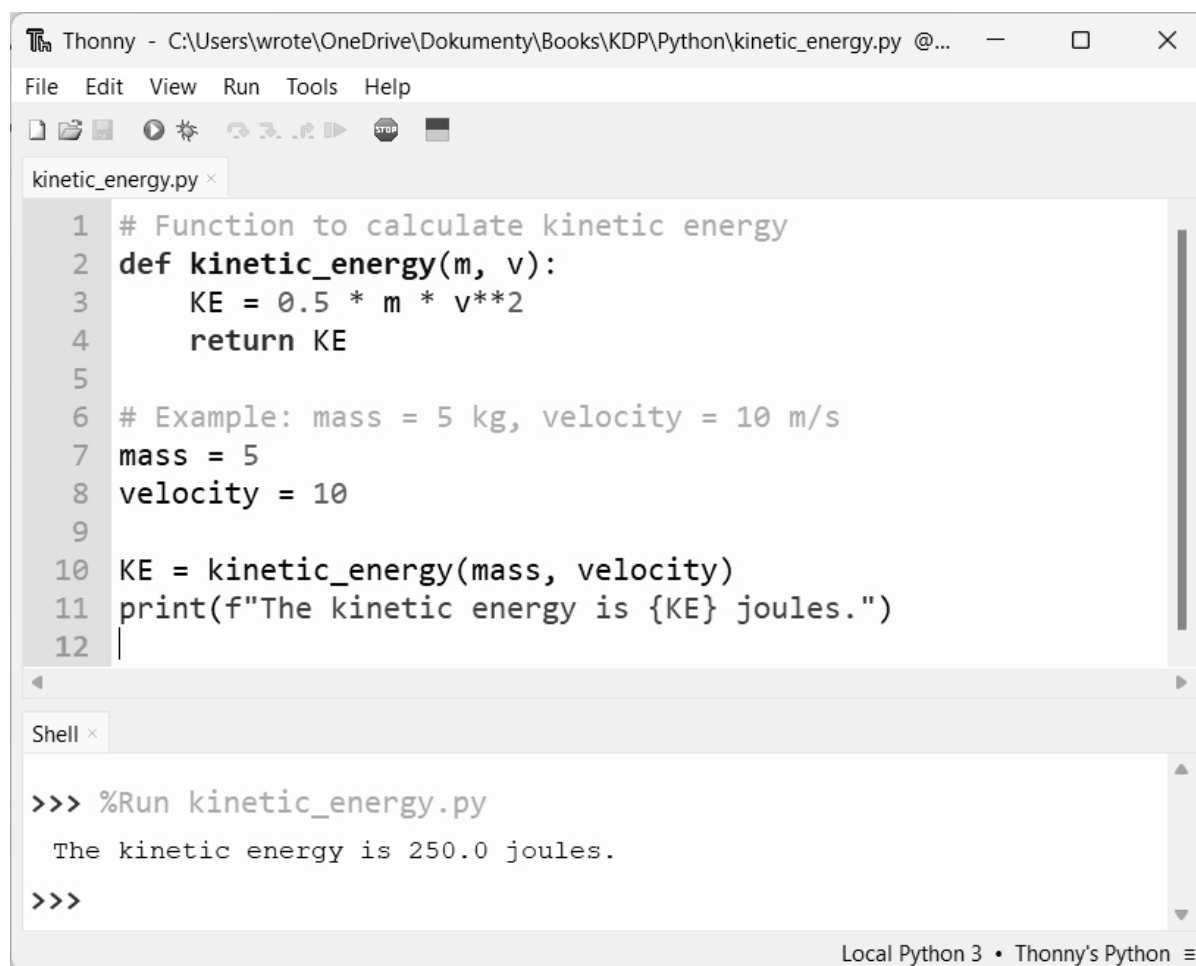
```
mass = 5
```

```
velocity = 10
```

```
KE = kinetic_energy(mass, velocity)
```

```
print(f"The kinetic energy is {KE} joules.")
```

This function helps calculate how much energy an object has based on its mass and velocity, reinforcing the connection between motion and energy.



The screenshot shows the Thonny Python IDE interface. The top window, titled 'kinetic_energy.py', contains the following Python code:

```
1 # Function to calculate kinetic energy
2 def kinetic_energy(m, v):
3     KE = 0.5 * m * v**2
4     return KE
5
6 # Example: mass = 5 kg, velocity = 10 m/s
7 mass = 5
8 velocity = 10
9
10 KE = kinetic_energy(mass, velocity)
11 print(f"The kinetic energy is {KE} joules.")
12
```

The bottom window, titled 'Shell', shows the execution output:

```
>>> %Run kinetic_energy.py
The kinetic energy is 250.0 joules.
>>>
```

The status bar at the bottom right indicates 'Local Python 3 • Thonny's Python'.

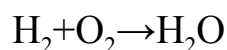
3. Solving Chemistry Problems with Python

Python is equally useful in chemistry, where we can use it to solve problems related to chemical reactions, temperature conversions, and more. Let's look at a couple of examples.

a) Balancing Chemical Equations

Balancing chemical equations is a fundamental skill in chemistry. It ensures that the number of atoms on both sides of the equation is the same. While Python can't balance equations by itself, libraries like **SymPy** can help automate this process.

Here's how we can use Python to balance a simple chemical equation:



```
from sympy import symbols, Eq, solve
```

```

# Define variables for the coefficients
H2, O2, H2O = symbols('H2 O2 H2O')

# Set up the equation (number of atoms on both sides must be equal)
equation = Eq(2 * H2, H2O) # Hydrogen balance
equation2 = Eq(2 * O2, H2O) # Oxygen balance

# Solve the system of equations
solution = solve([equation, equation2], (H2, O2, H2O))
print(f"Balanced equation: {solution}")

```

The screenshot shows the Thonny Python IDE interface. The top window, titled 'water.py', contains the following Python code:

```

1 from sympy import symbols, Eq, solve
2 # Define variables for the coefficients
3 H2, O2, H2O = symbols('H2 O2 H2O')
4
5 # Set up the equation (number of atoms on both sides must be equal)
6 equation = Eq(2 * H2, H2O) # Hydrogen balance
7 equation2 = Eq(2 * O2, H2O) # Oxygen balance
8
9 # Solve the system of equations
10 solution = solve([equation, equation2], (H2, O2, H2O))
11 print(f"Balanced equation: {solution}")
12

```

Below the code editor is a 'Shell' window showing the execution output:

```

>>> %Run water.py
Balanced equation: {H2: H2O/2, O2: H2O/2}
>>>

```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

b) Temperature Conversions

Temperature conversions are a common task in chemistry, especially when working with different units like Celsius, Fahrenheit, and Kelvin. Here's how we can use Python to convert between these units.

```

# Function to convert Celsius to Fahrenheit

```

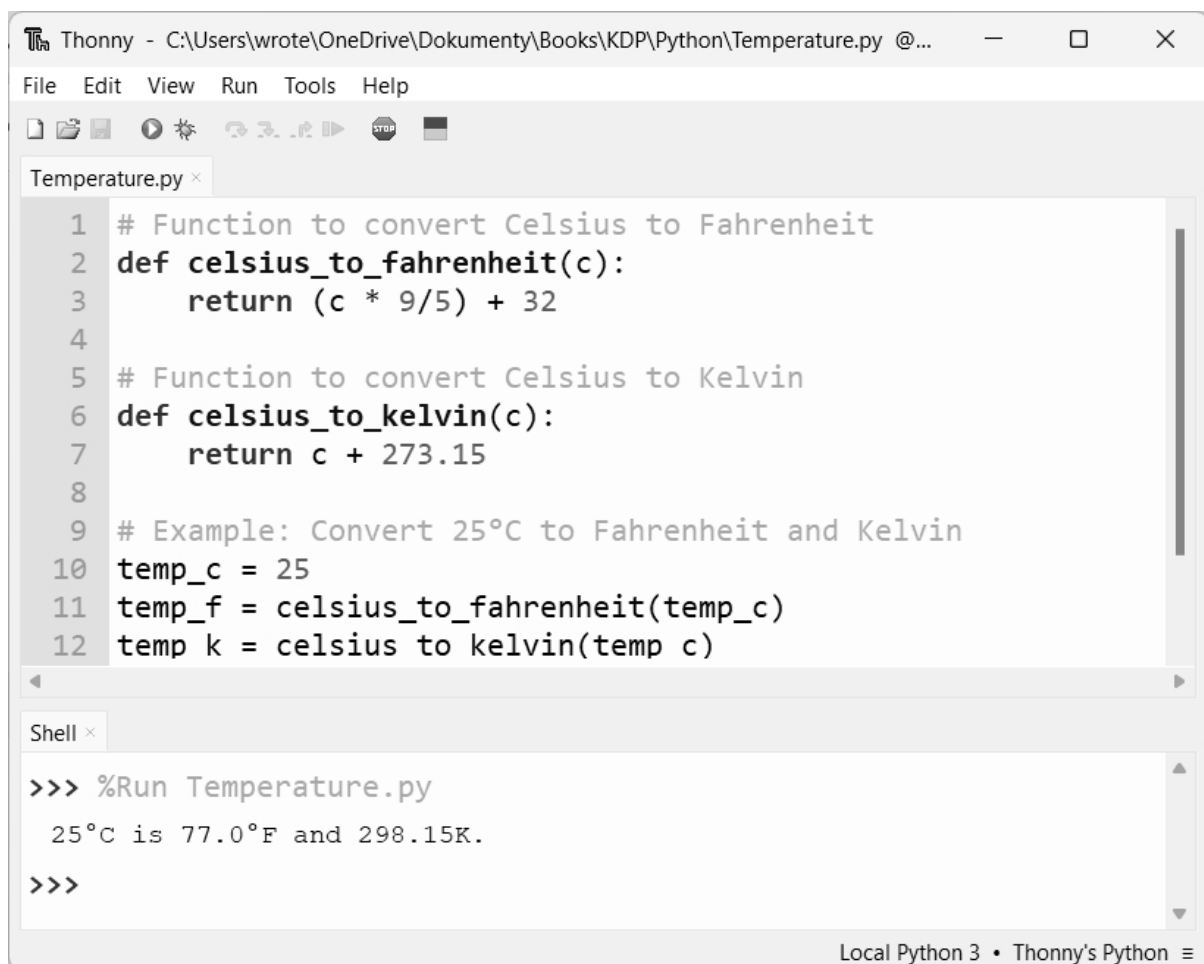
```
def celsius_to_fahrenheit(c):
    return (c * 9/5) + 32

# Function to convert Celsius to Kelvin
def celsius_to_kelvin(c):
    return c + 273.15

# Example: Convert 25°C to Fahrenheit and Kelvin
temp_c = 25
temp_f = celsius_to_fahrenheit(temp_c)
temp_k = celsius_to_kelvin(temp_c)

print(f'{temp_c}°C is {temp_f}°F and {temp_k}K.')
```

This simple function allows us to switch between temperature units, which is useful when performing experiments or analyzing chemical reactions.



The screenshot shows the Thonny Python IDE interface. The main editor window displays a Python script named 'Temperature.py' with the following code:

```
1 # Function to convert Celsius to Fahrenheit
2 def celsius_to_fahrenheit(c):
3     return (c * 9/5) + 32
4
5 # Function to convert Celsius to Kelvin
6 def celsius_to_kelvin(c):
7     return c + 273.15
8
9 # Example: Convert 25°C to Fahrenheit and Kelvin
10 temp_c = 25
11 temp_f = celsius_to_fahrenheit(temp_c)
12 temp_k = celsius_to_kelvin(temp_c)
```

Below the editor is a 'Shell' window showing the execution of the script:

```
>>> %Run Temperature.py
25°C is 77.0°F and 298.15K.
>>>
```

The status bar at the bottom indicates 'Local Python 3 • Thonny's Python'.

4. Why Python Is Valuable in Science

Python is a valuable tool for solving physics and chemistry problems because:

- **It simplifies complex calculations:** Instead of solving equations by hand, you can automate the process with Python.
- **It helps visualize concepts:** Python's plotting libraries make it easy to see how different variables affect physical and chemical processes.
- **It reinforces learning:** By writing code to solve scientific problems, you deepen your understanding of the underlying principles.

Summary

Using Python to solve physics and chemistry problems makes these subjects more interactive and accessible. From calculating motion and energy to balancing chemical equations and converting temperatures, Python helps students explore science in a hands-on way. By combining programming with scientific concepts, you can build a deeper understanding of how the world works and solve complex problems with ease. Keep practicing, and soon you'll be able to tackle even more advanced challenges in science and engineering!

Chapter 16: Working on Capstone Projects

How to Approach Larger Problems and Break Them Down

Tackling larger problems can seem overwhelming at first, especially when they involve many moving parts. Whether you're working on a capstone project, a classroom assignment, or any other complex task, the key to success is learning how to break big problems down into smaller, manageable pieces. In this section, we'll walk through strategies for approaching large problems with confidence, using Python and logical thinking to make the process smoother and more effective.

By the end of this chapter, you'll be equipped with practical techniques to break down larger projects into clear steps, making it easier to plan, execute, and complete tasks without feeling overwhelmed.

1. Why Break Problems Down?

When you look at a large problem, it can be easy to feel lost or unsure where to begin. Breaking it down helps because:

- **It simplifies the task:** Smaller pieces are easier to understand and solve.
- **It helps you focus:** By working on one part at a time, you can give your full attention to solving each step.
- **It provides a clear plan:** With a step-by-step approach, you know exactly what you need to do next.

Large problems often involve multiple components, but when each part is tackled individually, the overall project becomes much more manageable.

2. Steps to Breaking Down Large Problems

Here are some tried-and-true steps to help you break down larger problems into smaller, more manageable tasks:

Step 1: Understand the Problem

Before diving into the solution, take time to **fully understand the problem**. Read through the instructions carefully, and if necessary, ask questions or do some research to clarify the problem's requirements.

- **What is the problem asking you to solve?**
- **What are the inputs and outputs?**
- **What is the overall goal?**

For example, if you're tasked with creating a **Classroom Management System**, you need to understand what features are required (like student registration, grade tracking, etc.) and what the end result should look like.

Step 2: Break the Problem into Main Tasks

Once you understand the problem, break it down into **main tasks**. These are the big chunks of work that need to be done. Think of them as the high-level parts of the project.

For example, in a classroom management system, the main tasks might be:

1. **Create a student registration system.**
2. **Design a grade tracking system.**
3. **Build a user interface for teachers and students.**

Each of these main tasks represents a significant part of the project, and breaking them down helps you see the overall structure more clearly.

Step 3: Divide Main Tasks into Subtasks

After identifying the main tasks, break each one down into **smaller subtasks**. Subtasks are bite-sized actions that are easier to handle and solve

individually.

Let's take the first main task, "Create a student registration system," and break it down further:

1. Create a form for students to enter their details.
2. Validate the form input (e.g., check if fields are filled).
3. Store the student details in a database.
4. Display a confirmation message after successful registration.

Each of these subtasks is clear and focused, making it easier to complete one at a time. When you finish one subtask, you can move on to the next without feeling overwhelmed by the larger project.

Step 4: Prioritize and Plan

Now that you have a list of subtasks, **prioritize** them based on their importance and logical order. For example, you need to create the form before you can validate the input, so that comes first. By organizing your tasks in a logical sequence, you create a clear **action plan**.

Make sure to focus on **one task at a time** rather than trying to do everything at once. This focused approach helps you avoid getting stuck and keeps you moving forward step by step.

Step 5: Start with the Easiest or Most Important Task

When faced with a long list of subtasks, it's helpful to start with the **easiest** or **most important** one. This will give you a sense of progress early on, which can motivate you to keep going.

For example, if creating a form is straightforward, start with that. Once you have it done, you'll feel a sense of accomplishment and be ready to tackle the next task.

Alternatively, if one task is essential to the project's success (such as connecting to a database), it might make sense to prioritize that first.

Step 6: Test as You Go

As you work through the smaller tasks, make sure to **test** each one before moving on to the next. This way, you can catch errors early and ensure that

each part of the project is working as expected. For example, after creating the student registration form, test it to make sure it correctly collects data.

Testing as you go also helps prevent problems from building up. If you wait until the end to test everything, it can be harder to pinpoint where things went wrong.

Step 7: Iterate and Improve

Once all the subtasks are completed, take a step back and **review the entire project**. Are there any parts that could be improved or optimized? Sometimes, after completing the basic version of a project, you'll find ways to make it better.

For example, you might decide to add an extra feature to the classroom management system, such as the ability to export student data to a CSV file. Iterating and improving your work helps you refine the project and take it to the next level.

3. Real-World Example: Building a Classroom Management System

Let's apply these steps to a real-world project: building a **Classroom Management System**. This system will allow teachers to manage student data, track grades, and view reports.

Step 1: Understand the Problem

The goal is to build a system where teachers can:

- Register students and store their details.
- Enter and track grades for assignments and tests.
- View reports of student performance.

Step 2: Break the Problem into Main Tasks

The main tasks for this project might be:

1. **Student registration:** Allow teachers to register students and store their information.
2. **Grade tracking:** Build a system to track grades for various assignments.

3. **Report generation:** Create reports that summarize student performance.

Step 3: Divide Each Task into Subtasks

Now, let's break down one of the main tasks—**Student registration**—into smaller subtasks:

1. Create a form to input student details (name, age, class).
2. Validate the input (ensure fields are filled correctly).
3. Save the student data to a file or database.
4. Display a message confirming the registration was successful.

Step 4: Prioritize and Plan

Prioritize the subtasks: first create the form, then validate the input, followed by saving the data, and finally displaying the confirmation message. By focusing on one part at a time, you reduce the complexity of the problem.

Step 5: Start and Test

Start with the first subtask—creating the form. Once that's done, test it to make sure it works as expected before moving on to the next subtask. Continue this process until all tasks are completed.

Step 6: Review and Improve

After finishing the basic system, you might decide to add extra features, like the ability to edit or delete student records. Iterating on the project allows you to make improvements and build a more complete system.

4. Why This Approach Works

Breaking down large problems works because:

- **It reduces overwhelm:** Tackling one small task at a time feels less daunting.
- **It creates a clear path:** You know exactly what to do next, which helps maintain focus and momentum.

- **It allows for flexibility:** If something goes wrong, it's easier to fix a small part of the project than the entire thing.

By following this structured approach, you can tackle any large problem with confidence and efficiency.

Summary

When faced with large problems, breaking them down into smaller, manageable tasks is the key to success. By understanding the problem, dividing it into main tasks and subtasks, and working through each part step by step, you can tackle even the most complex projects with ease. Whether you're building a classroom management system or solving a challenging assignment, this method will help you stay organized, focused, and productive.

As you practice these techniques, you'll find that you can approach any large problem with confidence, turning it into a series of achievable goals that lead to a successful outcome. Keep practicing, and soon you'll master the art of problem-solving with Python!

Capstone Example: A Classroom Management System

A **Classroom Management System** is a practical and valuable project that can help teachers organize student data, track grades, and generate reports. It brings together many of the programming skills you've learned throughout this book—like working with data, creating user interfaces, and organizing complex tasks—into one cohesive project.

In this capstone example, we'll walk you through how to build a basic classroom management system using Python. By following along, you'll see how to break the project into smaller tasks and implement each feature step by step.

By the end of this chapter, you'll have a fully functional classroom management system that can be expanded with more features, giving you the opportunity to apply your Python skills to solve a real-world problem.

1. What Is a Classroom Management System?

A **Classroom Management System** is a tool that helps teachers keep track of important information about their students. This could include:

- **Student Registration:** Storing details like names, ages, and classes.
- **Grade Tracking:** Entering and calculating grades for assignments and tests.
- **Reports:** Generating summaries of student performance, attendance, or progress over time.

This system will allow teachers to automate much of their administrative work, making it easier to manage their classrooms efficiently.

2. Breaking Down the System into Key Features

To build a classroom management system, it's important to break the project down into smaller, manageable pieces. Here are the key features

we'll focus on:

1. **Student Registration:** Teachers can add students to the system, storing their names and other details.
2. **Grade Tracking:** Teachers can enter and store student grades for assignments, tests, and quizzes.
3. **Report Generation:** The system will generate reports that show student performance over time.

Each of these features will require several smaller tasks, which we'll tackle one at a time.

3. Step-by-Step Guide to Building the System

Let's go through the process of building each feature, step by step. We'll write Python code for each part and explain how it works.

Step 1: Setting Up Student Registration

The first feature we'll implement is **student registration**. This part of the system allows teachers to enter student information and save it for future use.

```
# Student data will be stored in a dictionary
students = {}

# Function to register a new student
def register_student():
    name = input("Enter the student's name: ")
    age = input("Enter the student's age: ")
    grade_level = input("Enter the student's grade level: ")

    # Store the student's information in the dictionary
    students[name] = {"Age": age, "Grade Level": grade_level}
    print(f"Student {name} has been registered.\n")

# Example: Registering students
register_student()
register_student()

# Display the list of registered students
```

```
print(students)
```

Explanation:

- The **students** dictionary stores student data, with the student's name as the key and their details (age and grade level) as values.
- **register_student()** prompts the teacher to input the student's information and stores it in the dictionary.

Step 2: Implementing Grade Tracking

Next, we'll add the ability to **track grades** for each student. The teacher will be able to enter grades for different assignments or tests, and the system will store them alongside each student's information.

```
# Function to add grades for a student
```

```
def add_grades(student_name):
```

```
    if student_name in students:
```

```
        # Get grades from the teacher
```

```
        math_grade = float(input(f'Enter {student_name}'s math grade: '))
```

```
        science_grade = float(input(f'Enter {student_name}'s science grade: '))
```

```
        # Store the grades in the student's record
```

```
        students[student_name]["Math"] = math_grade
```

```
        students[student_name]["Science"] = science_grade
```

```
        print(f'Grades have been added for {student_name}.\n')
```

```
    else:
```

```
        print(f'Student {student_name} not found.')
```

```
# Example: Adding grades for a student
```

```
add_grades("Alice")
```

```
add_grades("Bob")
```

```
# Display the students with their grades
```

```
print(students)
```

Explanation:

- **add_grades()** allows the teacher to input grades for a student. If the student is found in the system, their grades are stored in the dictionary under the subject names (Math, Science, etc.).
- This step allows teachers to track how each student performs in different subjects.

Step 3: Generating Reports

Finally, we'll add a feature to **generate reports**. The system will calculate the student's average grade and display their performance, helping teachers understand how students are doing over time.

```
# Function to generate a report for a student
def generate_report(student_name):
    if student_name in students:
        student_data = students[student_name]

        # Calculate the average grade if grades are available
        if "Math" in student_data and "Science" in student_data:
            average_grade = (student_data["Math"] + student_data["Science"]) / 2
            print(f"Report for {student_name}:")
            print(f"Age: {student_data['Age']}")
            print(f"Grade Level: {student_data['Grade Level']}")
            print(f"Math Grade: {student_data['Math']}")
            print(f"Science Grade: {student_data['Science']}")
            print(f"Average Grade: {average_grade:.2f}\n")
        else:
            print(f"No grades available for {student_name}.\n")
    else:
        print(f"Student {student_name} not found.")

# Example: Generating a report for a student
generate_report("Alice")
generate_report("Bob")
```

Explanation:

- **generate_report()** pulls the student's information from the dictionary and calculates their average grade. It then displays the student's details and grades in a clear format.
- This feature is essential for giving teachers an overview of each student's performance.

4. Putting It All Together

Now that we've implemented the three main features—student registration, grade tracking, and report generation—we can combine them into a complete classroom management system.

Here's an example of how you can structure the system to let teachers choose what they want to do:

```
# Main menu for the classroom management system
def main_menu():
    while True:
        print("Classroom Management System")
        print("1. Register a student")
        print("2. Add grades")
        print("3. Generate report")
        print("4. Exit")

        choice = input("Enter your choice: ")

        if choice == "1":
            register_student()
        elif choice == "2":
            student_name = input("Enter the student's name: ")
            add_grades(student_name)
        elif choice == "3":
            student_name = input("Enter the student's name: ")
            generate_report(student_name)
        elif choice == "4":
            print("Exiting the system.")
            break
```

```
else:  
    print("Invalid choice. Please try again.")
```

```
# Run the system  
main_menu()
```

Explanation:

- **main_menu()** presents the teacher with options to register a student, add grades, or generate reports. The system keeps running until the teacher chooses to exit.
- This complete system provides the core features needed for managing a classroom, and you can easily expand it by adding more functions, like editing or deleting student records.

5. Expanding the System

Once the basic classroom management system is in place, you can expand it with additional features to make it even more useful. Here are a few ideas:

- **Attendance tracking:** Add a feature that allows teachers to mark attendance and view attendance reports.
- **Advanced grading system:** Include different types of assignments (quizzes, homework, exams) and calculate weighted averages.
- **User authentication:** Add login functionality so that only authorized teachers can access the system.
- **Data storage:** Store student data in a database or a file, so it persists between sessions.

6. Why This Project Is Valuable

Building a classroom management system is a valuable project because:

- **It's practical:** The system solves real-world problems that teachers face every day.

- **It integrates multiple skills:** You'll apply Python concepts like data structures, functions, and loops while working on a meaningful project.
- **It's expandable:** You can easily add new features as you learn more advanced Python techniques.

This project gives you hands-on experience in building a complete application from start to finish, helping you strengthen your programming skills and apply them to solve real problems.

Summary

The **Classroom Management System** is a comprehensive capstone project that brings together many of the programming skills you've learned. By breaking the project into smaller tasks—like student registration, grade tracking, and report generation—you can build a functional system that helps teachers manage their classrooms more efficiently.

As you practice and expand the system, you'll continue developing your Python skills while creating a real-world tool that's both practical and rewarding. This project showcases how Python can be used to solve everyday problems, making it a valuable learning experience for both students and teachers. Keep building, and soon you'll be able to tackle even more complex projects with confidence!

Further Learning: Expanding Your Python Skills Beyond This Book

Congratulations on completing this book! You've learned a lot about Python and how it can be used to solve real-world problems, from basic programming concepts to building more advanced projects like a Classroom Management System. But this is just the beginning of your Python journey. There's so much more to explore, and with your solid foundation, you're ready to dive deeper into new topics and expand your skills.

In this section, we'll guide you on how to continue learning Python, introduce you to advanced concepts and libraries, and provide tips for applying your knowledge to real-world projects. Whether you're a student looking to advance your programming skills or a teacher seeking new ways to engage with Python, this chapter will help you discover what's next.

1. Exploring Advanced Python Concepts

Now that you have a good grasp of the basics, you can begin exploring more advanced Python topics. These concepts will help you write more efficient, powerful, and professional code.

a) Object-Oriented Programming (OOP)

While this book has introduced you to functions and data structures, learning **object-oriented programming (OOP)** will take your coding to the next level. OOP allows you to structure your programs using classes and objects, making it easier to organize and reuse code.

Here are some key concepts to explore in OOP:

- **Classes and Objects:** Learn how to create your own custom data types using classes.
- **Inheritance and Polymorphism:** Understand how you can extend existing classes and create more flexible programs.

- **Encapsulation:** Practice hiding the internal details of your code to make it easier to manage and modify.

b) Data Structures and Algorithms

Learning about **advanced data structures** and **algorithms** will sharpen your problem-solving skills and help you write more efficient programs. Some topics to focus on include:

- **Linked lists, stacks, and queues:** These are essential for understanding how data can be organized and accessed.
- **Trees and graphs:** These data structures are useful for solving complex problems like searching and pathfinding.
- **Sorting and searching algorithms:** Learn about common algorithms like merge sort, quicksort, and binary search to optimize your programs.

c) File Handling and Databases

While we've covered basic file handling, you can go further by learning how to store and retrieve larger amounts of data using **databases**. Python provides libraries like **SQLite** and **SQLAlchemy** that allow you to interact with databases in your programs. You can explore topics such as:

- **Creating and managing databases:** Learn how to design a database schema and use SQL queries to manage data.
- **Connecting Python to databases:** Practice using Python to insert, update, and retrieve data from a database.

2. Working with Python Libraries

One of Python's strengths is its vast ecosystem of libraries, which allow you to accomplish tasks quickly without reinventing the wheel. Here are some libraries you might want to explore as you continue learning:

a) NumPy and Pandas for Data Analysis

If you're interested in data science or working with large datasets, **NumPy** and **Pandas** are essential libraries. They provide powerful tools for:

- **Numerical computations:** NumPy allows you to perform complex mathematical operations on large arrays and matrices.
- **Data manipulation:** Pandas makes it easy to work with data tables, filter rows, and apply functions to your data.

b) Matplotlib and Seaborn for Data Visualization

To create clear and informative visualizations, **Matplotlib** and **Seaborn** are popular Python libraries. They allow you to generate a wide range of plots and graphs, making it easier to present data in a meaningful way.

Here's what you can do with these libraries:

- **Line plots, bar charts, histograms:** Use Matplotlib to create standard visualizations.
- **Advanced visualizations:** Seaborn simplifies the creation of heatmaps, pair plots, and violin plots for deeper data insights.

c) Web Development with Flask or Django

If you're interested in building web applications, you can explore Python web frameworks like **Flask** and **Django**. These frameworks allow you to create dynamic websites and web services using Python. Some topics to explore include:

- **Setting up a web server:** Learn how to handle requests and serve web pages.
- **Building a REST API:** Create web services that allow other programs to interact with your application.
- **Managing user authentication:** Implement login systems and manage user data securely.

d) Automation with Python

Python is also widely used for **automating repetitive tasks**. You can explore libraries like **Selenium** for automating browser tasks or **openpyxl** for automating Excel tasks.

3. Real-World Projects to Sharpen Your Skills

The best way to continue improving your Python skills is by working on real-world projects. Here are some project ideas that will challenge you to apply what you've learned and help you develop new skills:

a) Build a Personal Budget Tracker

Create a program that helps you track income and expenses. You can add features like generating reports, visualizing spending patterns with charts, and setting budget goals.

b) Create a Weather Dashboard

Use an API like the **OpenWeatherMap API** to fetch weather data and display it in a user-friendly interface. You can show current weather conditions, forecasts, and even weather alerts for a specific location.

c) Develop a To-Do List App

Design a simple to-do list app where users can add, edit, and mark tasks as completed. You can extend it by adding a calendar feature or integrating it with notifications.

d) Build a Chatbot

Create a chatbot using Python that can answer questions, provide information, or even help with customer service. You can use libraries like **NLTK** or **spaCy** to process and understand language.

4. Joining Python Communities

Learning doesn't stop once you finish this book. One of the best ways to grow as a programmer is by joining the **Python community**. Engaging with others who share your passion for coding can provide support, inspiration, and opportunities to learn.

Here's how you can connect with Python communities:

- **Online forums:** Websites like **Stack Overflow**, **Reddit's Python community**, and **Python Discord** offer places to ask questions, share your projects, and get feedback.
- **Contribute to open source:** Get involved in open-source projects on platforms like **GitHub**. Contributing to real-world projects is a great way to learn from others and improve your coding skills.
- **Attend Python meetups:** Check out local Python meetups or virtual conferences to network with other Python enthusiasts and learn from their experiences.

5. Expanding into New Areas of Python

Python is a versatile language with applications in many fields. As you continue learning, you might discover new areas that interest you, such as:

- **Artificial Intelligence and Machine Learning:** Python is the go-to language for machine learning, with libraries like **TensorFlow**, **Keras**, and **scikit-learn**. You can start building predictive models or even work on AI projects like image recognition or natural language processing.
- **Cybersecurity:** Python is also widely used in cybersecurity for writing scripts to detect vulnerabilities, automate security tasks, and analyze networks.
- **Game Development:** If you're interested in creating games, you can use libraries like **Pygame** to build your own 2D games.

6. Setting Learning Goals and Staying Motivated

As you continue learning Python, it's important to set clear **learning goals** and stay motivated. Here are some tips:

- **Set project-based goals:** Aim to complete a new project every few months, focusing on areas that interest you or challenge your skills.
- **Stay consistent:** Practice coding regularly, even if it's just for 30 minutes a day.
- **Track your progress:** Keep a journal of your coding journey or track the projects you've completed. This can help you see how far you've come and stay motivated to keep improving.

Summary

Learning Python doesn't stop when this book ends—there are endless opportunities to grow and expand your skills. By exploring advanced topics like object-oriented programming, working with powerful libraries, and building real-world projects, you'll continue to deepen your understanding of Python. Engaging with the Python community and setting goals for your learning will keep you motivated and open new doors to exciting possibilities.

With your solid foundation, you're ready to tackle more challenging projects, explore new areas like data science, web development, and machine learning, and apply your Python skills to solve real-world problems. Keep building, keep learning, and enjoy the journey!

Conclusion: Your Journey Ahead

How to Continue Problem Solving with Python

Now that you've learned the fundamentals of Python and how to apply it to solve real-world problems, your journey with Python is just beginning. The skills you've gained will help you approach new challenges with confidence, whether you're working on school projects, exploring personal interests, or preparing for a career in technology. But learning to solve problems with Python is an ongoing process, and there are many ways to continue growing your abilities.

In this section, we'll explore how you can keep building on what you've learned, develop new skills, and approach problem-solving with Python in even more effective ways.

1. Keep Practicing Through Projects

One of the best ways to improve your problem-solving skills is to keep coding regularly by working on new projects. The more you practice, the more comfortable you'll become with thinking like a programmer and breaking problems down into smaller, manageable steps.

Here are a few ideas for projects that will challenge you while applying what you've already learned:

- **Build a calculator:** Create a fully functional calculator that performs basic operations like addition, subtraction, multiplication, and division. You can extend it with more complex features, like handling square roots or exponentiation.

- **Create a personal journal app:** Design a program that allows you to enter daily journal entries and stores them for future reference. You can even add features like searching through entries or categorizing them by topic.
- **Solve coding challenges:** Websites like **LeetCode**, **HackerRank**, and **Codewars** offer coding problems at varying levels of difficulty. These platforms help you apply your knowledge to solve challenges in data structures, algorithms, and logic.

By constantly seeking out new projects, you'll discover new areas of interest and gain deeper insights into programming.

2. Explore More Advanced Topics

As you continue to develop your Python skills, you'll want to explore more advanced programming topics. These new concepts will help you approach problems from different angles and provide more efficient solutions.

Here are a few advanced areas worth exploring:

- **Recursion:** Recursion is a technique where a function calls itself to solve smaller parts of a larger problem. It's commonly used in problems like factorial calculations or traversing tree structures. Learning recursion will help you think about problems in a new way.
- **Algorithm design:** Algorithms are step-by-step procedures for solving problems. By learning about classic algorithms (such as sorting, searching, and dynamic programming), you can find optimal solutions to complex challenges.
- **Object-oriented programming (OOP):** OOP is a paradigm that allows you to create classes and objects, which makes your code more modular and reusable. Mastering OOP will

enable you to design more sophisticated programs and larger projects.

Each of these topics adds new tools to your problem-solving toolkit, giving you more ways to tackle complex challenges with Python.

3. Learn from Others

Problem-solving is often more effective when you're part of a learning community. By interacting with other Python learners, you can get fresh perspectives, exchange ideas, and even receive guidance on how to approach tricky problems. Engaging with a community also helps you stay motivated and inspired.

Here are some ways to connect with others:

- **Join online forums:** Platforms like **Stack Overflow**, **Reddit's r/learnpython**, and **Python Discord** are great places to ask questions, share your projects, and get advice from other Python learners.
- **Participate in coding competitions:** Websites like **Kaggle** and **TopCoder** host coding competitions where you can test your skills against real-world problems. These competitions not only challenge your problem-solving skills but also introduce you to new approaches and solutions.
- **Contribute to open-source projects:** Many Python projects are open-source, meaning anyone can contribute to their development. By contributing to an open-source project, you'll gain valuable experience, collaborate with experienced developers, and make meaningful contributions to the Python community.

By learning from others and sharing your knowledge, you'll strengthen your problem-solving skills and become part of a supportive programming community.

4. Develop a Problem-Solving Mindset

Beyond learning new techniques and tools, problem-solving is about developing the right mindset. Effective problem solvers approach challenges with a structured and logical way of thinking. Here are a few key habits to develop that will help you continue solving problems with Python:

a) Break Down Problems

When faced with a large problem, always break it down into smaller, more manageable parts. Solving each part individually makes the overall challenge less overwhelming. For example, if you're tasked with building a game, focus on one aspect at a time, such as designing the game's rules before moving on to the user interface.

b) Test and Debug Regularly

Don't wait until the end to test your code. Test each piece as you build it to catch bugs early. Debugging your code as you go will make the process smoother and help you understand where problems might arise. If your program isn't working as expected, don't get frustrated—debugging is part of the learning process and helps you improve your skills.

c) Stay Curious and Experiment

Keep exploring new ways to solve problems. If you encounter a new concept, take time to experiment with it. Write small programs to test out ideas and see how different solutions work in practice. The more curious and creative you are, the more confident you'll become as a problem solver.

d) Persevere Through Challenges

Problem-solving isn't always easy, and you may encounter obstacles along the way. The key is to persevere and keep trying different approaches. When you're stuck, step back, take a break, and return to the problem with fresh eyes. Remember, every challenge you overcome is an opportunity to learn and grow.

5. Apply Python to Real-World Problems

As you continue learning Python, look for opportunities to apply your skills to real-world problems. Whether it's automating a repetitive task, analyzing data, or developing tools for others, solving real problems with Python will

give you valuable experience and make your learning journey more rewarding.

Here are a few ways you can use Python in real-world situations:

- **Automation:** Use Python to automate tasks like renaming files, generating reports, or processing large amounts of data. Automating tasks saves time and shows the practical value of your coding skills.
- **Data analysis:** If you're interested in data science, practice analyzing real datasets using Python's libraries like **Pandas** and **NumPy**. This could involve anything from analyzing school grades to examining weather patterns.
- **Build helpful tools:** If you have a hobby or interest, think about how Python could be used to build tools that make it easier or more fun. For example, you could create a workout tracker, a recipe organizer, or even a language-learning app.

By solving real-world problems, you'll deepen your understanding of Python and see how powerful and flexible the language can be.

Summary

Continuing your journey of problem-solving with Python is an exciting opportunity to build on the skills you've learned. By practicing regularly through projects, exploring advanced topics, and learning from others, you'll keep growing as a programmer. As you apply Python to real-world problems and develop a problem-solving mindset, you'll discover new ways to tackle challenges and unlock your full potential as a coder.

Remember, Python is a tool that empowers you to create, explore, and solve. Keep learning, stay curious, and enjoy the journey ahead!

Recommended Resources and Communities for Python Learners

As you continue your journey with Python, there are many resources and communities that can help you expand your knowledge, improve your skills, and stay motivated. Whether you're looking for tutorials, coding challenges, or a place to ask questions, the Python community is full of learners and experts eager to share their knowledge. In this section, we'll introduce you to some of the best resources and communities that will support you as you grow as a Python programmer.

1. Learning Platforms and Tutorials

There are countless online platforms that offer Python tutorials, courses, and coding exercises. These resources range from beginner to advanced levels, so you can find content that matches your skill level and goals.

a) Codecademy

Codecademy is an interactive platform that offers a Python course designed for beginners. The lessons are hands-on, allowing you to write Python code directly in your browser while receiving real-time feedback. Codecademy's step-by-step approach makes it easy to grasp the fundamentals while building your confidence through practice.

Website: www.codecademy.com

b) Real Python

Real Python is a website dedicated to teaching Python through in-depth tutorials, articles, and video lessons. It covers a wide range of topics, from Python basics to more advanced subjects like web development, data analysis, and automation. Real Python also has a beginner-friendly guide that will help you set up your coding environment and get started.

Website: www.realpython.com

c) Python.org

The official Python website, **python.org**, is a great resource for learning and documentation. It offers an excellent tutorial for beginners, along with reference guides, community forums, and news on the latest Python updates. As you advance, you'll find the documentation here invaluable for understanding how to use Python's built-in libraries and features.

Website: www.python.org

d) Kaggle

For learners interested in data science, **Kaggle** provides a range of free courses and tutorials that focus on Python for data analysis, machine learning, and artificial intelligence. Kaggle also offers real-world datasets that you can practice with, along with coding competitions where you can test your skills against other learners.

Website: www.kaggle.com

e) SoloLearn

SoloLearn is a mobile-friendly platform offering bite-sized lessons in Python and other programming languages. It's great for learning on the go, and its interactive challenges and quizzes help reinforce concepts in a fun way. You can also engage with the community by sharing your own projects and solutions.

Website: www.sololearn.com

2. Communities for Python Learners

Engaging with other Python learners and developers is one of the best ways to stay motivated, find support, and share knowledge. By joining a community, you'll have a place to ask questions, get feedback on your code, and learn from others' experiences.

a) Stack Overflow

Stack Overflow is one of the largest online communities for programmers. It's a question-and-answer site where developers from around the world come to ask questions and provide solutions to coding problems. Whether you're stuck on a specific error or need advice on how to approach a problem, Stack Overflow is an excellent place to find answers.

Website: www.stackoverflow.com

b) Reddit: r/learnpython

The **r/learnpython** community on Reddit is a welcoming space for Python beginners and intermediate learners. You can ask questions, share your projects, and get feedback from others. The community is active, and you'll often find helpful discussions on a wide range of Python topics, from beginner exercises to advanced projects.

Website: www.reddit.com/r/learnpython

c) Python Discord

Python Discord is a large community of Python enthusiasts who gather to chat, learn, and collaborate. The Discord server offers dedicated channels for different Python topics, coding challenges, and live coding help. Python Discord also hosts regular events like coding jams and competitions, which are great for testing your skills and connecting with others.

Website: www.pythondiscord.com

d) GitHub

GitHub is a platform for collaborating on open-source projects. By joining the Python community on GitHub, you can explore a wide variety of Python projects, contribute to existing ones, or start your own. It's an excellent way to practice coding in a real-world environment and learn from others' code.

Website: www.github.com

e) Exercism

Exercism is an open-source platform that offers coding challenges in Python and other languages. What sets Exercism apart is its mentorship feature—you can submit your solutions and receive feedback from experienced Python developers. This is a fantastic way to improve your code, learn new techniques, and engage with the Python community.

Website: www.exercism.io

3. YouTube Channels for Python Learners

Sometimes, watching a tutorial is the best way to learn. YouTube is full of channels that offer high-quality Python lessons, tutorials, and coding

challenges. Here are a few channels you can explore:

a) Corey Schafer

Corey Schafer's YouTube channel is known for its clear and in-depth Python tutorials. His videos cover everything from basic Python programming to web development, automation, and data science. Each tutorial is beginner-friendly but comprehensive, making it easy to follow along and learn new concepts.

Channel: [Corey Schafer on YouTube](#)

b) Tech With Tim

Tech With Tim is another great YouTube channel for Python learners. Tim covers a wide range of Python topics, including game development with Pygame, web scraping, and machine learning. His tutorials are easy to follow, and he often shares tips and tricks to improve your coding workflow.

Channel: [Tech With Tim on YouTube](#)

c) CS Dojo

CS Dojo focuses on Python programming, algorithms, and coding interview preparation. If you're interested in building your problem-solving skills and preparing for coding challenges or technical interviews, this channel provides excellent tutorials and explanations.

Channel: [CS Dojo on YouTube](#)

4. Books and Other Reading Materials

In addition to online courses and videos, there are many books that can help deepen your Python knowledge. Here are a few highly recommended books for learners at different levels:

a) "Python Crash Course" by Eric Matthes

This is one of the most popular books for beginners, offering a fast-paced, no-nonsense introduction to Python. It covers all the basics and includes projects that help solidify your understanding. By the end of the book, you'll have built real-world applications like a simple game and a data visualization tool.

b) “Automate the Boring Stuff with Python” by Al Sweigart

If you’re looking to use Python to solve everyday tasks, this book is perfect. It teaches you how to automate repetitive tasks, such as organizing files, scraping websites, and processing Excel data. It’s practical, easy to follow, and full of examples that demonstrate Python’s versatility.

c) “Fluent Python” by Luciano Ramalho

For more advanced learners, “Fluent Python” dives deep into Python’s features and best practices. It’s a great resource if you want to write more efficient, readable, and idiomatic Python code. The book covers topics like data structures, object-oriented programming, and concurrency in detail.

Summary

As you continue your Python journey, you’ll find plenty of resources and communities to support your learning. Whether you prefer interactive platforms, books, videos, or forums, there are countless ways to deepen your knowledge and stay motivated. Engaging with the Python community can help you find answers to questions, get feedback on your code, and connect with others who share your passion for programming.

By using these resources and staying involved with the community, you’ll continue growing as a Python programmer and open the door to new opportunities and challenges. Keep exploring, keep coding, and enjoy the process!

A Note to Teachers: How to Incorporate This Book into Your Curriculum

This book is designed not just for students but also as a resource for teachers looking to integrate Python into their classrooms. Whether you're teaching computer science, mathematics, or another subject, Python is a powerful tool for problem-solving, creativity, and engagement. The book provides hands-on projects, practical examples, and clear explanations that can easily be adapted to different learning environments and student skill levels.

In this section, we'll explore how you, as a teacher, can use this book effectively in your curriculum, incorporating Python into lessons in a way that is both engaging and educational for students.

1. Using the Book's Structure for Lesson Planning

The structure of the book is designed to gradually build students' knowledge and confidence in Python. Here's how you can use this structure to guide your lesson planning:

a) Part I: Python Basics for Problem Solving

The first part of the book introduces the fundamentals of Python. These lessons are perfect for introducing new students to programming. As a teacher, you can use this section to:

- **Start with the basics:** Use the chapters on variables, data types, and control structures to introduce students to Python in a step-by-step way.
- **Encourage hands-on coding:** Each chapter includes practical examples and exercises that students can code themselves. Set these exercises as in-class activities or homework assignments.

- **Focus on problem-solving:** Emphasize how Python can be used to solve real-world problems by relating coding tasks to problems students might encounter in other subjects, like math or science.

b) Part II: Problem-Solving with Python

This section focuses on applying Python to real-world scenarios, making it ideal for project-based learning. You can encourage students to:

- **Solve math and science problems with Python:** The chapters on math and geometry problems can be used in cross-curricular lessons, showing students how programming helps in solving complex problems.
- **Assign practical projects:** Use the real-world applications (such as building a simple quiz or solving geometry problems) as class projects. These projects will not only solidify their Python skills but also teach them how to break down larger problems into smaller, manageable parts.

c) Part III and IV: Intermediate Python and Advanced Projects

Once students have a strong foundation, you can introduce them to more advanced topics. This is where students can start working on bigger, more independent projects.

- **Teach libraries like NumPy and Pandas:** Use these chapters to introduce students to data analysis, showing them how to manipulate and analyze datasets.
- **Introduce web scraping or automation:** If your students are interested in real-world applications, these chapters can help them learn how to collect data from websites or automate repetitive tasks.

- **Encourage independent projects:** The advanced chapters provide the perfect opportunity for students to design and build their own projects, based on their interests.

d) Capstone Projects

The capstone project ideas in the book can be adapted into major class assignments or even end-of-term projects. For example, the **Classroom Management System** project can be a collaborative project that teaches students about planning, designing, and building a complete application.

2. Adapting the Content for Different Learning Levels

Python is accessible for learners of all levels, and this book can be adapted to suit both beginners and more advanced students. Here's how you can tailor the material based on your students' skills:

a) For Beginners

- **Focus on fundamentals:** Introduce Python concepts slowly, ensuring that students grasp the basics before moving on to more advanced topics.
- **Use simpler projects:** The early chapters on loops, conditionals, and functions provide simple, ready-made examples that beginners can follow and modify. Encourage students to experiment with these examples.

b) For Intermediate Learners

- **Encourage problem-solving:** Once students are familiar with basic syntax, use the book's problem-solving examples to help them think more critically. Ask students to apply their knowledge to new problems, encouraging creativity and independent thinking.
- **Introduce libraries:** Intermediate students will benefit from learning Python libraries like **Matplotlib** for data visualization

or **Selenium** for web automation. The book's chapters on these topics provide accessible introductions.

c) For Advanced Students

- **Capstone projects:** Advanced students should be encouraged to work on more complex, independent projects. The capstone examples can serve as inspiration for custom projects that challenge their understanding of Python and problem-solving.
- **Encourage collaboration:** Use the capstone project ideas to form small teams and encourage students to collaborate. This will help them develop not only their technical skills but also teamwork and communication skills.

3. Integrating Python Across Different Subjects

Python isn't limited to computer science classes. It can be used across a variety of subjects to enhance learning and problem-solving. Here's how you can integrate Python into other parts of your curriculum:

a) Mathematics

Python is a fantastic tool for solving math problems, visualizing data, and performing complex calculations. Use the chapters on **solving math problems with Python** to show students how programming can help solve equations, work with geometry, and analyze data.

b) Science

Python is widely used in fields like physics, chemistry, and biology for data analysis and simulation. The **physics and chemistry** chapters will allow you to create lessons that connect Python with scientific concepts, showing students how they can use programming to solve science problems.

c) Data Science and Research Projects

If you teach a course that involves research or data analysis, Python is an invaluable tool. Introduce students to data science with the chapters on **NumPy** and **Pandas**, and encourage them to apply these skills to real datasets, whether for school projects or personal research.

d) Engineering

Python is widely used in engineering fields to automate tasks, simulate systems, and manage data. The book's chapters on **automation** and **machine learning** can provide an entry point for students interested in using Python for engineering applications.

4. Encouraging Student Collaboration and Problem-Solving

One of the best ways to engage students with Python is by encouraging them to collaborate and solve problems together. Here's how you can foster teamwork and creativity in the classroom:

a) Group Projects

Assign group projects where students must collaborate to design and build a Python application. For example, you could have them work together on a capstone project like the **Classroom Management System** or a simple game using Python.

b) Coding Challenges

Host coding challenges or hackathons where students solve problems within a set time limit. These events foster a competitive yet collaborative spirit and encourage students to think creatively under pressure.

c) Peer Code Review

Encourage students to review each other's code. This helps them learn from their peers and improves their coding practices. You can organize code review sessions where students present their work and receive feedback from classmates.

5. Providing Resources for Self-Paced Learning

Not all students learn at the same pace, so it's important to provide resources for those who want to move ahead or revisit earlier topics. Here's how you can help students continue learning outside the classroom:

a) Online Resources

Point students to the **recommended resources and communities** mentioned earlier in the book. Websites like **Codecademy**, **Kaggle**, and

Real Python offer great opportunities for students to learn at their own pace and deepen their knowledge.

b) Supplementary Exercises

Provide additional exercises or challenges for students who finish their work early. These can include coding puzzles, algorithm challenges, or more advanced Python projects.

c) Project-Based Learning

Encourage students to pursue their own Python projects outside of class. Whether it's a game, a personal tool, or a data analysis project, letting students follow their interests will keep them motivated and engaged with Python.

Summary

As a teacher, this book gives you the flexibility to incorporate Python into your curriculum in a way that's engaging and educational for students of all levels. By following the book's structure, adapting the content to your students' needs, and integrating Python across different subjects, you can create a learning environment that fosters problem-solving, creativity, and collaboration.

Encouraging students to work on projects, participate in coding challenges, and collaborate with their peers will not only improve their Python skills but also teach them the critical thinking and teamwork skills they need to succeed in today's world.